
IFM 3.1: Compilerbau (Winter 2025/26)

BC George & Carsten Gips

23. Juni 2026

Inhaltsverzeichnis

I. Syllabus	1
1. Syllabus	2
1.1. Kursbeschreibung	2
1.2. Überblick Modulinhalte	2
1.3. Team	3
1.4. Kursformat	3
1.5. Fahrplan	3
1.6. Prüfungsform, Note und Credits	5
1.7. Materialien	6
1.8. Förderungen und Kooperationen	6
1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)	6
1.9. LICENSE	6
II. Vorlesungsunterlagen	7
2. Überblick	8
2.1. Struktur eines Compilers	8
2.1.1. Sprachen verstehen, Texte transformieren	8
2.1.2. Compiler: Big Picture	9
2.1.3. Lexikalische Analyse: Wörter (“Token”) erkennen	11
2.1.4. Syntaxanalyse: Sätze erkennen	12
2.1.5. Vorschau: Parser implementieren	12
2.1.6. Semantische Analyse: Bedeutung erkennen	13
2.1.7. Zwischencode generieren	14
2.1.8. Code optimieren	14
2.1.9. Code generieren	14
2.1.10. Probleme	15
2.1.11. Wrap-Up	15
2.2. Bandbreite der Programmiersprachen	16
2.2.1. 99 Bottles of Beer	16
2.2.2. Imperativ, Hardwarenah: C	16
2.2.3. Imperativ, Objektorientiert: Java	17
2.2.4. Logisch: Prolog	18
2.2.5. Funktional: Haskell	18

2.2.6. Brainfuck	19
2.2.7. Programmiersprache Lox	19
2.2.8. Wrap-Up	20
2.3. Anwendungen	21
2.3.1. Anwendung: Compiler	21
2.3.2. Anwendung: Interpreter	21
2.3.3. Anwendung: Virtuelle Maschinen	22
2.3.4. Anwendung: C-Toolchain	22
2.3.5. Anwendung: C++-Compiler	22
2.3.6. Anwendung: Bugfinder	23
2.3.7. Anwendung: Pandoc	23
2.3.8. Was bringt mir das?	24
2.3.9. Wrap-Up	26
3. Lexikalische Analyse	28
3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)	28
3.1.1. Motivation	28
3.1.2. Endliche Automaten	28
3.1.3. Reguläre Sprachen	30
3.1.4. Wrap-Up	33
3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)	33
3.2.1. Wiederholung	33
3.2.2. Motivation	34
3.2.3. Lexer	34
3.2.4. Wrap-Up	35
3.3. Lexer mit ANTLR generieren	36
3.3.1. Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom	36
3.3.2. Definition wichtiger Begriffe	37
3.3.3. Typische Muster für Erstellung von Token	37
3.3.4. "Hello World" mit ANTLR (Lexer)	38
3.3.5. Verhalten des Lexers	45
3.3.6. Attribute und Aktionen	46
3.3.7. Wrap-Up	47
3.4. Lexer: Handcodierte Implementierung	49
3.4.1. Lexer: Erzeugen eines Token-Stroms aus einem Zeichenstrom	50
3.4.2. Manuelle Implementierung: Rekursiver Abstieg	50
3.4.3. Read-Ahead: Unterscheiden von "<" und "<="	51
3.4.4. Puffern des Input-Stroms: Double Buffering	52
3.4.5. Typische Muster für Erstellung von Token	53
3.4.6. Fehler bei der Lexikalischen Analyse	54
3.4.7. Wrap-Up	55

4. Syntaktische Analyse	57
4.1. CFG	57
4.1.1. Wiederholung	57
4.1.2. Motivation	57
4.1.3. Kontextfreie Grammatiken und Sprachen	59
4.1.4. Syntaxanalyse	60
4.1.5. Wrap-Up	60
4.2. Parser mit ANTLR generieren	61
4.2.1. Hello World	62
4.2.2. Expressions und Vorrang (Operatoren)	66
4.2.3. Kontext-Objekte für Parser-Regeln	71
4.2.4. Benannte Regel-Elemente oder Alternativen	71
4.2.5. Arbeiten mit ANTLR-Listeners	72
4.2.6. Arbeiten mit dem Visitor-Pattern	73
4.2.7. Arbeiten mit Pattern Matching	75
4.2.8. Vom Parse-Tree zum AST	76
4.2.9. Eingebettete Aktionen und Attribute	78
4.2.10. Ausblick	79
4.2.11. Wrap-Up	79
4.3. LL-Parser	80
4.3.1. Wiederholung	80
4.3.2. Motivation	81
4.3.3. Syntaxanalyse	81
4.3.4. LL(k)-Grammatiken	81
4.3.5. Wrap-Up	84
4.4. LL-Parser selbst implementiert	85
4.4.1. Erinnerung Lexer: Zeichenstrom => Tokenstrom	86
4.4.2. Grundidee LL-Parser	86
4.4.3. Alternative Subregeln	87
4.4.4. Optionale Subregeln: Eins oder keins	87
4.4.5. Optionale Subregeln: Mindestens eins	87
4.4.6. LL(1)-Parser	88
4.4.7. Detail: <i>match()</i> und <i>consume()</i>	88
4.4.8. Vorrangregeln	88
4.4.9. Linksrekursion	89
4.4.10. Assoziativität	90
4.4.11. LL(k)-Parser	90
4.4.12. LL(k)-Parser: Implementierung mit Ringpuffer	90
4.4.13. Wrap-Up	91
5. Semantische Analyse	93
5.1. SymbTab0: Überblick Symboltabellen	93
5.1.1. Was passiert nach der Syntaxanalyse?	93
5.1.2. Semantische Analyse und Symboltabellen	95

5.1.3.	Syntax und Semantik	95
5.1.4.	Aufgaben der semantischen Analyse	96
5.1.5.	Einfache Verwaltung von Variablen primitiven Typs	97
5.1.6.	Was kann jetzt weiter passieren?	99
5.1.7.	Wo werden Verweise in Symboltabellen gebraucht?	100
5.1.8.	Grenzen der semantischen Analyse	100
5.1.9.	Wrap-Up	100
5.2.	SymbTab1: Nested Scopes	101
5.2.1.	Scopes und Name Spaces	101
5.2.2.	Symbole und (nested) Scopes	101
5.2.3.	Nested Scopes: Symbole und Scopes	103
5.2.4.	Nested Scopes: Definieren und Auflösen von Namen	104
5.2.5.	Nested Scopes: Listener	105
5.2.6.	Löschen von Symboltabellen	106
5.2.7.	Wrap-Up	106
5.3.	SymbTab2: Funktionen	107
5.3.1.	Funktionen und Scopes	107
5.3.2.	Erweiterung des Klassendiagramms für Funktions-Scopes	109
5.3.3.	Funktionen sind Symbole <i>und</i> Scopes	109
5.3.4.	Funktionen: Listener	109
5.3.5.	Wrap-Up	110
5.4.	SymbTab3: Strukturen und Klassen	111
5.4.1.	Strukturen	112
5.4.2.	Strukturen: Erweiterung der Symbole und Scopes	113
5.4.3.	Strukturen: Auflösen von Namen	113
5.4.4.	Klassen	113
5.4.5.	Klassen: Erweiterung der Symbole und Scopes	115
5.4.6.	Klassen: Auflösen von Namen	115
5.4.7.	Wrap-Up	116
6.	Interpreter	118
6.1.	AST-basierte Interpreter: Basics	118
6.1.1.	Aufgaben im Interpreter	119
6.1.2.	AST-basierte Interpreter: Visitor-Dispatcher	119
6.1.3.	Auswertung von Literalen und Ausdrücken	120
6.1.4.	Kontrollstrukturen	121
6.1.5.	Zustände: Auswerten von Anweisungen	121
6.1.6.	Detail: Felder im Interpreter	122
6.1.7.	Ausführen einer Variablendeklaration	123
6.1.8.	Ausführen einer Zuweisung	123
6.1.9.	Blöcke: Umgang mit verschachtelten Environments	124
6.1.10.	Wrap-Up	124
6.2.	AST-basierte Interpreter: Funktionen und Klassen	125
6.2.1.	Funktionen	126

6.2.2.	Ausführen einer Funktionsdeklaration	126
6.2.3.	Ausführen eines Funktionsaufrufs	127
6.2.4.	Funktionsaufruf: Rückgabewerte	127
6.2.5.	Native Funktionen	128
6.2.6.	Klassen und Instanzen I	129
6.2.7.	Klassen und Instanzen II	130
6.2.8.	Zugriff auf Methoden (und Attribute)	130
6.2.9.	Methoden und <i>this</i> oder <i>self</i>	130
6.2.10.	Wrap-Up	131
7.	Programmiersprachen und -konzepte	133
7.1.	Einführung in C++ (Erinnerungen an C)	133
7.1.1.	Warum?	135
7.1.2.	Hello World!	135
7.1.3.	Variablen, Operatoren, Kontrollfluss	137
7.1.4.	Ein- und Ausgabe mit <i>printf</i> und <i>cin/cout</i>	137
7.1.5.	Sichtbarkeit und Gültigkeit und Namespaces	138
7.1.6.	Arrays und Vektoren in C++	139
7.1.7.	Alias-Namen für Typen mit <i>typedef</i> und <i>using</i>	140
7.1.8.	Erinnerungen an C - Vergleich mit C++	140
7.1.9.	Wrap-Up	152
7.2.	C++: Pointer und Referenzen	154
7.2.1.	Virtueller Speicher	156
7.2.2.	Konzept eines Pointers	157
7.2.3.	Speicherverwaltung	160
7.2.4.	Pointer und Arrays	165
7.2.5.	Referenzen in C++	167
7.2.6.	Call-by-Reference Semantik in C++	169
7.2.7.	Vergleich Pointer mit Referenzen	171
7.2.8.	Wrap-Up	172
7.3.	C++: Klassen	176
7.3.1.	OOP in C++	177
7.3.2.	Objekte erzeugen mit Konstruktoren	179
7.3.3.	Default-Konstruktoren	179
7.3.4.	C++: Trennung .h und .cpp	179
7.3.5.	Konstruktoren: Normale (Java-like) Initialisierung	180
7.3.6.	Konstruktoren: Initialisierungslisten	180
7.3.7.	Zwang zu Initialisierungslisten	181
7.3.8.	C++11 und delegierende Konstruktoren	181
7.3.9.	C++ und explizite Konstruktoren	182
7.3.10.	Wrap-Up	182
7.4.	C++: Big 3	183
7.4.1.	Big Three	184
7.4.2.	delete this?	187

7.4.3.	C++11: <i>default</i> und <i>delete</i>	188
7.4.4.	Statische Methoden und Attribute	188
7.4.5.	Konstante Methoden und Kontexte	189
7.4.6.	Wrap-Up	189
7.5.	C++: Operatoren	191
7.5.1.	Überladen von Operatoren <i>in</i> Klassen	192
7.5.2.	Überladen von Operatoren <i>außerhalb</i> von Klassen	192
7.5.3.	Überladen von Operatoren <i>außerhalb</i> von Klassen (cnt.)	193
7.5.4.	Meine Freunde dürfen in mein Wohnzimmer	193
7.5.5.	(Fast) alle Operatoren lassen sich überladen	194
7.5.6.	Implizite Typkonvertierungen bei Aufruf	194
7.5.7.	Anmerkung zu “++” und “-” Operatoren: Präfix und Postfix	195
7.5.8.	Weitere Anmerkungen	195
7.5.9.	Wrap-Up	196
7.6.	C++: Vererbung und Polymorphie	197
7.6.1.	Vererbung: “IS-A”-Beziehung zw. Klassen	198
7.6.2.	Abstrakte Klassen	199
7.6.3.	Polymorphie: Was passiert im folgenden Beispiel?	200
7.6.4.	Polymorphie: statisch und dynamisch	200
7.6.5.	Dynamisches Binden geht auch in C++ ...	200
7.6.6.	Vorsicht Slicing	201
7.6.7.	Anmerkungen zu Polymorphie in C++	202
7.6.8.	Mehrfachvererbung in C++	202
7.6.9.	Mehrfachvererbung in C++: Virtuelle Basisklassen	204
7.6.10.	Sonderregeln bei virtueller Ableitung	204
7.6.11.	Mehrfachvererbung in C++ ist ein recht kompliziertes Thema	205
7.6.12.	Wrap-Up	205

III. Praktikum 207

8. Blatt 01: Reguläre Sprachen 209

8.1.	A1.1: Sprachen von regulären Ausdrücken (1P)	209
8.2.	A1.2: Bezeichner in Programmiersprachen (3P)	209
8.3.	A1.3: Gleitkommazahlen in Programmiersprachen (2P)	209
8.4.	A1.4: Mailadressen? (1P)	210
8.5.	A1.5: Der zweitletzte Buchstabe (1P)	210
8.6.	A1.6: Sprache einer regulären Grammatik (2P)	210

9. Blatt 02: CFG 211

9.1.	A2.1: PDA (3P)	211
9.2.	A2.2: Akzeptierte Sprache (2P)	211
9.3.	A2.3: Kontextfreie Sprache (2P)	212
9.4.	A2.4: Kontextfreie Grammatik (3P)	212

10. Blatt 03: ANTLR	213
10.1. Zusammenfassung	213
10.2. Methodik	213
10.3. Sprachdefinition	214
10.3.1. Anweisungen (<i>Statements</i>)	214
10.3.2. Ausdrücke (<i>Expressions</i>)	214
10.3.3. Bezeichner	215
10.3.4. Variablen	215
10.3.5. Kommentare	215
10.3.6. Kontrollstrukturen	215
10.3.7. Datentypen	216
10.3.8. Beispiele	217
10.4. Aufgaben	217
10.4.1. A3.1: Grammatik (4P)	217
10.4.2. A3.2: Pretty Printer (3P)	217
10.4.3. A3.3: AST (3P)	218
11. Blatt 04: Lexer und Parser selbst implementiert	220
11.1. Zusammenfassung	220
11.2. Methodik	220
11.3. Sprachdefinition	220
11.3.1. Eingebaute Funktionen	221
11.3.2. Operatoren	221
11.3.3. Kontrollstrukturen (If-Else)	222
11.3.4. Variablen: Bindings mit <i>def</i> anlegen	223
11.3.5. Funktionen mit <i>defn</i> definieren	223
11.3.6. Lokale Scopes mit <i>let</i>	223
11.3.7. Rekursion	224
11.3.8. Datenstrukturen	224
11.4. Aufgaben	224
11.4.1. A4.1: Kontextfreie Grammatik (1P)	224
11.4.2. A4.2: Grammatik (2P)	225
11.4.3. A4.3: Lexer (2P)	225
11.4.4. A4.4: Parser mit <i>recursive descent</i> (3P)	226
11.4.5. A4.5: AST (1P)	226
11.4.6. A4.6: Recherche und Diskussion (1P)	226
12. Blatt 05: Semantische Analyse	227
12.1. Zusammenfassung	227
12.2. Methodik	227
12.3. Sprachdefinition	227
12.4. Semantik	228
12.5. Aufgaben	229
12.5.1. A5.1: Grammatik und Sprache (1P)	229

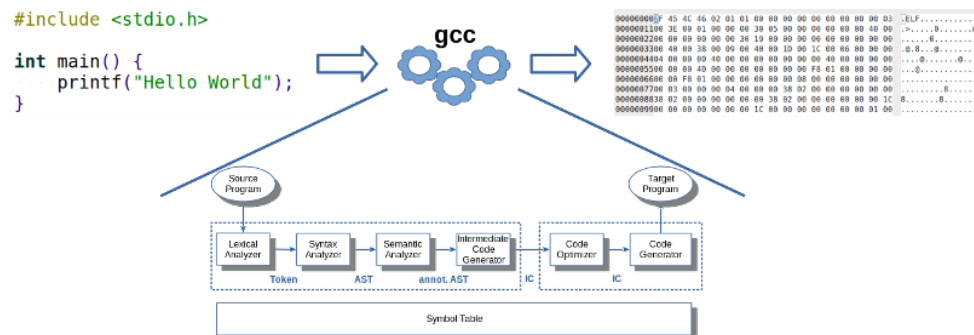
12.5.2. A5.1: AST (2P)	229
12.5.3. A5.2: Aufbau der Symboltabelle (5P)	231
12.5.4. A5.3: Symboltabelle: Referenzierungen und Funktionsaufrufe (2P)	232
13. Blatt 06: Interpreter	233
13.1. Zusammenfassung	233
13.2. Methodik	233
13.3. Sprachdefinition	234
13.4. Aufgaben	234
13.4.1. A6.1: AST und Syntactic Sugar (1P)	234
13.4.2. A6.2: Tree-Walking-Interpreter (4P)	234
13.4.3. A6.3: Semantische Analyse (3P)	234
13.4.4. A6.4: Interaktiver Interpreter (1P)	234
13.4.5. A6.5: Auswirkungen der Grammatik auf den Interpreter (1P)	235
14. Blatt 07: C++	237
14.1. Zusammenfassung	237
14.2. Methodik	237
14.3. Speicherverwaltung in C/C++	237
14.3.1. Beispiel für eine Tokenizer-Funktion	237
14.3.2. Smart-Pointer als Lösung	239
14.3.3. Reference Counting	241
14.3.4. Dereferenzierung von Smart-Pointern	242
14.4. Aufgaben	243
14.4.1. A7.1: Klasse für Token (1P)	243
14.4.2. A7.2: Implementierung eines einfachen Tokenizers (1P)	244
14.4.3. A7.3: Reference Counter (1P)	244
14.4.4. A7.4: Smart-Pointer (3P)	244
14.4.5. A7.5: Ringpuffer (4P)	246
15. Blatt 08: Mini-Projekt C++-Interpreter	248
15.1. Aufgabe	248
15.1.1. Betrachtete Sprache: Sub-Dialekt von C++	248
15.1.2. REPL-Modell	252
15.1.3. Tests	253
15.2. Projektvorstellung: Walk-Through statt Präsentation	253
15.3. Abgabeformat	253
IV. Organisatorisches	254
16. Prüfungsvorbereitung	255
16.1. Elektronische Klausur: Termin, Materialien	256
16.1.1. Termin	256
16.1.2. Zugelassene Hilfsmittel	256

16.1.3. Einsicht	256
16.2. Technische Vorbereitungen	257
16.3. Bearbeitung des E-Assessment	258
16.4. Fragetypen-Demo	258
16.5. Hinweise zu den Inhalten	258
16.6. Beispiele für mögliche Fragen	259

Teil I.

Syllabus

1. Syllabus



1.1. Kursbeschreibung

Der Compiler ist das wichtigste Werkzeug in der Informatik. In der Königsdisziplin der Informatik schließt sich der Kreis, hier kommen die unterschiedlichen Algorithmen und Datenstrukturen und Programmiersprachenkonzepte zur Anwendung.

In diesem Modul geht es um ein grundlegendes Verständnis für die wichtigsten Konzepte im Compilerbau. Wir schauen uns dazu relevante aktuelle Tools und Frameworks an und setzen diese bei der Erstellung eines kleinen Compiler-Frontends für C++ ein.

1.2. Überblick Modulinhalte

1. Lexikalische Analyse: Scanner/Lexer

- Reguläre Sprachen
- Generierung mit ANTLR

2. Syntaxanalyse: Parser

- Kontextfreie Grammatiken (CFG)
- LL-Parser (Top-Down-Parser)
- Generierung mit ANTLR

3. Semantische Analyse: Symboltabellen

- Namen und Scopes
- Typen, Klassen, Polymorphie

4. Interpreter: AST-Traversierung
5. C++ als zu verarbeitende Programmiersprache

1.3. Team

- [BC George](#)
- [Carsten Gips](#) (Sprechstunde nach Vereinbarung)
- Alesia Herbertz (Tutorin)

1.4. Kursformat

Vorlesung (2 SWS)	Praktikum (2 SWS)
Do, 15:30 - 17:00 Uhr (Zoom)	G1: Mi, 09:45 - 11:15 Uhr (Zoom)
	G2: Mi, 11:30 - 13:00 Uhr (Zoom)
	G3: Mi, 09:45 - 11:15 Uhr (Präsenz D320)
	G4: Mi, 11:30 - 13:00 Uhr (Präsenz D320)

Achtung: Ab dem 23.10. fangen wir die Vorlesung eine Viertelstunde früher an als im Stundenplan eingetragen, um allen die Chance zu geben, den letzten Uni-Bus um 17:30 Uhr noch rechtzeitig zu erreichen.

Durchführung der Vorlesung als *Flipped Classroom* (Carsten) bzw. als *reguläre Vorlesung* (BC). Zugangsdaten Zoom siehe [ILIAS](#).

1.5. Fahrplan

Hier finden Sie einen abonnierbaren [Google Kalender](#) mit allen Terminen der Veranstaltung zum Einbinden in Ihre Kalender-App.

Abgabe der Post Mortems jeweils **Mittwoch bis 09:00 Uhr** im [ILIAS](#). Vorstellung der Lösung im jeweiligen Praktikum in der Abgabewoche.

Woche	Vorlesung: Selbststudium (Vorbereitung)	Vorlesung: Gemeinsame Sitzung	Praktikum	Edmonton/Minden- Meetings
06. - 10.10.	Überblick Sprachen Anwendungen	Orga (Zoom)		
13. - 17.10.		Reguläre Sprachen 1		

Woche	Vorlesung: Selbststudium (Vorbereitung)	Vorlesung: Gemeinsame Sitzung	Praktikum	Edmonton/Minden- Meetings
20. - 24.10.		Reguläre Sprachen 2 CFG	B01 Reguläre Sprachen	
27. - 31.10.	Lexer mit ANTLR	Parser mit ANTLR	B02 CFG	
03. - 07.11.		LL-Parser (Theorie)	B03 ANTLR	Mi, 05.11., 18:00 - 19:00 Uhr (online): Edmon- ton/Minden: ANTLR + Live-Coding
10. - 14.11.	Lexer (Implementierung)	LL-Parser (Implementierung)		
17. - 21.11.	Symboltabellen: Überblick Scopes	Funktionen Klassen	B04 LL hand coded	
24. - 28.11.	AST-basierte Interpreter 1	AST-basierte Interpreter 2	B05 Seman- tische Analyse	
01. - 05.12.	C++ I: Basics	Pointer & Referenzen		Mo, 01.12., 18:00 - 19:00 Uhr (online): Edmon- ton/Minden: Minden Presentations
08. - 12.12.	C++ II: Klassen Big Three Operatoren	Vererbung & Polymorphie	B06 Interpreter	Mo, 08.12., 18:00 - 19:00 Uhr (online): Edmon- ton/Minden: Edmonton Presentations
15. - 19.12.		Sprechstunde <i>Mathe-3-Irgendwas</i>	B07 C++	
22. - 26.12.		<i>Weihnachtspause</i>		
29.12. -		<i>Weihnachtspause</i>		
02.01.				

Woche	Vorlesung: Selbststudium (Vorbereitung)	Vorlesung: Gemeinsame Sitzung	Praktikum	Edmonton/Minden- Meetings
05. - 09.01.		<i>Sprechstunde</i>	<i>Freies Arbeiten (B08)</i>	
12. - 16.01.		<i>Sprechstunde</i>	<i>Freies Arbeiten (B08)</i>	
19. - 23.01.	Prüfungsvorbereitung	Rückblick (Zoom)	B08 Mini- Projekt	

1.6. Prüfungsform, Note und Credits

(Digitale) Klausur plus Studienleistung (Portfolio), 5 ECTS

- **Studienleistung:** “Portfolio” - Kriterien je Person:
 1. Teilnahme an mind. zwei Edmonton/Minden-Terminen mit aktiver Beteiligung
 2. Mind. vier der Übungsblätter B01..B07 erfolgreich bearbeitet
 3. Abschlussvortrag zum erfolgreich bearbeiteten Mini-Projekt (B08) am Semesterende (21.01.) a 15 Minuten (pro Team)

Je Kriterium: Abgabe eines Post Mortem im ILIAS (**jede Person individuell**)

- **Gesamtnote:** (Digitale) Klausur im B40 (90 Minuten)

Hinweise

- Die Bearbeitung der Leistungen erfolgt im Team.
- Ein Team umfasst 3 Personen.
- Die Post Mortems sind individuell zu erstellen und abzugeben.
- “Aktive Beteiligung” umfasst Anwesenheit und sachbezogene Beiträge; Anwesenheit/Beteiligung werden dokumentiert.
- “Erfolgreiche Bearbeitung” eines Blattes umfasst Bearbeitung im Team, Bearbeitung aller Aufgaben des Blattes, fristgerechte Abgabe des ausreichenden Post Mortems im ILIAS. Die intensive Beschäftigung mit den Aufgaben muss erkennbar sein.
- **Post Mortem:** Jede Person beschreibt individuell(!) die Bearbeitung des jeweiligen Kriteriums bzw. die Teilnahme an den Edmonton/Minden-Meetings zurückblickend mit mind. 150 bis max. 400 Wörtern (Nutzlast; Überschriften und Links zählen nicht mit). Gehen Sie dabei aussagekräftig und nachvollziehbar auf folgende Punkte ein:
 1. Zusammenfassung: Was wurde gemacht bzw. was wurde auf dem Meeting besprochen?
 2. Details: Kurze Beschreibung besonders interessanter Aspekte.

3. Reflexion: Was war der schwierigste Teil? Wie haben Sie dieses Problem gelöst?
4. Reflexion: Was haben Sie gelernt oder (besser) verstanden?
5. Team: Mit wem haben Sie zusammengearbeitet?
6. Link zu Ihrem Repo mit den relevanten Artefakten (Lösung, Slides für den Vortrag, ...).

Für die Edmonton/Minden-Meetings passen Sie bitte die Punkte (1) bis (4) entsprechend inhaltlich an, (5) und (6) entfallen.

Die Post Mortems geben Sie bitte pro Person bis spätestens zur jeweiligen Deadline im [ILIAS](#) ab.

Siehe auch <https://github.com/Compiler-CampusMinden/CB-Vorlesung-Bachelor-W25/discussions/3>.

1.7. Materialien

1. **“Compilers: Principles, Techniques, and Tools”**. Aho, A. V. und Lam, M. S. und Sethi, R. und Ullman, J. D. and Bansal, S., Pearson India, 2023. ISBN 978-9-3570-5488-1. [Online](#) über die [O’Reilly-Lernplattform](#).
2. **“Crafting Interpreters”**. Nystrom, R., Genever Benning, 2021. ISBN 978-0-9905829-3-9. [Online](#).
3. **“The Definitive ANTLR 4 Reference”**. Parr, T., Pragmatic Bookshelf, 2014. ISBN 978-1-9343-5699-9. [Online](#) über die [O’Reilly-Lernplattform](#).
4. **“Writing a C Compiler”**. Sandler, N., No Starch Press, 2024. ISBN 978-1-0981-8222-9. [Online](#) über die [O’Reilly-Lernplattform](#).

1.8. Förderungen und Kooperationen

1.8.1. Kooperation mit University of Alberta, Edmonton (Kanada)

Über das Projekt **“We CAN virtuOWL”** der Fachhochschule Bielefeld ist im Frühjahr 2021 eine Kooperation mit der [University of Alberta](#) (Edmonton/Alberta, Kanada) im Modul “Compilerbau” gestartet.

Wir freuen uns, auch in diesem Semester wieder drei gemeinsame Sitzungen für beide Hochschulen anbieten zu können. (Diese Termine werden in englischer Sprache durchgeführt.)

1.9. LICENSE



Unless otherwise noted, [this work](#) by [BC George](#), [Carsten Gips](#) and [contributors](#) is licensed under [CC BY-SA 4.0](#). See the [credits](#) for a detailed list of contributing projects.

Teil II.

Vorlesungsunterlagen

2. Überblick

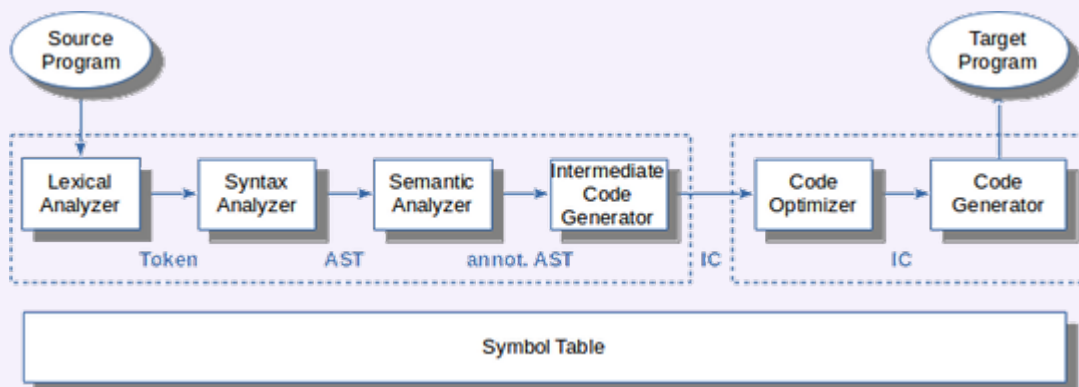
Was ist ein Compiler? Welche Bausteine lassen sich identifizieren, welche Aufgaben haben diese?

2.1. Struktur eines Compilers

TL;DR

Compiler übersetzen (formalen) Text in ein anderes Format.

Typischerweise kann man diesen Prozess in verschiedene Stufen/Phasen einteilen. Dabei verarbeitet jede Phase den Output der vorangegangenen Phase und erzeugt ein (kompakteres) Ergebnis, welches an die nächste Phase weitergereicht wird. Dabei nimmt die Abstraktion von Stufe zu Stufe zu: Der ursprüngliche Input ist ein Strom von Zeichen, daraus wird ein Strom von Wörtern (Token), daraus ein Baum (Parse Tree), Zwischencode (IC), ...



Die gezeigten Phasen werden traditionell unterschieden. Je nach Aufgabe können verschiedene Stufen zusammengefasst werden oder sogar gar nicht auftreten.

Videos

- [VL Überblick](#)

2.1.1. Sprachen verstehen, Texte transformieren

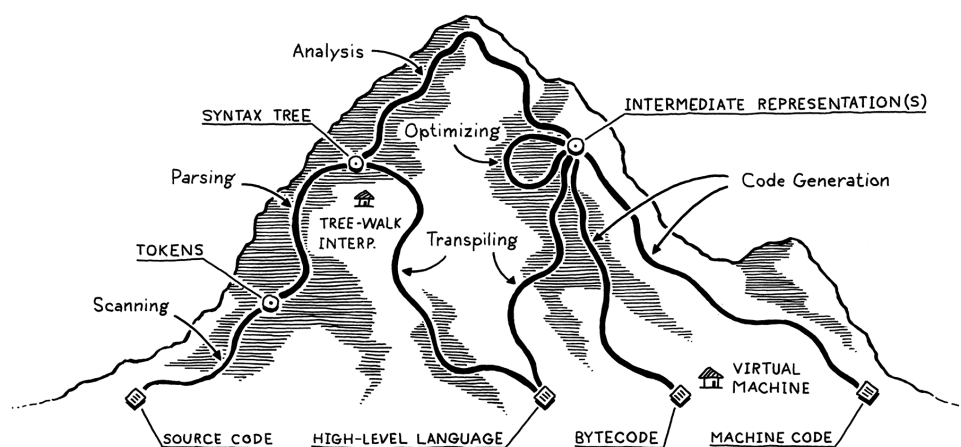
The cat runs quickly.

=> Struktur? Bedeutung?

Wir können hier (mit steigender Abstraktionsstufe) unterscheiden:

- Sequenz von Zeichen
 - Wörter: Zeichenketten mit bestimmten Buchstaben, getrennt durch bestimmte andere Zeichen; Wörter könnten im Wörterbuch nachgeschlagen werden
 - Sätze: Anordnung von Wörtern nach einer bestimmten Grammatik, Grenze: Satzzeichen
- Hier (vereinfacht): Ein Satz besteht aus Subjekt und Prädikat. Das Subjekt besteht aus einem oder keinen Artikel und einem Substantiv. Das Prädikat besteht aus einem Verb und einem oder keinem Adverb.
- Sprache: Die Menge der in einer Grammatik erlaubten Sätze

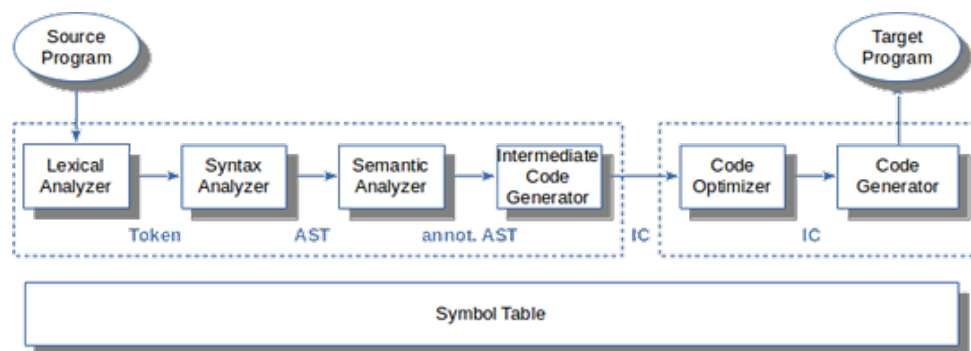
2.1.2. Compiler: Big Picture



Quelle: A Map of the Territory (mountain.png) by Bob Nystrom on Github.com (MIT)

Begriffe und Phasen

Die obige Bergsteige-Metapher kann man in ein nüchternes Ablaufdiagramm mit verschiedenen Stufen und den zwischen den Stufen ausgetauschten Artefakten übersetzen:



2.1.2.1. Frontend, Analyse

Die ersten Stufen eines Compilers, die mit der **Analyse** des Inputs beschäftigt sind. Dies sind in der Regel der Scanner, der Parser und die semantische Analyse.

- Scanner, Lexer, Tokenizer, Lexikalische Analyse

Zerteilt den Zeichenstrom in eine Folge von Wörtern. Mit regulären Ausdrücken kann definiert werden, was Klassen gültiger Wörter ("Token") sind. Ein Token hat i.d.R. einen Namen und einen Wert.

- Parser, Syntaxanalyse

Der Parser erhält als Eingabe die Folge der Token und versucht mit Hilfe einer Grammatik zu bestimmen, ob es sich bei der Tokensequenz um gültige Sätze im Sinne der Grammatik handelt. Hier gibt es viele Algorithmen, die im Wesentlichen in die Klassen "top-down" und "bottom-up" fallen.

- Semantische Analyse, Kontexthandling

In den vorigen Stufen wurde eher lokal gearbeitet. Hier wird über den gesamten Baum und die Symboltabelle hinweg geprüft, ob beispielsweise Typen korrekt verwendet wurden, in welchen Scope ein Name gehört etc. Mit diesen Informationen wird der AST angereichert.

- Symboltabellen

Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.2. Backend, Synthese

Die hinteren Stufen eines Compilers, die mit der **Synthese** der Ausgabe beschäftigt sind. Dies sind in der Regel verschiedene Optimierungen und letztlich die Code-Generierung

- Codegenerierung

Erzeugung des Zielprogramms aus der (optimierten) Zwischendarstellung. Dies ist oft Maschinencode, kann aber auch C-Code oder eine andere Ziel-Sprache sein.

- Optimierung

Diverse Maßnahmen, um den resultierenden Code kleiner und/oder schneller zu gestalten.

- Symboltabellen

Datenstrukturen, um Namen, Werte, Scopes und weitere Informationen zu speichern. Die Symboltabellen werden vor allem beim Parsen befüllt und bei der semantischen Analyse gelesen, aber auch der Lexer benötigt u.U. diese Informationen.

2.1.2.3. Weitere Begriffe

- Parse Tree, Concrete Syntax Tree

Repräsentiert die Struktur eines Satzes, wobei jeder Knoten dem Namen einer Regel der Grammatik entspricht. Die Blätter bestehen aus den Token samt ihren Werten.

- AST, (Abstract) Syntax Tree

Vereinfachte Form des Parse Tree, wobei der Bezug auf die Element der Grammatik (mehr oder weniger) weggelassen wird.

- Annotierter AST

Anmerkungen am AST, die für spätere Verarbeitungsstufen interessant sein könnten: Typ-Informationen, Optimierungsinformationen, ...

- Zwischen-Code, IC

Zwischensprache, die abstrakter ist als die dem AST zugrunde liegenden Konstrukte der Ausgangssprache. Beispielsweise könnten **while**-Schleifen durch entsprechende Label und Sprünge ersetzt werden. Wie genau dieser Zwischen-Code aussieht, muss der Compilerdesigner entscheiden. Oft findet man den Assembler-ähnlichen "3-Adressen-Code".

- Sprache

Eine Sprache ist eine Menge gültiger Sätze. Die Sätze werden aus Wörtern gebildet, diese wiederum aus Zeichenfolgen.

- Grammatik

Eine Grammatik beschreibt formal die Syntaxregeln für eine Sprache. Jede Regel in der Grammatik beschreibt dabei die Struktur eines Satzes oder einer Phrase.

2.1.3. Lexikalische Analyse: Wörter ("**Token**") erkennen

Die lexikalische Analyse (auch *Scanner* oder *Lexer* oder *Tokenizer* genannt) zerteilt den Zeichenstrom in eine Folge von Wörtern ("**Token**"). Die geschieht i.d.R. mit Hilfe von *regulären Ausdrücken*.

Dabei müssen unsinnige/nicht erlaubte Wörter erkannt werden.

Überflüssige Zeichen (etwa Leerzeichen) werden i.d.R. entfernt.

```
1 sp = 100;
```

```
1 <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

Anmerkung: In der obigen Darstellung werden die Werte der Token (“*Lexeme*”) zusammen mit den Token “gespeichert”. Alternativ können die Werte der Token auch direkt in der Symboltabelle gespeichert werden und in den Token nur der Verweis auf den jeweiligen Eintrag in der Tabelle.

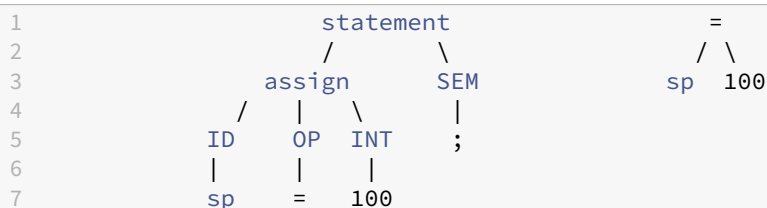
2.1.4. Syntaxanalyse: Sätze erkennen

In der Syntaxanalyse (auch *Parser* genannt) wird die Tokensequenz in gültige Sätze unterteilt. Dazu werden in der Regel *kontextfreie Grammatiken* und unterschiedliche Parsing-Methoden (*top-down*, *bottom-up*) genutzt.

Dabei müssen nicht erlaubte Sätze erkannt werden.

```
1 <ID, sp>, <OP, =>, <INT, 100>, <SEM>
```

```
1 statement : assign SEM ;
2 assign : ID OP INT ;
```



Mit Hilfe der Produktionsregeln der Grammatik wird versucht, die Tokensequenz zu erzeugen. Wenn dies gelingt, ist der Satz (also die Tokensequenz) ein gültiger Satz im Sinne der Grammatik. Dabei sind die Token aus der lexikalischen Analyse die hier betrachteten Wörter!

Dabei entsteht ein sogenannter *Parse-Tree* (oder auch “*Syntax Tree*”; in der obigen Darstellung der linke Baum). In diesen Bäumen spiegeln sich die Regeln der Grammatik wider, d.h. zu einem Satz kann es durchaus verschiedene Parse-Trees geben.

Beim AST (“*Abstract Syntax Tree*”) werden die Knoten um alle später nicht mehr benötigten Informationen bereinigt (in der obigen Darstellung der rechte Baum).

Anmerkung: Die Begriffe werden oft nicht eindeutig verwendet. Je nach Anwendung ist das Ergebnis des Parsers ein AST oder ein Parse-Tree.

Anmerkung: Man könnte statt **OP** auch etwa ein **ASSIGN** nutzen und müsste dann das “=” nicht extra als Inhalt speichern, d.h. man würde die Information im Token-Typ kodieren.

2.1.5. Vorschau: Parser implementieren

```

1 stat : assign | ifstat | ... ;
2 assign : ID '=' expr ';' ;

```

```

1 void stat() {
2     switch (<<current token>>) {
3         case ID : assign(); break;
4         case IF : ifstat(); break;
5         ...
6         default : <<raise exception>>
7     }
8 }
9 void assign() {
10    match(ID);
11    match('=');
12    expr();
13    match(';');
14 }

```

Der gezeigte Parser ist ein sogenannter “LL(1)”-Parser und geht von oben nach unten vor, d.h. ist ein Top-Down-Parser.

Nach dem Betrachten des aktuellen Tokens wird entschieden, welche Alternative vorliegt und in die jeweilige Methode gesprungen.

Die `match()`-Methode entspricht dabei dem Erzeugen von Blättern, d.h. hier werden letztlich die Token der Grammatik erkannt.

2.1.6. Semantische Analyse: Bedeutung erkennen

In der semantischen Analyse (auch *Context Handling* genannt) wird der AST zusammen mit der Symboltabelle geprüft. Dabei spielen Probleme wie Scopes, Namen und Typen eine wichtige Rolle.

Die semantische Analyse ist direkt vom Programmierparadigma der zu übersetzenden Sprache abhängig, d.h. müssen wir beispielsweise das Konzept von Klassen verstehen?

Als Ergebnis dieser Phase entsteht typischerweise ein *annotierter AST*.

```

1 {
2     int x = 42;
3     {
4         int x = 7;
5         x += 3;    // ???
6     }
7 }

```

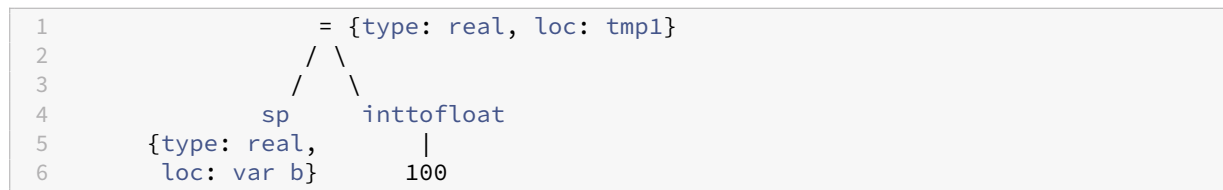
```

1                                     = {type: real, loc: tmp1}
2     sp = 100;                       / \
3                                     sp  inttofloat
4                                     / \   |
5     {type: real,                  {type: real,
6     loc: var b}                  loc: var b}  100

```

2.1.7. Zwischencode generieren

Aus dem annotierten AST wird in der Regel ein Zwischencode (“*Intermediate Code*”, auch “IC”) generiert. oft findet man hier den Assembler-ähnlichen “3-Adressen-Code”, in manchen Compilern wird als IC aber auch der AST selbst genutzt.



=> `t1 = inttofloat(100)`

2.1.8. Code optimieren

An dieser Stelle verlassen wir das Compiler-Frontend und begeben uns in das sogenannte *Backend*. Die Optimierung des Codes kann sehr unterschiedlich ausfallen, beispielsweise kann man den Zwischencode selbst optimieren, dann nach sogenanntem “Targetcode” übersetzen und diesen weiter optimieren, bevor das Ergebnis im letzten Schritt in Maschinencode übersetzt wird.

Die Optimierungsphase ist sehr stark abhängig von der Zielhardware. Hier kommen fortgeschrittene Mengen- und Graphalgorithmen zur Anwendung. Die Optimierung stellt den wichtigsten Teil aktueller Compiler dar.

Aus zeitlichen und didaktischen Gründen werden wir in dieser Veranstaltung den Fokus auf die Frontend-Phasen legen und die Optimierung nur grob streifen.

`t1 = inttofloat(100)` => `t1 = 100.0`

`x = y*0;` => `x = 0;`

2.1.9. Code generieren

- Maschinencode:

```
1  STD  t1, 100.0
```

- Andere Sprache:

- Bytecode
- C
- ...

2.1.10. Probleme

```
1 5*4+3
```

AST?

Problem: Vorrang von Operatoren

- Variante 1: $+(*(5, 4), 3)$
- Variante 2: $*(5, +(4, 3))$

```
1 stat : expr ';'
2      | ID '(' ')' ';'
3      ;
4 expr : ID '(' ')'
5      | INT
6      ;
```

2.1.11. Wrap-Up

- Compiler übersetzen Text in ein anderes Format
- Typische Phasen:
 1. Lexikalische Analyse
 2. Syntaxanalyse
 3. Semantische Analyse
 4. Generierung von Zwischencode
 5. Optimierung des (Zwischen-) Codes
 6. Codegenerierung

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k2: Ich kann die Struktur eines Compilers und die verschiedenen Phasen und deren Aufgaben erklären

2.2. Bandbreite der Programmiersprachen

TL;DR

Am Beispiel des Abzählreims “99 Bottles of Beer” werden (ganz kurz) verschiedene Programmiersprachen betrachtet. Jede der Sprachen hat ihr eigenes Sprachkonzept (Programmierparadigma) und auch ein eigenes Typ-System sowie ihre eigene Strategie zur Speicherverwaltung und Abarbeitung.

Auch wenn die Darstellung längst nicht vollständig ist, macht sie doch deutlich, dass Compiler teilweise sehr unterschiedliche Konzepte “verstehen” müssen.

Videos

- [VL Programmiersprachen](#)

2.2.1. 99 Bottles of Beer

99 bottles of beer on the wall, 99 bottles of beer. Take one down and pass it around, 98 bottles of beer on the wall.

98 bottles of beer on the wall, 98 bottles of beer. Take one down and pass it around, 97 bottles of beer on the wall.

[...]

2 bottles of beer on the wall, 2 bottles of beer. Take one down and pass it around, 1 bottle of beer on the wall.

1 bottle of beer on the wall, 1 bottle of beer. Take one down and pass it around, no more bottles of beer on the wall.

No more bottles of beer on the wall, no more bottles of beer. Go to the store and buy some more, 99 bottles of beer on the wall.

Quelle: Abzählreim “99 Bottles of Beer” nach “Lyrics of the song 99 Bottles of Beer” on 99-bottles-of-beer.net

2.2.2. Imperativ, Hardwarenah: C

```
1  #define MAXBEER (99)
2  void chug(int beers);
3  main() {
4      register beers;
5      for(beers = MAXBEER; beers; chug(beers--)) puts("");
6      puts("\nTime to buy more beer!\n");
7  }
8  void chug(register beers) {
9      char howmany[8], *s;
10     s = beers != 1 ? "s" : "";
```

```

11     printf("%d bottle%s of beer on the wall,\n", beers, s);
12     printf("%d bottle%s of beeeer . . . ,\n", beers, s);
13     printf("Take one down, pass it around,\n");
14     if(--beers) sprintf(howmany, "%d", beers); else strcpy(howmany, "No more");
15     s = beers != 1 ? "s" : "";
16     printf("%s bottle%s of beer on the wall.\n", howmany, s);
17 }

```

Quelle: "Language C" by Bill Wein on 99-bottles-of-beer.net

- Imperativ
- Procedural
- Statisches Typsystem
- Ressourcenschonend, aber "unsicher": Programmierer muss wissen, was er tut
- Relativ hardwarenah
- Einsatz: Betriebssysteme, Systemprogrammierung

2.2.3. Imperativ, Objektorientiert: Java

```

1  class bottles {
2      public static void main(String args[]) {
3          String s = "s";
4          for (int beers=99; beers>-1;) {
5              System.out.print(beers + " bottle" + s + " of beer on the wall, ");
6              System.out.println(beers + " bottle" + s + " of beer, ");
7              if (beers==0) {
8                  System.out.print("Go to the store, buy some more, ");
9                  System.out.println("99 bottles of beer on the wall.\n");
10                 System.exit(0);
11             } else
12                 System.out.print("Take one down, pass it around, ");
13             s = (--beers == 1)?"": "s";
14             System.out.println(beers + " bottle" + s + " of beer on the wall.\n");
15         }
16     }
17 }

```

Quelle: "Language Java" by Sean Russell on 99-bottles-of-beer.net

- Imperativ
- Objektorientiert
- Multi-Threading
- Basiert auf C/C++
- Statisches Typsystem
- Automatische Garbage Collection

- “Sichere” Architektur: Laufzeitumgebung fängt viele Probleme ab
- Architekturneutral: Nutzt Bytecode und eine JVM
- Einsatz: High-Level All-Purpose Language

2.2.4. Logisch: Prolog

```

1 bottles :-
2     bottles(99).
3
4 bottles(1) :-
5     write('1 bottle of beer on the wall, 1 bottle of beer, '), nl,
6     write('Take one down, and pass it around, '), nl,
7     write('Now they are all gone. '), nl,!.
8 bottles(X) :-
9     write(X), write(' bottles of beer on the wall, '), nl,
10    write(X), write(' bottles of beer, '), nl,
11    write('Take one down and pass it around, '), nl,
12    NX is X - 1,
13    write(NX), write(' bottles of beer on the wall. '), nl, nl,
14    bottles(NX).

```

Quelle: "Language Prolog" by M@ on 99-bottles-of-beer.net

- Deklarativ
- Logisch: Definition von Fakten und Regeln; eingebautes Beweissystem
- Einsatz: Theorem-Beweisen, Natural Language Programming (NLP), Expertensysteme, ...

2.2.5. Funktional: Haskell

```

1 bottles 0 = "no more bottles"
2 bottles 1 = "1 bottle"
3 bottles n = show n ++ " bottles"
4
5 verse 0   = "No more bottles of beer on the wall, no more bottles of beer.\n"
6           ++ "Go to the store and buy some more, 99 bottles of beer on the wall."
7
8 verse n   = bottles n ++ " of beer on the wall, " ++ bottles n ++ " of beer.\n"
9           ++ "Take one down and pass it around, " ++ bottles (n-1)
10                                     ++ " of beer on the wall.\n"
11
12 main      = mapM (putStrLn . verse) [99,98..0]

```

Quelle: "Language Haskell" by Iavor on 99-bottles-of-beer.net

- Deklarativ
- Funktional

- Lazy, pure
- Statisches Typsystem
- Typinferenz
- Algebraische Datentypen, Patternmatching
- Einsatz: Compiler, DSL, Forschung

2.2.6. Brainfuck

Quelle: Screenshot of "Language Brainfuck" by Michal Wojciech Tarnowski on 99-bottles-of-beer.net

- Imperativ
- Feldbasiert (analog zum Band der Turingmaschine)
- 8 Befehle: Zeiger und Zellen inkrementieren/dekrementieren, Aus- und Eingabe, Sprungbefehle

2.2.7. Programmiersprache Lox

```

1 fun fib(x) {
2   if (x == 0) {
3     return 0;
4   } else {
5     if (x == 1) {
6       return 1;
7     } else {
8       fib(x - 1) + fib(x - 2);
9     }

```

```
10     }  
11 }  
12  
13 var wuppie = fib;  
14 wuppie(4);
```

- Die Sprache “Lox” finden Sie hier: craftinginterpreters.com/the-lox-language.html
- C-ähnliche Syntax
- Imperativ, objektorientiert, Funktionen als *First Class Citizens*, Closures
- Dynamisch typisiert
- Garbage Collector
- Statements und Expressions
- (Kleine) Standardbibliothek eingebaut

Die Sprache ähnelt stark anderen modernen Sprachen und ist gut geeignet, um an ihrem Beispiel Themen wie Scanner/Parser/AST, Interpreter, Object Code und VM zu studieren :)

2.2.8. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format
- Berücksichtigung von unterschiedlichen
 - Sprachkonzepten (Programmierparadigmen)
 - Typ-Systemen
 - Speicherverwaltungsstrategien
 - Abarbeitungsstrategien

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Beispiele für verschiedene Programmiersprachen und Paradigmen

2.3. Anwendungen

TL;DR

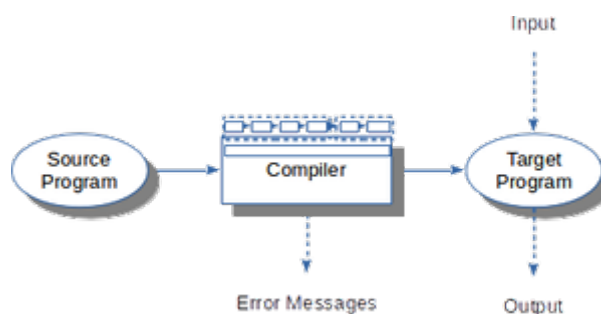
Es gibt verschiedene Anwendungsmöglichkeiten für Compiler. Je nach Bedarf wird dabei die komplette Toolchain durchlaufen oder es werden Stufen ausgelassen. Häufig genutzte Varianten sind dabei:

- “Echte” Compiler: Übersetzen Sourcecode nach ausführbarem Maschinencode
- Interpreter: Interaktive Ausführung von Sourcecode
- Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
- Transpiler: Übersetzen formalen Text nach formalem Text
- Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Videos

- [VL Anwendungen](#)

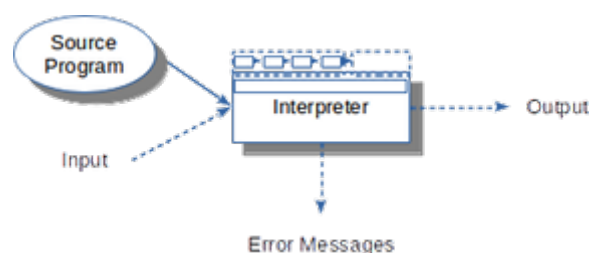
2.3.1. Anwendung: Compiler



Wie oben diskutiert: Der Sourcecode durchläuft alle Phasen des Compilers, am Ende fällt ein ausführbares Programm heraus. Dieses kann man starten und ggf. mit Inputdaten versehen und erhält den entsprechenden Output. Das erzeugte Programm läuft i.d.R. nur auf einer bestimmten Plattform.

Beispiele: gcc, clang, ...

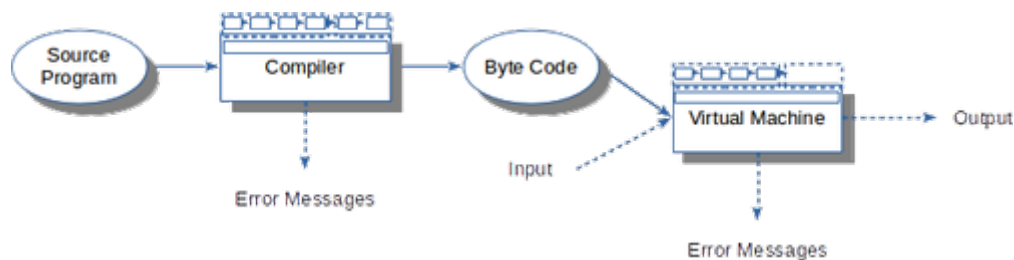
2.3.2. Anwendung: Interpreter



Beim Interpreter durchläuft der Sourcecode nur das Frontend, also die Analyse. Es wird kein Code erzeugt, stattdessen führt der Interpreter die Anweisungen im AST bzw. IC aus. Dazu muss der Interpreter mit den Eingabedaten beschickt werden. Typischerweise hat man hier eine “Read-Eval-Print-Loop” (*REPL*).

Beispiele: Python

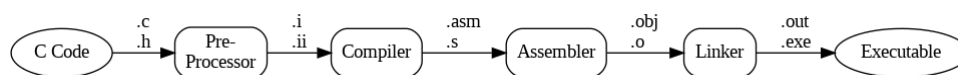
2.3.3. Anwendung: Virtuelle Maschinen



Hier liegt eine Art Mischform aus Compiler und Interpreter vor: Der Compiler übersetzt den Quellcode in ein maschinenunabhängiges Zwischenformat (“Byte-Code”). Dieser wird von der virtuellen Maschine (“VM”) gelesen und ausgeführt. Die VM kann also als Interpreter für Byte-Code betrachtet werden.

Beispiel: Java mit seiner JVM

2.3.4. Anwendung: C-Toolchain

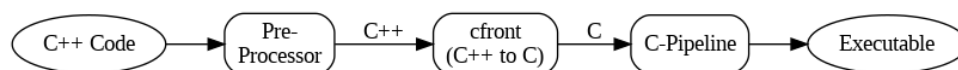


Erinnern Sie sich an die LV “Systemprogrammierung” im dritten Semester :-)

Auch wenn es so aussieht, als würde der C-Compiler aus dem Quelltext direkt das ausführbare Programm erzeugen, finden hier dennoch verschiedene Stufen statt. Zuerst läuft ein Präprozessor über den Quelltext und ersetzt alle `#include` und `#define` etc., danach arbeitet der C-Compiler, dessen Ausgabe wiederum durch einen Assembler zu ausführbarem Maschinencode transformiert wird.

Beispiele: gcc, clang, ...

2.3.5. Anwendung: C++-Compiler



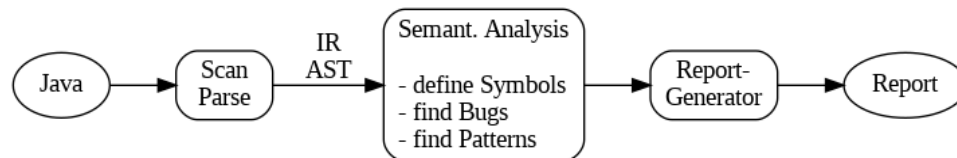
C++ hat meist keinen eigenen (vollständigen) Compiler :-)

In der Regel werden die C++-Konstrukte durch `cfront` nach C übersetzt, so dass man anschließend auf die etablierten Tools zurückgreifen kann.

Dieses Vorgehen werden Sie relativ häufig finden. Vielleicht sogar in Ihrem Projekt ...

Beispiel: g++

2.3.6. Anwendung: Bugfinder



Tools wie FindBugs analysieren den (Java-) Quellcode und suchen nach bekannten Fehlermustern. Dazu benötigen sie nur den Analyse-Teil eines Compilers!

Auf dem AST kann dann nach vorab definierten Fehlermustern gesucht werden (Stichwort “Graphmatching”). Dazu fällt die semantische Analyse entsprechend umfangreicher aus als normal.

Zusätzlich wird noch eine Reporting-Komponente benötigt, da die normalen durch die Analyseketten erzeugten Fehlermeldungen nicht helfen (bzw. sofern der Quellcode wohlgeformter Code ist, würden ja keine Fehlermeldungen durch die Analyseeinheit generiert).

Beispiele: SpotBugs, Checkstyle, ESLint, ...

2.3.7. Anwendung: Pandoc

[Pandoc](#) ist ein universeller und modular aufgebauter Textkonverter, der mit Hilfe verschiedener *Reader* unterschiedliche Textformate einlesen und in ein Zwischenformat (hier JSON) transformieren kann. Über verschiedene *Writer* können aus dem Zwischenformat dann Dokumente in den gewünschten Zielformaten erzeugt werden.

Die Reader entsprechen der Analyse-Phase und die Writer der Synthese-Phase eines Compilers. Anstelle eines ausführbaren Programms (Maschinencode) wird ein anderes Textformat erstellt/ausgegeben.

Beispielsweise wird aus diesem Markdown-Schnipsel ...

```

1 Dies ist ein Satz mit
2 * einem Stichpunkt, und
3 * einem zweiten Stichpunkt.
  
```

... dieses Zwischenformat erzeugt, ...

```

1 {"blocks":[{"t":"Para","c":[{"t":"Str","c":"Dies"}, {"t":"Space"},
2 {"t":"Str","c":"ist"}, {"t":"Space"}, {"t":"Str","c":"ein"},
3 {"t":"Space"}, {"t":"Str","c":"Satz"}, {"t":"Space"},
4 {"t":"Str","c":"mit"}]},
5 {"t":"BulletList","c":[{"t":"Plain","c":[{"t":"Str","c":"einem"}, {"t":"Space"}, {"t":"Str","c":"Stichpunkt,"}, {"t":"Space"}, {"t":"Str","c":"und"}]}, [{"t":"Plain","c":[{"t":"Str","c":"einem"}, {"t":"Space"}, {"t":"Str","c":"zweiten"}, {"t":"Space"}, {"t":"Str","c":"Stichpunkt."}]}]}]}], "pandoc-api-version": [1, 17, 0, 4], "meta": {}}
```

... und daraus schließlich dieser TeX-Code.

```
1 Dies ist ein Satz mit
2 \begin{itemize}
3 \tightlist
4 \item einem Stichpunkt, und
5 \item einem zweiten Stichpunkt.
6 \end{itemize}
```

Im Prinzip ist Pandoc damit ein Beispiel für Compiler, die aus einem formalen Text nicht ein ausführbares Programm erzeugen (Maschinencode), sondern einen anderen formalen Text. Dieser werden häufig auch “Transpiler” genannt.

Weitere Beispiele:

- Lexer-/Parser-Generatoren: ANTLR, Flex, Bison, ...: formale Grammatik nach Sourcecode
- CoffeeScript: CoffeeScript (eine Art “JavaScript light”) nach JavaScript
- Emscripten: C/C++ nach LLVM nach WebAssembly (tatsächlich kann LLVM-IR auch direkt als Input verwendet werden)
- Fitness: Word/Wiki nach ausführbare Unit-Tests

2.3.8. Was bringt mir das?

Beschäftigung mit dem schönsten Thema in der Informatik ;-)

2.3.8.1. Auswahl einiger Gründe für den Besuch des Moduls “Compilerbau”

- Erstellung eigener kleiner Interpreter/Compiler
 - Einlesen von komplexen Daten
 - DSL als Brücke zwischen Stakeholdern
 - DSL zum schnelleren Programmieren (denken Sie etwa an [CoffeeScript](#) ...)
- Wie funktionieren FindBugs, Lint und ähnliche Tools?
 - Statische Codeanalyse: Dead code elimination
- Language-theoretic Security: [LangSec](#)
- Verständnis für bestimmte Sprachkonstrukte und -konzepte (etwa `virtual` in C++)
- Vertiefung durch Besuch “echter” Compilerbau-Veranstaltungen an Uni möglich :-)
- Wie funktioniert:
 - ein Python-Interpreter?
 - das Syntaxhighlighting in einem Editor oder in Doxygen?
 - ein Hardwarecompiler (etwa VHDL)?
 - ein Text-Formatierer (TeX, LaTeX, ...)?
 - CoffeeScript oder Emscripten?
- Wie kann man einen eigenen Compiler/Interpreter basteln, etwa für

- MiniJava (mit C-Backend)
 - Brainfuck
 - Übersetzung von JSON nach XML
- Um ein tiefes Verständnis von Programmiersprachen zu erlangen, ist eine Beschäftigung mit ihrer Implementierung unerlässlich.
 - Viele Grundtechniken der Informatik und elementare Datenstrukturen wie Keller, Listen, Abbildungen, Bäume, Graphen, Automaten etc. finden im Compilerbau Anwendung. Dadurch schließt sich in gewisser Weise der Kreis in der Informatikausbildung...
 - Aufgrund seiner Reife gibt es hervorragende Beispiele von formaler Spezifikation im Compilerbau.
 - Mit dem Gebiet der formalen Sprachen berührt der Compilerbau interessante Aspekte moderner Linguistik. Damit ergibt sich letztlich eine Verbindung zur KI...
 - Die Unterscheidung von Syntax und Semantik ist eine grundlegende Technik in fast allen formalen Systemen.

2.3.8.2. Parser-Generatoren (Auswahl)

Diese Tools könnte man beispielsweise nutzen, um seine eigene Sprache zu basteln.

- ANTLR (ANother Tool for Language Recognition) is a powerful parser generator for reading, processing, executing, or translating structured text or binary files: github.com/antlr/antlr4
- Grammars written for ANTLR v4; expectation that the grammars are free of actions: github.com/antlr/grammars-v4
- An incremental parsing system for programmings tools: github.com/tree-sitter/tree-sitter
- Flex, the Fast Lexical Analyzer - scanner generator for lexing in C and C++: github.com/westes/flex
- Bison is a general-purpose parser generator that converts an annotated context-free grammar into a deterministic LR or generalized LR (GLR) parser employing LALR(1) parser tables: gnu.org/software/bison
- Parser combinators for binary formats, in C: github.com/UpstandingHackers/hammer
- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.3. Statische Analyse, Type-Checking und Linter

Als Startpunkt für eigene Ideen. Oder Verbessern/Erweitern der Projekte...

- Pluggable type-checking for Java: github.com/typetools/checker-framework
- SpotBugs is FindBugs' successor. A tool for static analysis to look for bugs in Java code: github.com/spotbugs/spotbugs
- An extensible cross-language static code analyzer: github.com/pmd/pmd
- Checkstyle is a development tool to help programmers write Java code that adheres to a coding standard: github.com/checkstyle/checkstyle
- JaCoCo - Java Code Coverage Library: github.com/jacoco/jacoco
- Sanitizers: memory error detector: github.com/google/sanitizers
- JSHint is a tool that helps to detect errors and potential problems in your JavaScript code: github.com/jshint/jshint
- Haskell source code suggestions: github.com/ndmitchell/hlint
- Syntax checking hacks for vim: github.com/vim-syntastic/syntastic

2.3.8.4. DSL (Domain Specific Language)

- NVIDIA Material Definition Language SDK: github.com/NVIDIA/MDL-SDK
- FitNesse – The Acceptance Test Wiki: github.com/unclebob/fitnesse

Hier noch ein Framework, welches auf das Erstellen von DSL spezialisiert ist:

- Eclipse Xtext is a language development framework: github.com/eclipse/xtext

2.3.8.5. Konverter von X nach Y

- Emscripten: An LLVM-to-JavaScript Compiler: github.com/kripken/emscripten
- “Unfancy JavaScript”: github.com/jashkenas/coffeescript
- Universal markup converter: github.com/jgm/pandoc
- Übersetzung von JSON nach XML

2.3.8.6. Odds and Ends

- How to write your own compiler: staff.polito.it/silvano.rivoira/HowToWriteYourOwnCompiler.htm
- Building a modern functional compiler from first principles: github.com/sdiehl/write-you-a-haskell
- Language-theoretic Security: [LangSec](https://www.langsec.org/)
- Generierung von automatisierten Tests mit Esprima: heise.de/-4129726
- Eigener kleiner Compiler/Interpreter, etwa für
 - MiniJava mit C-Backend oder sogar [LLVM](https://llvm.org/)-Backend
 - Brainfuck

2.3.8.7. Als weitere Anregung: Themen der Mini-Projekte im W17

- Java2UMLet
- JavaDoc-to-Markdown
- Validierung und Übersetzung von Google Protocol Buffers v3 nach JSON
- svg2tikz
- SwaggerLang – Schreiben wie im Tagebuch
- Markdown zu LaTeX
- JavaDocToLaTeX
- MySQL2REDIS-Parser

2.3.9. Wrap-Up

- Compiler übersetzen formalen Text in ein anderes Format
- Nicht alle Stufen kommen immer vor => unterschiedliche Anwendungen

- “Echte” Compiler: Sourcecode nach Maschinencode
- Interpreter: Interaktive Ausführung
- Virtuelle Maschinen als Zwischending zwischen Compiler und Interpreter
- Transpiler: formaler Text nach formalem Text
- Analysetools: Parsen den Sourcecode, werten die Strukturen aus

Zum Nachlesen

- Aho u. a. (2023): Kapitel 1 Introduction
- Grune u. a. (2012): Kapitel 1 Introduction

Lernziele

- k1: Ich kenne verschiedene Anwendungen für Compiler durch Einsatz bestimmter Stufen der Compiler-Pipeline

3. Lexikalische Analyse

In der lexikalischen Analyse soll ein Lexer (auch “Scanner”) den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden in der Regel reguläre Ausdrücke verwendet.

3.1. Reguläre Sprachen, Ausdrucksstärke (Teil 1)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.1.1. Motivation

3.1.1.1. Was muss ein Compiler wohl als erstes tun?

Hier entsteht ein Tafelbild.

3.1.1.2. Themen für heute

- Lexer
- Endliche Automaten
- Reguläre Sprachen

3.1.2. Endliche Automaten

3.1.2.1. Alphabete

Def.: Ein **Alphabet** Σ ist eine endliche, nicht-leere Menge von Symbolen. Die Symbole eines Alphabets heißen *Buchstaben*.

Def.: Ein **Wort** w über einem Alphabet Σ ist eine endliche Folge von Symbolen aus Σ . ϵ ist das leere Wort. Die *Länge* $|w|$ eines Wortes w ist die Anzahl von Buchstaben, die es enthält (Kardinalität).

Def.: Eine **Sprache** L über einem Alphabet Σ ist eine Menge von Wörtern über diesem Alphabet. Sprachen können endlich oder unendlich viele Wörter enthalten.

3.1.2.2. State machine

Hier entsteht ein Tafelbild.

3.1.2.3. Deterministische endliche Automaten

Bestimmte State machines:

- Eingaben bestimmen Zustandsübergänge
- Zustandsübergänge sind eindeutig
- Es gibt Anfang(szustand) und End(zustand)e

3.1.2.4. Wie definieren wir das formal?

Hier entsteht ein Tafelbild.

3.1.2.5. Def.: Deterministischer endlicher Automat

Def.: Ein **deterministischer endlicher Automat** (DFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$, δ kann partiell sein
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.6. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.7. Eingabewörter statt Buchstaben

Def.: Wir definieren $\delta^* : (Q \times \Sigma^*) \rightarrow Q$: induktiv wie folgt:

- Basis: $\delta^*(q, \epsilon) = q \ \forall q \in Q$
- Induktion: $\delta^*(q, a_1, \dots, a_n) = \delta(\delta^*(q, a_1, \dots, a_{n-1}), a_n)$

Def.: Ein DFA akzeptiert ein Wort $w \in \Sigma^*$ genau dann, wenn $\delta^*(q_0, w) \in F$.

3.1.2.8. Beispiel

Hier entsteht ein Tafelbild.

3.1.2.9. Nichtdeterministische endliche Automaten

Hier entsteht ein Tafelbild.

3.1.2.10. Def.: Nichtdeterministischer Automat

Def.: Ein **nichtdeterministischer endlicher Automat** (NFA) ist ein 5-Tupel $A = (Q, \Sigma, \delta, q_0, F)$ mit

- Q : endliche Menge von **Zuständen**
- Σ : Alphabet von **Eingabesymbolen**
- δ : die (eventuell partielle) **Übergangsfunktion** $(Q \times \Sigma) \rightarrow Q$
- $q_0 \in Q$: der **Startzustand**
- $F \subseteq Q$: die Menge der **Endzustände**

3.1.2.11. Akzeptierte Sprachen

Def.: Sei A ein DFA oder ein NFA. Dann ist $L(A)$ die von A akzeptierte Sprache, d. h.

$$L(A) = \{\text{Wörter } w \mid \delta^*(q_0, w) \in F\}$$

3.1.2.12. Wozu NFAs im Compilerbau?

Pattern Matching (Erkennung von Schlüsselwörtern, Bezeichnen, ...) geht mit NFAs.

NFAs sind so nicht zu programmieren, aber:

Satz: Eine Sprache L wird von einem NFA akzeptiert $\Leftrightarrow L$ wird von einem DFA akzeptiert.

D. h. es existieren Algorithmen zur

- Umwandlung von NFAs in DFAS
- Minimierung von DFAs

3.1.3. Reguläre Sprachen

3.1.3.1. Reguläre Ausdrücke definieren Sprachen

Def.: Induktive Definition von **regulären Ausdrücken** (regex) und der von ihnen repräsentierten Sprache **L**:

- Basis:
 - ϵ und $\emptyset$ sind reguläre Ausdrücke mit $L(\epsilon) = \{\epsilon\}$, $L(\emptyset) = \emptyset$
 - Sei a ein Symbol $\Rightarrow a$ ist ein regex mit $L(a) = \{a\}$
- Induktion: Seien E, F reguläre Ausdrücke. Dann gilt:
 - $E + F$ ist ein regex und bezeichnet die Vereinigung $L(E + F) = L(E) \cup L(F)$
 - EF ist ein regex und bezeichnet die Konkatenation $L(EF) = L(E)L(F)$
 - E^* ist ein regex und bezeichnet die Kleene-Hülle $L(E^*) = (L(E))^*$
 - (E) ist ein regex mit $L((E)) = L(E)$

Vorrangregeln der Operatoren für reguläre Ausdrücke: $*$, Konkatenation, $+$

3.1.3.2. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.3. Wichtige Identitäten

Satz: Sei A ein DFA $\Rightarrow \exists$ regex R mit $L(A) = L(R)$.

Satz: Sei E ein regex $\Rightarrow \exists$ DFA A mit $L(E) = L(A)$.

3.1.3.4. Formale Grammatiken

Hier entsteht ein Tafelbild.

3.1.3.5. Formale Definition formaler Grammatiken

Def.: Eine *formale Grammatik* ist ein 4-Tupel $G = (N, T, P, S)$ aus

- N : endliche Menge von **Nichtterminalen**
- T : endliche Menge von **Terminalen**, $N \cap T = \emptyset$
- $S \in N$: **Startsymbol**
- P : endliche Menge von **Produktionen** der Form

$$X \rightarrow Y \text{ mit } X \in (N \cup T)^* N (N \cup T)^*, Y \in (N \cup T)^*$$

3.1.3.6. Ableitungen

Def.: Sei $G = (N, T, P, S)$ eine Grammatik, sei $\alpha A \beta$ eine Zeichenkette über $(N \cup T)^*$ und sei $A \rightarrow \gamma$ eine Produktion von G .

Wir schreiben: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ ($\alpha A \beta$ leitet $\alpha \gamma \beta$ ab).

Def.: Wir definieren die Relation $\Rightarrow^*$ induktiv wie folgt:

- Basis: $\forall \alpha \in (N \cup T)^* \alpha \xRightarrow{*} \alpha$ (Jede Zeichenkette leitet sich selbst ab.)
- Induktion: Wenn $\alpha \xRightarrow{*} \beta$ und $\beta \Rightarrow \gamma$ dann $\alpha \xRightarrow{*} \gamma$

Def.: Sei $G = (N, T, P, S)$ eine formale Grammatik. Dann ist $L(G) = \{\text{Wörter } w \text{ über } T \mid S \xRightarrow{*} w\}$ die von G erzeugte Sprache.

3.1.3.7. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.8. Reguläre Grammatiken

Def.: Eine **reguläre (oder type-3-) Grammatik** ist eine formale Grammatik mit den folgenden Einschränkungen:

- Alle Produktionen sind entweder von der Form
 - $X \rightarrow aY$ mit $X \in N, a \in T, Y \in N$ (*rechtsreguläre Grammatik*) oder
 - $X \rightarrow Ya$ mit $X \in N, a \in T, Y \in N$ (*linksreguläre Grammatik*)
- $X \rightarrow \epsilon$ ist erlaubt

3.1.3.9. Beispiel

Hier entsteht ein Tafelbild.

3.1.3.10. Reguläre Sprachen

Satz: Die von endlichen Automaten akzeptierte Sprachklasse, die von regulären Ausdrücken beschriebene Sprachklasse und die von regulären Grammatiken erzeugte Sprachklasse sind identisch und heißen **reguläre Sprachen**.

3.1.4. Wrap-Up

3.1.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- DFAs und NFAs
- Reguläre Ausdrücke
- Reguläre Grammatiken
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne DFAs
- k1: Ich kenne reguläre Ausdrücke
- k1: Ich kenne reguläre Grammatiken
- k2: Ich kann die Zusammenhänge und Gesetzmäßigkeiten bzgl. der oben genannten Konstrukte an einem Beispiel erklären
- k3: Ich kann für eine Fragestellung DFAs, reguläre Ausdrücke, reguläre Grammatiken entwickeln
- k3: Ich kann einen DFA entwickeln, der alle Schlüsselwörter, Namen und weitere Symbole einer Programmiersprache akzeptiert

3.2. Reguläre Sprachen, Ausdrucksstärke (Teil 2)

Weitere Unterlagen

- [Annotierte Folien: Reguläre Sprachen, Ausdrucksstärke](#)

3.2.1. Wiederholung

3.2.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Wie sind DFAs und NFAs definiert?
- Was sind reguläre Ausdrücke?
- Was sind formale und reguläre Grammatiken?
- In welchem Zusammenhang stehen all diese Begriffe?

3.2.2. Motivation

3.2.2.1. Was haben reguläre Sprachen mit Compilern zu tun?

Hier entsteht ein Tafelbild.

3.2.2.2. Themen für heute

- Reguläre Sprachen
- Lexer
- Grenzen regulärer Sprachen

3.2.2.3. Wozu reguläre Sprachen im Compilerbau?

Reguläre Ausdrücke

- definieren Schlüsselwörter und alle weiteren Symbole einer Programmiersprache, z. B. den Aufbau von Gleitkommazahlen
- werden (oft von einem Generator) in DFAs umgewandelt
- sind die Basis des *Scanners* oder *Lexers*

3.2.3. Lexer

3.2.3.1. Ein Lexer ist mehr als ein DFA

Ein **Lexer**

- kann aus regulären Ausdrücken automatisch generiert werden
- wandelt mittels DFAs aus regulären Ausdrücken die Folge von Zeichen der Quelldatei in eine Folge von sog. Token um
- bekommt als Input eine Liste von Paaren aus regulären Ausdrücken und Tokennamen, z. B. (“while”, WHILE)
- Kommentare und Strings müssen richtig erkannt werden. (Schachtelungen)
- liefert Paare von Token und deren Werte, sofern benötigt, z. B. (WHILE, _), oder (IDENTIFIER, “radius”) oder (INTEGERZAHL, “334”)

3.2.3.2. Wofür reichen reguläre Sprachen nicht?

Für z. B. alle Sprachen, in deren Wörtern Zeichen über eine Konstante hinaus gezählt werden müssen. Diese Sprachen lassen sich oft mit Variablen im Exponenten beschreiben, die unendlich viele Werte annehmen können.

- $a^i b^{2*i}$ ist nicht regulär

- $a^i b^{2*i}$ für $0 \leq i \leq 3$ ist regulär
- Wo finden sich die oben genannten Variablen bei einem DFA wieder?
- Warum ist die erste Sprache oben nicht regulär, die zweite aber?

3.2.3.3. Wie geht es weiter?

Ein Parser

- führt mit Hilfe des Tokenstreams vom Lexer die Syntaxanalyse durch
- basiert auf einer sog. kontextfreien Grammatik, deren Terminale die Token sind
- liefert die syntaktische Struktur in Form eines Ableitungsbaums (**syntax tree, parse tree**), bzw. einen **AST** (abstract syntax tree) ohne redundante Informationen im Ableitungsbaum (z. B. Semikolons)
- liefert evtl. Fehlermeldungen

3.2.4. Wrap-Up

3.2.4.1. Wrap-Up

- Definition und Aufgaben von Lexern
- Zusammenhänge zwischen diesen Mechanismen und Lexern, bzw. Lexergeneratoren
- Grenzen regulärer Sprachen

Zum Nachlesen

- Aho u. a. (2023): Abschnitt 2.6 und Kapitel 3
- Torczon und Cooper (2012): Kapitel 2
- Parr (2014)

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k1: Ich kenne die Zusammenhänge zwischen DFAs, regulären Ausdrücken, regulären Grammatiken und Lexern
- k2: Ich kann für eine Beispielsprache begründen, warum sie nicht mit einem der oben genannten Mechanismen beschrieben werden kann

3.3. Lexer mit ANTLR generieren

TL;DR

ANTLR ist ein Parser-Generator, der aus einer Grammatik einen Parser in verschiedenen Zielsprachen (Java, Python, C++, ...) generieren kann.

In der ANTLR-Grammatik werden die Parser-Regeln klein geschrieben, die Lexer-Regeln werden mit **Großbuchstaben** geschrieben. Jede Lexer-Regel liefert ein Token zurück, dabei ist der Tokenname die linke Seite der Regel. Wie bei Flex gewinnt der längste Match, und bei Gleichstand (mehrere längste Regeln matchen) gewinnt die zuerst definierte Regel.

Die Lexer-Regeln können mit Aktionen annotiert werden, die beim Matchen der jeweiligen Regel abgearbeitet werden. Diese Aktionen müssen in der Zielprogrammiersprache formuliert werden, da sie in die generierte Lexerklasse in die jeweiligen Methoden eingebettet werden.

Videos

- [VL Lexer mit ANTLR](#)
- [Demo ANTLR Basics](#)
- [Demo Verhalten Lexer-Regeln](#)
- [Demo Lexer-Regeln mit Aktionen](#)

Aus dem Eingabe(-quell-)text

```
1  /* demo */
2  a= [5 , 6] ;
```

erstellt der Lexer (oder auch Scanner genannt) eine Sequenz von Token:

```
1  <ID, "a"> <ASSIGN> <LBRACK> <NUM, 5> <COMMA> <NUM, 6> <RBRACK> <SEMICOL>
```

- Input: Zeichenstrom (Eingabedatei o.ä.)
- Verarbeitung: Finden sinnvoller Sequenzen im Zeichenstrom ("Lexeme"), Einteilung in Kategorien und Erzeugen von Token (Paare: Typ/Name, Wert)
- Ausgabe: Tokenstrom

Normalerweise werden für spätere Phasen unwichtige Elemente wie White-Space oder Kommentare entfernt.

Durch diese Vorverarbeitung wird eine höhere Abstraktionsstufe erreicht und es können erste grobe Fehler gefunden werden. Dadurch kann der Parser auf einer abstrakteren Stufe arbeiten und muss nicht mehr den gesamten ursprünglichen Zeichenstrom verarbeiten.

Anmerkung: In dieser Phase steht die Geschwindigkeit stark im Vordergrund: Der Lexer "sieht" *alle* Zeichen im Input. Deshalb findet man häufig von Hand kodierte Lexer, obwohl die Erstellung der Lexer auch durch Generatoren erledigt werden könnte ...

Anmerkung: Die Token sind die Terminalsymbole in den Parserregeln (Grammatik).

3.3.2. Definition wichtiger Begriffe

- **Token:** Tupel (Tokenname, optional: Wert)

Der Tokenname ist ein abstraktes Symbol, welches eine lexikalische Einheit repräsentiert (Kategorie). Die Tokennamen sind die Eingabesymbole für den Parser.

Token werden i.d.R. einfach über ihren Namen referenziert. Token werden häufig zur Unterscheidung von anderen Symbolen in der Grammatik in Fettschrift oder mit großen Anfangsbuchstaben geschrieben.

Ein Token kann einen Wert haben, etwa eine Zahl oder einen Bezeichner, der auf das zum Token gehörende Pattern gematcht hatte (also das Lexem). Wenn der Wert des Tokens eindeutig über den Namen bestimmt ist (im Beispiel oben beim Komma oder den Klammern), dann wird häufig auf den Wert verzichtet.

- **Lexeme:** Sequenz von Zeichen im Eingabestrom, die auf ein Tokenpattern matcht und vom Lexer als Instanz dieses Tokens identifiziert wird.
- **Pattern:** Beschreibung der Form eines Lexems

Bei Schlüsselwörtern oder Klammern etc. sind dies die Schlüsselwörter oder Klammern selbst. Bei Zahlen oder Bezeichnern (Namen) werden i.d.R. reguläre Ausdrücke zur Beschreibung der Form des Lexems formuliert.

3.3.3. Typische Muster für Erstellung von Token

1. Schlüsselwörter

- Ein eigenes Token (RE/DFA) für jedes Schlüsselwort, oder
- Erkennung als Name (**ID**) und nachträglich Vergleich mit Wörterbuch sowie Korrektur des Tokentyps

Wenn Schlüsselwörter über je ein eigenes Token abgebildet werden, benötigt man für jedes Schlüsselwort einen eigenen RE bzw. DFA. Die Erkennung als Bezeichner und das Nachschlagen in einem Wörterbuch (geeignete Hashtabelle) sowie die entsprechende nachträgliche Korrektur des Tokentyps kann die Anzahl der Zustände im Lexer signifikant reduzieren!

2. Operatoren

- Ein eigenes Token für jeden Operator, oder
- Gemeinsames Token für jede Operatoren-Klasse

3. Bezeichner: Ein gemeinsames Token für alle Namen

4. Zahlen: Ein gemeinsames Token für alle numerischen Konstante (ggf. Integer und Float unterscheiden)

Für Zahlen führt man oft ein Token "**NUM**" ein. Als Attribut speichert man das Lexem i.d.R. als String. Alternativ kann man (zusätzlich) das Lexem in eine Zahl konvertieren und als (zusätzliches) Attribut speichern. Dies kann in späteren Stufen viel Arbeit sparen.

5. String-Literale: Ein gemeinsames Token

6. Komma, Semikolon, Klammern, ...: Je ein eigenes Token

7. Regeln für White-Space und Kommentare etc. ...

Normalerweise benötigt man Kommentare und White-Spaces in den folgenden Stufen nicht und entfernt diese deshalb aus dem Eingabestrom. Dabei könnte man etwa White-Spaces in den Pattern der restlichen Token berücksichtigen, was die Pattern aber sehr komplex macht. Die Alternative sind zusätzliche Pattern, die auf die White-Space und anderen nicht benötigten Inhalt matchen und diesen "geräuschlos" entfernen. Mit diesen Pattern werden keine Token erzeugt, d.h. der Parser und die anderen Stufen bemerken nichts von diesem Inhalt.

Gelegentlich benötigt man aber auch Informationen über White-Spaces, beispielsweise in Python. Dann müssen diese Token wie normale Token an den Parser weitergereicht werden.

Jedes Token hat i.d.R. ein Attribut, in dem das Lexem gespeichert wird. Bei eindeutigen Token (etwa bei eigenen Token je Schlüsselwort oder bei den Interpunktions-Token) kann man sich das Attribut auch sparen, da das Lexem durch den Tokennamen eindeutig rekonstruierbar ist.

Token	Beschreibung	Beispiel-Lexeme
if	Zeichen i und f	if
relop	< oder > oder <= oder >= oder == oder !=	<, <=
id	Buchstabe, gefolgt von Buchstaben oder Ziffern	pi , count , x3
num	Numerische Konstante	42, 3.14159, 0
literal	Alle Zeichen außer " , in " eingeschlossen	"core dumped"

Anmerkung: Wenn es mehrere matchende REs gibt, wird in der Regel das längste Lexem bevorzugt. Wenn es mehrere gleich lange Alternativen gibt, muss man mit Vorrangregeln bzgl. der Token arbeiten.

3.3.4. "Hello World" mit ANTLR (Lexer)

```

1 grammar Hello;
2
3 start      : 'hello' GREETING ;
4
5 GREETING   : [a-zA-Z]+ ;
6 WHITESPACE : [ \t\n]+ -> skip ;

```

Konsole: Hello (Classpath, Aliase, grun, Main, Dateien, Ausgabe)

3.3.4.1. Hinweis zur Grammatik (Regeln)

- **start** ist eine Parser-Regel => Eine Parser-Regel pro Grammatik wird benötigt, damit man den generierten Parser am Ende auch starten kann ...
- Die anderen beiden Regeln (mit großem Anfangsbuchstaben) aus der obigen Grammatik zählen zum Lexer

3.3.4.2. ANTLR einrichten

- Lokal für die Nutzung in der Konsole:
 - Es wird ein JDK benötigt. Installieren Sie am besten die aktuelle LTS-Version.
 - Aktuelle Version herunterladen: antlr.org, für Java als Zielsprache: “Complete ANTLR 4.x Java binaries jar”
 - CLASSPATH setzen: `export CLASSPATH=".:<pathToJar>/antlr-4.13.2-complete.jar:$CLASSPATH"`
 - Aliase einrichten (`.bashrc`):
 - * `alias antlr='java org.antlr.v4.Tool'`
 - * `alias grun='java org.antlr.v4.gui.TestRig'`
- Alternativ für Java-Projekte mit Gradle (empfehlenswert):

```
1 plugins {
2     id 'java'
3     id 'antlr'
4 }
5
6 repositories {
7     mavenCentral()
8 }
9
10 dependencies {
11     antlr 'org.antlr:antlr4:4.13.2'
12 }
```

Der Eintrag in `plugins` sorgt dafür, dass Gradle beim Bauen des Java-Projekts die ANTLR-Grammatiken übersetzt, der Eintrag in `dependencies` lädt automatisch das Jar-File für ANTLR herunter und bindet es entsprechend im CLASSPATH ein.

- Alternativ das [ANTLR tool \(JAR\)](#) manuell herunterladen und in der IDE als Library hinzufügen (bitte nur als Hilfslösung, der Weg über das Build-Tool ist deutlich besser)
- Im Web ohne lokale Installation: [ANTLR Lab](#) (nur HTTP)

(vgl. github.com/antlr/antlr4/blob/master/doc/getting-started.md)

Tip: Für IntelliJ gibt es ein sehr gutes [ANTLR-Plugin](#). Dieses erlaubt das interaktive Experimentieren mit ANTLR-Grammatiken, ohne dass man ANTLR selbst installiert haben muss oder Code geschrieben haben muss. Für die Arbeit mit Grammatiken ist dieses Plugin unbedingt empfehlenswert! (Für das normale Arbeiten sollten Sie aber die Gradle-Konfiguration nutzen!)

Hinweis: Im Beispiel-Projekt [student-support-code-template](#) finden Sie ein fertig eingerichtetes Java-Projekt mit Gradle und ANTLR. Sie können dieses Projekt als Template nutzen und in Ihrer IDE importieren, d.h. Sie brauchen nur ein installiertes JDK (Version 25 ist die aktuelle LTS) und eine Java-IDE, der Rest (Gradle, ANTLR) kommt über die Einstellungen im Beispiel-Projekt. Durch die Gradle-Settings sollten anschließend alle nötigen Einstellungen und Abhängigkeiten automatisch in der IDE korrekt gesetzt bzw. aufgelöst werden. Im `Main.main` () finden Sie eine kurze Demo, wie man einen generierten Lexer/Parser mit einbindet und aufruft. Grammatiken

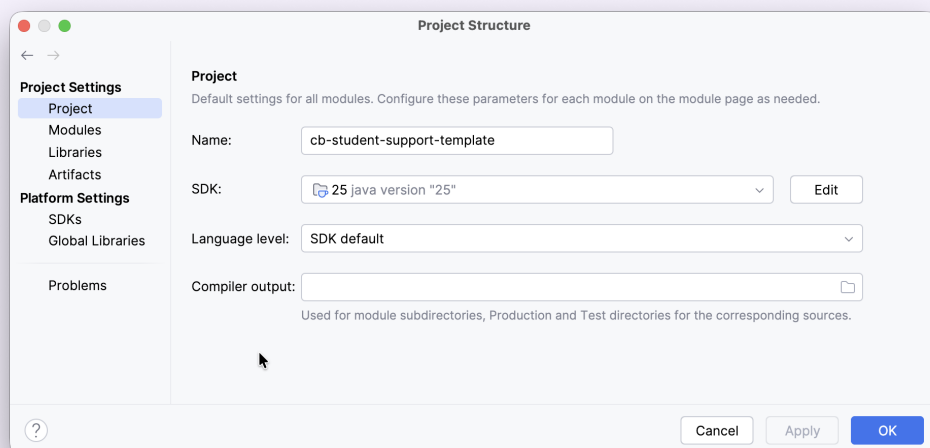
können unter `src/main/antlr/` abgelegt werden, wo sie automatisch vom ANTLR-Gradle-Plugin beim Build-Prozess (`./gradlew build`) gefunden und bearbeitet werden. Die daraus generierten ANTLR-Lexer und -Parser werden im Build-Ordner `build/generated-src/antlr/main/` abgelegt und stehen in der IDE damit automatisch zur Verfügung, ohne dass die generierten Dateien im versionierten Source-Tree auftauchen und diesen “verschmutzen”. **Wichtig:** So lange wie die generierten Dateien nicht erzeugt wurden, zeigen die IDE für diese Klassen und Interfaces entsprechend Fehler an. Ein Übersetzen des Projekts oder ein explizites `./gradlew generateGrammarSource` generiert die fehlenden Dateien und die Fehlermeldungen sollten verschwinden.

Important

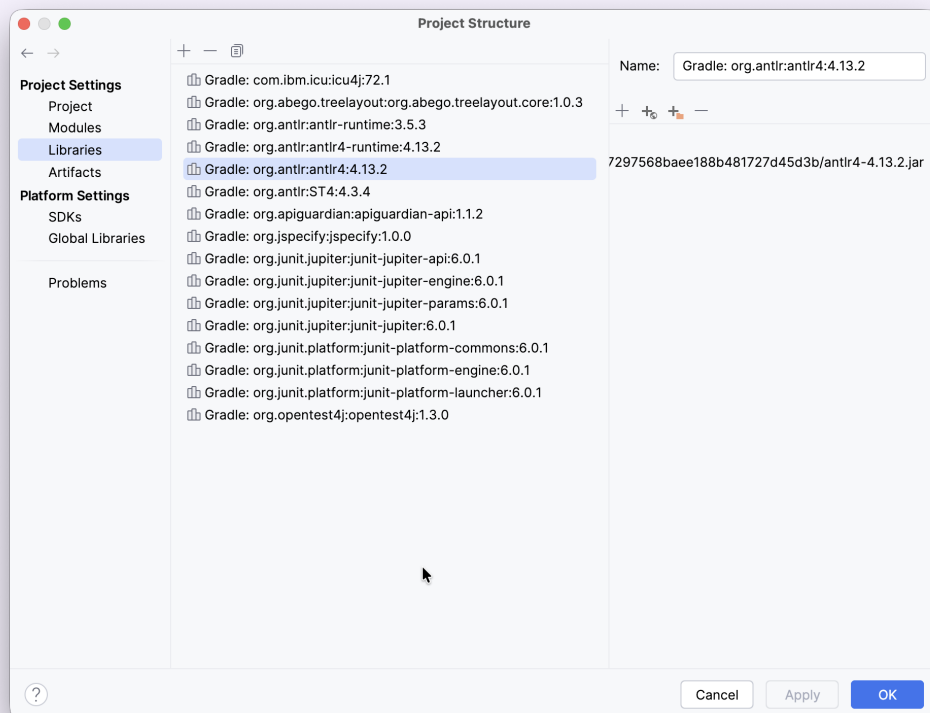
Hinweis: Sorgen Sie dafür, dass Ihre IDE tatsächlich auch die **Projekteinstellungen von Gradle übernommen** hat und auch **mit Gradle baut!**

1. Check, ob die **Projekteinstellungen** in IntelliJ passen:

1. Menü `File > Project Structure > Project Settings > Project` sollte für Ihr Projekt als SDK ein “Java 25” zeigen:

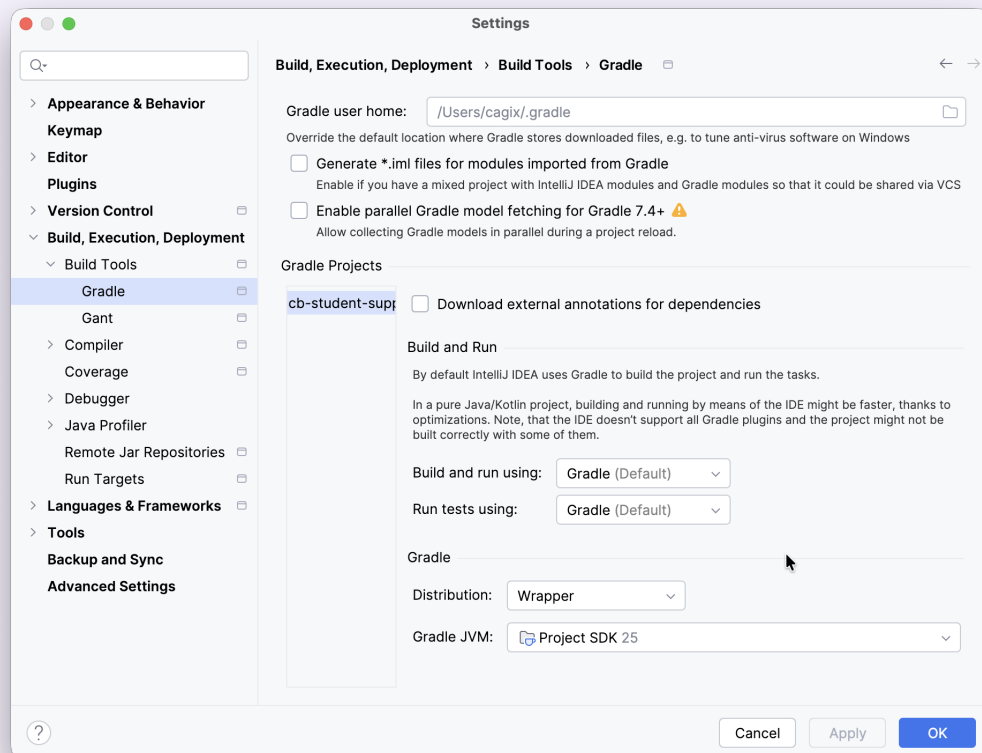


2. Menü `File > Project Structure > Project Settings > Libraries` sollte für Ihr Projekt Jar-Files für ANTLR4 zeigen:



2. Check, ob **IntelliJ mit Gradle baut:**

Menü `File > Settings > Build, Execution, Deployment > Build Tools > Gradle` sollte auf Gradle umgestellt sein:



Unter **“Build & Run”** sollte **“Gradle”** ausgewählt sein, die **“Distribution”** sollte auf **“Wrapper”** stehen, und als **“Gradle JVM”** sollte die für das Projekt verwendete JVM eingestellt sein, d.h. aktuell Java 25.

Siehe auch [Compiler-CampusMinden/student-support-code-template](https://compiler-campusminden.de/student-support-code-template).

3.3.4.3. “Hello World” übersetzen und ausführen

1. Grammatik übersetzen und Code generieren: `antlr Hello.g4`
2. Java-Code kompilieren: `javac *.java`
3. Lexer ausführen:

- `grun Hello start -tokens` (Grammatik “Hello”, Startregel “start”)
- Alternativ mit kleinem Java-Programm:

```

1  import org.antlr.v4.runtime.*;
2
3  public class Main {
4      public static void main(String[] args) throws Exception {
5          CharStream input = CharStreams.fromString(IO.readln("?> "));
6          Lexer l = new HelloLexer(input);
7          Token t = l.nextToken();

```

```
8         while (t.getType() != Token.EOF) {
9             System.out.println(t);
10            t = l.nextToken();
11        }
12    }
13 }
```

3.3.4.4. Generierte Dateien und Klassen

Nach dem Übersetzen finden sich folgende Dateien und Klassen vor:

```
1  .
2  |-- bin
3  |   |-- HelloBaseListener.class
4  |   |-- HelloBaseVisitor.class
5  |   |-- HelloLexer.class
6  |   |-- HelloListener.class
7  |   |-- HelloParser.class
8  |   |-- HelloParser$RContext.class
9  |   |-- HelloVisitor.class
10 |   |-- Main.class
11 |-- Hello.g4
12 |-- src
13 |   |-- HelloBaseListener.java
14 |   |-- HelloBaseVisitor.java
15 |   |-- HelloLexer.java
16 |   |-- HelloLexer.tokens
17 |   |-- HelloListener.java
18 |   |-- HelloParser.java
19 |   |-- Hello.tokens
20 |   |-- HelloVisitor.java
21 |   |-- Main.java
```

Anmerkung: Die Ordnerstruktur wurde durch ein ANTLR-Plugin für Eclipse erzeugt. Bei Ausführung in der Konsole liegen alle Dateien in einem Ordner.

Anmerkung: Per Default werden nur die Listener angelegt, für die Visitoren muss eine extra Option mitgegeben werden.

Die Dateien `Hello.tokens` und `HelloLexer.tokens` enthalten die Token samt einer internen Nummer. (Der Inhalt beider Dateien ist identisch.)

Die Datei `HelloLexer.java` enthält den generierten Lexer, der eine Spezialisierung der abstrakten Basisklasse `Lexer` darstellt. Über den Konstruktor wird der zu scannende `CharStream` gesetzt. Über die Methode `Lexer#nextToken()` kann man sich die erkannten Token der Reihe nach zurückgeben lassen. (Diese Methode wird letztlich vom Parser benutzt.)

Die restlichen Dateien werden für den Parser und verschiedene Arten der Traversierung des AST generiert (vgl. AST-basierte Interpreter).

3.3.4.5. Bedeutung der Ausgabe

Wenn man dem Hello-Lexer die Eingabe

```
1 hello world
2 <EOF>
```

(das <EOF> wird durch die Tastenkombination **STRG-D** erreicht) gibt, dann lautet die Ausgabe

```
1 $ grun Hello start -tokens
2 hello world
3 <EOF>
4 [@0,0:4='hello',<'hello'>,1:0]
5 [@1,6:10='world',<GREETING>,1:6]
6 [@2,12:11='<EOF>',<EOF>,2:0]
```

Die erkannten Token werden jeweils auf einer eigenen Zeile ausgegeben.

- @0: Das erste Token (fortlaufend nummeriert, beginnend mit 0)
- 0:4: Das Token umfasst die Zeichen 0 bis 4 im Eingabestrom
- ='hello': Das gefundene Lexem (Wert des Tokens)
- <'hello'>: Das Token (Name/Typ des Tokens)
- 1:0: Das Token wurde in Zeile 1 gefunden (Start der Nummerierung mit Zeile 1), und startet in dieser Zeile an Position 0

Entsprechend bekommt man mit

```
1 $ grun Hello start -tokens
2 hello
3   world
4
5 <EOF>
6 [@0,0:4='hello',<'hello'>,1:0]
7 [@1,8:12='world',<GREETING>,2:2]
8 [@2,15:14='<EOF>',<EOF>,4:0]
```

3.3.4.6. ANTLR-Grammatik für die Lexer-Generierung

- Start der Grammatik mit dem Namen “XYZ” mit

```
1 grammar XYZ;
```

oder (nur Lexer)

```
1 lexer grammar XYZ;
```

- Token und Lexer-Regeln starten mit *großen Anfangsbuchstaben* (Ausblick: Parser-Regeln starten mit kleinen Anfangsbuchstaben)

Format: `TokenName : Alternative1 | ... | AlternativeN ;`

Rekursive Lexer-Regeln sind erlaubt. **Achtung:** Es dürfen keine *links-rekursiven* Regeln genutzt werden, etwa wie `ID : ID '*' ID ; ...` (Eine genauere Definition und die Transformation in nicht-linksrekursive Regeln siehe CFG).

- Alle Literale werden in *einfache* Anführungszeichen eingeschlossen (es erfolgt keine Unterscheidung zwischen einzelnen Zeichen und Strings wie in anderen Sprachen)
- Zeichenmengen: `[a-z\n]` umfasst alle Zeichen von `'a'` bis `'z'` sowie `'\n'`
`'a' .. 'z'` ist identisch zu `[a-z]`
- Schlüsselwörter: Die folgenden Strings stellen reservierte Schlüsselwörter dar und dürfen nicht als Token, Regel oder Label genutzt werden:

```
1 import, fragment, lexer, parser, grammar, returns, locals, throws, catch,
  finally, mode, options, tokens
```

Anmerkung: `rule` ist zwar kein Schlüsselwort, wird aber als Methodenname bei der Codegenerierung verwendet. => Wie ein Schlüsselwort behandeln!

(vgl. github.com/antlr/antlr4/blob/master/doc/lexicon.md)

3.3.4.7. Greedy und Non-greedy Lexer-Regeln

Die regulären Ausdrücke `(...)?`, `(...)*` und `(...)+` sind *greedy* und versuchen soviel Input wie möglich zu matchen.

Falls dies nicht sinnvoll sein sollte, kann man mit einem weiteren `?` das Verhalten auf *non-greedy* umschalten. Allerdings können non-greedy Regeln das Verhalten des Lexers u.U. schwer vorhersehbar machen!

Die Empfehlung ist, non-greedy Lexer-Regeln nur sparsam einzusetzen (vgl. github.com/antlr/antlr4/blob/master/doc/wildcard.md)

3.3.5. Verhalten des Lexers

3.3.5.1. 1. Längster Match

Primäres Ziel: Erkennen der längsten Zeichenkette

```
1 CHARS : [a-z]+ ;
2 DIGITS : [0-9]+ ;
3 FOO : [a-z]+ [0-9]+ ;
```

Die Regel, die den längsten Match für die aktuelle Eingabesequenz produziert, "gewinnt".

Im Beispiel würde ein "foo42" als `FOO` erkannt und nicht als `CHARS DIGITS`.

3.3.5.2. 2. Reihenfolge

Reihenfolge in Grammatik definiert Priorität

```

1 FOO      : 'f' .*? 'r' ;
2 BAR      : 'foo' .*? 'bar' ;

```

Falls mehr als eine Lexer-Regel die selbe Inputsequenz matcht, dann hat die in der Grammatik zuerst genannte Regel Priorität.

Im Beispiel würden für die Eingabe “foo42bar” beide Regeln den selben längsten Match liefern - die Regel **FOO** ist in der Grammatik früher definiert und “gewinnt”.

3.3.5.3. 3. Non-greedy Regeln

Non-greedy Regeln versuchen so *wenig* Zeichen wie möglich zu matchen

```

1 FOO      : 'foo' .*? 'bar' ;
2 BAR      : 'bar' ;

```

Hier würde ein “foo42barbar” zu **FOO** gefolgt von **BAR** erkannt werden.

Achtung: Nach dem Abarbeiten einer non-greedy Sub-Regel in einer Lexer-Regel gilt “*first match wins*”

`.*? ( '4' | '42' )`

=> Der Teil `'42'` auf der rechten Seite ist “toter Code” (wegen der non-greedy Sub-Regel `.*?`)!

Die Eingabe “x4” würde korrekt erkannt, währende “x42” nur als “x4” erkannt wird und für die verbleibende “2” würde ein *token recognition error* geworfen.

(vgl. github.com/antlr/antlr4/blob/master/doc/wildcard.md)

3.3.6. Attribute und Aktionen

```

1 grammar Demo;
2
3 @header {
4   import java.util.*;
5 }
6
7 @members {
8   String s = "";
9 }
10
11 start      : TYPE ID '=' INT ';' ;
12
13 TYPE       : ('int' | 'float') {s = getText();} ;
14 INT        : [0-9]+             {System.out.println(s+"="+Integer.valueOf(getText(
15   )));} ;
16 ID         : [a-z]+             {setText(String.valueOf(getText().charAt(0)));} ;
17 WS         : [ \t\n]+ -> skip ;

```

3.3.6.1. Attribute bei Token (Auswahl)

Token haben Attribute, die man abfragen kann. Dies umfasst u.a. folgende Felder:

- `text`: Das gefundene Lexem als String
- `type`: Der Token-Typ als Integer
- `index`: Das wievielte Token (als Integer)

(vgl. github.com/antlr/antlr4/blob/master/doc/actions.md)

Zur Auswertung in den Lexer-Regeln muss man anders vorgehen als in Parser-Regeln: Nach der Erstellung eines Tokens kann man die zum Attribut gehörenden `getX()` und `setX()`-Methoden aufrufen, um die Werte abzufragen oder zu ändern.

Dies passiert im obigen Beispiel für das Attribut `text`: Abfrage mit `getText()`, Ändern/Setzen mit `setText()`.

Die Methodenaufrufe wirken sich immer auf das gerade erstellte Token aus.

Achtung: Bei Aktionen in Parser-Regeln gelten andere Spielregeln!

3.3.6.2. Aktionen mit den Lexer-Regeln

Aktionen für Lexer-Regeln sind Code-Blöcke in der Zielsprache, eingeschlossen in geschweifte Klammern. Die Code-Blöcke werden direkt in die generierten Lexer-Methoden kopiert.

Zusätzlich:

- `@header`: Package-Deklarationen und/oder Importe (wird vor der Klassendefinition eingefügt)
- `@members`: zusätzliche Attribute für die generierten Lexer- (und Parser-) Klassen.

Mit `@lexer::header` bzw. `@lexer::members` werden diese Codeblöcke nur in den generierten Lexer eingefügt.

Anmerkung: Lexer-Aktionen müssen am Ende der äußersten Alternative erscheinen. Wenn eine Lexer-Regel mehr als eine Alternative hat, müssen diese in runde Klammern eingeschlossen werden.

(vgl. github.com/antlr/antlr4/blob/master/doc/grammars.md)

3.3.7. Wrap-Up

Lexer mit ANTLR generieren: Lexer-Regeln werden mit **Großbuchstaben** geschrieben

- Längster Match gewinnt, Gleichstand: zuerst definierte Regel
- *non greedy*-Regeln: versuche so *wenig* Zeichen zu matchen wie möglich
- Aktionen beim Matchen

Zum Nachlesen

- Parr (2014)

Lernziele

- k3: Ich kann Lexer-Regeln in ANTLR formulieren und einsetzen
- k2: Ich kann das Verhalten des Lexers erklären, insbesondere längste Matches und der Einfluss der Reihenfolge der Regeln
- k3: Ich kann Lexer-Aktionen einsetzen

Challenges

Token und Lexer-Regeln mit ANTLR

Formulieren Sie für ANTLR Lexer-Regeln, mit denen folgende Token erkannt werden:

- White-Space: Leerzeichen, Tabs, Zeilenumbrüche
- Vergleichsoperatoren: <, >, <=, >=, ==, <>
- If: **if**
- Then: **then**
- Else: **else**
- Namen: Ein Buchstabe, gefolgt von beliebig vielen weiteren Buchstaben und/oder Ziffern
- Numerische Konstanten: Mindestens eine Ziffer, gefolgt von maximal einem Paar bestehend aus einem Punkt und mindestens einer Ziffer, gefolgt von maximal einem Paar bestehend aus dem Buchstaben "E" gefolgt von einem "+" oder "-" und mindestens einer Ziffer.

Formulieren Sie Hilfskonstrukte zur Verwendung in mehreren Lexer-Regeln als ANTLR-Fragmente.

White-Spaces sollen entfernt werden und nicht als Token weitergereicht werden.

Real-World-Lexer mit ANTLR: Programmiersprache Lox

Betrachten Sie folgenden Code-Schnipsel in der Sprache "Lox":

```
1 fun fib(x) {
2   if (x == 0) {
3     return 0;
4   } else {
5     if (x == 1) {
6       return 1;
7     } else {
8       fib(x - 1) + fib(x - 2);
9     }
10  }
11 }
12
13 var wuppie = fib(4);
```

Erstellen Sie für diese fiktive Sprache einen Lexer mit ANTLR. Die genauere Sprachdefinition finden Sie unter craftinginterpreters.com/the-lox-language.html.

Pig-Latin mit ANTLR-Lexer

Schreiben Sie eine Lexer-Grammatik mit eingebetteten Aktionen für ANTLR sowie ein passendes Programm zur Einbindung des generierten Lexers, welches einen Text nach [Pig Latin](#) übersetzt:

- Ist der erste Buchstabe eines Wortes ein Konsonant, schiebe ihn ans Ende des Wortes und füge "ay" an.
- Ist der erste Buchstabe eines Wortes ein Vokal, hänge an das Wort ein "ay" an.

Lexing mit ANTLR

In einem Telefonbuch sind zeilenweise Namen und Telefonnummern gespeichert.

Definieren Sie eine Lexer-Grammatik für ANTLR, mit der Sie die Zeilen einlesen können. Können Sie dabei verschiedene Formate der Telefonnummern berücksichtigen?

```
1 Heinz 030 5346 983
2 Kalle +49 30 1234 567
3 Lina +49.571.8385-255
4 Rosi (0571) 8385-268
```

Können Sie die Grammatik so anpassen, dass Sie nur möglichst wenige verschiedene Token an den Parser weitergeben?

Ergänzen Sie Ihre Grammatik um Lexer-Aktionen, so dass Sie die Zeilen, die Zeichen (in den Namen) und die Ziffern (in den Telefonnummern) zählen können.

Lexing mit ANTLR

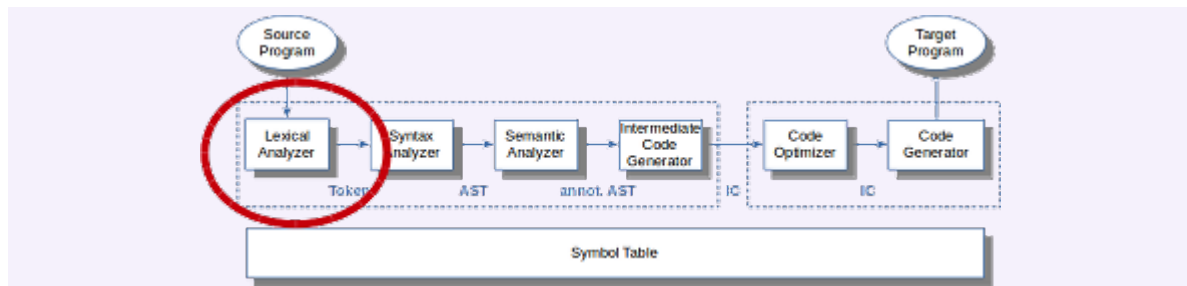
IBAN für Deutschland bestehen aus dem Kürzel "DE" sowie einer zweistelligen Checksumme, gefolgt von 2x 4 Ziffern für die Bank (ehemalige Bankleitzahl) sowie 2x 4 Ziffern für die ehemalige Kontonummer sowie zwei weiteren Ziffern. Typisch sind zwei Formate:

- Menschenlesbares Format: DEcc bbbb bbbb kkkk kkkk xx
- Maschinenlesbares Format: DEccbbbbbbkkkkkkkkxx

Definieren Sie eine Lexer-Grammatik für ANTLR, mit der Sie die verschiedenen IBAN-Formate für Deutschland einlesen können.

3.4. Lexer: Handcodierte Implementierung

TL;DR



Der Lexer (auch “Scanner”) soll den Zeichenstrom in eine Folge von Token zerlegen. Zur Spezifikation der Token werden reguläre Ausdrücke verwendet.

Von Hand implementierte Lexer arbeiten üblicherweise rekursiv und verarbeiten immer das nächste Zeichen im Eingabestrom. Die Arbeitsweise erinnert an LL-Parser (vgl. LL-Parser).

Lexer müssen sehr effizient sein, da sie noch direkt auf der niedrigsten Abstraktionsstufe arbeiten und u.U. oft durchlaufen werden. Deshalb setzt man hier gern spezielle Techniken wie Puffern von Zeichen über einen Doppel-Puffer ein.

Die Palette an Fehlerbehandlungsstrategien im Lexer reichen von “aufgeben” über den “Panic Mode” (“gobbeln” von Zeichen, bis wieder eines passt) und Ein-Schritt-Transformationen bis hin zu speziellen Lexer-Regeln, die beispielsweise besonders häufige Typos abfangen.

Videos

- [VL Handcodierte Lexer](#)

Aus dem Eingabe(-quell-)text

```
1  /* demo */
2  a= [5 , 6] ;
```

erstellt der Lexer (oder auch Scanner genannt) eine Sequenz von Token:

```
1  <ID, "a"> <ASSIGN> <LBRACK> <NUM, 5> <COMMA> <NUM, 6> <RBRACK> <SEMICOL>
```

3.4.2. Manuelle Implementierung: Rekursiver Abstieg

```
1  def nextToken():
2      while (peek != EOF): # globale Variable, über consume()
3          switch (peek):
4              case ' ': case '\t': case '\n': WS(); continue
5              case '[': consume(); return Token(LBRACK, '[')
6              ...
7          default:
8              if isLetter(peek): return NAME()
9              raise Error("invalid character: "+peek)
10     return Token(EOF_Type, "<EOF>")
```

```

11
12 def WS():
13     while (peek == ' ' || peek == '\t' || ...): consume()
14
15 def NAME():
16     buf = StringBuilder()
17     do { buf.append(peek); consume(); } while (isLetter(peek))
18     return Token(NAME, buf.toString())

```

Die manuelle Implementierung “denkt” nicht in den Zuständen des DFA, sondern orientiert sich immer am aktuellen Zeichen “**peek**”. Abhängig von dessen Ausprägung wird entweder direkt ein Token erzeugt und das Zeichen aus dem Eingabestrom entfernt sowie das nächste Zeichen eingelesen (mittels der Funktion `consume()`, nicht dargestellt im Beispiel), oder man ruft weitere Funktionen auf, die das Gewünschte erledigen, beispielsweise um White-Spaces zu entfernen oder um einen Namen einzulesen: Nach einem Buchstaben werden alle folgenden Buchstaben dem Namen (Bezeichner) hinzugefügt. Sobald ein anderes Zeichen im Eingabestrom erscheint, wird das Namen-Token erzeugt.

Die Funktion `consume()` “verbraucht” das aktuelle Zeichen “**peek**” und holt das nächste Zeichen aus dem Eingabestrom.

Anmerkung: Häufig findet man im Lexer keinen “schönen” objektorientierten Ansatz. Dies ist i.d.R. Geschwindigkeitsgründen geschuldet...

3.4.3. Read-Ahead: Unterscheiden von “<” und “<=”

```

1 def nextToken():
2     while (peek != EOF): # globale Variable
3         switch (peek):
4             case '<':
5                 if match('='): consume(); return Token(LE, "<=")
6                 else: consume(); return Token(LESS, '<')
7             ...
8     return Token(EOF_Type, "<EOF>")
9
10 def match(c): # Lookahead: Ein Zeichen
11     consume()
12     if (peek == c): return True
13     else: rollBack(); return False

```

Um die Token “<” und “<=” unterscheiden zu können, müssen wir ein Zeichen vorausschauen: Wenn nach dem “<” noch ein “=” kommt, ist es “<=”, sonst “<”.

Erinnerung: Die Funktion `consume()` liest das nächste Zeichen aus dem Eingabestrom und speichert den Wert in der globalen Variable `peek`.

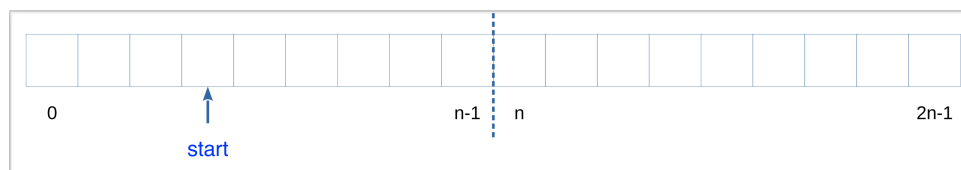
Für das Read-Ahead wird die Funktion `match()` definiert, die zunächst das bereits bekannte Zeichen, in diesem Fall das “<” durch das nächste Zeichen im Eingabestrom ersetzt (Aufruf von `consume()`). Falls der Vergleich des Lookahead-Zeichens mit dem gesuchten Zeichen erfolgreich ist, liegt das “größere” Token vor, also “<=”. Dann wird noch das “=” durch das nächste Zeichen ersetzt und das Token `LE` gebildet. Anderenfalls muss das zuviel gelesene Zeichen wieder in den Eingabestrom zurückgelegt werden (`rollBack()`).

3.4.4. Puffern des Input-Stroms: Double Buffering

Das Einlesen einzelner Zeichen führt zwar zu eleganten algorithmischen Lösungen, ist aber zur Laufzeit deutlich “teurer” als das Einlesen mit gepufferten I/O-Operationen, die eine ganze Folge von Zeichen einlesen (typischerweise einen ganzen Disk-Block, beispielsweise 4096 Zeichen).

Dazu kann man einen Ringpuffer nutzen, den man mit Hilfe von zwei gleich großen **char**-Puffern mit jeweils der Länge N simulieren kann. (N sollte dann der Länge eines Disk-Blocks entsprechen.)

Vergleiche auch [Wikipedia: “Circular Buffer”](#).



```

1  start = 0; end = 0; fill(buffer[0:n])
2
3  def consume():
4      peek = buffer[start]
5      start = (start+1) mod 2n
6      if (start mod n == 0):
7          fill(buffer[start:start+n-1])
8          end = (start+n) mod 2n
9
10 def rollBack():
11     if (start == end): raise Error("roll back error")
12     start = (start-1) mod 2n

```

Zunächst wird nur der vordere Pufferteil durch einen passenden Systemaufruf gefüllt.

Beim Weiterschalten im simulierten DFA oder im manuell kodierten Lexer (Funktionsaufruf von `consume()`) wird das nächste Zeichen aus dem vorderen Pufferteil zurückgeliefert. Über die Modulo-Operation bleibt der Pointer `start` immer im Speicherbereich der beiden Puffer.

Wenn man das Ende des vorderen Puffers erreicht, wird der hintere Puffer mit einem Systemaufruf gefüllt. Gleichzeitig wird ein Hilfspointer `end` auf den Anfang des vorderen Puffers gesetzt, um Fehler beim Roll-Back zu erkennen.

Wenn man das Ende des hinteren Puffers erreicht, wird der vordere Puffer nachgeladen und der Hilfspointer auf den Anfang des hinteren Puffers gesetzt.

Im Grunde ist also immer ein Puffer der “Arbeitspuffer” und der andere enthält die bereits gelesene (verarbeitete) Zeichenkette. Wenn beim Nachladen weniger als N Zeichen gelesen werden, liefert der Systemaufruf als letztes “Zeichen” ein EOF. Beim Verarbeiten wird `peek` entsprechend diesen Wert bekommen und der Lexer muss diesen Wert abfragen und berücksichtigen.

Für das Roll-Back wird der `start`-Pointer einfach dekrementiert (und mit einer Modulo-Operation auf den Speicherbereich der beiden Puffer begrenzt). Falls dabei der `end`-Pointer “eingeholt” wird, ist der `start`-Pointer

durch beide Puffer zurückgelaufen und es gibt keinen früheren Input mehr. In diesem Fall wird entsprechend ein Fehler gemeldet.

Anmerkung: In der Regel sind die Lexeme kurz und man muss man nur ein bis zwei Zeichen im Voraus lesen. Dann ist eine Puffergröße von 4096 Zeichen mehr als ausreichend groß und man sollte nicht in Probleme laufen. Wenn der nötige Look-Ahead aber beliebig groß werden kann, etwa bei Sprachen ohne reservierte Schlüsselwörtern oder bei Kontext-sensitiven Lexer-Grammatiken (denken Sie etwa an die Einrücktiefe bei Python), muss man andere Strategien verwenden. ANTLR beispielsweise vergrößert in diesem Fall den Puffer dynamisch, alternativ könnte man die Auflösung zwischen Schlüsselwörtern und Bezeichnern dem Parser überlassen.

3.4.5. Typische Muster für Erstellung von Token

1. Schlüsselwörter

- Ein eigenes Token (RE/DFA) für jedes Schlüsselwort, oder
- Erkennung als Name (**ID**) und nachträglich Vergleich mit Wörterbuch sowie Korrektur des Tokentyps

Wenn Schlüsselwörter über je ein eigenes Token abgebildet werden, benötigt man für jedes Schlüsselwort einen eigenen RE bzw. DFA. Die Erkennung als Bezeichner und das Nachschlagen in einem Wörterbuch (geeignete Hashtabelle) sowie die entsprechende nachträgliche Korrektur des Tokentyps kann die Anzahl der Zustände im Lexer signifikant reduzieren!

2. Operatoren

- Ein eigenes Token für jeden Operator, oder
- Gemeinsames Token für jede Operatoren-Klasse

3. Bezeichner: Ein gemeinsames Token für alle Namen

4. Zahlen: Ein gemeinsames Token für alle numerischen Konstante (ggf. Integer und Float unterscheiden)

Für Zahlen führt man oft ein Token "**NUM**" ein. Als Attribut speichert man das Lexem i.d.R. als String. Alternativ kann man (zusätzlich) das Lexem in eine Zahl konvertieren und als (zusätzliches) Attribut speichern. Dies kann in späteren Stufen viel Arbeit sparen.

5. String-Literale: Ein gemeinsames Token

6. Komma, Semikolon, Klammern, ...: Je ein eigenes Token

7. Regeln für White-Space und Kommentare etc. ...

Normalerweise benötigt man Kommentare und White-Spaces in den folgenden Stufen nicht und entfernt diese deshalb aus dem Eingabestrom. Dabei könnte man etwa White-Spaces in den Pattern der restlichen Token berücksichtigen, was die Pattern aber sehr komplex macht. Die Alternative sind zusätzliche Pattern, die auf die White-Space und anderen nicht benötigten Inhalt matchen und diesen "geräuschlos" entfernen. Mit diesen Pattern werden keine Token erzeugt, d.h. der Parser und die anderen Stufen bemerken nichts von diesem Inhalt.

Gelegentlich benötigt man aber auch Informationen über White-Spaces, beispielsweise in Python. Dann müssen diese Token wie normale Token an den Parser weitergereicht werden.

Jedes Token hat i.d.R. ein Attribut, in dem das Lexem gespeichert wird. Bei eindeutigen Token (etwa bei eigenen Token je Schlüsselwort oder bei den Interpunktions-Token) kann man sich das Attribut auch sparen, da das Lexem durch den Tokennamen eindeutig rekonstruierbar ist.

Token	Beschreibung	Beispiel-Lexeme
if	Zeichen i und f	if
relop	< oder > oder <= oder >= oder == oder !=	<, <=
id	Buchstabe, gefolgt von Buchstaben oder Ziffern	pi , count , x3
num	Numerische Konstante	42, 3.14159, 0
literal	Alle Zeichen außer " , in " eingeschlossen	"core dumped"

Anmerkung: Wenn es mehrere matchende REs gibt, wird in der Regel das längste Lexem bevorzugt. Wenn es mehrere gleich lange Alternativen gibt, muss man mit Vorrangregeln bzgl. der Token arbeiten.

3.4.6. Fehler bei der Lexikalischen Analyse

Problem: Eingabestrom sieht so aus: **fi** (**a**==42) { ... }

Der Lexer kann nicht erkennen, ob es sich bei **fi** um ein vertipptes Schlüsselwort handelt oder um einen Bezeichner: Es könnte sich um einen Funktionsaufruf der Funktion **fi** () handeln ... Dieses Problem kann erst in der nächsten Stufe sinnvoll erkannt und behoben werden.

=> Was tun, wenn keines der Pattern auf den Anfang des Eingabestroms passt?

Optionen:

- Aufgeben ...

Eventuell vielleicht sogar die beste und einfachste Variante :-)

- "Panic Mode": Entferne so lange Zeichen, bis ein Pattern passt.

Das verwirrt u.U. den Parser, kann aber insbesondere in interaktiven Umgebungen hilfreich sein. Ggf. kann man dem Parser auch signalisieren, dass hier ein Problem vorlag.

- Ein-Schritt-Transformationen:

- Füge fehlendes Zeichen in Eingabestrom ein.
- Entferne ein Zeichen aus Eingabestrom.
- Vertausche ein Zeichen:
 - * Ersetze ein Zeichen durch ein anderes.
 - * Vertausche zwei benachbarte Zeichen.

Diese Transformationen versuchen, den Input in einem Schritt zu reparieren. Das ist durchaus sinnvoll, da in der Praxis die meisten Fehler in dieser Stufe durch ein einzelnes Zeichen hervorgerufen werden: Es fehlt ein Zeichen oder es ist eines zuviel im Input. Es liegt ein falsches Zeichen vor (Tippfehler) oder zwei benachbarte Zeichen wurden verdreht ...

Im Prinzip könnte man auch eine allgemeinere Strategie versuchen, indem man diejenige Transformation mit der *kleinsten Anzahl von Schritten* zur Fehlerbehebung bestimmt. Beispiele dafür finden sich im Bereich Natural Language Processing (NLP), etwa die Levenshtein-Distanz oder der SoundEx-Algorithmus oder sogar Hidden-Markov-Modelle. Allerdings muss man sich in Erinnerung rufen, dass gerade in dieser ersten Phase eines Compilers die Geschwindigkeit stark im Fokus steht und eine ausgefeilte Fehlerkorrekturstrategie die vielen kleinen Optimierungen schnell wieder zunichte machen kann.

- Fehler-Regeln: Matche typische Typos

Gelegentlich findet man in den Grammatiken für den Lexer extra Regeln, die häufige bzw. typische Typos matchen und dann passend darauf reagieren.

3.4.7. Wrap-Up

- Zusammenhang DFA, RE und Lexer
- Implementierungsansatz: Manuell codiert (rekursiver Abstieg)
- Read-Ahead
- Puffern mit Doppel-Puffer-Strategie
- Typische Fehler beim Scannen

Zum Nachlesen

- Nystrom (2021): Kapitel 4
- Parr (2010): Kapitel 2 ("Pattern 2: LL(1) Recursive-Decent Lexer")

Lernziele

- k1: Ich kenne die Aufgaben eines Lexers
- k2: Ich kann die manuelle Implementierung eines Lexers mit Hilfe von rekursivem Abstieg erklären
- k2: Ich kann den Umgang mit dem Doppel-Puffer erklären
- k2: Ich kann die Varianten bei der Erkennung von Schlüsselwörtern an einem Beispiel verdeutlichen
- k2: Ich kann typische Fehler und Lösungsansätze in der lexikalischen Analyse erklären
- k3: Ich kann für ein Problem eine typische Einteilung von Token vornehmen
- k3: Ich kann einen Top-Down-Lexer mit Read-Ahead und intelligenter Pufferung implementieren

Challenges

Manuell implementierter Lexer

Betrachten Sie die folgende einfache Sprache:

```
1  a = 10 - 5      # Zuweisung des Ausdrucks 10-5 (Integer-Wert 5) an Variable
    a
2  b = a + 2 * 3    # Zuweisung von 16 an Variable b
3  c = a != b       # Zuweisung eines boolschen Werts an c
```

Es gibt nur Statements und Expressions:

- Statement: Zuweisung; jedes Statement endet mit einem NL
- Expression: Zahl, Variable, Addition, Subtraktion, Multiplikation (mit üblichem Vorrang), Vergleich

Aufgaben:

- Geben Sie eine ANTLR-Grammatik für diese Sprache an.
- Implementieren Sie analog zum Vorgehen in der Vorlesung einen Lexer für diese Sprache. (Nur den Lexer, den Parser besprechen wir in einer anderen Sitzung.)

4. Syntaktische Analyse

In der syntaktischen Analyse arbeitet ein Parser mit dem Tokenstrom, der vom Lexer kommt. Mit Hilfe einer Grammatik wird geprüft, ob gültige Sätze im Sinne der Sprache/Grammatik gebildet wurden. Der Parser erzeugt dabei den Parse-Tree. Man kann verschiedene Parser unterscheiden, beispielsweise die LL- und die LR-Parser.

4.1. CFG

Weitere Unterlagen

- [Annotierte Folien: CFG, LL-Parser](#)

4.1.1. Wiederholung

4.1.1.1. Endliche Automaten, reguläre Ausdrücke, reguläre Grammatiken, reguläre Sprachen

- Was ist ein Lexer?
- In welchem Zusammenhang stehen Lexer und reguläre Sprachen?
- Was können Lexer nicht?

4.1.2. Motivation

4.1.2.1. Was brauchen wir jetzt?

4.1.2.2. Themen für heute

- PDAs: mächtiger als DFAs, NFAs
- kontextfreie Grammatiken und Sprachen: mächtiger als reguläre Grammatiken und Sprachen
- DPDAs und deterministisch kontextfreie Grammatiken: die Grundlage der Syntaxanalyse im Compilerbau
- Syntaxanalyse

4.1.2.3. Einordnung: Erweiterung der Automatenklasse DFA, um komplexere Sprachen als die regulären akzeptieren zu können

Wir spendieren den DFAs einen möglichst einfachen, aber beliebig großen, Speicher, um zählen und matchen zu können. Wir suchen dabei konzeptionell die "kleinstmögliche" Erweiterung, die die akzeptierte Sprachklasse

gegenüber DFAs vergrößert.

- Der konzeptionell einfachste Speicher ist ein Stack. Wir haben keinen wahlfreien Zugriff auf die gespeicherten Werte.
- Es soll eine deterministische und eine indeterministische Variante der neuen Automatenklasse geben.
- In diesem Zusammenhang wird der Stack auch Keller genannt.

4.1.2.4. Kellerautomaten (Push-Down-Automata, PDAs)

Def.: Ein Kellerautomat (PDA) $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist ein Septupel mit:

Q :	eine endliche Menge von Zuständen
Σ :	eine endliche Menge von Eingabesymbolen
Γ :	ein endliches Kelleralphabet
δ :	die Übergangsfunktion
	$\delta : Q \times \Sigma \cup \{\epsilon\} \times \Gamma^* \rightarrow 2^{Q \times \Gamma^*}$
q_0 :	der Startzustand
$\perp \in \Gamma$:	der anfängliche Kellerinhalt, symbolisiert den leeren Keller ($\perp$ = bottom)
$F \subseteq Q$:	die Menge von Endzuständen

Abbildung 4.1.: Definition eines PDAs

Ein PDA ist per Definition nichtdeterministisch und kann spontane Zustandsübergänge durchführen.

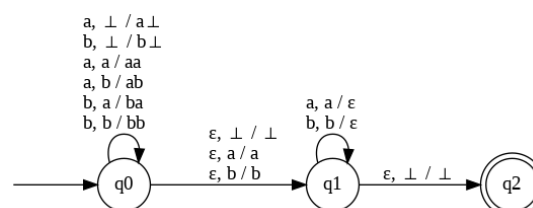
4.1.2.5. Was kann man damit akzeptieren?

Strukturen mit paarweise zu matchenden Symbolen.

Bei jedem Zustandsübergang wird ein Zeichen (oder ϵ) aus der Eingabe gelesen, ein Symbol von Keller genommen. Diese und das Eingabezeichen bestimmen den Folgezustand und eine Zeichenfolge, die auf den Stack gepackt wird. Dabei wird ein Symbol, (z. B. eines, das später mit einem Eingabesymbol zu matchen ist,) auf den Stack gepackt. Soll das automatisch vom Stack genommene Symbol auf dem Stack bleiben, muss es wieder gepusht werden.

4.1.2.6. Beispiel

Ein PDA für $L = \{ww^R \mid w \in \{a, b\}^*\}$:



4.1.2.7. Noch ein Beispiel

Hier entsteht ein Tafelbild.

4.1.2.8. Deterministische PDAs

Def. Ein PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, \perp, F)$ ist *deterministisch* $:\Leftrightarrow$

- $\delta(q, a, X)$ hat höchstens ein Element für jedes $q \in Q, a \in \Sigma$ oder ($a = \epsilon$ und $X \in \Gamma$).
- Wenn $\delta(q, a, x)$ nicht leer ist für ein $a \in \Sigma$, dann muss $\delta(q, \epsilon, x)$ leer sein.

Deterministische PDAs werden auch *DPDAs* genannt.

4.1.2.9. Der kleine Unterschied

Satz: Die von DPDAs akzeptierten Sprachen sind eine echte Teilmenge der von PDAs akzeptierten Sprachen.

Die regulären Sprachen sind eine echte Teilmenge der von DPDAs akzeptierten Sprachen.

4.1.3. Kontextfreie Grammatiken und Sprachen

4.1.3.1. Kontextfreie Grammatiken

Def. Eine *kontextfreie* (cf-) Grammatik ist ein 4-Tupel $G = (N, T, P, S)$ mit N, T, S wie in (formalen) Grammatiken und P ist eine endliche Menge von Produktionen der Form:

$X \rightarrow Y$ mit $X \in N, Y \in (N \cup T)^*$.

$\Rightarrow, \Rightarrow^*$ sind definiert wie bei regulären Sprachen. Bei cf-Grammatiken nennt man die Ableitungsbäume oft *Parse trees*.

4.1.3.2. Beispiel

$S \rightarrow a \mid S + S \mid S * S$
Hier entsteht ein Tafelbild.

Ableitungsbäume für $a + a * a$:

Hier entsteht ein Tafelbild.

4.1.3.3. Was ist hier los?

4.1.3.4. Nicht jede kontextfreie Grammatik ist eindeutig

Def.: Gibt es in einer von einer kontextfreien Grammatik erzeugten Sprache ein Wort, für das mehr als ein Ableitungsbaum existiert, so heißt diese Grammatik *mehrdeutig*. Anderenfalls heißt sie *eindeutig*.

Satz: Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.

Satz: Es gibt kontextfreie Sprachen, für die keine eindeutige Grammatik existiert.

4.1.3.5. Kontextfreie Grammatiken und PDAs

Satz: Die kontextfreien Sprachen und die Sprachen, die von PDAs akzeptiert werden, sind dieselbe Sprachklasse.

Satz: Eine von einem DPDA akzeptierte Sprache hat eine eindeutige Grammatik.

Vorgehensweise im Compilerbau: Eine (cf) Grammatik für die gewünschte Sprache definieren und schauen, ob sich daraus ein DPDA generieren lässt (automatisch).

4.1.4. Syntaxanalyse

4.1.4.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

4.1.4.2. Ziele der Syntaxanalyse

- Bestimmung der syntaktischen Struktur eines Programms
- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche

4.1.4.3. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (= Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

4.1.5. Wrap-Up

4.1.5.1. Das sollen Sie mitnehmen

- Die Struktur von gängigen Programmiersprachen lässt sich nicht mit regulären Ausdrücken beschreiben und damit nicht mit DFAs akzeptieren.
- Das Automatenmodell der DFAs wird um einen endlosen Stack erweitert, das ergibt PDAs.
- Kontextfreie Grammatiken (CFGs) erweitern die regulären Grammatiken.
- PDAs akzeptieren kontextfreie Sprachen.
- Deterministisch parsbare Sprachen haben eine eindeutige kontextfreie Grammatik, aber nicht für jede eindeutige kontextfreie Grammatik lässt sich ein deterministischer PDA finden.

- Es ist nicht entscheidbar, ob eine gegebene kontextfreie Grammatik eindeutig ist.
- Syntaxanalyse wird mit (möglichst deterministisch) kontextfreien Grammatiken durchgeführt.
- In der Praxis werden aus kontextfreien Grammatiken Parser automatisch generiert.

Zum Nachlesen

- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne PDAs
- k1: Ich kenne deterministische PDAs
- k1: Ich kenne kontextfreie Grammatiken
- k1: Ich kenne deterministisch kontextfreie Grammatiken
- k2: Ich kann den Zusammenhang zwischen PDAs und kontextfreien Grammatiken an einem Beispiel erklären
- k2: Ich kann PDAs entwickeln
- k2: Ich kann kontextfreie Grammatiken entwickeln

4.2. Parser mit ANTLR generieren

TL;DR

Mit ANTLR kann aus einer Grammatik ein LL(*)-Parser generiert werden. Die Parser-Regeln in der Grammatik fangen dabei mit einem **Kleinbuchstaben** an (Erinnerung: Lexer-Regel starten mit einem Großbuchstaben).

Regeln haben einen Namen (linke Seite) und eine Produktion (rechte Seite). Dabei können beliebige Abfolgen von Lexer- und Parser-Regeln auf der rechten Seite einer Parser-Regel auftauchen. Die Token müssen jeweils matchen, die Parser-Regeln werden in einen Aufruf der jeweiligen generierten Funktion übersetzt.

Parser-Regeln können aus mehreren Alternativen bestehen, diese werden per `|` separiert. Dabei hat bei Mehrdeutigkeiten die erste passende Alternative Vorrang. Wie bei Lexer-Regeln können Teile per `?` ein- oder keinmal vorkommen, per `*` beliebig oft oder per `+` ein- oder mehrfach.

ANTLR erlaubt im Gegensatz zu allgemeinen LL-Parsern direkte Links-Rekursion. (Indirekte Links-Rekursion funktioniert allerdings nicht.)

Der von ANTLR generierte Parser erzeugt auf der Eingabe einen Parse-Tree, der die Strukturen der Grammatik widerspiegelt: Die Token bilden die Blätter und jede erfolgreich durchlaufene Parser-Regel bildet einen entsprechenden Knoten im Baum.

Für die Traversierung des Parse-Tree kann man die generierten Listener- oder Visitor-Klassen nutzen. Beim Einsatz der Listener nutzt man die vorgegebene Klasse `ParseTreeWalker`, die mit dem Parse-Tree

und dem Listener den Baum per Tiefensuche traversiert und immer die jeweiligen `enterRegel-` und `exitRegel-` Methoden aufruft. Beim Visitor muss die Traversierung selbst erledigt werden, hier steht die aus der Klassenhierarchie geerbte Methode `visit` als Startpunkt zur Verfügung. In dieser Methode wird basierend auf dem Knotentyp die in den Visitor-Klassen implementierte `visitRegel-` Methode aufgerufen und man muss darauf achten, die Kindknoten durch passende Aufrufe zu traversieren. Sowohl bei den generierten Listener- als auch den Visitor-Klassen kann man die leeren Defaultmethoden bei Bedarf selbst überschreiben. Für den Zugriff auf die Regel-Elemente werden die sogenannten Kontextobjekte als Parameter übergeben.

Benannte Alternativen und Regel-Elemente sind nützlich, weil für die benannten Alternativen zusätzliche Kontextklassen erzeugt werden, über die dann auf die Bestandteile der Alternativen zugegriffen werden kann. Außerdem werden zusätzlich passende `enterAlternative-` und `exitAlternative-` bzw. `visitAlternative-` Methoden generiert. Für benannte Regel-Elemente wird ein entsprechend benanntes Attribut im Kontextobjekt angelegt, welches `public` sichtbar ist.

Videos

- [VL Parser mit ANTLR](#)
- [Demo ANTLR Parser](#)

4.2.1. Hello World

```

1  grammar Hello;
2
3  start : stmt* ;
4
5  stmt  : ID '=' expr ';' | expr ';' ;
6
7  expr  : term ('+' term)* ;
8  term  : atom ('*' atom)* ;
9
10 atom  : ID | NUM ;
11
12 ID    : [a-z][a-zA-Z]* ;
13 NUM   : [0-9]+ ;
14 WS    : [ \t\n]+ -> skip ;

```

[Konsole: Hello \(grun, Parse-Tree\)](#)

4.2.1.1. Starten des Parsers

1. Grammatik übersetzen und Code generieren: `antlr Hello.g4`
2. Java-Code kompilieren: `javac *.java`
3. Parser ausführen:
 - `grun Hello start -tree` oder `grun Hello start -gui` (Grammatik "Hello", Startregel "start")

- Alternativ mit kleinem Java-Programm:

```

1  import org.antlr.v4.runtime.CharStreams;
2  import org.antlr.v4.runtime.CommonTokenStream;
3  import org.antlr.v4.runtime.tree.ParseTree;
4
5  public class Main {
6      public static void main(String[] args) throws Exception {
7          CharStream input = CharStreams.fromString(IO.readln("expr?> "))
8          ;
9          HelloLexer lexer = new HelloLexer(input);
10         CommonTokenStream tokens = new CommonTokenStream(lexer);
11         HelloParser parser = new HelloParser(tokens);
12
13         ParseTree tree = parser.start(); // Start-Regel
14         System.out.println(tree.toStringTree(parser));
15     }
16 }

```

4.2.1.2. Startregeln

- `start` ist eine Parser-Regel => Eine Parser-Regel pro Grammatik wird benötigt, damit man den generierten Parser am Ende auch starten kann ...
- Alle Regeln mit kleinem Anfangsbuchstaben sind Parser-Regeln
- Alle Regeln mit großem Anfangsbuchstaben sind Lexer-Regeln

4.2.1.3. Formen der Subregeln

```
1 stmt : ID '=' expr ';' ;
```

Um die Regel `stmt` anwenden zu können, müssen alle Elemente auf der rechten Seite der Regel erfüllt werden. Dabei müssen die Token wie `ID`, `=` und `;` matchen und die Subregel `expr` muss erfüllt werden können. Beachten Sie das abschließende Semikolon am Ende einer ANTLR-Regel!

```
1 stmt : ID '=' expr ';' | expr ';' ;
```

Alternativen werden durch ein `|` getrennt. Hier muss genau eine Alternative erfüllt werden. Falls nötig, trennt man die Alternativen durch Einschließung in runden Klammern vom Rest der Regel ab: `r : a (b | c) d ;`.

```
1 expr : term ('+' term)* ;
```

Der durch den `*` gekennzeichnete Teil kann beliebig oft vorkommen oder auch fehlen. Bei einem `+` müsste der Teil mind. einmal vorkommen und bei einem `?` entsprechend einmal oder keinmal.

Auch hier kann man die Operatoren durch ein zusätzliches `?` auf non-greedy umschalten (analog zu den Lexer-Regeln).

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

4.2.1.4. Reihenfolge in Grammatik definiert Priorität

Falls mehr als eine Parser-Regel die selbe Input-Sequenz matcht, löst ANTLR diese Mehrdeutigkeit auf, indem es die erste Alternative nimmt, die an der Entscheidung beteiligt ist.

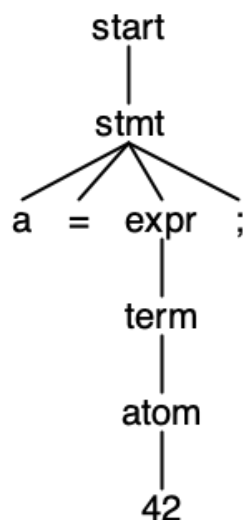
```
1 start : stmt ;  
2  
3 stmt  : expr | ID ;  
4 expr  : ID | NUM ;
```

Bei der Eingabe “foo” würde die Alternative **ID** in der Regel **expr** “gewinnen”, weil sie in der Grammatik vor der Alternative **ID** in der Regel **stmt** kommt und damit Vorrang hat.

4.2.1.5. Parse-Tree

Betrachten wir erneut die obige Grammatik.

Die Eingabe von “a = 42 ;” führt zu folgendem Parse-Tree:



Diese Eingabe führt zur Erkennung der Token [**ID**, **WS**, **=**, **WS**, **NUM**, **;**], wobei die **WS**-Token verworfen werden und der Parser den Tokenstream [**ID**, **=**, **NUM**, **;**] erhält.

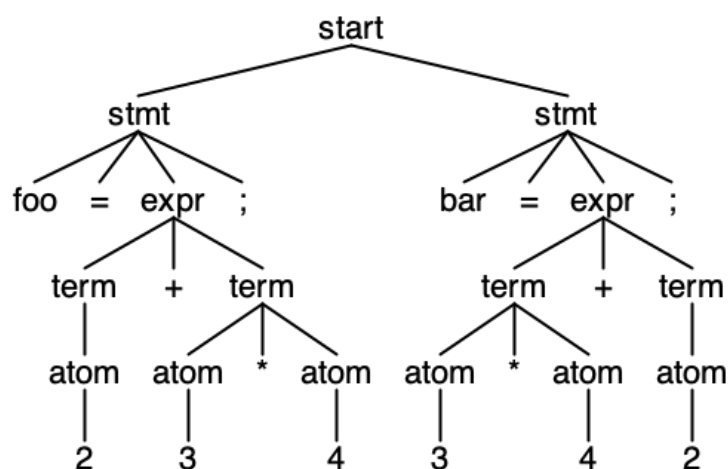
Die Startregel hat auf der rechten Seite kein oder mehrere **stmt**-Regeln. Die **stmt**-Regel fordert auf der rechten Seite entweder die Token **ID** und **=** sowie die Regel **expr** gefolgt vom Token **;**, oder die Regel **expr** gefolgt vom Token **;**. In unserem Beispiel kann für das “a” das Token **ID** produziert werden, das “=” matcht ebenfalls. Die “42” wird erklärt, indem für **expr** ein **term** und dort ein **atom** aufgerufen wird. Für das **atom** muss entweder ein Token **ID** oder **NUM** als nächstes Token kommen - hier wird die “42” als Token **NUM** verarbeitet. Da die

weiteren Regelteile in `term` und `expr` optional sind, haben wir damit ein `expr` erfüllt und das nachfolgende `;`-Token schließt die erste Alternative der Regel `stmt` erfolgreich ab.

Im entstehenden Parse-Tree sind diese Abläufe und grammatikalischen Strukturen direkt erkennbar. Jede erfolgreich durchlaufene Parserregel wird zu einem Knoten im Parse-Tree. Die Token werden als Terminale (Blätter) in den Baum eingehängt.

Anmerkung: Der Parse-Tree ist das Ergebnis der Parsers-Phase im Compiler und dient damit als Input für die folgenden Compilerstufen. In der Regel benötigt man die oft recht komplexen Strukturen aber später nicht mehr und vereinfacht den Baum zu einem *Abstract Syntax Tree* (AST). Im Beispiel könnte man den Zweig `stmt` - `expr` - `term` - `atom` - 42 zu `stmt` - 42 vereinfachen.

Betrachten wir nun die Eingabe `foo = 2+3*4; bar = 3*4+2;`. Diese führt zu folgendem Parse-Tree:



Wie man sehen kann, sind in der Grammatik die üblichen Vorrangregeln für die Operationen `+` und `*` berücksichtigt - die Multiplikation wird in beiden Fällen korrekt “unter” der Addition im Baum eingehängt.

4.2.1.6. To EOF not to EOF?

Startregeln müssen nicht unbedingt den gesamten Input “konsumieren”. Sie müssen per Default nur eine der Alternativen in der Startregel erfüllen.

Betrachten wir noch einmal einen leicht modifizierten Ausschnitt aus der obigen Grammatik:

```
1 start : stmt ;
```

Die Startregel wurde so geändert, dass sie nur noch genau ein Statement akzeptieren soll.

In diesem Fall würde die Startregel bei der Eingabe “aa; bb;” nur den ersten Teil “aa;” konsumieren (als Token `ID`) und das folgende “bb;” ignorieren. Das wäre in diesem Fall aber auch kein Fehler.

Wenn der gesamte Eingabestrom durch die Startregel erklärt werden soll, dann muss das vordefinierte Token **EOF** am Ende der Startregel eingesetzt werden:

```
1 start : stmt EOF;
```

Hier würde die Eingabe “aa; bb;” zu einem Fehler führen, da nur der Teil “aa;” durch die Startregel abgedeckt ist (Token **ID**), und der Rest “bb;” zwar sogar ein gültiges Token wären (ebenfalls **ID** und **;**), aber eben nicht mehr von der Startregel akzeptiert. Durch das **EOF** soll die Startregel aber den gesamten Input konsumieren und erklären, was hier nicht geht und entsprechend zum Fehler führt.

(vgl. github.com/antlr/antlr4/blob/master/doc/parser-rules.md)

4.2.2. Expressions und Vorrang (Operatoren)

Tip

Exkurs Expressions (Ausdrücke) vs. Statements (Anweisungen)

In Programmiersprachen unterscheiden wir häufig **Expressions** (*Ausdrücke*) und **Statements** (*Anweisungen*).

Expressions sind dabei syntaktische Konstrukte einer Programmiersprache, die (in einem gegebenen Kontext) zu einem Wert **evaluiert** werden können. Typische Expressions sind beispielsweise Ausdrücke wie 2×3 oder `foo(42)`; ... In manchen Sprachen sind beispielsweise auch Zuweisungen Expressions: `v = 42 + 7`; würde in C der Variablen `v` den Wert 49 zuweisen, dies ist gleichzeitig auch der Wert des gesamten Ausdrucks. Man könnte in C also Dinge formulieren wie `if (v = 42 + 7) ...` (wobei das Interpretieren eines Integers in einem bool'schen Kontext nochmal ein anderes Problem ist).

Statements sind syntaktische Konstrukte in Programmiersprachen, die **ausgeführt** werden können und dabei in der Regel einen Zustand im Programm verändern, also einen Seiteneffekt haben. Die Ausführung eines Statements hat normalerweise keinen Wert an sich. Typische Beispiele sind Zuweisungen `v = 7`, Kontrollfluss `if (...) then {...} else {...}`, Schleifen `for x in foo: ...`, `switch / case`-Statements. (Es gibt aber auch Programmiersprachen, wo ein `if/then/else`-Konstrukt eine Expression ist, also bei der Ausführung einen Wert ergibt.) In den meisten Programmiersprachen können Expressions Teile von Statements bilden: In `v = 42 + 7` ist die gesamte Zuweisung eine Anweisung (Seiteneffekt: die Variable `v` hat danach einen anderen Zustand), und der Teil `42 + 7` ist ein Ausdruck, der ausgewertet werden kann und üblicherweise den Wert 49 ergibt (außer man beauftragt ein LLM mit der Auswertung). In C-ähnlichen Sprachen kann durch Hinzufügen eines Semikolons aus dem Ausdruck `42 + 7` eine Anweisung gemacht werden...

Vergleiche auch Nystrom (2021), Kapitel 6 “Parsing Expressions”, Kapitel 7 “Evaluating Expressions” und Kapitel 8 “Statements and State”, aber auch [Wikipedia: Expression](#) und [Wikipedia: Statement](#).

Betrachten wir noch einmal den Ausschnitt für die Ausdrücke (*Expressions*) in der obigen Beispielgrammatik:

```
1 expr : term ('+' term)* ;
2 term : atom ('*' atom)* ;
```

```
3 atom : ID ;
```

Diese typische, etwas komplex anmutende Struktur soll sicher stellen, dass die Vorrangregeln für Addition und Multiplikation korrekt beachtet werden, d.h. dass $2+3*4$ als $2+(3*4)$ geparkt wird und nicht fälschlicherweise als $(2+3)*4$ erkannt wird.

Zusätzlich muss bei LL-Parsern Links-Rekursion vermieden werden: Die Parser-Regeln werden in Funktionsaufrufe übersetzt, d.h. bei einer Links-Rekursion würde man die selbe Regel immer wieder aufrufen, ohne ein Token aus dem Token-Strom zu entnehmen.

ANTLR (ab Version 4) kann mit beiden Aspekten automatisch umgehen:

- ANTLR kann direkte Linksrekursion automatisch auflösen. Die Regel $r : r T U \mid V ;$ kann also in ANTLR verarbeitet werden.
- ANTLR besitzt einen Mechanismus zur Auflösung von Mehrdeutigkeiten. Wie oben geschrieben, wird bei der Anwendbarkeit von mehreren Alternativen die erste Alternative genutzt.

Damit lässt sich die typische Struktur für Expression-Grammatiken deutlich lesbarer gestalten:

```
1 expr : expr '*' expr
2      | expr '+' expr
3      | ID
4      ;
```

Die Regel `expr` ist links-rekursiv, was normalerweise bei LL-Parsern problematisch ist. ANTLR löst diese Links-Rekursion automatisch auf (vgl. github.com/antlr/antlr4/blob/master/doc/left-recursion.md).

Da bei Mehrdeutigkeit in der Grammatik, also bei der Anwendbarkeit mehrerer Alternativen stets die erste Alternative genommen wird, lassen sich die Vorrangregeln durch die Reihenfolge der Alternativen in der `expr`-Regel implementieren: Die Multiplikation hat Vorrang von der Addition, und diese hat wiederum Vorrang von einer einfachen `ID`.

4.2.2.1. Direkte vs. indirekte Links-Rekursion

ANTLR kann nur *direkte* Links-Rekursion auflösen. Regeln wie $r : r T U \mid V ;$ stellen in ANTLR also kein Problem dar.

Indirekte Links-Rekursion erkennt ANTLR dagegen nicht:

```
1 r : s T U | V ;
2 s : r W X ;
```

Hier würden sich die Regeln `r` und `s` gegenseitig aufrufen und kein Token aus dem Tokenstrom entfernen, so dass der generierte LL-Parser hier in einer Endlosschleife stecken bleiben würde. Mit indirekter Links-Rekursion kann ANTLR nicht umgehen.

4.2.2.2. Konflikte in Regeln und implizite Token

Wenn mehrere Alternativen einer Regel anwendbar sind, entscheidet sich ANTLR für die erste Alternative.

Wenn sich mehrere Tokenregeln überlappen, “gewinnt” auch hier die zuerst definierte Regel.

```
1 def : 'func' ID '(' ')' block ;
2
3 FOR : 'for' ;
4 ID  : [a-z][a-zA-Z]* ;
```

Hier werden ein implizites Token `'func'` sowie die expliziten Token `FOR` und `ID` definiert. Dabei sind die Lexeme für `'func'` und `FOR` auch in `ID` enthalten. Dennoch werden `'func'` und `FOR` erkannt und nicht über `ID` gematcht, weil sie *vor* der Regel `ID` definiert sind.

Tatsächlich sortiert ANTLR die Regeln intern um, so dass alle Parser-Regeln *vor* den Lexer-Regeln definiert sind. Die impliziten Token werden dabei noch *vor* den expliziten Token-Regeln angeordnet. Im obigen Beispiel hat also `'func'` eine höhere Priorität als `FOR`, und `FOR` hat eine höhere Priorität als `ID`. Aus diesem Grund gibt es die Konvention, die Parser-Regeln in der Grammatik vor den Lexer-Regeln zu definieren - dies entspricht quasi der Anordnung, die ANTLR bei der Verarbeitung sowieso erzeugen würde.

Aus diesem Grund würde auch eine Umsortierung der obigen Grammatik funktionieren:

```
1 FOR : 'for' ;
2 ID  : [a-z][a-zA-Z]* ;
3
4 def : 'func' ID '(' ')' block ;
```

Intern würde ANTLR die Parser-Regel `def` wieder vor den beiden Lexer-Regeln anordnen, und zwischen den Parser-Regeln und den Lexer-Regeln die impliziten Token (hier `'func'`).

4.2.2.3. Lustige Probleme mit überlappenden Token (typische Stolperfalle)

Die folgende Grammatik sieht harmlos aus:

```
1 foo : (ID '(' ')' '=' '0' | ID | NUM) EOF ;
2
3 ID   : [a-z][a-zA-Z]* ;
4 NUM  : [0-9]+ ;
5 WS   : [ \t\n]+ -> skip ;
```

Was passiert bei der Eingabe von `10` und `01` und `a() = 0`, wie sieht der Parse-Tree aus? Was passiert bei der Eingabe von `0`?

Antwort: `10` wird als `foo -> NUM(10)` erkannt, `01` als `foo -> NUM(01)`, und `a() = 0` wird zu einem `foo -> ID(a), (, ), =, 0`. Dagegen ist die Eingabe `0` ein Fehler!

Das liegt hier an überlappenden Token-Definitionen: Die `0` wird als implizites Token definiert, während die Integerzahlen als explizites Token `NUM` definiert werden. Damit ist die `0` in `NUM` enthalten. Da ANTLR die impliziten Token intern vor den expliziten Token definiert (egal, in welcher Reihenfolge wir die Grammatik aufbauen), fällt die Eingabe `“0”` an das implizite Token `0` und nicht an `NUM`. Da es keine Regel gibt, wo eine einzelne `“0”` erlaubt ist, bekommen wir einen Fehler. Sobald die Eingabe länger wird, greift wieder die Regel des längsten Matches und `NUM` “gewinnt”.

4.2.2.4. Vorrang-Regeln bei links-rekursiven Parser-Regeln

Betrachten Sie die folgende typische ANTLR-Grammatik für Expressions:

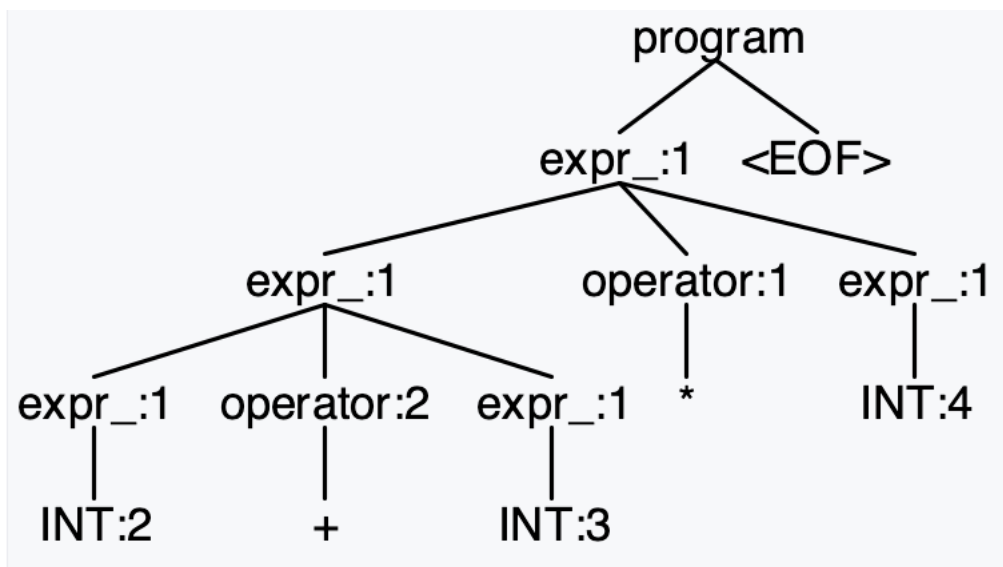
```

1 program : expr EOF ;
2
3 expr
4 : expr ('*' | '/') expr
5 | expr ('+' | '-') expr
6 | INT
7 ;

```

Wie oben dargestellt, ist die Regel `expr` *direkt links-rekursiv*, womit ANTLR4 aber umgehen kann.

In der gezeigten Schreibweise wird implizit ein Vorrang unter den Alternativen dieser direkt links-rekursiven Regel `expr` definiert: Beim internen Auflösen der Linksrekursion erhalten die Regeln eine Präzedenz in der Reihenfolge ihrer Auflistung (von oben nach unten). Die weiter oben aufgelisteten Alternativen der Regel `expr` haben eine höhere Präzedenz als die weiter unten aufgelisteten Regeln, d.h. die Multiplikation/Division hat Vorrang vor der Addition/Subtraktion. Entsprechend wird die Eingabe $2+3*4$ korrekt als $2+(3*4)$ geparkt und es entsteht der korrekte Parse-Tree



Wenn man nun beispielsweise versucht, die Operatoren auszuklammern und in eine eigene Regel zu verlagern und damit die ANTLR-Grammatik vermeintlich zu verbessern, klappt das nicht mehr. Betrachten Sie die folgende leicht geänderte Grammatik:

```

1 program : expr_ EOF ;
2
3 expr_
4 : expr_ operator expr_
5 | INT
6 ;
7 operator
8 : ('*' | '/')

```

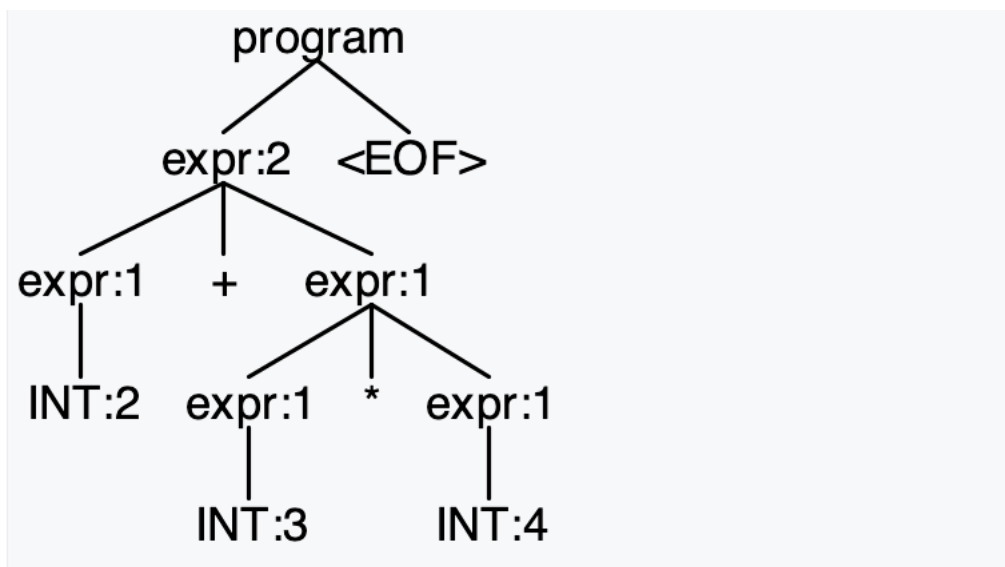
```

9 | ('+' | '-')
10 ;

```

Rein von der Grammatik her ist kein Unterschied in der erkannten Sprache zu sehen.

Der Vorrang-Mechanismus in ANTLR funktioniert aber nun nicht mehr, da die Alternativen nicht mehr in der linksrekursiven Regel `expr` definiert sind. Die Eingabe `2+3*4` wird jetzt inkorrekt als `"(2+3)*4"` geparkt:



Error

Präzedenz entsteht in ANTLR4 nur durch die Reihenfolge der linksrekursiven Alternativen in *derselben* Regel (ggf. mit `<assoc=right>` für Rechtsassoziativität).

Info

Bitte daran denken, dass die Reihenfolge der linksrekursiven Alternativen den Vorrang definiert.

Betrachten Sie die folgende Änderung der Grammatik von oben:

```

1  expr
2  : expr '*' expr
3  | expr '/' expr
4  | expr '+' expr
5  | expr '-' expr
6  | INT
7  ;

```

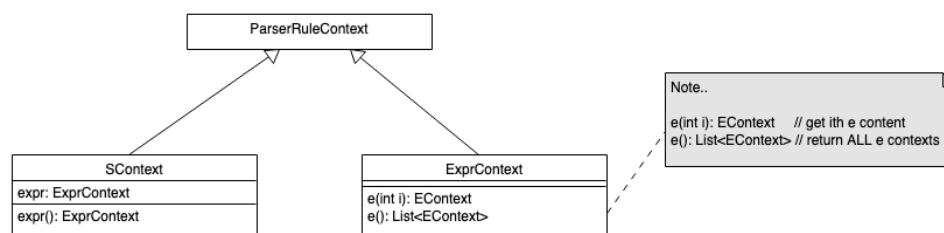
Das funktioniert, hier wird den Alternativen implizit ein Vorrang zugeordnet. Aber jetzt bekommt die Multiplikation Vorrang vor der Division, und auch die Addition bekommt Vorrang vor der Subtraktion. Ist das wirklich so gewollt?

4.2.3. Kontext-Objekte für Parser-Regeln

```

1 s      : expr          {List<EContext> x = $expr.ctx.e();}
2      ;
3 expr : e '*' e ;

```



Jede Regel liefert ein passend zu dieser Regel generiertes Kontext-Objekt zurück. Darüber kann man das/die Kontextobjekt(e) der Sub-Regeln abfragen.

Die Regel `s` liefert entsprechend ein `SContext`-Objekt und die Regel `expr` liefert ein `ExprContext`-Objekt zurück.

In der Aktion fragt man das Kontextobjekt über `ctx` ab, in den Listener- und Visitor-Methoden erhält man die Kontextobjekte als Parameter.

Für einfache Regel-Aufrufe liefert die parameterlose Methode nur ein einziges Kontextobjekt (statt einer Liste) zurück.

Anmerkung: ANTLR generiert nur dann *Felder* für die Regel-Elemente im Kontextobjekt, wenn diese in irgendeiner Form referenziert werden. Dies kann beispielsweise durch Benennung (Definition eines Labels, siehe nächste Folie) oder durch Nutzung in einer Aktion (siehe obiges Beispiel) geschehen.

4.2.4. Benannte Regel-Elemente oder Alternativen

```

1 stat : 'return' value=e ';' # Return
2      | 'break' ';'         # Break
3      ;

```

```

1 public static class StatContext extends ParserRuleContext { ... }
2 public static class ReturnContext extends StatContext {
3     public EContext value;
4     public EContext e() { ... }
5 }
6 public static class BreakContext extends StatContext { ... }

```

Mit `value=e` wird der Aufruf der Regel `e` mit dem Label `value` belegt, d.h. man kann mit `$e.text` oder `$value.text` auf das `text`-Attribut von `e` zugreifen. Falls es in einer Produktion mehrere Aufrufe einer anderen Regel gibt, **muss** man für den Zugriff auf die Attribute eindeutige Label vergeben.

Analog wird für die beiden Alternativen je ein eigener Kontext erzeugt.

4.2.5. Arbeiten mit ANTLR-Listeners

ANTLR (generiert auf Wunsch) zur Grammatik passende Listener (Interface und leere Basisimplementierung). Beim Traversieren mit dem Default-`ParseTreeWalker` wird der Parse-Tree mit Tiefensuche abgelaufen und jeweils beim Eintritt in bzw. beim Austritt aus einen/m Knoten der passende Listener mit dem passenden Kontext-Objekt aufgerufen.

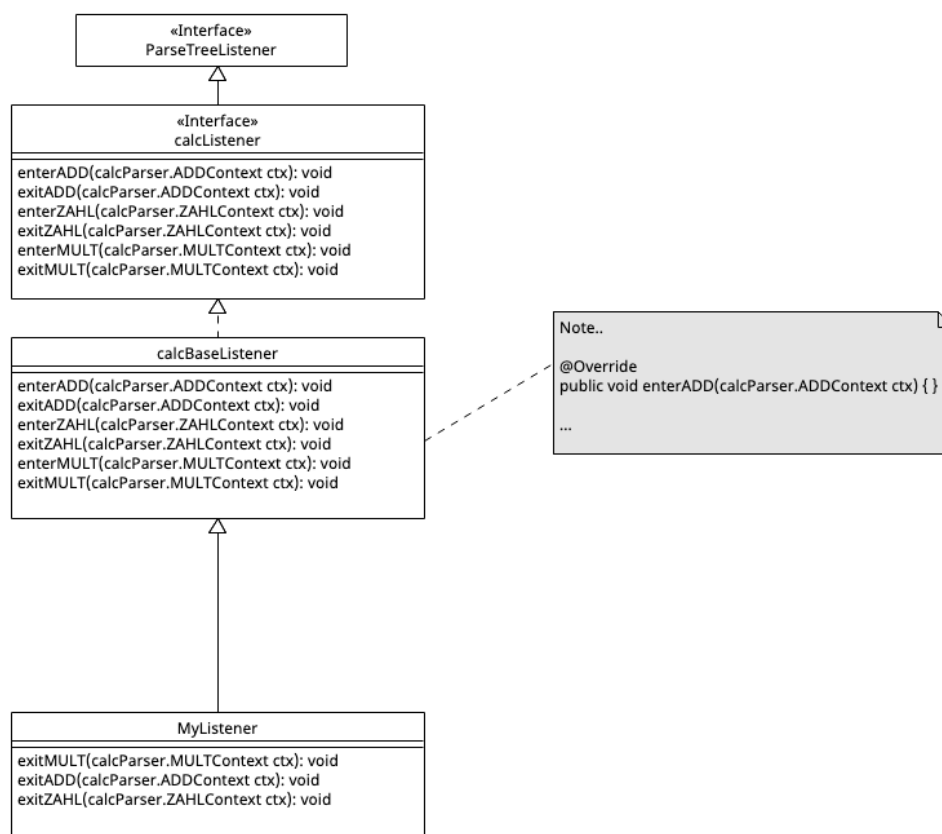
Damit kann man die Grammatik “für sich” halten, d.h. unabhängig von einer konkreten Zielsprache und die Aktionen über die Listener (oder Visitors, s.u.) ausführen.

```

1  expr : e1=expr '*' e2=expr      # MULT
2      | e1=expr '+' e2=expr      # ADD
3      | DIGIT                    # ZAHL
4      ;

```

ANTLR kann zu dieser Grammatik `calc.g4` einen passenden Listener (Interface `calcListener`) generieren (Option `-listener` beim Aufruf von `antlr`). Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseListener`):



(Nur “interessante” Methoden gezeigt.)

Von dieser Basisklasse leitet man einen eigenen Listener ab und implementiert die Methoden, die man benötigt.

```

1  public static class MyListener extends calcBaseListener {
2      public void exitMULT(calcParser.MULTContext ctx) {
3          ...
4      }
5      public void exitADD(calcParser.ADDContext ctx) {
6          ...
7      }
8      public void exitZAHL(calcParser.ZAHLContext ctx) {
9          ...
10     }
11 }

```

Anschließend baut man das alles in eine Traversierung des Parse-Trees ein:

```

1  public class TestMyListener {
2      public static class MyListener extends calcBaseListener {
3          ...
4      }
5
6      public static void main(String[] args) throws Exception {
7          CharStream input = CharStreams.fromString(IO.readln("expr?> "));
8          calcLexer lexer = new calcLexer(input);
9          CommonTokenStream tokens = new CommonTokenStream(lexer);
10         calcParser parser = new calcParser(tokens);
11
12         ParseTree tree = parser.s();    // Start-Regel
13
14         ParseTreeWalker walker = new ParseTreeWalker();
15         MyListener eval = new MyListener();
16         walker.walk(eval, tree);
17     }
18 }

```

Beispiel: TestMyListener.java und calc.g4

4.2.6. Arbeiten mit dem Visitor-Pattern

ANTLR (generiert ebenfalls auf Wunsch) zur Grammatik passende Visatoren (Interface und leere Basisimplementierung).

Hier muss man im Gegensatz zu den Listeners allerdings selbst für eine geeignete Traversierung des Parse-Trees sorgen. Dafür hat man mehr Freiheiten im Vergleich zum Einsatz von Listeners, insbesondere im Hinblick auf Rückgabewerte.

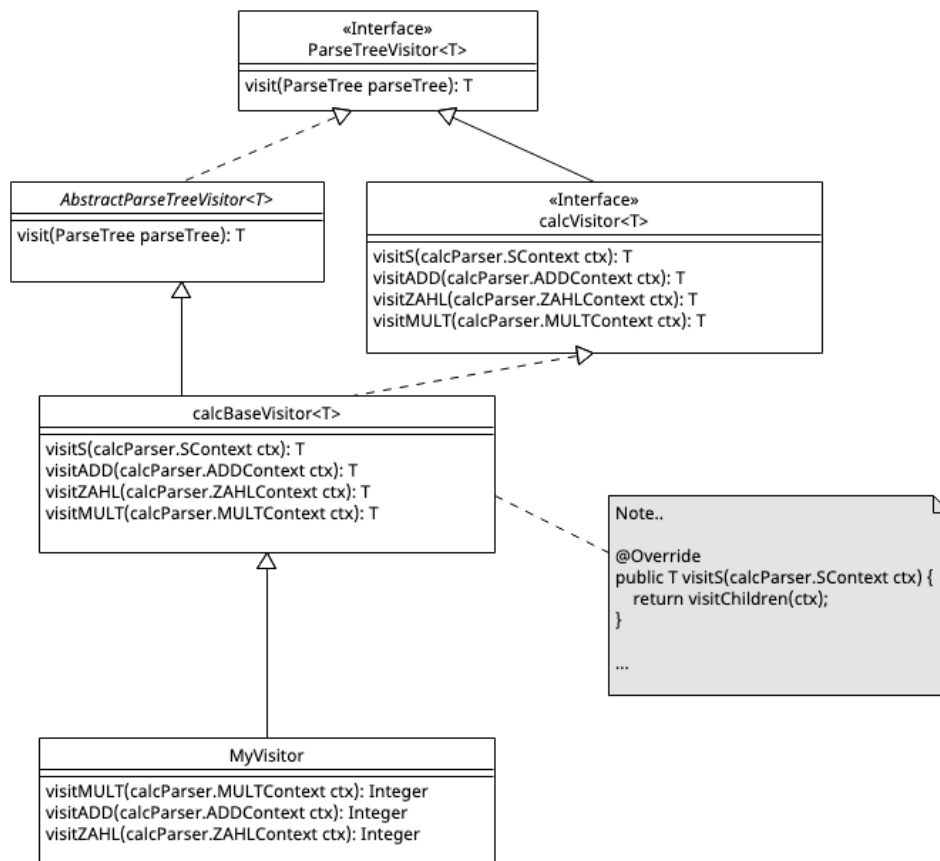
Erinnerung: Das Visitor-Pattern wurde in Prog2 besprochen: <https://github.com/Programmiermethoden-CampusMinden/Prog2-Lecture/blob/master/lecture/pattern/visitor.md>.

```

1  expr : e1=expr '*' e2=expr      # MULT
2      | e1=expr '+' e2=expr      # ADD
3      | DIGIT                    # ZAHL
4      ;

```

ANTLR kann zu dieser Grammatik einen passenden Visitor (Interface `calcVisitor<T>`) generieren (Option `-visitor` beim Aufruf von `antlr`). Weiterhin generiert ANTLR eine leere Basisimplementierung (Klasse `calcBaseVisitor<T>`):



(Nur “interessante” Methoden gezeigt.)

Von dieser Basisklasse leitet man einen eigenen Visitor ab und überschreibt die Methoden, die man benötigt. Wichtig ist, dass man selbst für das “Besuchen” der Kindknoten sorgen muss (rekursiver Aufruf der geerbten Methode `visit()`).

```

1 public static class MyVisitor extends calcBaseVisitor<Integer> {
2     public Integer visitMULT(calcParser.MULTContext ctx) {
3         return ...
4     }
5     public Integer visitADD(calcParser.ADDContext ctx) {
6         return ...
7     }
8     public Integer visitZahl(calcParser.ZahlContext ctx) {
9         return ...
10    }
11 }

```

Anschließend baut man das alles in eine manuelle Traversierung des Parse-Trees ein:

```

1 public class TestMyVisitor {
2     public static class MyVisitor extends calcBaseVisitor<Integer> {
3         ...
4     }
5
6     public static void main(String[] args) throws Exception {
7         CharStream input = CharStreams.fromString(IO.readln("expr?> "));
8         calcLexer lexer = new calcLexer(input);
9         CommonTokenStream tokens = new CommonTokenStream(lexer);
10        calcParser parser = new calcParser(tokens);
11
12        ParseTree tree = parser.s();    // Start-Regel
13
14        MyVisitor eval = new MyVisitor();
15        eval.visit(tree);
16    }
17 }

```

Beispiel: TestMyVisitor.java und calc.g4

4.2.7. Arbeiten mit Pattern Matching

ANTLR generiert zu jeder Regel `rule` eine Klasse `ruleContext`. Diese Klassen sind im generierten Parser definiert und werden für den Parse-Tree genutzt. Seit Java 25 ist *Pattern Matching* über Klassen und Records so weit ausgebaut, dass man es als Alternative zum Visitor-Pattern oder zu den ANTLR-Listenern nutzen kann.

```

1 expr : e1=expr '*' e2=expr      # MULT
2       | e1=expr '+' e2=expr     # ADD
3       | DIGIT                  # ZAHL
4       ;

```

```

1 public static class PatternMatching {
2     static Integer eval(calcParser.ExprContext e) {
3         return switch (e) {
4             case calcParser.MULTContext m -> eval(m.e1) * eval(m.e2);
5             case calcParser.ADDContext a -> eval(a.e1) + eval(a.e2);
6             case calcParser.ZAHLContext n -> Integer.parseInt(n.DIGIT().getText());
7             default -> throw new IllegalStateException();
8         };
9     }
10 }

```

Anschließend baut man das alles in eine manuelle Traversierung des Parse-Trees ein:

```

1 public class TestMyPM {
2     public static class PatternMatching {
3         ...
4     }
5
6     public static void main(String[] args) throws Exception {
7         CharStream input = CharStreams.fromString(IO.readln("expr?> "));

```

```

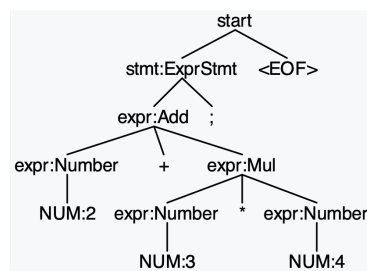
8      calcLexer lexer = new calcLexer(input);
9      CommonTokenStream tokens = new CommonTokenStream(lexer);
10     calcParser parser = new calcParser(tokens);
11
12     calcParser.SContext tree = parser.s(); // Start-Regel
13
14     PM.eval(tree.expr());
15 }
16 }

```

Beispiel: TestMyPM.java und calc.g4

4.2.8. Vom Parse-Tree zum AST

Der Parse-Tree spiegelt direkt die Strukturen der Grammatik wider, die beim Parsen gematcht haben. Normalerweise braucht man diesen Detailgrad später nicht mehr und baut den Parse-Tree zu einem abstrakteren AST (*Abstract Syntax Tree*) um.



Dieser Parse-Tree entstand mit einer einfachen, nachfolgend aufgeführten Expression-Grammatik und der Eingabe "2+3*4;".

Man erkennt gut die Grammatik-Regeln:

```

1  grammar MyLang;
2
3  start
4      : stmt* EOF
5      ;
6
7  stmt
8      : id=ID '=' value=expr ';'      # Assign
9      | expr ';'                      # ExprStmt
10     ;
11
12  expr
13      : lhs=expr '*' rhs=expr          # Mul
14      | lhs=expr '+' rhs=expr          # Add
15      | ID                             # Name
16      | NUM                             # Number
17      ;
18
19
20  ID : [a-zA-Z_] [a-zA-Z_0-9]* ;

```

```

21 NUM : [0-9]+ ;
22
23 WS  : [ \t\r\n]+ -> skip ;

```

Für das weitere Arbeiten ist aber nicht mehr relevant, ob da ein EOF-Token war oder nicht - der Parser würde eine Eingabe ohne dieses Token am Ende ja ablehnen. Auch ist die Stufe `expr: NUM`; nicht notwendig, und statt des Tokens möchte man eigentlich den Integerwert haben.

Es bietet sich also an, einige wenige Typen zu definieren, mit denen man diesen Baum darstellen kann.

```

1 sealed interface Stmt permits Stmt.Assign, Stmt.ExprStmt {
2     record Assign(String id, Expr value) implements Stmt {}
3     record ExprStmt(Expr expr) implements Stmt {}
4 }
5
6 sealed interface Expr permits Expr.Mul, Expr.Add, Expr.Name, Expr.Number {
7     record Mul(Expr lhs, Expr rhs) implements Expr {}
8     record Add(Expr lhs, Expr rhs) implements Expr {}
9     record Name(String id) implements Expr {}
10    record Number(int value) implements Expr {}
11 }

```

Statements sind syntaktische Strukturen, die ausgeführt werden und i.d.R. keinen Wert ergeben. In der obigen Grammatik gibt es zwei verschiedene Statements, die man auch später noch unterscheiden möchte: Zuweisungen und Ausdrücke (mit einem Semikolon abgeschlossen als eigenständiges Statement). Dies wird in der obigen Modellierung entsprechend berücksichtigt: Es gibt ein Interface für Statements und genau zwei Klassen, die dieses Interface implementieren. Bei einer Zuweisung werden später der Name der Variablen (linke Seite der Anweisung) und die Expression auf der rechten Seite der Anweisung benötigt, die restlichen Informationen aus dem Parse-Tree sind nach dem erfolgreichen Parsen nicht mehr interessant.

Expressions sind syntaktische Strukturen, die ausgewertet werden können und dabei einen Wert ergeben. Auch hier wird wieder ein gemeinsames Interface definiert und je Expression-Variante eine konkrete Datenklasse. Auch hier werden wieder nur die wirklich notwendigen Daten übernommen. Man könnte sogar noch überlegen, ob man die beiden `Mul` und `Add` zu einer gemeinsamen Klasse zusammenfassen möchte, dann müsste man aber noch die Operation als weiteres Attribut anlegen, und später müsste eine zusätzliche Fallunterscheidung anhand der Operation erfolgen, während man mit der obigen Modellierung per `switch/case` auf den Klassen direkt die gesuchte Information erhält.

```

1 static Stmt toAst(MyLangParser.StmtContext s) {
2     return switch (s) {
3         case MyLangParser.AssignContext a -> new Stmt.Assign(a.id.getText(),
4             toAst(a.value));
5         case MyLangParser.ExprStmtContext e -> new Stmt.ExprStmt(toAst(e.expr()))
6         ;
7         default -> throw new IllegalStateException();
8     };
9 }
10
11 static Expr toAst(MyLangParser.ExprContext e) {
12     return switch (e) {
13         case MyLangParser.MulContext m -> new Expr.Mul(toAst(m.lhs), toAst(m.rhs))
14         ;
15     };
16 }

```

```

12     case MyLangParser.AddContext a -> new Expr.Add(toAst(a.lhs), toAst(a.rhs)
13         );
14     case MyLangParser.NameContext n -> new Expr.Name(n.ID().getText());
15     case MyLangParser.NumberContext n -> new Expr.Number(Integer.parseInt(n.
16         NUM().getText()));
17     default -> throw new IllegalStateException();
18 };
19 }

```

Mit Hilfe der beiden oben gezeigten Methoden und dem folgenden Code kann der Parse-Tree traversiert werden. Dabei kommt das *Pattern Matching* auf Klassen zur Anwendung, welches in der Funktionalen Programmierung schon lange bekannt ist und nun endlich auch Einzug in die OOP-Welt hält.

In jedem einzelnen Knoten im Parse-Tree entscheidet man, ob und welchen neuen Knoten für den AST man erzeugen möchte und übernimmt die entsprechenden Informationen.

Hier noch der restliche “Starter-Code”:

```

1 public class AstBuilder {
2     static void main(String... args) {
3         CharStream input = CharStreams.fromString(IO.readln("expr?> "));
4         MyLangLexer lexer = new MyLangLexer(input);
5         CommonTokenStream tokens = new CommonTokenStream(lexer);
6         MyLangParser parser = new MyLangParser(tokens);
7
8         MyLangParser.StartContext tree = parser.start();
9
10        IO.println(toAst(tree));
11    }
12
13    static List<Stmt> toAst(MyLangParser.StartContext s) {
14        return s.stmt().stream()
15            .map(AstBuilder::toAst)
16            .collect(Collectors.toCollection(ArrayList::new));
17    }
18
19    ... // die beiden statischen Methoden von oben
20 }

```

4.2.9. Eingebettete Aktionen und Attribute

```

1 s : expr {System.err.println($expr.v);}
2 ;
3
4 expr returns [int v]
5 : e1=expr '*' e2=expr {$v = $e1.v * $e2.v;}
6 ;

```

Auch die Parser-Regeln können mit eingebetteten Aktionen ergänzt werden, die in die (für die jeweilige Regel) generierte Methode eingefügt werden und bei erfolgreicher Anwendung der Parser-Regel ausgeführt werden.

Über `returns [int v]` fügt man der Regel `expr` ein Attribut `v` (Integer) hinzu, welches man im jeweiligen Kontext abfragen bzw. setzen kann (agiert als Rückgabewert der generierten Methode). Auf diesen Wert kann in

den Aktionen mit `$v` zugegriffen werden.

Anmerkung: Durch den Einsatz von eingebetteten Aktionen und Attributen wird die Grammatik abhängig von der Zielsprache des generierten Lexers/Parsers!

4.2.10. Ausblick

Damit haben wir die sprichwörtliche “Spitze des Eisbergs” gesehen. Mit ANTLR sind noch viele weitere Dinge möglich. Bitte nutzen Sie aktiv die Dokumentation auf github.com/antlr/antlr4.

4.2.11. Wrap-Up

Parser mit ANTLR generieren: Parser-Regeln werden mit **Kleinbuchstaben** geschrieben

- Regeln können Lexer- und Parser-Regeln “aufrufen”
- Regeln können Alternativen haben
- Bei Mehrdeutigkeit: Vorrang für erste Alternative
- ANTLR erlaubt direkte Links-Rekursion
- ANTLR erzeugt Parse-Tree
- Benannte Alternativen und Regel-Elemente
- Traversierung des Parse-Tree: Listener oder Visatoren oder *Pattern Matching*, Zugriff auf Kontextobjekte

Zum Nachlesen

- Parr (2014)

Lernziele

- k2: Ich kann den Aufbau der Parser-Regeln an einem Beispiel erklären
- k3: Ich kann Alternativen und optionale/mehrfache Regelteile in Parser-Regeln umsetzen
- k3: Ich kann den Vorrang von Alternativen (bei Mehrdeutigkeiten) definieren
- k3: Ich kann Benannte Alternativen und Regel-Elemente zielgerichtet einsetzen
- k2: Ich kann den Aufbau des Parse-Tree von ANTLR erklären
- k3: Ich kann mit den Kontext-Objekten von ANTLR umgehen
- k3: Ich kann den Parse-Tree mit den generierten Listenern oder Visitors traversieren und bearbeiten

Challenges

Lexer und Parser mit ANTLR

Betrachten Sie den folgenden Programmschnipsel:

```
1 result99 = acc_2*ACC_2 + spillover7 + bonus_1*3 + inc_0;
2 calc_42 = __9 * zZ_1 + 5 + FooBar_42 * bar_7 + q0;
3 _ExprLine + A_1 * bB_2 + cc3 * 7 + 11;
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Listener oder Visitoren einen einfachen Pretty-Printer.

Lexer und Parser mit ANTLR: Programmiersprache Lox

Betrachten Sie folgenden Code-Schnipsel in der Sprache “Lox”:

```
1 fun fib(x) {
2     if (x == 0) {
3         return 0;
4     } else {
5         if (x == 1) {
6             return 1;
7         } else {
8             fib(x - 1) + fib(x - 2);
9         }
10    }
11 }
12
13 var wuppie = fib(4);
```

Erstellen Sie für diese fiktive Sprache einen Lexer+Parser mit ANTLR. Implementieren Sie mit Hilfe des Parse-Trees und der Listener oder Visitoren einen einfachen Pretty-Printer.

(Die genauere Sprachdefinition finden Sie bei Bedarf unter craftinginterpreters.com/the-lox-language.html.)

4.3. LL-Parser

Weitere Unterlagen

- [Annotierte Folien: LL-Parser](#)

4.3.1. Wiederholung

4.3.1.1. PDAs und kontextfreie Grammatiken

- Warum reichen uns DFAs nicht zum Matchen von Eingabezeichen?
- Wie können wir sie minimal erweitern?
- Sind PDAs deterministisch?
- Wie sind kontextfreie Grammatiken definiert?
- Sind kontextfreie Grammatiken eindeutig?

4.3.2. Motivation

4.3.2.1. Was brauchen wir für die Syntaxanalyse von Programmen?

- einen Grammatiktypen, aus dem sich manuell oder automatisiert ein Programm zur deterministischen Syntaxanalyse (=Parser) erstellen lässt
- einen Algorithmus zum Parsen von Programmen mit Hilfe einer solchen Grammatik

4.3.2.2. Themen für heute

- Automatische Generierung von Top-Down-Parsern aus LL-Grammatiken

4.3.3. Syntaxanalyse

4.3.3.1. Syntax

Wir verstehen unter Syntax eine Menge von Regeln, die die Struktur von Daten (z. B. Programmen) bestimmen.

Syntaxanalyse ist die Bestimmung, ob Eingabedaten einer vorgegebenen Syntax entsprechen.

Diese vorgegebene Syntax wird im Compilerbau mit einer kontextfreien Grammatik beschrieben und mit einem sogenannten **Parser** analysiert.

Wir beschäftigen uns heute mit LL-Parsing, mit dem man eine Teilmenge der eindeutigen kontextfreien Grammatiken syntaktisch analysieren kann.

Der Ableitungsbaum wird von oben nach unten aufgebaut.

4.3.3.2. Ziele der Syntaxanalyse

- aussagekräftige Fehlermeldungen, wenn ein Eingabeprogramm syntaktisch nicht korrekt ist
- evtl. Fehlerkorrektur
- Bestimmung der syntaktischen Struktur eines Programms
- Erstellung des AST (abstrakter Syntaxbaum): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. Semikolons, manche Schlüsselwörter)
- die Symboltabelle(n) mit Informationen bzgl. Bezeichner (Variable, Funktionen und Methoden, Klassen, benutzerdefinierte Typen, Parameter, ...), aber auch die Gültigkeitsbereiche.

4.3.4. LL(k)-Grammatiken

4.3.4.1. First-Mengen

$$S \rightarrow A \mid B \mid C$$

Welche Produktion nehmen?

Wir brauchen die “terminalen k-Anfänge” von Ableitungen von Nichtterminalen, um eindeutig die nächste zu benutzende Produktion festzulegen. k ist dabei die Anzahl der Vorschautoken.

Def.: Wir definieren *First* - Mengen einer Grammatik wie folgt:

- $a \in T^*, |a| \leq k : First_k(a) = \{a\}$
- $a \in T^*, |a| > k : First_k(a) = \{v \in T^* \mid a = vw, |v| = k\}$
- $\alpha \in (N \cup T)^* \setminus T^* : First_k(\alpha) = \{v \in T^* \mid \alpha \xRightarrow{*} w, \text{ mit } w \in T^*, First_k(w) = \{v\}\}$

4.3.4.2. Linksableitungen

Def.: Bei einer kontextfreien Grammatik G ist die *Linksableitung* von $\alpha \in (N \cup T)^*$ die Ableitung, die man erhält, wenn in jedem Schritt das am weitesten links stehende Nichtterminal in α abgeleitet wird.

Man schreibt $\alpha \xRightarrow{*}_l \beta$.

4.3.4.3. LL(k)-Grammatiken

Def.: Eine kontextfreie Grammatik $G = (N, T, P, S)$ ist genau dann eine $LL(k)$ -Grammatik, wenn für alle Linksableitungen der Form:

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\alpha\gamma \xRightarrow{*}_l wx$$

und

$$S \xRightarrow{*}_l wA\gamma \Rightarrow_l w\beta\gamma \xRightarrow{*}_l wy$$

mit $(w, x, y \in T^*, \alpha, \beta, \gamma \in (N \cup T)^*, A \in N)$ und $First_k(x) = First_k(y)$ gilt:

$$\alpha = \beta$$

4.3.4.4. LL(1)-Grammatiken

Hier entsteht ein Tafelbild.

4.3.4.5. LL(k)-Sprachen

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der deterministisch parsbaren Sprachen.

Die von $LL(k)$ -Grammatiken erzeugten Sprachen sind eine echte Teilmenge der von $LL(k+1)$ -Grammatiken erzeugten Sprachen.

Für eine kontextfreie Grammatik G ist nicht entscheidbar, ob es eine $LL(1)$ - Grammatik G' gibt mit $L(G) = L(G')$.

In der Praxis reichen $LL(1)$ - Grammatiken oft. Hier gibt es effiziente Parsergeneratoren (hier: ANTLR), deren Eingabe eine $LL(k)$ - (meist $LL(1)$ -) Grammatik ist, und die als Ausgabe den Quellcode eines (effizienten) tabellengesteuerten Parsers generieren.

4.3.4.6. Algorithmus: Konstruktion einer LL-Parsertabelle

Eingabe: Eine Grammatik $G = (N, T, P, S)$

Ausgabe: Eine Parsertabelle P

```

for each production  $X \rightarrow \alpha$ 
  for each  $a \in First(\alpha)$ 
    add  $X \rightarrow \alpha$  to  $P[X, a]$ 

  if  $\epsilon \in First(\alpha)$ 
    for each  $b \in Follow(\alpha)$ 
      add  $X \rightarrow \alpha$  to  $P[X, b]$ 
      if  $\epsilon \in First(\alpha)$  and  $\perp \in Follow(X)$ 
        add  $X \rightarrow \alpha$  to  $P[X, \perp]$ 

```

Abbildung 4.2.: Algorithmus zur Generierung einer LL-Parsertabelle

Hier ist $\perp$ das Endezeichen des Inputs. Statt $First_1(\alpha)$ wird oft nur $First(\alpha)$ geschrieben.

4.3.4.7. LL-Parsertabellen

Hier entsteht ein Tafelbild.

4.3.4.8. LL-Parsertabellen

Rekursive Programmierung bedeutet, dass das Laufzeitsystem einen Stack benutzt. Diesen Stack kann man auch "selbst programmieren", d. h. einen PDA implementieren. Dabei wird ebenfalls die oben genannte Tabelle zur Bestimmung der nächsten anzuwendenden Produktion benutzt. Der Stack enthält die zu erwartenden Eingabezeichen, wenn immer eine Linksableitung gebildet wird. Diese Zeichen im Stack werden mit dem Input gematcht.

4.3.4.9. Algorithmus: Tabellengesteuertes LL-Parsen mit einem PDA

Eingabe: Eine Grammatik $G = (N, T, P, S)$, eine Parsertabelle P mit " $w \perp$ " als initialem Kellerinhalt

Ausgabe: Wenn $w \in L(G)$, eine Linksableitung von w , Fehler sonst

```

a = next_token()
X = top of stack // entfernt X vom Stack

while X  $\neq$   $\perp$ 

    if X = a
        a = next_token()

    else if X  $\in$  T
        error

    else if P[X,a] leer
        error

    else if P[X,a] =  $X \rightarrow Y_1 Y_2 \dots Y_k$ 
        process_production( $X \rightarrow Y_1 Y_2 \dots Y_k$ )
        push( $Y_1 Y_2 \dots Y_k$ ) //  $Y_1$  = top of stack

X = top of stack

```

Abbildung 4.3.: Algorithmus zum tabellengesteuerten LL-Parsen

4.3.4.10. Ergebnisse der Syntaxanalyse

- eventuelle Syntaxfehler mit Angabe der Fehlerart und des -Ortes
- Fehlerkorrektur
- Format für die Weiterverarbeitung:
 - Ableitungsbaum oder Syntaxbaum oder Parse Tree
 - abstrakter Syntaxbaum (AST): Der Parse Tree ohne Symbole, die nach der Syntaxanalyse inhaltlich irrelevant sind (z. B. ;, Klammern, manche Schlüsselwörter, . . .)
- Symboltabelle

4.3.5. Wrap-Up

4.3.5.1. Wrap-Up

- Syntaxanalyse wird mit deterministisch kontextfreien Grammatiken durchgeführt.
- Eine Teilmenge der dazu gehörigen Sprachen lässt sich top-down parsen.
- Ein effizienter LL(k)-Parser realisiert einen DPDA und kann automatisch aus einer LL(k)-Grammatik generiert werden.
- Der Parser liefert in der Regel einen abstrakten Syntaxbaum.

Zum Nachlesen

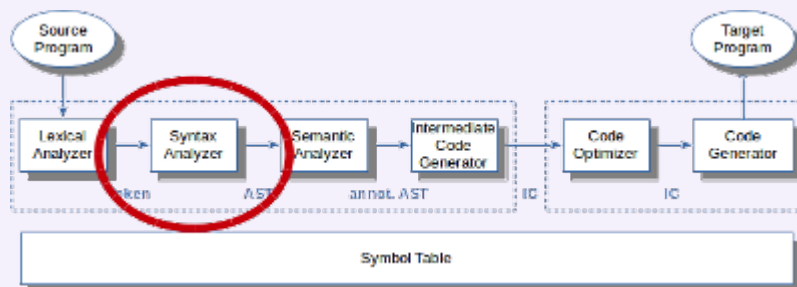
- Aho u. a. (2023)
- Hopcroft u. a. (2003)

Lernziele

- k1: Ich kenne die Top-Down-Analyse
- k1: Ich kenne LL-Parser
- k2: Ich kann den algorithmischen Ablauf von LL-Parsern an einem Beispiel erklären

4.4. LL-Parser selbst implementiert

TL;DR



LL-Parser können über einen “rekursiven Abstieg” direkt aus einer Grammatik implementiert werden:

- Zu jeder Produktionsregel erstellt man eine gleichnamige Funktion.
- Wenn in der Produktionsregel andere Regeln “aufgerufen” werden, erfolgt in der Funktion an dieser Stelle der entsprechende Funktionsaufruf.
- Bei Terminalsymbolen wird das erwartete Token geprüft.

Dabei findet man wie bereits im Lexer die Funktionen `match` und `consume`, die sich hier aber auf den Tokenstrom beziehen. LL(1)-Parser schauen dabei immer das nächste Token an, LL(k) haben ein entsprechendes Look-Ahead von k Token. Dies kann man mit einem Ringpuffer für die Token realisieren.

Zur Beachtung der Vorrang- und Assoziativitätsregeln muss die Grammatik entsprechend umgebaut werden. LL-Parser haben durch die Betrachtung des aktuellen Vorschau-Tokens ein Problem mit Linksrekursion in der Grammatik, diese muss zunächst beseitigt werden. (ANTLR bietet hier gewisse Vereinfachungen an, kann aber mit indirekter Linksrekursion auch nicht umgehen.)

Eine gute Darstellung, die sich am zeichen- bzw. tokenweisen Einlesen orientiert, finden Sie in (Parr 2010) (Kapitel 2). Eine weitere sehr gut lesbare Darstellung ist in [Nystrom2021] (Kapitel 4, 5 und 6) zu finden. Dort wird die Eingabe komplett eingelesen und daraus eine Liste aller Token erzeugt, auf der der Parser operiert. Der Beispiel-Code in dieser Vorlesung orientiert sich an der zeichenweisen bzw. tokenweisen Verarbeitung.

Videos

- VL LL-Parser selbst implementiert

4.4.1. Erinnerung Lexer: Zeichenstrom => Tokenstrom

```

1  def nextToken():
2      while (peek != EOF): # globale Variable, über consume()
3          switch (peek):
4              case ' ': case '\t': case '\n': WS(); continue
5              case '[': consume(); return Token(LBRACK, '[')
6              ...
7              default: raise Error("invalid character: "+peek)
8      return Token(EOF_Type, "<EOF>")
9
10 def match(c): # Lookahead: Ein Zeichen
11     consume()
12     if (peek == c): return True
13     else: rollBack(); return False
14
15 def consume():
16     peek = buffer[start]
17     start = (start+1) mod 2n
18     if (start mod n == 0):
19         fill(buffer[start:start+n-1])
20     end = (start+n) mod 2n

```

Erinnerung: Der Lexer arbeitet direkt auf dem Zeichenstrom und versucht über längste Matches daraus einen Tokenstrom zu erzeugen. Dabei wird immer das nächste Zeichen angeschaut (Funktion `match`) und mit `consume` das aktuelle Zeichen “verbraucht” und das nächste Zeichen geladen. Hier kann man über die Doppel-Puffer-Strategie das Einlesen einzelner Zeichen aus einer Datei vermeiden und immer blockweise in den Puffer einlesen.

4.4.2. Grundidee LL-Parser

Grammatik:

```
1 r : X s ;
```

LL-Implementierung:

```

1 def r():
2     match(X)
3     s()

```

- Für jede Regel in der Grammatik wird eine Methode/Funktion mit dem selben Namen definiert
- Referenzen auf ein Token `T` werden durch den Aufruf der Methode `match(T)` aufgelöst
 - `match(T)` “konsumiert” das aktuelle Token, falls dieses mit `T` übereinstimmt
 - Anderenfalls löst `match()` eine Exception aus

- Referenzen auf Regeln `s` werden durch Methodenaufrufe `s()` aufgelöst

Erinnerung: In ANTLR werden Parser-Regeln (non-Terminale) mit einem kleinen und Lexer-Regeln (Terminale/Token) mit einem großen Anfangsbuchstaben geschrieben.

4.4.3. Alternative Subregeln

```
1 a | b | c
```

kann zu einem **switch**-Konstrukt aufgelöst werden:

```
1 switch (lookahead):
2     case predicting_a:
3         a(); break;
4     case predicting_b:
5         b(); break;
6     ...
7     default: raise Error()
```

Dabei ist `lookahead` eine globale Variable, die das zu betrachtende Token enthält (vergleichbar mit `peek` beim Lexer).

Das Prädikat `predicting_a` soll andeuten, dass man mit dem aktuellen Token eine Vorhersage für die Regel `a` versucht (hier kommen die FIRST- und FOLLOW-Mengen ins Spiel ...). Wenn das der Fall ist, springt man entsprechend in die Funktion bzw. Methode `a()`.

4.4.4. Optionale Subregeln: Eins oder keins

```
1 (T)?
```

wird zu einem **if** ohne **else**-Teil:

```
1 if lookahead.predicting_T: match(T)
```

4.4.5. Optionale Subregeln: Mindestens eins

```
1 (T)+
```

wird zu einer **do-while**-Schleife (mind. ein Durchlauf):

```
1 while True:
2     match(T)
3     if not lookahead.predicting_T: break
```

4.4.6. LL(1)-Parser

Beispiel: Parsen von Listen, also Sequenzen wie `[1, 2, 3, 4]`:

```
1 list      : '[' elements ']' ;
2 elements : INT (',' INT)* ;
3
4 INT       : ('0'..'9')+ ;
```

Formal berechnet man die Lookahead-Mengen mit **FIRST** und **FOLLOW**, um eine Entscheidung für die nächste Regel zu treffen. Praktisch betrachtet kann man sich fragen, welche(s) Token eine Phrase in der aktuellen Alternative starten können.

Für LL(1)-Parser betrachtet man immer das **aktuelle** Token (**genau EIN Lookahead-Token**), um eine Entscheidung zu treffen.

```
1 def list():
2     match(LBRACK); elements(); match(RBRACK);
3
4 def elements():
5     match(INT)
6     while lookahead == COMMA: # globale Variable, über consume()
7         match(COMMA); match(INT)
```

4.4.7. Detail: *match()* und *consume()*

```
1 def match(x):
2     if lookahead == x: consume()
3     else: raise Exception()
4
5 def consume():
6     lookahead = lexer.nextToken()
```

Quelle: Eigener Code basierend auf einer Idee nach (Parr 2010, p. 43)

Dabei setzt man in der Klasse **Parser** zwei Attribute voraus:

```
1 class Parser:
2     Lexer lexer
3     Token lookahead
```

Starten würde man den Parser nach dem Erzeugen einer Instanz (dabei wird ein Lexer mit durchgereicht) über den Aufruf der Start-Regel, also beispielsweise `parser.list()`.

Anmerkung: Mit dem generierten Parse-Tree bzw. AST beschäftigen wir uns später ($\Rightarrow$ AST-basierte Interpreter).

4.4.8. Vorrangregeln

```
1 1+2*3 == 1+(2*3) != (1+2)*3
```

Die Eingabe $1+2*3$ muss als $1+(2*3)$ interpretiert werden, da $*$ Vorrang vor $+$ hat.

Dies formuliert man üblicherweise in der Grammatik:

```
1  expr : expr '+' term
2      | term
3      ;
4  term : term '*' INT
5      | INT
6      ;
```

ANTLR nutzt die Strategie des *“precedence climbing”* und löst nach der *Reihenfolge der Alternativen* in einer Regel auf. Entsprechend könnte man die obige Grammatik unter Beibehaltung der Vorrangregeln so in ANTLR (v4) formulieren:

```
1  expr : expr '*' expr
2      | expr '+' expr
3      | INT
4      ;
```

4.4.9. Linksrekursion

Normalerweise sind linksrekursive Grammatiken nicht mit einem LL-Parser behandelbar. Man muss die Linksrekursion manuell auflösen und die Grammatik umschreiben.

Beispiel:

```
1  expr : expr '*' expr | expr '+' expr | INT ;
```

Diese linksrekursive Grammatik könnte man (unter Beachtung der Vorrangregeln) etwa so umformulieren:

```
1  expr      : addExpr ;
2  addExpr   : multExpr ('+' multExpr)* ;
3  multExpr  : INT ('*' INT)* ;
```

ANTLR (v4) kann Grammatiken mit *direkter* Linksrekursion auflösen. Für frühere Versionen von ANTLR muss man die Rekursion manuell beseitigen.

Vergleiche *“ALL(*)”* bzw. *“Adaptive LL(*)”*.

Achtung: Mit *indirekter* Linksrekursion kann ANTLR (v4) *nicht* umgehen:

```
1  expr : expM | ... ;
2  expM : expr '*' expr ;
```

=> *Nicht* erlaubt!

4.4.10. Assoziativität

Die Eingabe 2^3^4 sollte als 2^3^4 geparkt werden. Analog sollte $a=b=c$ in C als $a=(b=c)$ verstanden werden.

Per Default werden Operatoren wie $+$ in ANTLR *links-assoziativ* behandelt, d.h. die Eingabe $1+2+3$ wird als $(1+2)+3$ gelesen. Für *rechts-assoziative* Operatoren muss man ANTLR dies in der Grammatik mitteilen:

```
1 expr : expr '^' <assoc=right> expr
2       | INT
3       ;
```

Anmerkung: Laut [Doku](#) gilt die Angabe `<assoc=right>` immer für die jeweilige Alternative und muss seit Version 4.2 an den Alternativen-Operator `|` geschrieben werden. In der Übergangsphase sei die Annotation an Tokenreferenzen noch zulässig, würde aber ignoriert?!

4.4.11. LL(k)-Parser

```
1 expr : ID '++' // x++
2       | ID '--' // x--
3       ;
```

Die obige Regel ist für einen LL(1)-Parser nicht deterministisch behandelbar, da die Alternativen mit dem selben Token beginnen (die Lookahead-Mengen überlappen sich). Entweder benötigt man zwei Lookahead-Tokens, also einen LL(2)-Parser, oder man muss die Regel in eine äquivalente LL(1)-Grammatik umschreiben:

```
1 expr : ID ('++' | '--') ; // x++ oder x--
```

4.4.12. LL(k)-Parser: Implementierung mit Ringpuffer

Für einen größeren Lookahead benötigt man einen Puffer für die Token. Für einen Lookahead von k Token (also einen LL(k)-Parser) würde man einen Puffer mit k Plätzen anlegen und diesen wie einen Ringpuffer benutzen. Dabei ist `start` der Index des aktuellen Lookahead-Tokens. Über die neue Funktion `lookahead(1)` ist dieses aktuelle Lookahead-Token abrufbar.

```
1 class Parser:
2     Lexer lexer
3     k = 3           # Lookahead: 3 Token => LL(3)
4     start = 0       # aktuelle Tokenposition im Ringpuffer
5     Token[k] lookahead # Ringpuffer mit k Plätzen (vorbefüllt via Konstruktor)
```

```
1 def match(x):
2     if lookahead(1) == x: consume()
3     else: raise Exception()
4
5 def consume():
```

```

6     lookahead[start] = lexer.nextToken()
7     start = (start+1) % k
8
9     def lookahead(i):
10        return lookahead[(start+i-1) % k] # i==1: start

```

Quelle: Eigener Code basierend auf einer Idee nach (Parr 2010, p. 47)

4.4.13. Wrap-Up

- LL(1) und LL(k) mit festem Lookahead
- Implementierung von Vorrang- und Assoziativitätsregeln
- Beachtung und Auflösung von Linksrekursion

Zum Nachlesen

- Nystrom (2021): Kapitel 5 und 6
- Parr (2010): Kapitel 2 ("Pattern 3: LL(1) Recursive-Decent Parser")

Lernziele

- k2: Ich kann den prinzipiellen Aufbau von LL-Parsern am Beispiel erklären
- k3: Ich kann LL(1)- und LL(k)-Parser implementieren
- k3: Ich kann Vorrang und Assoziativität bei der Implementierung korrekt umsetzen
- k3: Ich kann mit Linksrekursion umgehen und diese ggf. auflösen

Challenges

Quizzfragen:

- Wie kann man aus einer LL(1)-Grammatik einen LL(1)-Parser mit rekursivem Abstieg implementieren? Wie "übersetzt" man dabei Token und Regeln?
- Wie geht man mit Alternativen um? Wie mit optionalen Subregeln?
- Warum ist Linksrekursion i.A. bei LL-Parsern nicht erlaubt? Wie kann man Linksrekursion beseitigen?
- Wie kann man Vorrangregeln implementieren?
- Wann braucht man mehr als ein Token Lookahead? Geben Sie ein Beispiel an.

Manuell implementierter Parser

Betrachten Sie erneut die folgende einfache Sprache:

```

1  a = 10 - 5      # Zuweisung des Ausdruckes 10-5 (Integer-Wert 5) an Variable
    a
2  b = a + 2 * 3    # Zuweisung von 16 an Variable b
3  c = a != b       # Zuweisung eines booleschen Werts an c

```

Es gibt nur Statements und Expressions:

- Statement: Zuweisung; jedes Statement endet mit einem NL
- Expression: Zahl, Variable, Addition, Subtraktion, Multiplikation (mit üblichem Vorrang), Vergleich

Aufgaben:

In den Challenges von LL Lexer haben Sie eine Grammatik definiert und einen Lexer implementiert.

- Geben Sie nun geeignete Datenstrukturen für den AST an.
- Implementieren Sie analog zum Vorgehen in der Vorlesung einen Parser mit *recursive descent* für diese Sprache.
- Was müssten Sie anpassen bzw. ergänzen, wenn Sie beispielsweise weitere Statements wie eine **if**-Abfrage oder eine **while**-Schleife mit einbauen wollten?

5. Semantische Analyse

Auf die lexikalische Analyse und die Syntaxanalyse folgt die semantische Analyse. Nach dem Parsen steht fest, dass ein Programm syntaktisch korrekt ist. Nun muss geprüft werden, ob es auch semantisch korrekt ist. Dazu gehören u.a. die Identifikation und Sammlung von Bezeichnern und die Zuordnung zur richtigen Ebene (Scopes) sowie die Typ-Prüfung und -Inferenz.

In dieser Phase zeigen sich die Eigenschaften der zu verarbeitenden Sprache sehr deutlich, beispielsweise müssen Bezeichner deklariert sein vor der ersten Benutzung, welche Art von Scopes soll es geben, gibt es Klassen und Vererbung ...

Da hier der Kontext der Symbole eine Rolle spielt, wird diese Phase oft auch "Context Handling" oder "Kontext Analyse" bezeichnet. Neben attributierten Grammatiken sind die Symboltabellen wichtige Werkzeuge.

5.1. SymbTab0: Überblick Symboltabellen

TL;DR

Auf die lexikalische Analyse und die Syntaxanalyse folgt die semantische Analyse. Nach dem Parsen steht fest, dass ein Programm syntaktisch korrekt ist. Nun muss geprüft werden, ob es auch semantisch korrekt ist. Dies umfasst in der Regel die Identifikation und Sammlung von Bezeichnern und die Zuordnung zur richtigen Ebene (Scopes). Außerdem muss die Nutzung von Symbolen validiert werden: Je nach Sprache müssen beispielsweise Variablen und Funktionen vor ihrer Benutzung zumindest deklariert sein; Funktionen sollten sich nicht wie Variablen benutzen lassen, ...

Als Werkzeug werden (hierarchische) Tabellen eingesetzt, um die verschiedenen Symbole und Informationen darüber zu verwalten. Dabei werden die Symboltabelleneinträge oft an verschiedenen Stellen im Compiler generiert und benutzt.

Videos

- [VL Symboltabellen \(Intro\)](#)

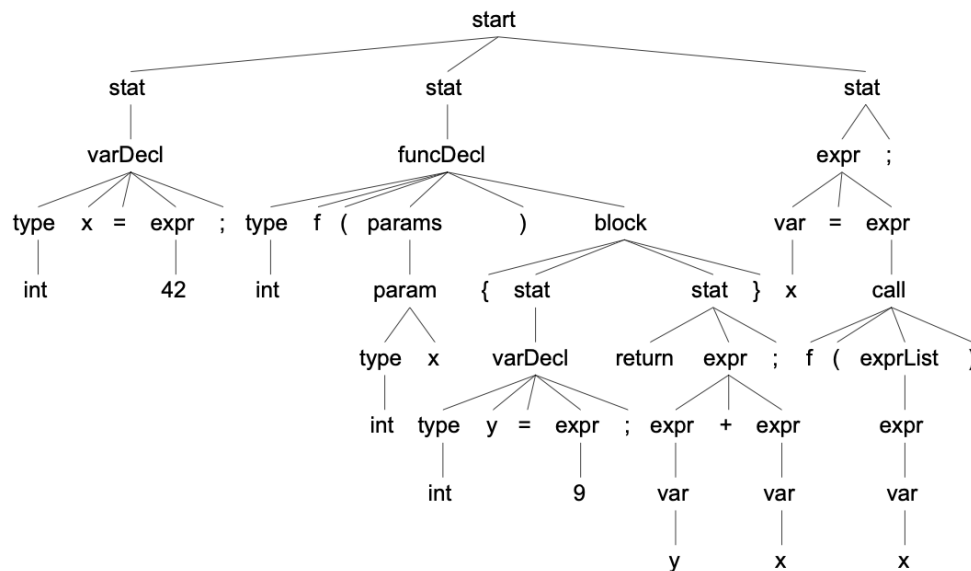
5.1.1. Was passiert nach der Syntaxanalyse?

```
1 int x = 42;  
2 int f(int x) {
```

```

3   int y = 9;
4   return y+x;
5 }
6
7 x = f(x);

```



Nach der Syntaxanalyse braucht der Compiler für die darauf folgenden Phasen **semantische Analyse**, Optimierung und Codegenerierung **Informationen über Bezeichner**, z.B.

- Welcher Bezeichner ist gemeint?
- Welchen Typ hat ein Bezeichner?

Auf dem Weg zum Interpreter/Compiler müssen die Symbole im AST korrekt zugeordnet werden. Dies geschieht über Symboltabellen. Im Folgenden werden wir verschiedene Aspekte von Symboltabellen betrachten und eine mögliche Implementierung erarbeiten, bevor wir uns um die Auswertung (Interpretation) des AST kümmern können.

5.1.1.1. Logische Compilerphasen

- Die lexikalische Analyse generiert eine Folge von Token.
- Die Syntaxanalyse generiert einen Parse Tree.
- Die semantische Analyse macht folgendes:
 - Der Parse Tree wird in einen abstrakten Syntaxbaum (AST) umgewandelt.
 - Dieser wird häufig mit Attributen annotiert.
 - Dabei sind oft mehrere Baumdurchläufe nötig (z.B. wegen der Abhängigkeiten der Attribute).
- Nachfolgende Stufen:

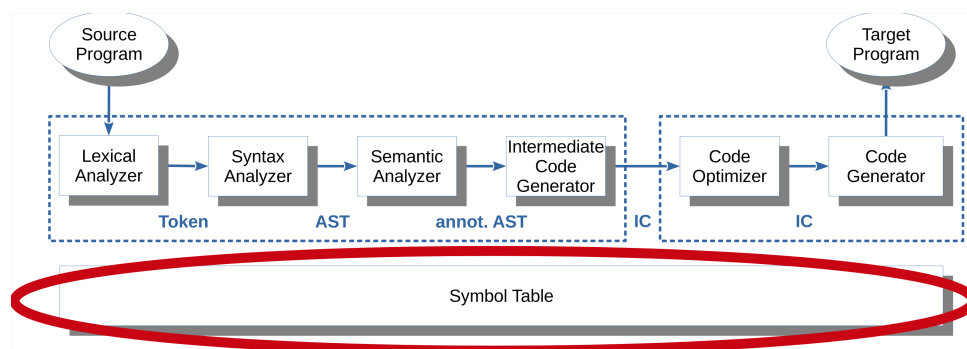
- Der AST wird in einen Zwischencode umgewandelt mit Registern und virtuellen Adressen.
- Der Zwischencode wird optimiert.
- Aus dem optimierten Zwischencode wird der endgültige Code, aber immer noch mit virtuellen Adressen, generiert.
- Der generierte Code wird nachoptimiert.
- Der Linker ersetzt die virtuellen Adressen durch reale Adressen.

5.1.1.2. Abgrenzung der Phasen

Diese Phasen sind oft nicht klar unterscheidbar. Schon allein zur Verbesserung der Laufzeit baut der Parser oft schon den abstrakten Syntaxbaum auf, der Lexer trägt schon Bezeichner in Symboltabellen ein, der Parser berechnet beim Baumaufbau schon Attribute, ...

Oft werden gar nicht alle Phasen und alle Zwischendarstellungen benötigt.

5.1.2. Semantische Analyse und Symboltabellen



5.1.3. Syntax und Semantik

- **Syntaxregeln:** Formaler Aufbau eines Programms
- **Semantik:** Bedeutung eines (syntaktisch korrekten) Programms

=> Keine Codegenerierung für syntaktisch/semantisch inkorrekte Programme!

Zur Erinnerung: Die *Syntaxregeln* einer Programmiersprache bestimmen den formalen Aufbau eines zu übersetzenden Programms. Die *Semantik* gibt die Bedeutung eines syntaktisch richtigen Programms an.

Lexikalische und syntaktische Analyse können formalisiert mit regulären Ausdrücken und endlichen Automaten, sowie mit CFG und Parsern durchgeführt werden.

Die Durchführung der semantischen Analyse ist stark von den Eigenschaften der zu übersetzenden Sprache, sowie der Zielsprache abhängig und kann hier nur beispielhaft für einige Eigenschaften erklärt werden.

Es darf kein lauffähiges Programm erstellt werden können, dass nicht syntaktisch und semantisch korrekt ist. Ein lauffähiges Programm muss syntaktisch und semantisch korrekt sein!

5.1.4. Aufgaben der semantischen Analyse

- Identifikation und Sammlung der Bezeichner
- Zuordnung zur richtigen Ebene (Scopes)
- Typ-Inferenz
- Typkonsistenz (Ausdrücke, Funktionsaufrufe, ...)
- Validieren der Nutzung von Symbolen
 - Vermeidung von Mehrfachdefinition
 - Zugriff auf nicht definierte Bezeichner
 - (Lesender) Zugriff auf nicht initialisierte Bezeichner
 - Funktionen werden nicht als Variablen genutzt
 - ...

Die semantische Analyse überprüft die Gültigkeit eines syntaktisch korrekten Programms bzgl. statischer semantischer Eigenschaften und liefert die Grundlage für die (Zwischen-) Codeerzeugung und -optimierung. Insbesondere wird hier die Typkonsistenz (in Ausdrücken, von Parametern, ...) überprüft, und implizite Typumwandlungen werden vorgenommen. Oft müssen Typen automatisch bestimmt werden (z.B. bei Polymorphie, Typinferenz). Damit Typen bestimmt oder angepasst werden können, müssen Bezeichner zunächst identifiziert werden, d.h. bei namensgleichen Bezeichnern der richtige Bezug bestimmt werden.

=> Ein wichtiges Hilfsmittel dazu sind **Symboltabellen**

5.1.4.1. Identifizierung von Objekten

Beim Compilervorgang müssen Namen immer wieder den dazugehörigen Definitionen zugeordnet, ihre Eigenschaften gesammelt und geprüft und darauf zugegriffen werden. Symboltabellen werden im Compiler fast überall gebraucht (siehe Abbildung unter "Einordnung").

Welche Informationen zu einem Bezeichner gespeichert und ermittelt werden, ist dann abhängig von der Klasse des Bezeichners.

5.1.4.2. Validieren der Nutzung von Symbolen

Hier sind unendlich viele Möglichkeiten denkbar. Dies reicht von den unten aufgeführten Basisprüfungen bis hin zum Prüfen der Typkompatibilität bei arithmetischen Operationen. Dabei müssen für alle Ausdrücke die Ergebnistypen berechnet werden und ggf. automatische Konvertierungen vorgenommen werden, etwa bei $3+4 \cdot 1 \dots$

- Zugriff auf Variablen: Müssen sichtbar sein
- Zugriff auf Funktionen: Vorwärtsreferenzen sind OK
- Variablen werden nicht als Funktionen genutzt
- Funktionen werden nicht als Variablen genutzt

Da Funktionen bereits vor dem Bekanntmachen der Definition aufgerufen werden dürfen, bietet sich ein **zweimaliger Durchlauf** (*pass*) an: Beim ersten Traversieren des AST werden alle Definitionen in der Symboltabelle gesammelt. Beim zweiten Durchlauf werden dann die Referenzen aufgelöst.

5.1.4.3. Das Mittel der Wahl: Tabellen für die Symbole (= Bezeichner)

Def.: *Symboltabellen* sind die zentrale Datenstruktur zur Identifizierung und Verwaltung von bezeichneten Elementen.

Die Organisation der Symboltabellen ist stark anwendungsabhängig. Je nach Sprachkonzept gibt es eine oder mehrere Symboltabellen, deren Einträge vom Lexer oder Parser angelegt werden. Die jeweiligen Inhalte jedes einzelnen Eintrags kommen aus den verschiedenen Phasen der Compilierung. Symboltabellen werden oft als Hashtables oder auch als Bäume implementiert, manchmal als verkettete Listen. In seltenen Fällen kommt man auch mit einem Stack aus.

Eine Symboltabelle enthält benutzerdefinierte Bezeichner (oder Verweise in eine Hashtable mit allen vorkommenden Namen), manchmal auch die Schlüsselwörter der Programmiersprache. Die einzelnen Felder eines Eintrags variieren stark, abhängig vom Typ des Bezeichners (= Bezeichnerklasse).

Manchmal gibt es für Datentypen eine Extra-Tabelle, ebenso eine für die Werte von Konstanten.

Manchmal werden die Namen selbst in eine (Hash-) Tabelle geschrieben. Die Symboltabelle enthält dann statt der Namen Verweise in diese (Hash-) Tabelle.

5.1.5. Einfache Verwaltung von Variablen primitiven Typs

```
1  int x = 0;  
2  int i = 0;  
3  
4  for (i=0; i<10; i++) {  
5      x++;  
6  }
```

	Name	Type
0	int	
1	for	
2	x	int
3	i	int

Bsp.: Die zu übersetzende Sprache hat nur einen (den globalen) Scope und kennt nur Bezeichner für Variablen.

- **Eine** Symboltabelle für **alle** Bezeichner
- Jeder Bezeichner ist der Name einer Variablen
- Symboltabelle wird evtl. mit Einträgen aller Schlüsselwörter initialisiert – warum?
- Scanner erkennt Bezeichner und sucht ihn in der Symboltabelle
- Ist der Bezeichner nicht vorhanden, wird ein (bis auf den Namen leerer) Eintrag angelegt
- Scanner übergibt dem Parser das erkannte Token und einen Verweis auf den Symboltabelleneintrag

Die Symboltabelle könnte hier eine (Hash-) Tabelle oder eine einfache verkettete Liste sein.

5.1.6. Was kann jetzt weiter passieren?

```
1  int x = 0;
2  int i = 0;
3
4  for (i=0; i<10; i++) {
5      x++;
6  }
7
8  a = 42;
```

In vielen Sprachen muss überprüft werden, ob es ein definierendes Vorkommen des Bezeichners oder ein angewandtes Vorkommen ist.

5.1.6.1. Definitionen und Deklarationen von Bezeichnern

Def.: Die *Definition* eines (bisher nicht existenten) Bezeichners in einem Programm generiert einen neuen Bezeichner und legt für ihn seinem Typ entsprechend Speicherplatz an.

Def.: Unter der *Deklaration* eines (bereits existierenden) Bezeichners verstehen wir seine Bekanntmachung, damit er benutzt werden kann. Er ist oft in einem anderen Scope definiert und bekommt dort Speicherplatz zugeteilt.

Insbesondere werden auch Typen deklariert. Hier gibt es in der Regel gar keine Speicherplatzzuweisung.

Ein Bezeichner kann beliebig oft deklariert werden, während er in einem Programm nur einmal definiert werden kann. Oft wird bei der Deklaration eines Elements sein Namensraum mit angegeben.

Vorsicht: Die Begriffe werden auch anders verwendet. Z.B. findet sich in der Java-Literatur der Begriff *Deklaration* anstelle von *Definition*.

Anmerkung: Deklarationen beziehen sich auf Definitionen, die woanders in einer Symboltabelle stehen, evtl. in einer anderen Datei, also in diesem Compilerlauf nicht zugänglich sind und erst von Linker aufgelöst werden können. Beim Auftreten einer Deklaration muss die dazugehörige Definition gesucht werden, und wenn vorhanden, im Symboltabelleneintrag für den deklarierten Bezeichner festgehalten werden. Hier ist evtl. ein zweiter Baumdurchlauf nötig, um alle offenen Deklarationen, die sich auf Definitionen in derselben Datei beziehen, aufzulösen.

Wird bei objektorientierten Sprachen ein Objekt definiert, dessen Klassendefinition in einer anderen Datei liegt, kann man die Definition des Objekts gleichzeitig als Deklaration der Klasse auffassen (Java).

5.1.7. Wo werden Verweise in Symboltabellen gebraucht?

=> Parse Tree und AST enthalten Verweise auf Symboltabelleneinträge

- Im Parse Tree enthält der Knoten für einen Bezeichner einen Verweis auf den Symboltabelleneintrag.
- Parser und semantische Analyse (AST) vervollständigen die Einträge.
- Attribute des AST können Feldern der Symboltabelle entsprechen, bzw. sich aus ihnen berechnen.
- Für Debugging-Zwecke können die Symboltabellen die ganze Compilierung und das Linken überleben.

5.1.8. Grenzen der semantischen Analyse

Welche semantischen Eigenschaften einer Sprache kann die semantische Analyse nicht überprüfen?

- Wer ist dann dafür verantwortlich?
- Wie äußert sich das im Fehlerfall?

Dinge, die erst durch eine Ausführung/Interpretation eines Programms berechnet werden können.

Beispielsweise können Werte von Ausdrücken oft erst zur Laufzeit bestimmt werden. Insbesondere kann die semantische Analyse in der Regel nicht feststellen, ob ein Null-Pointer übergeben wird und anschließend dereferenziert wird.

5.1.9. Wrap-Up

- Semantische Analyse:
 - Identifikation und Sammlung der Bezeichner
 - Zuordnung zur richtigen Ebene (Scopes)
 - Validieren der Nutzung von Symbolen
 - Typ-Inferenz
 - Typkonsistenz (Ausdrücke, Funktionsaufrufe, ...)
- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Symboltabelleneinträge werden an verschiedenen Stellen des Compilers generiert und benutzt

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k2: Ich kann die Aufgaben der semantischen Analyse an einem Beispiel erklären
- k2: Ich kann die Bedeutung von Symboltabellen erklären und die Aufgaben und Verbindung zu den Compiler-Phasen verdeutlichen

5.2. SymbTab1: Nested Scopes

TL;DR

In Symboltabellen werden Informationen über Bezeichner verwaltet. Wenn es in der zu übersetzenden Sprache *Nested Scopes* gibt, spiegelt sich dies in den Symboltabellen wider: Auch hier wird eine entsprechende hierarchische Organisation notwendig. In der Regel nutzt man Tabellen, die untereinander verlinkt sind.

Eine wichtige Aufgabe ist das Binden von Bezeichner gleichen Namens an ihren jeweiligen Scope => `bind()`. Zusätzlich müssen Symboltabellen auch das Abrufen von Bezeichnern aus dem aktuellen Scope oder den Elternscopes unterstützen => `resolve()`.

Videos

- [VL Nested Scopes](#)

5.2.1. Scopes und Name Spaces

Def.: Unter dem *Gültigkeitsbereich* (Sichtbarkeitsbereich, Scope) eines Bezeichners versteht man den Programabschnitt, in dem der Bezeichner sichtbar und nutzbar ist. Das ist oft der kleinste umgebende Block, außer darin enthaltene Scopes, die ein eigenes Element dieses Namens benutzen.

Scopes sind fast immer hierarchisch angeordnet.

Def.: Unter einem *Namensraum* (name space) versteht man die Menge der zu einem Zeitpunkt sichtbaren Bezeichner.

Es gibt Sprachen, in denen man eigene Namensräume explizit definieren kann (z.B. C++).

Vorsicht: Diese Begriffe werden nicht immer gleich definiert und auch gerne verwechselt.

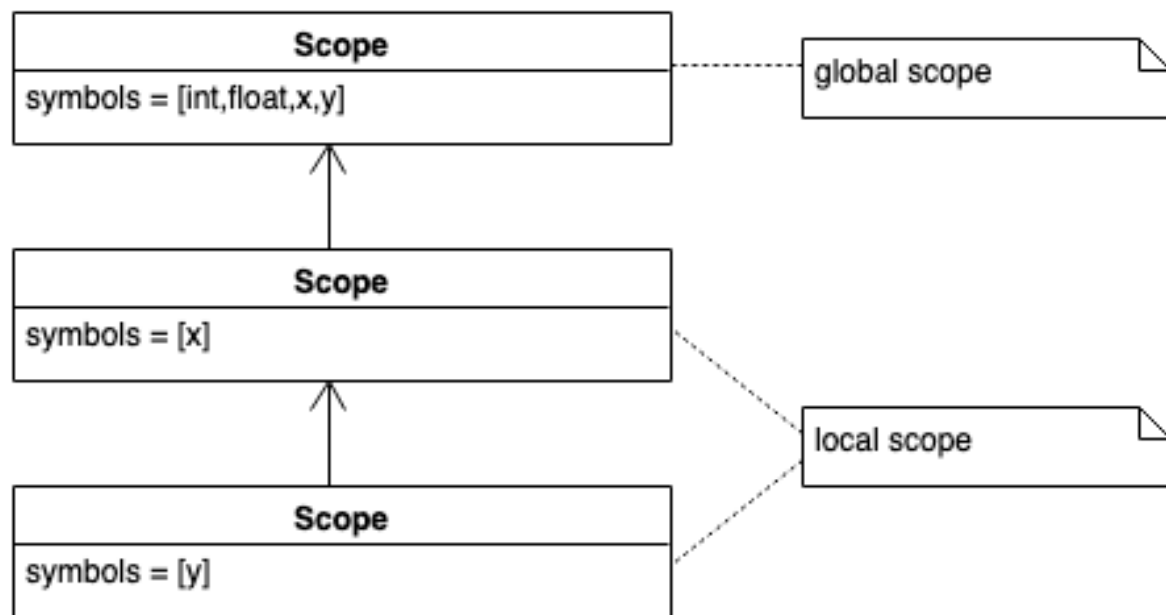
5.2.2. Symbole und (nested) Scopes

```
1 int x = 42;  
2 float y;  
3 {  
4     int x;
```

```

5   x = 1;
6   y = 2;
7   { int y = x; }
8   }

```



Aufgaben:

- `bind()`: Symbole im Scope definieren
- `resolve()`: Symbole aus Scope oder Eltern-Scope abrufen

5.2.2.1. Hinzunahme von Scopes

Bsp.: Die zu übersetzende Sprache ist scope-basiert und kennt nur Bezeichner für Variablen

Scopes können ineinander verschachtelt sein. Die Spezifikation der zu übersetzenden Sprache legt fest, in welcher Reihenfolge Scopes zu durchsuchen sind, wenn auf einen Bezeichner Bezug genommen wird, der nicht im aktuellen Scope definiert ist.

Insgesamt bilden die Scopes oft eine Baumstruktur, wobei jeder Knoten einen Scope repräsentiert und seine Kinder die direkt in ihm enthaltenen Scopes sind. Dabei ist es in der Regel so, dass Scopes sich entweder vollständig überlappen oder gar nicht. Wenn ein Bezeichner nicht im aktuellen Scope vorhanden ist, muss er in der Regel in umschließenden Scopes gesucht werden. Hier kann ein Stack aller "offenen" Scopes benutzt werden.

5.2.2.2. Grundlegendes Vorgehen

Das Element, das einen neuen Scope definiert, steht selbst in dem aktuell behandelten Scope. Wenn dieses Element selbst ein Bezeichner ist, gehört dieser in den aktuellen Scope. Nur das, was nur innerhalb des oben genannten Elements oder Bezeichners definiert wird, gehört in den Scope des Elements oder Bezeichners.

5.2.3. Nested Scopes: Symbole und Scopes

5.2.3.1. Implementierung mit hierarchischen (verketteten) Tabellen

Pro Scope wird eine Symboltabelle angelegt, dabei enthält jede Symboltabelle zusätzlich einen Verweis auf ihre Vorgängersymboltabelle für den umgebenden Scope. Die globale Symboltabelle wird typischerweise mit allen Schlüsselwörtern initialisiert.

- Wenn ein neuer Scope betreten wird, wird eine neue Symboltabelle erzeugt.
- Scanner: Erkennt Bezeichner und sucht ihn in der Symboltabelle des aktuellen Scopes bzw. trägt ihn dort ein und übergibt dem Parser das erkannte Token und einen Verweis auf den Symboltabelleneintrag (**Erinnerung:** Der Scanner wird i.d.R. vom Parser aus aufgerufen, d.h. der Parser setzt den aktuellen Scope!)
- Parser:
 - Wird ein neues Element (ein Bezeichner) definiert, muss bestimmt werden, ob es einen eigenen Scope hat. Wenn ja, wird eine neue Symboltabelle für den Scope angelegt. Sie enthält alle Definitionen von Elementen, die in diesem Scope liegen. Der Bezeichner selbst wird in die aktuelle Symboltabelle eingetragen mit einem Verweis auf die neue Tabelle, die all die Bezeichner beinhaltet, die außerhalb dieses Scopes nicht sichtbar sein sollen. Die Tabellen werden untereinander verzeigert.
 - Wird ein Element deklariert oder benutzt, muss sein Eintrag in allen sichtbaren Scopes in der richtigen Reihenfolge entlang der Verzeigerung gesucht (und je nach Sprachdefinition auch gefunden) werden.
- Der Parse-Tree enthält im Knoten für den Bezeichner den Verweis in die Symboltabelle

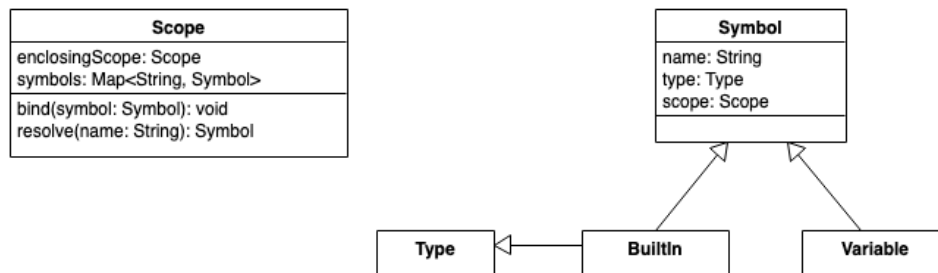
5.2.3.2. Klassenhierarchie für Scopes

Für die Scopes wird eine Klasse `Scope` definiert mit den Methoden `bind()` (zum Definieren von Symbolen im Scope) und `resolve()` (zum Abrufen von Symbolen aus dem Scope oder dem umgebenden Scope).

Für lokale Scopes wird eine Instanz dieser Klasse angelegt, die eine Referenz auf den einschließenden Scope im Attribut `enclosingScope` hält. Für den globalen Scope ist diese Referenz einfach leer (`None`).

5.2.3.3. Klassen und Interfaces für Symbole

Für die Symbole gibt es die Klasse `Symbol`, wo für jedes Symbol Name und Typ gespeichert wird. Variablen-symbole leiten direkt von dieser Klasse ab. Für die eingebauten Typen wird ein "Marker-Interface" `Type` erstellt, um Variablen- und Typ-Symbole unterscheiden zu können.



Quelle: Nested Scopes: Eigene Modellierung nach einer Idee in (Parr 2010, p. 142)

5.2.3.4. Alternative Implementierung über einen Stack

- Der Parse Tree bzw. der AST enthalten an den Knoten, die jeweils einen ganzen Scope repräsentieren, einen Verweis auf die Symboltabelle dieses Scopes.
- Die Scopes werden in einem Stack verwaltet.
- Wird ein Scope betreten beim Baumdurchlauf, wird ein Verweis auf seine Symboltabelle auf den Stack gepackt.
- Die Suche von Bezeichnern in umliegenden Scopes erfordert ein Durchsuchen des Stacks von oben nach unten.
- Beim Verlassen eines Scopes beim Baumdurchlauf wird der Scope vom Stack entfernt.

5.2.4. Nested Scopes: Definieren und Auflösen von Namen

```

1 class Scope:
2     Scope enclosingScope # None if global (outermost) scope
3     Symbol<String, Symbol> symbols
4
5     def resolve(name):
6         # do we know "name" here?
7         if symbols[name]: return symbols[name]
8         # if not here, check any enclosing scope
9         if enclosingScope: return enclosingScope.resolve(name)
10        else: return None # not found
11
12    def bind(symbol):
13        symbols[symbol.name] = symbol
14        symbol.scope = self # track the scope in each symbol
  
```

Quelle: Nested Scopes: Eigene Implementierung nach einer Idee in (Parr 2010, p. 169)

Anmerkung: In der Klasse `Symbol` kann man ein Feld `scope` vom Typ `Scope` implementieren. Damit “weiss” jedes Symbol, in welchem Scope es definiert ist und man muss sich auf der Suche nach dem Scope eines Symbols ggf. nicht erst durch die Baumstruktur hangeln. Aus technischer Sicht verhindert das Attribut das Aufräumen eines lokalen Scopes durch den Garbage Collector, wenn man den lokalen Scope wieder verlässt: Jeder Scope hat eine Referenz auf den umgebenden (Eltern-) Scope (Feld `enclosingScope`). Wenn man den aktuellen Scope “nach oben” verlässt, würde der eben verlassene lokale Scope bei nächster Gelegenheit aufgeräumt,

wenn es keine weiteren Referenzen auf diesen gäbe. Da nun aber die Symbole, die in diesem Scope definiert wurden, auf diesen verweisen, passiert das nicht :)

5.2.5. Nested Scopes: Listener

Mit einem passenden Listener kann man damit die nötigen Scopes aufbauen:

- `enterStart`:
 - erzeuge neuen globalen Scope
 - definiere und pushe die eingebauten Typen
- `exitVarDecl`:
 - löse den Typ der Variablen im aktuellen Scope auf
 - definiere ein neues Variablensymbol im aktuellen Scope
- `exitVar`:
 - löse die Variable im aktuellen Scope auf
- `enterBlock`:
 - erzeuge neuen lokalen Scope, wobei der aktuelle Scope der Elternscope ist
 - ersetze den aktuellen Scope durch den lokalen Scope
- `exitBlock`:
 - ersetze den aktuellen Scope durch dessen Elternscope

```

1  start   :  stat+ ;
2
3  stat    :  block | varDecl | expr ';' ;
4  block   :  '{' stat* '}' ;
5
6  varDecl :  type ID ('=' expr)? ';' ;
7  expr    :  var '=' INT ;
8
9  var     :  ID ;
10 type    :  'float' | 'int' ;

```

Relevanter Ausschnitt aus der Grammatik

```

1  int x = 42;
2
3  { int y = 9; x = 7; }

```

```

1  class MyListener(BaseListener):
2      Scope scope
3
4      def enterStart(Parser.FileContext ctx):
5          globals = Scope()
6          globals.bind(BuiltIn("int"))

```

```

7      globals.bind(BuiltIn("float"))
8      scope = globals
9
10     def enterBlock(Parser.BlockContext ctx):
11         scope = Scope(scope)
12     def exitBlock(Parser.BlockContext ctx):
13         scope = scope.enclosingScope
14
15     def exitVarDecl(Parser.VarDeclContext ctx):
16         t = scope.resolve(ctx.type().getText())
17         var = Variable(ctx.ID().getText(), t)
18         scope.bind(var)
19     def exitVar(Parser.VarContext ctx):
20         name = ctx.ID().getText()
21         var = scope.resolve(name)
22         if var == None: error("no such var: " + name)

```

Anmerkung: Um den Code auf die Folie zu bekommen, ist dies ein Mix aus Java und Python geworden. Sry ;)

In der Methode `exitVar()` wird das Variablensymbol beim Ablaufen des AST lediglich aufgelöst und ein Fehler geworfen, wenn das Variablensymbol (noch) nicht bekannt ist. Hier könnte man weiteres Type-Checking und/oder -Propagation ansetzen.

Später im Interpreter muss an dieser Stelle dann aber auch der **Wert** der Variablen abgerufen werden ...

5.2.6. Löschen von Symboltabellen

Möglicherweise sind die Symboltabellen nach der Identifizierungsphase der Elemente überflüssig, weil die zusammengetragenen Informationen als Attribute im AST stehen. Die Knoten enthalten dann Verweise auf definierende Knoten von Elementen, nicht mehr auf Einträge in den Symboltabellen. In diesem Fall können die Symboltabellen nach der Identifizierung gelöscht werden, wenn sie nicht z.B. für einen symbolischen Debugger noch gebraucht werden.

5.2.7. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Blöcke: Nested Scopes => hierarchische Organisation
- Binden von Bezeichner gleichen Namens an ihren jeweiligen Scope => `bind()`
- Abrufen von Bezeichnern aus dem aktuellen Scope oder den Elternscopes => `resolve()`

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Symbole über die Scopes auflösen
- k3: Ich kann einfache statische Prüfungen anhand der Symboltabellen durchführen

5.3. SymbTab2: Funktionen

TL;DR

Eine Funktion sind selbst ein Symbol, welches in einem Scope gilt und entsprechend in der Symboltabelle eingetragen wird. Darüber hinaus bildet sie einen neuen verschachtelten Scope, in dem die Funktionsparameter und der Funktionskörper definiert werden müssen.

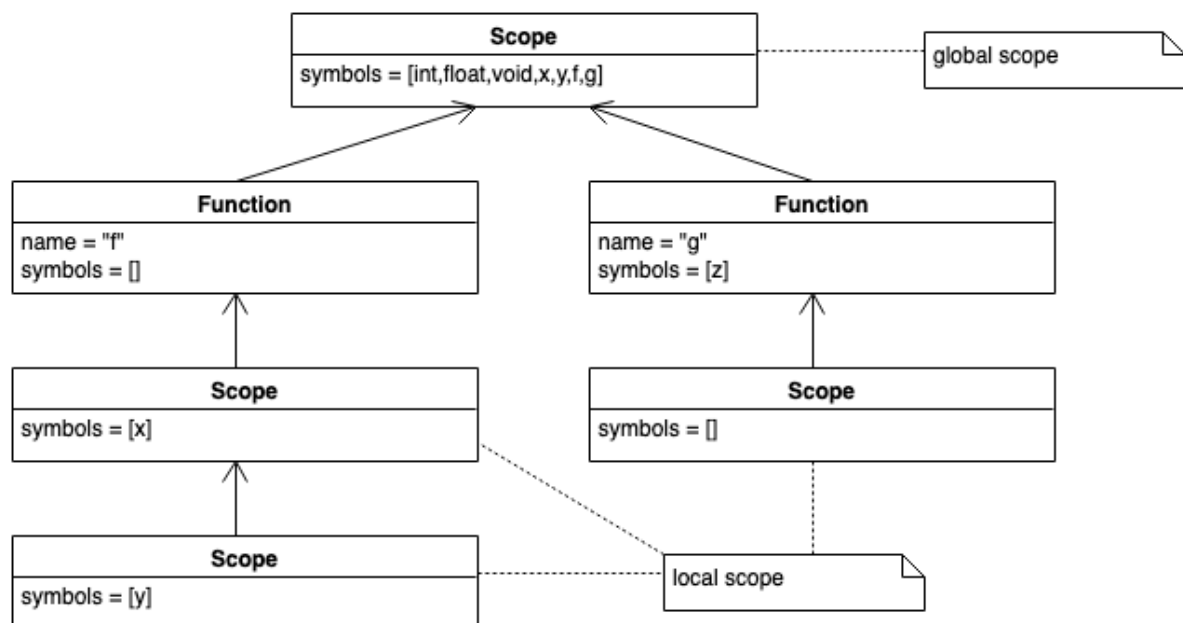
Entsprechend müssen die Strukturen für die Symboltabellen sowie das Eintragen und das Auflösen von Symbolen erweitert werden.

Videos

- [VL Funktionen](#)

5.3.1. Funktionen und Scopes

```
1  int x = 42;
2  int y;
3  void f() {
4      int x;
5      x = 1;
6      y = 2;
7      { int y = x; }
8  }
9  void g(int z){}
```



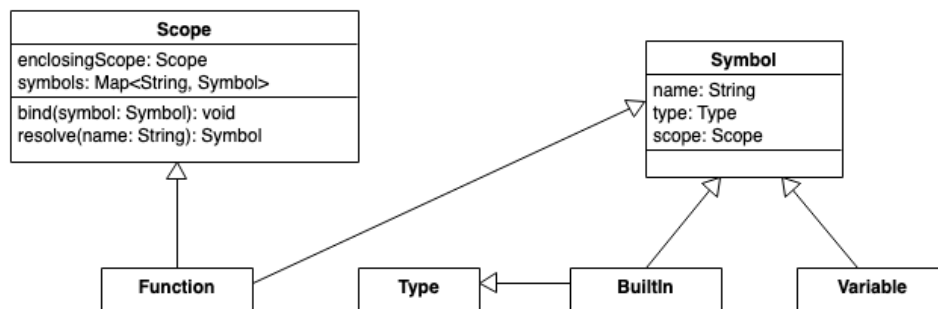
5.3.1.1. Behandlung von Funktionsdefinitionen

- Jeder Symboltabelleneintrag braucht ein Feld, das angibt, ob es sich um eine Variable, eine Funktion, ... handelt. Alternativ eine eigene Klasse ableiten ...
- Der Name der Funktion steht als Bezeichner in der Symboltabelle des Scopes, in dem die Funktion definiert wird.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Verweise auf die Parameter.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Angaben über den Rückgabebetypen.
- Jede Funktion wird grundsätzlich wie ein neuer Scope behandelt.
- Die formalen Parameter werden als Einträge in der Symboltabelle für den Scope der Funktion angelegt und entsprechend als Parameter gekennzeichnet.

5.3.1.2. Behandlung von Funktionsaufrufen

- Der Name der Funktion steht als Bezeichner in der Symboltabelle des Scopes, in dem die Funktion aufgerufen wird und wird als Aufruf gekennzeichnet.
- Der Symboltabelleneintrag für den Funktionsnamen enthält Verweise auf die aktuellen Parameter.
- Die Definition der Funktion wird in den zugänglichen Scopes gesucht (wie oben) und ein Verweis darauf in der Symboltabelle gespeichert.

5.3.2. Erweiterung des Klassendiagramms für Funktions-Scopes



Quelle: Funktions-Scopes: Eigene Modellierung nach einer Idee in (Parr 2010, p. 147)

5.3.3. Funktionen sind Symbole *und* Scopes

```

1 class Function(Scope, Symbol):
2     def __init__(name, retType, enclScope):
3         Symbol.__init__(name, retType) # we are "Symbol" ...
4         enclosingScope = enclScope    # ... and "Scope"
  
```

5.3.4. Funktionen: Listener

Den Listener zum Aufbau der Scopes könnte man entsprechend erweitern:

- `enterFuncDecl`:
 - löse den Typ der Funktion im aktuellen Scope auf
 - lege neues Funktionssymbol an, wobei der aktuelle Scope der Elternscope ist
 - definiere das Funktionssymbol im aktuellen Scope
 - ersetze den aktuellen Scope durch das Funktionssymbol
- `exitFuncDecl`:
 - ersetze den aktuellen Scope durch dessen Elternscope
- `exitParam`: analog zu `exitVarDecl`
 - löse den Typ der Variablen im aktuellen Scope auf
 - definiere ein neues Variablensymbol im aktuellen Scope
- `exitCall`: analog zu `exitVar`
 - löse das Funktionssymbol (und die Argumente) im aktuellen Scope auf

```

1 funcDecl : type ID '(' params? ')' block ;
2 params  : param (',' param)* ;
3 param   : type ID ;
4
5 call    : ID '(' exprList? ')' ;
6 exprList : expr (',' expr)* ;

```

Relevanter Ausschnitt aus der Grammatik

```

1 int f(int x) {
2     int y = 9;
3 }
4
5 int x = f(x);

```

```

1 def enterFuncDecl(Parser.FuncDeclContext ctx):
2     name = ctx.ID().getText()
3     type = scope.resolve(ctx.type().getText())
4     func = Function(name, type, scope)
5     scope.bind(func)
6     # change current scope to function scope
7     scope = func
8
9 def exitFuncDecl(Parser.FuncDeclContext ctx):
10    scope = scope.enclosingScope
11
12 def exitParam(Parser.ParamContext ctx):
13    t = scope.resolve(ctx.type().getText())
14    var = Variable(ctx.ID().getText(), t)
15    scope.bind(var)
16
17 def exitCall(Parser.CallContext ctx):
18    name = ctx.ID().getText()
19    func = scope.resolve(name)
20    if func == None:
21        error("no such function: " + name)
22    if func.type == Variable:
23        error(name + " is not a function")

```

Anmerkung: Um den Code auf die Folie zu bekommen, ist dies wieder ein Mix aus Java und Python geworden. Sry ;)

Im Vergleich zu den einfachen *nested scopes* kommt hier nur ein weiterer Scope für den Funktionskopf dazu. Dieser spielt eine Doppelrolle: Er ist sowohl ein Symbol (welches im Elternscope bekannt ist) als auch ein eigener (lokaler) Scope für die Funktionsparameter.

Um später im Interpreter eine Funktion tatsächlich auswerten zu können, muss im Scope der Funktion zusätzlich der AST-Knoten der Funktionsdefinition gespeichert werden (weiteres Feld/Attribut in `Function`)!

5.3.5. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Funktionen: Nested Scopes => hierarchische Organisation

- Umgang mit dem Funktionsnamen, den Parametern und dem Funktionskörper

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Parameter und lokale Variablen von Funktionen auflösen

Challenges

Diskutieren Sie folgende Fragen:

- Warum werden überhaupt Symboltabellen eingesetzt?
- Warum muss man zwischen Deklaration und Definition unterscheiden?
- Erklären Sie die Verbindung einer Symboltabelle zu den einzelnen Phasen einer Compiler-Pipeline.
- Wo liegen die Grenzen der semantischen Analyse?
- Warum kann man im Allgemeinen nicht die Symboltabellen nutzen, um die Werte von Symbolen mit zu speichern?
- Wieso sind Funktionen Scope und Symbol gleichzeitig?
- Skizzieren Sie für eine Funktionsdeklaration mit Parametern die resultierende Symboltabelle.
- Erklären Sie, wie man beim Funktionsaufruf vorgehen würde. Werden dabei Einträge in der Symboltabelle erzeugt?

5.4. SymbTab3: Strukturen und Klassen

TL;DR

Strukturen und Klassen bilden jeweils einen eigenen verschachtelten Scope, worin die Attribute und Methoden definiert werden.

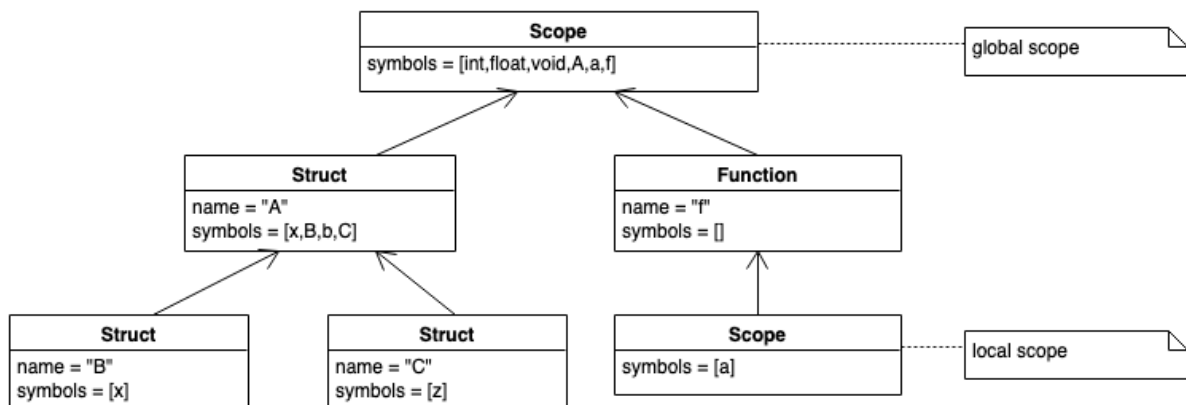
Bei der Namensauflösung muss man dies beachten und darf beim Zugriff auf Attribute und Methoden nicht einfach in den übergeordneten Scope schauen. Zusätzlich müssen hier Vererbungshierarchien in der Struktur der Symboltabelle berücksichtigt werden.

Videos

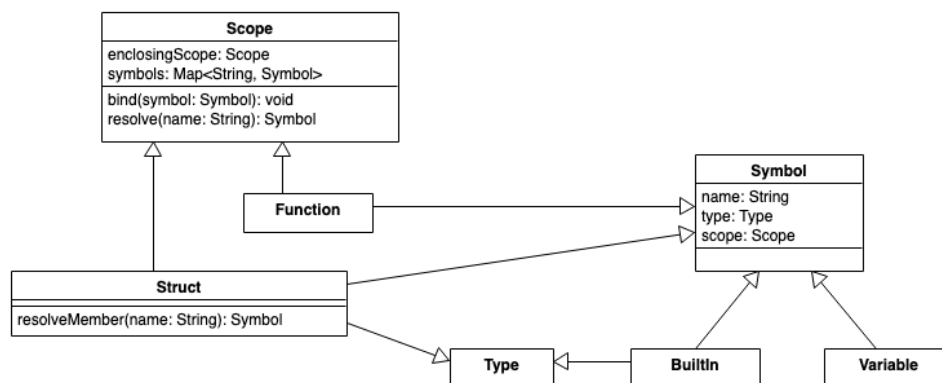
- [VL Strukturen und Klassen](#)

5.4.1. Strukturen

```
1 struct A {  
2     int x;  
3     struct B {int x;};  
4     B b;  
5     struct C {int z;};  
6 };  
7 A a;  
8 void f() {  
9     A a;  
10    a.b.x = 42;  
11 }
```



5.4.2. Strukturen: Erweiterung der Symbole und Scopes



Quelle: Structs: Eigene Modellierung nach einer Idee in (Parr 2010, p. 162)

Strukturen stellen wie Funktionen sowohl einen Scope als auch ein Symbol dar.

Zusätzlich stellt eine Struktur (-definition) aber auch einen neuen Typ dar, weshalb **Struct** auch noch das Interface **Type** "implementiert".

5.4.3. Strukturen: Auflösen von Namen

```

1 class Struct(Scope, Symbol, Type):
2     def resolveMember(name):
3         return symbols[name]
  
```

=> Auflösen von "a.b"(im Listener in `exitMember()`):

- a im "normalen" Modus mit `resolve()` über den aktuellen Scope
- Typ von a ist **Struct** mit Verweis auf den eigenen Scope
- b nur innerhalb des **Struct**-Scopes mit `resolveMember()`

In der Grammatik würde es eine Regel `member` geben, die auf eine Struktur der Art `ID.ID` anspricht (d.h. eigentlich den Teil `.ID`), und entsprechend zu Methoden `enterMember()` und `exitMember()` im Listener führt.

Das Symbol für a hat als `type`-Attribut eine Referenz auf die **Struct**, die ja einen eigenen Scope hat (`symbols`-Map). Darin muss dann b aufgelöst werden.

5.4.4. Klassen

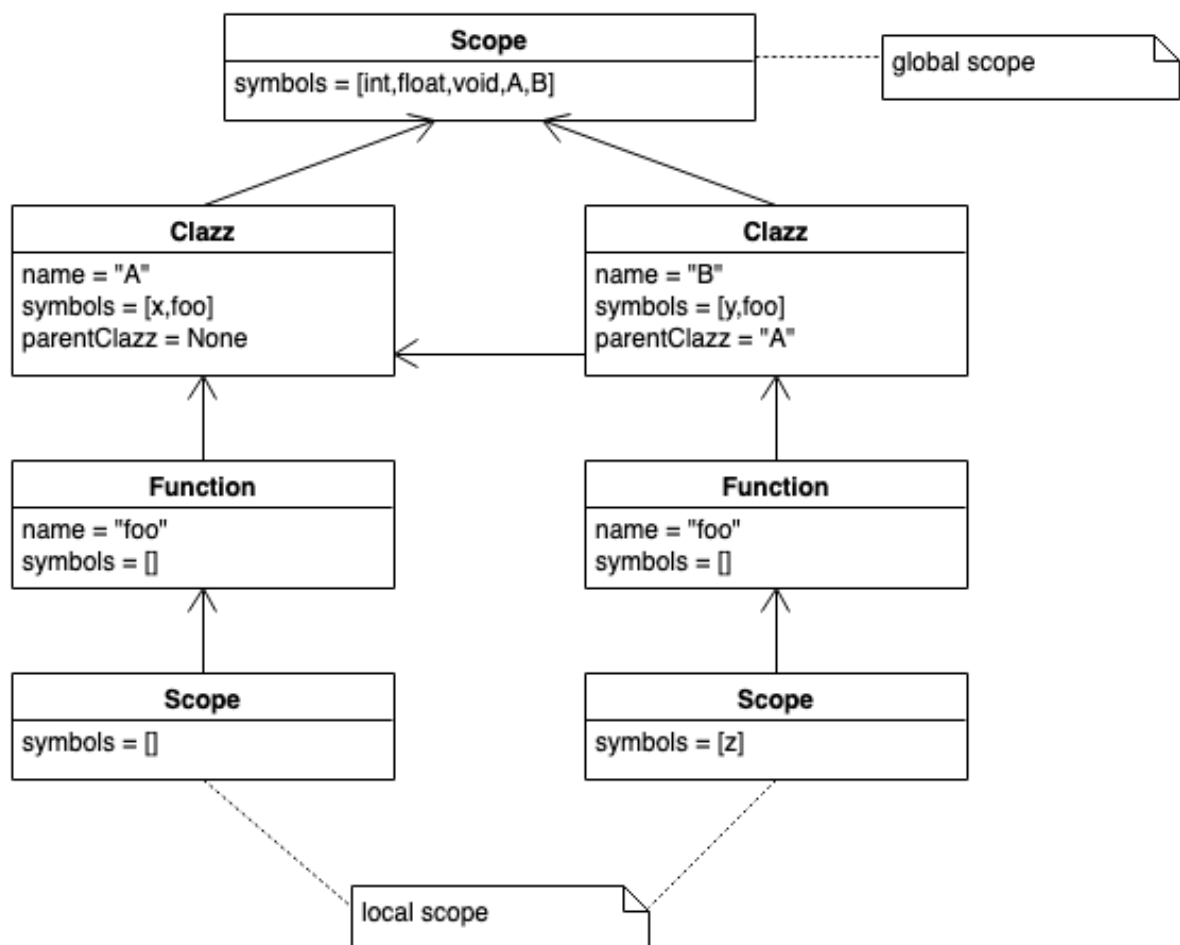
```

1 class A {
2     public:
3         int x;
4         void foo() { ; }
  
```

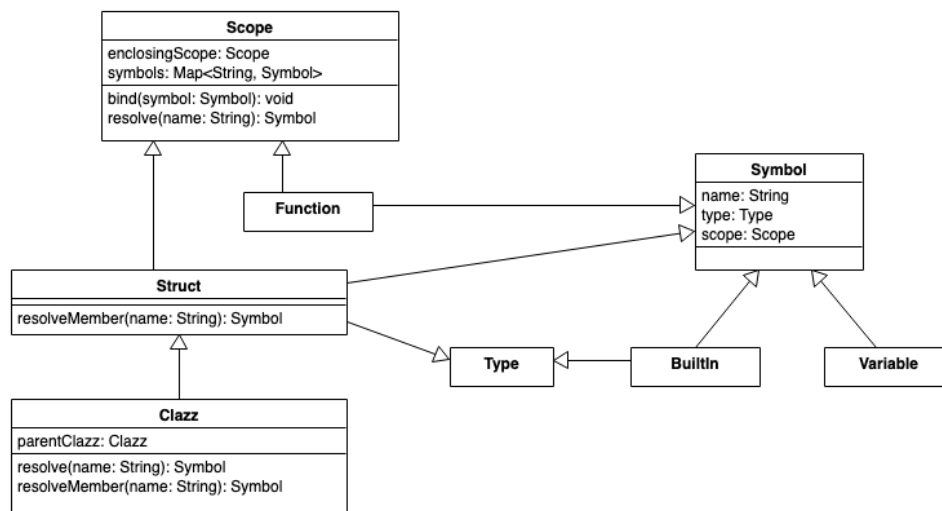
```

5  };
6  class B : public A {
7  public
8      int y;
9      void foo() {
10         int z = x+y;
11     }
12 };

```



5.4.5. Klassen: Erweiterung der Symbole und Scopes



Quelle: Klassen: Eigene Modellierung nach einer Idee in (Parr 2010, p. 167)

Bei Klassen kommt in den Tabellen ein weiterer Pointer `parentClazz` auf die Elternklasse hinzu (in der Superklasse ist der Wert `None`).

5.4.6. Klassen: Auflösen von Namen

```

1  class Clazz(Struct):
2      Clazz parentClazz    # None if base class
3
4      def resolve(name):
5          # do we know "name" here?
6          if symbols[name]: return symbols[name]
7          # NEW: if not here, check any parent class ...
8          if parentClazz and parentClazz.resolve(name): return parentClazz.
              resolve(name)
9          else:
10             # ... or enclosing scope if base class
11             if enclosingScope: return enclosingScope.resolve(name)
12             else: return None    # not found
13
14     def resolveMember(name):
15         if symbols[name]: return symbols[name]
16         # NEW: check parent class
17         if parentClazz: return parentClazz.resolveMember(name)
18         else: return None
  
```

Quelle: Klassen: Eigene Implementierung nach einer Idee in (Parr 2010, p. 172)

Hinweis: Die obige Implementierungsskizze soll vor allem das Prinzip demonstrieren - sie ist aus Gründen der Lesbarkeit nicht besonders effizient: beispielsweise wird `parentClazz.resolve(name)` mehrfach evaluiert...

Beim Auflösen von Attributen oder Methoden muss zunächst in der Klasse selbst gesucht werden, anschließend in der Elternklasse.

Beispiel (mit den obigen Klassen A und B):

```
1 B foo;
2 foo.x = 42;
```

Hier wird analog zu den Structs zuerst `foo` mit `resolve()` im lokalen Scope aufgelöst. Der Typ des Symbols `foo` ist ein `Clazz`, was zugleich ein Scope ist. In diesem Scope wird nun mit `resolveMember()` nach dem Symbol `x` gesucht. Falls es hier nicht gefunden werden kann, wird in der Elternklasse (sofern vorhanden) weiter mit `resolveMember()` gesucht.

Die normale Namensauflösung wird ebenfalls erweitert um die Auflösung in der Elternklasse.

Beispiel:

```
1 int wuppie;
2 class A {
3     public:
4         int x;
5         void foo() { ; }
6 };
7 class B : public A {
8     public
9         int y;
10        void foo() {
11            int z = x+y+wuppie;
12        }
13 };
```

Hier würde `wuppie` als Symbol im globalen Scope definiert werden. Beim Verarbeiten von `int z = x+y+wuppie;` würde mit `resolve()` nach `wuppie` gesucht: Zuerst im lokalen Scope unterhalb der Funktion, dann im Funktions-Scope, dann im Klassen-Scope von B. Hier sucht `resolve()` auch zunächst lokal, geht dann aber die Vererbungshierarchie entlang (sofern wie hier vorhanden). Erst in der Superklasse (wenn der `parentClazz`-Zeiger `None` ist), löst `resolve()` wieder normal auf und sucht um umgebenden Scope. Auf diese Weise kann man wie gezeigt in Klassen (Methoden) auf globale Variablen verweisen ...

Anmerkung: Durch dieses Vorgehen wird im Prinzip in Methoden aus dem Zugriff auf ein Feld `x` implizit ein `this.x` aufgelöst, wobei `this` die Klasse auflöst und `x` als Attribut darin.

5.4.7. Wrap-Up

- Symboltabellen: Verwaltung von Symbolen und Typen (Informationen über Bezeichner)
- Strukturen und Klassen bilden eigenen Scope
- Strukturen/Klassen lösen etwas anders auf: Zugriff auf Attribute und Methoden

Zum Nachlesen

- Mogensen (2017): Kapitel 3
- Parr (2014): Kapitel 6.4 und 8.4
- Parr (2010): Kapitel 6, 7 und 8

Lernziele

- k3: Ich kann Symboltabellen für Nested Scopes implementieren unter Nutzung von passenden Strukturen/Klassen und einem Listener
- k3: Ich kann Attribute von Klassen und Strukturen auflösen

Challenges

Symboltabellen praktisch

Betrachten Sie folgenden Java-Code:

1. Umkreisen Sie alle Symbole.
2. Zeichnen Sie Pfeile von Symbol-Referenzen zur jeweiligen Definition (falls vorhanden).
3. Identifizieren Sie alle benannten Scopes.
4. Identifizieren Sie alle anonymen Scopes.
5. Geben Sie die resultierende Symboltabelle an (Strukturen wie in VL besprochen).

```
1 package a.b;
2
3 import u.Y;
4
5 class X extends Y {
6     int f(int x) {
7         int x,y;
8         { int x; x = y + 1; }
9         x = y + 1;
10    }
11 }
12
13 class Z {
14     class W extends X {
15         int x;
16         void foo() { f(34); }
17     }
18     int x,z;
19     int f(int x) {
20         int y;
21         y = x;
22         z = x;
23     }
24 }
```

6. Interpreter

Ein Interpreter erzeugt keinen Code, sondern führt Source-Code (interaktiv) aus. Die einfachste Möglichkeit ist der Einsatz von attribuierten Grammatiken, wo der Code bereits beim Parsen ausgeführt wird (“syntaxgesteuerte Interpretation”). Mehr Möglichkeiten hat man dagegen bei der Traversierung des AST, beispielsweise mit dem Visitor-Pattern. Auch die Abarbeitung von Bytecode in einer Virtuellen Maschine (VM) zählt zur Interpretation.

(Register- und Stack-basierte Interpreter betrachten wir im Rahmen der Veranstaltung aktuell nicht.)

6.1. AST-basierte Interpreter: Basics

TL;DR

Ein AST-basierter Interpreter besteht oft aus einem “Visitor-Dispatcher”: Man traversiert mit einer `eval()`-Funktion den AST und ruft je nach Knotentyp die passende Funktion auf. Dabei werden bei Ausdrücken (*Expressions*) Werte berechnet und zurückgegeben, d.h. hier hat man einen Rückgabewert und ein entsprechendes `return` im `switch/case`, während man bei Anweisungen (*Statements*) keinen Rückgabewert hat.

Der Wert von Literalen ergibt sich direkt durch die Übersetzung des jeweiligen Werts in den passenden Typ der Implementierungssprache. Bei Ausdrücken interpretiert zunächst die Teilausdrücke durch den Aufruf von `eval()` für die jeweiligen AST-Kindknoten und berechnet daraus das gewünschte Ergebnis.

Für Blöcke und Variablen muss man analog zum Aufbau von Symboltabellen wieder Scopes berücksichtigen, d.h. man benötigt Strukturen ähnlich zu den Symboltabellen (hier “Umgebung” (*Environment*) genannt). Es gibt eine globale Umgebung, und mit dem Betreten eines neuen Blocks wird eine neue Umgebung aufgemacht, deren Eltern-Umgebung die bisherige Umgebung ist.

Zu jedem Namen kann man in einer Umgebung einen Wert definieren bzw. abrufen. Dabei muss man je nach Semantik der zu interpretierenden Sprache unterscheiden zwischen der “Definition” und der “Zuweisung” einer Variablen: Die Definition erfolgt i.d.R. in der aktuellen Umgebung, bei der Zuweisung sucht man ausgehend von der aktuellen Umgebung bis hoch zur globalen Umgebung nach dem ersten Vorkommen der Variablen und setzt den Wert in der gefundenen Umgebung. Bei Sprachen, die Variablen beim ersten Zugriff definieren, muss man dieses Verhalten entsprechend anpassen.

Videos

- [VL AST-basierte Interpreter \(Basics\)](#)

6.1.1. Aufgaben im Interpreter

Im Allgemeinen reichen einfache syntaxgesteuerte Interpreter nicht aus. Normalerweise simuliert ein Interpreter die Ausführung eines Programms durch den Computer. D.h. der Interpreter muss über die entsprechenden Eigenschaften verfügen: Prozessor, Code-Speicher, Datenspeicher, Stack ...

```

1  int x = 42;
2  int f(int x) {
3      int y = 9;
4      return y+x;
5  }
6
7  x = f(x);

```

- Aufbauen des AST ... => Lexer+Parser
- Auflösen von Symbolen/Namen ... => Symboltabellen, Resolving
- Type-Checking und -Inference ... => Semantische Analyse (auf Symboltabellen)
- Speichern von Daten: Name+Wert vs. Adresse+Wert (Erinnerung: Data-Segment und Stack im virtuellen Speicher)
- Ausführen von Anweisungen Text-Segment im virtuellen Speicher; hier über den AST
- Aufruf von Funktionen und Methoden Kontextwechsel nötig: Was ist von wo aus sichtbar?

6.1.2. AST-basierte Interpreter: Visitor-Dispatcher

```

1  def eval(self, AST t):
2      if t.type == Parser.BLOCK : block(t)
3      elif t.type == Parser.ASSIGN : assign(t)
4      elif t.type == Parser.RETURN : ret(t)
5      elif t.type == Parser.IF : ifstat(t)
6      elif t.type == Parser.CALL : return call(t)
7      elif t.type == Parser.ADD : return add(t)
8      elif t.type == Parser.MUL : return mul(t)
9      elif t.type == Parser.INT : return Integer.parseInt(t.getText())
10     elif t.type == Parser.ID : return load(t)
11     else : ... # catch unhandled node types
12     return None;

```

Nach dem Aufbau des AST durch Scanner und Parser und der semantischen Analyse anhand der Symboltabellen müssen die Ausdrücke (*expressions*) und Anweisungen (*statements*) durch den Interpreter ausgewertet werden. Eine Möglichkeit dazu ist das Traversieren des AST mit dem Visitor-Pattern. Basierend auf dem Typ des aktuell betrachteten AST-Knotens wird entschieden, wie damit umgegangen werden soll. Dies erinnert an den Aufbau der Symboltabellen ...

Tip

Exkurs Expressions (Ausdrücke) vs. Statements (Anweisungen)

In Programmiersprachen unterscheiden wir häufig **Expressions** (*Ausdrücke*) und **Statements** (*Anweisungen*).

Expressions sind dabei syntaktische Konstrukte einer Programmiersprache, die (in einem gegebenen Kontext) zu einem Wert **evaluiert** werden können. Typische Expressions sind beispielsweise Ausdrücke wie $2 * 3$ oder `foo(42)`; ... In manchen Sprachen sind beispielsweise auch Zuweisungen Expressions: `v = 42 + 7`; würde in C der Variablen `v` den Wert 49 zuweisen, dies ist gleichzeitig auch der Wert des gesamten Ausdrucks. Man könnte in C also Dinge formulieren wie `if (v = 42 + 7) ...` (wobei das Interpretieren eines Integers in einem bool'schen Kontext nochmal ein anderes Problem ist).

Statements sind syntaktische Konstrukte in Programmiersprachen, die **ausgeführt** werden können und dabei in der Regel einen Zustand im Programm verändern, also einen Seiteneffekt haben. Die Ausführung eines Statements hat normalerweise keinen Wert an sich. Typische Beispiele sind Zuweisungen `v = 7`, Kontrollfluss `if (...) then {...} else {...}`, Schleifen `for x in foo: ...`, `switch / case`-Statements. (Es gibt aber auch Programmiersprachen, wo ein `if/then/else`-Konstrukt eine Expression ist, also bei der Ausführung einen Wert ergibt.) In den meisten Programmiersprachen können Expressions Teile von Statements bilden: In `v = 42 + 7` ist die gesamte Zuweisung eine Anweisung (Seiteneffekt: die Variable `v` hat danach einen anderen Zustand), und der Teil `42 + 7` ist ein Ausdruck, der ausgewertet werden kann und üblicherweise den Wert 49 ergibt (außer man beauftragt ein LLM mit der Auswertung). In C-ähnlichen Sprachen kann durch Hinzufügen eines Semikolons aus dem Ausdruck `42 + 7` eine Anweisung gemacht werden...

Vergleiche auch Nystrom (2021), Kapitel 6 "Parsing Expressions", Kapitel 7 "Evaluating Expressions" und Kapitel 8 "Statements and State", aber auch [Wikipedia: Expression](#) und [Wikipedia: Statement](#).

Die `eval()`-Methode bildet das Kernstück des (AST-traversierenden) Interpreters. Hier wird passend zum aktuellen AST-Knoten die passende Methode des Interpreters aufgerufen.

Hinweis: Im obigen Beispiel wird nicht zwischen der Auswertung von Ausdrücken und Anweisungen unterschieden, es wird die selbe Methode `eval()` genutzt. Allerdings liefern Ausdrücke einen Wert zurück (erkennbar am `return` im jeweiligen `switch/case`-Zweig), während Anweisungen keinen Wert liefern.

In den folgenden Beispielen wird davon ausgegangen, dass ein komplettes Programm eingelesen, geparkt, vorverarbeitet und dann interpretiert wird.

Für einen interaktiven Interpreter würde man in einer Schleife die Eingaben lesen, parsen und vorverarbeiten und dann interpretieren. Dabei würde jeweils der AST und die Symboltabelle *ergänzt*, damit die neuen Eingaben auf frühere verarbeitete Eingaben zurückgreifen können. Durch die Form der Schleife "Einlesen – Verarbeiten – Auswerten" hat sich auch der Name "Read-Eval-Loop" bzw. "Read-Eval-Print-Loop" (**REPL**) eingebürgert.

6.1.3. Auswertung von Literalen und Ausdrücken

- Typen mappen: Zielsprache => Implementierungssprache

Die in der Zielsprache verwendeten (primitiven) Typen müssen auf passende Typen der Sprache, in der der Interpreter selbst implementiert ist, abgebildet werden.

Beispielsweise könnte man den Typ `nil` der Zielsprache auf den Typ `null` des in Java implementierten Interpreters abbilden, oder den Typ `number` der Zielsprache auf den Typ `Double` in Java mappen.

- Literale auswerten:

```
1 INT: [0-9]+ ;
```

```
1 elif t.type == Parser.INT : return Integer.parseInt(t.getText())
```

Das ist der einfachste Teil ... Die primitiven Typen der Zielsprache, für die es meist ein eigenes Token gibt, müssen als Datentyp der Interpreter-Programmiersprache ausgewertet werden.

- Ausdrücke auswerten:

```
1 add: e1=expr "+" e2=expr ;
```

```
1 def add(self, AST t):
2     lhs = eval(t.e1())
3     rhs = eval(t.e2())
4     return (double)lhs + (double)rhs # Semantik!
```

Die meisten möglichen Fehlerzustände sind bereits durch den Parser und bei der semantischen Analyse abgefangen worden. Falls zur Laufzeit die Auswertung der beiden Summanden keine Zahl ergibt, würde eine Java-Exception geworfen, die man an geeigneter Stelle fangen und behandeln muss. Der Interpreter soll sich ja nicht mit einem Stack-Trace verabschieden, sondern soll eine Fehlermeldung präsentieren und danach normal weiter machen ...

6.1.4. Kontrollstrukturen

```
1 ifstat: 'if' expr 'then' s1=stat ('else' s2=stat)? ;
```

```
1 def ifstat(self, AST t):
2     if eval(t.expr()): eval(t.s1())
3     else:
4         if t.s2(): eval(t.s2())
```

Analog können die anderen bekannten Kontrollstrukturen umgesetzt werden, etwa `switch/case`, `while` oder `for`.

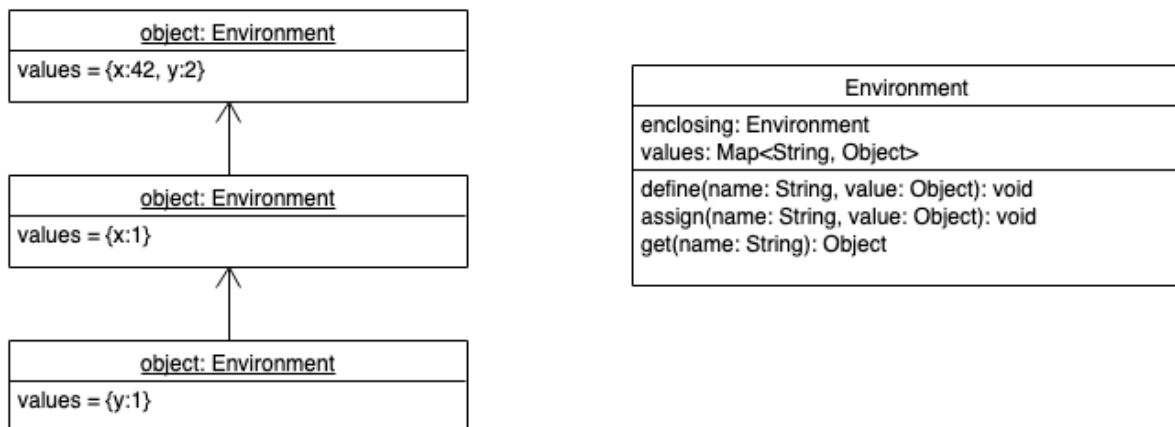
Dabei können erste Optimierungen vorgenommen werden: Beispielsweise könnten `for`-Schleifen im Interpreter in `while`-Schleifen transformiert werden, wodurch im Interpreter nur ein Schleifenkonstrukt implementiert werden müsste.

6.1.5. Zustände: Auswerten von Anweisungen

```

1  int x = 42;
2  float y;
3  {
4      int x;
5      x = 1;
6      y = 2;
7      { int y = x; }
8  }

```



Das erinnert nicht nur zufällig an den Aufbau der Symboltabellen :-)

Und so lange es nur um Variablen ginge, könnte man die Symboltabellen für das Speichern der Werte nutzen. Allerdings müssen wir noch Funktionen und Strukturen bzw. Klassen realisieren, und spätestens dann kann man die Symboltabelle nicht mehr zum Speichern von Werten einsetzen. Also lohnt es sich, direkt neue Strukturen für das Halten von Variablen und Werten aufzubauen.

6.1.6. Detail: Felder im Interpreter

Eine mögliche Implementierung für einen Interpreter basierend auf einem ANTLR-Visitor ist nachfolgend gezeigt.

Hinweis: Bei der Ableitung des `BaseVisitor<T>` muss der Typ `T` festgelegt werden. Dieser fungiert als Rückgabebetyp für die Visitor-Methoden. Entsprechend können alle Methoden nur einen gemeinsamen (Ober-) Typ zurückliefern, weshalb man sich an der Stelle oft mit `Object` behilft und dann manuell den konkreten Typ abfragen und korrekt casten muss.

```

1  class Interpreter(BaseVisitor<Object>):
2      __init__(self, AST t):
3          BaseVisitor<Object>.__init__(self)
4          self.root = t
5          self.env = Environment()

```

Quelle: AST-Interpreter: Eigener Code basierend auf einer Idee nach [Interpreter.java](#) by Bob Nystrom on Github.com (MIT)

6.1.7. Ausführen einer Variablendeklaration

```
1 varDecl: "var" ID ("=" expr)? ";" ;
```

```
1 def varDecl(self, AST t):
2     # deklarierte Variable (String)
3     name = t.ID().getText()
4
5     value = None; # TODO: Typ der Variablen beachten (Defaultwert)
6     if t.expr(): value = eval(t.expr())
7
8     self.env.define(name, value)
9
10    return None
```

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Variablendeklaration vorbeikommen, also etwa `int x;` oder `int x = wuppie + fluppie;`, dann wird im **aktuellen** Environment der String "x" sowie der Wert (im zweiten Fall) eingetragen.

6.1.8. Ausführen einer Zuweisung

```
1 assign: ID "=" expr;
```

```
1 def assign(self, AST t):
2     lhs = t.ID().getText()
3     value = eval(t.expr())
4
5     self.env.assign(lhs, value) # Semantik!
6 }
7
8 class Environment:
9     def assign(self, String n, Object v):
10        if self.values[n]: self.values[n] = v
11        elif self.enclosing: self.enclosing.assign(n, v)
12        else: raise RuntimeError(n, "undefined variable")
```

Quelle: Evaluieren einer Zuweisung: Eigener Code basierend auf einer Idee nach [Environment.java](#) by Bob Nystrom on Github.com (MIT)

Wenn wir bei der Traversierung des AST mit `eval()` bei einer Zuweisung vorbeikommen, also etwa `x = 7;` oder `x = wuppie + fluppie;`, dann wird zunächst im aktuellen Environment die rechte Seite der Zuweisung ausgewertet (Aufruf von `eval()`). Anschließend wird der Wert für die Variable im Environment eingetragen: Entweder sie wurde im aktuellen Environment früher bereits definiert, dann wird der neue Wert hier eingetragen. Ansonsten wird entlang der Verschachtelungshierarchie gesucht und entsprechend eingetragen. Falls die Variable nicht gefunden werden kann, wird eine Exception ausgelöst.

An dieser Stelle kann man über die Methode `assign` in der Klasse `Environment` dafür sorgen, dass nur bereits deklarierte Variablen zugewiesen werden dürfen. Wenn man stattdessen wie etwa in Python das implizite Erzeugen neuer Variablen erlauben möchte, würde man statt `Environment#assign` einfach `Environment#define` nutzen ...

Anmerkung: Der gezeigte Code funktioniert nur für normale Variablen, nicht für Zugriffe auf Attribute einer Struct oder Klasse!

6.1.9. Blöcke: Umgang mit verschachtelten Environments

```
1 block: '{' stat* '}' ;
```

```
1 def block(self, AST t):
2     prev = self.env
3
4     try:
5         self.env = Environment(self.env)
6         for s in t.stat(): eval(s)
7     finally: self.env = prev
8
9     return None;
```

Quelle: Nested Environments: Eigener Code basierend auf einer Idee nach [Interpreter.java](#) by Bob Nystrom on Github.com (MIT)

Beim Interpretieren von Blöcken muss man einfach nur eine weitere Verschachtelungsebene für die Environments anlegen und darin dann die Anweisungen eines Blockes auswerten ...

Wichtig: Egal, was beim Auswerten der Anweisungen in einem Block passiert: Es muss am Ende die ursprüngliche Umgebung wieder hergestellt werden (**finally**-Block).

6.1.10. Wrap-Up

- Interpreter simulieren die Programmausführung
 - Namen und Symbole auflösen
 - Speicherbereiche simulieren
 - Code ausführen: Read-Eval-Loop
- Traversierung des AST: `eval (AST t)` als Visitor-Dispatcher
- Scopes mit `Environment` (analog zu Symboltabellen)
- Interpretation von Blöcken und Variablen (Deklaration, Zuweisung)

Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter, insb. 8. Statements and State
- Grune u. a. (2012): Kapitel 6
- Mogensen (2017): Kapitel 4

Lernziele

- k3: Ich kann die Traversierung von Parse-Trees implementieren und dabei mit Hilfe des Visitor-Patterns Aktionen ausführen
- k3: Ich kann Environment-Strukturen analog zu den Symboltabellen aufbauen, um Namen und Werte dynamisch zu speichern
- k3: Ich kann eine Read-Eval-Schleife implementieren und dabei durch Traversierung des AST die dort abgelegten Anweisungen und Ausdrücke und Kontrollstrukturen ausführen

6.2. AST-basierte Interpreter: Funktionen und Klassen

TL;DR

Üblicherweise können Funktionen auf die Umgebung zurückgreifen, in der die Definition der Funktion erfolgt ist (“**Closure**”). Deshalb wird beim Interpretieren einer Funktionsdefinition der jeweilige AST-Knoten (mit dem Block des Funktionskörpers) und die aktuelle Umgebung in einer Struktur zusammengefasst. Zusätzlich muss in der aktuellen Umgebung der Name der Funktion zusammen mit der eben erzeugten Struktur (“Funktionsobjekt”) als Wert definiert werden.

Beim Funktionsaufruf löst man den Funktionsnamen in der aktuellen Umgebung auf und erhält das Funktionsobjekt mit dem AST der Funktion und der Closure. Die Funktionsparameter werden ebenfalls in der aktuellen Umgebung aufgelöst (Aufruf von `eval()` für die AST-Kindknoten des Funktionsaufrufs). Zur Interpretation der Funktion legt man sich eine neue Umgebung an, deren Eltern-Umgebung die Closure der Funktion ist, definiert die Funktionsparameter (Name und eben ermittelter Wert) in dieser neuen Umgebung und interpretiert dann den AST-Kindknoten des Funktionsblocks in dieser neuen Umgebung. Für den Rückgabewert muss man ein wenig tricksen: Ein Block hat normalerweise keinen Wert. Eine Möglichkeit wäre, bei der Interpretation eines **return**-Statements eine Exception mit dem Wert des Ausdruck hinter dem “**return**” zu werfen und im `eval()` des Funktionsblock zu fangen.

Für Klassen kann man analog verfahren. Methoden sind zunächst einfach Funktionen, die in einem Klassenobjekt gesammelt werden. Das Erzeugen einer Instanz einer Klasse ist die Interpretation eines “Aufrufs” der Klasse (analog zum Aufruf einer Funktion): Dabei wird ein spezielles Instanzobjekt erzeugt, welches auf die Klasse verweist und welches die Werte der Attribute hält. Beim Aufruf von Methoden auf einem Instanzobjekt wird der Name der Funktion über das Klassenobjekt aufgelöst, eine neue Umgebung erzeugt mit der Closure der Funktion als Eltern-Umgebung und das Instanzobjekt wird in dieser Umgebung definiert als “**this**” oder “**self**”. Anschließend wird ein neues Funktionsobjekt mit der eben erzeugten Umgebung und dem Funktions-AST erzeugt und zurückgeliefert. Dieses neue Funktionsobjekt wird dann wie eine normale Funktion aufgerufen (interpretiert, s.o.). Der Zugriff in der Methode auf die Attribute der Klasse erfolgt dann über **this** bzw. **self**, welche in der Closure der Funktion nun definiert sind und auf das Instanzobjekt mit den Attributen verweisen.

Videos

- [VL AST-basierte Interpreter \(Funktionen, Klassen\)](#)

6.2.1. Funktionen

```

1  int foo(int a, int b, int c) {
2      print a + b + c;
3  }
4
5  foo(1, 2, 3);

```

```

1  def makeCounter():
2      var i = 0
3      def count():
4          i = i + 1
5          print i
6      return count;
7
8  counter = makeCounter()
9  counter()    # "1"
10 counter()    # "2"

```

Die Funktionsdeklaration muss im aktuellen Kontext abgelegt werden, dazu wird der AST-Teilbaum der Deklaration benötigt.

Beim Aufruf muss man das Funktionssymbol im aktuellen Kontext suchen, die Argumente auswerten, einen neuen lokalen Kontext anlegen und darin die Parameter definieren (mit den eben ausgewerteten Werten) und anschließend den AST-Teilbaum des Funktionskörpers im Interpreter mit `eval()` auswerten ...

6.2.2. Ausführen einer Funktionsdeklaration

```

1  funcDecl : type ID '(' params? ')' block ;
2  funcCall : ID '(' exprList? ')' ;

```

```

1  def funcDecl(self, AST t):
2      fn = Fun(t, self.env)
3      self.env.define(t.ID().getText(), fn)

```

Fun
decl: AST closure: Environment

Quelle: Funktionsdeklaration: Eigener Code basierend auf einer Idee nach `LoxFunction.java` by Bob Nystrom on Github.com (MIT)

Man definiert im aktuellen Environment den Funktionsnamen und hält dazu den aktuellen Kontext (aktuelles Environment) sowie den AST-Knoten mit der eigentlichen Funktionsdefinition fest.

Für *Closures* ist der aktuelle Kontext wichtig, sobald man die Funktion ausführen muss. In (Parr 2010, 236) wird beispielsweise einfach nur ein neuer Memory-Space (entspricht ungefähr hier einem neuen lokalen Environment) angelegt, in dem die im Funktionskörper definierten Symbole angelegt werden. Die Suche nach Symbolen erfolgt dort nur im Memory-Space (Environment) der Funktion bzw. im globalen Scope (Environment).

6.2.3. Ausführen eines Funktionsaufrufs

```
1 funcDecl : type ID '(' params? ')' block ;
2 funcCall : ID '(' exprList? ')' ;
```

```
1 def funcCall(self, AST t):
2     fn = (Fun)eval(t.ID())
3     args = [eval(a) for a in t.exprList()]
4
5     prev = self.env; self.env = Environment(fn.closure)
6     for i in range(args.size()):
7         self.env.define(fn.decl.params()[i].getText(), args[i])
8
9     eval(fn.decl.block())
10    self.env = prev
```

Quelle: Funktionsaufruf: Eigener Code basierend auf einer Idee nach `LoxFunction.java` by Bob Nystrom on Github.com (MIT)

Zunächst wird die `ID` im aktuellen Kontext ausgewertet. In der obigen Grammatik ist dies tatsächlich nur ein Funktionsname, aber man könnte über diesen Mechanismus auch Ausdrücke erlauben und damit Funktionspointer bzw. Funktionsreferenzen realisieren ... Im Ergebnis hat man das Funktionsobjekt mit dem zugehörigen AST-Knoten und dem Kontext zur Deklarationszeit.

Die Argumente der Funktion werden nacheinander ebenfalls im aktuellen Kontext ausgewertet.

Um den Funktionsblock auszuwerten, legt man einen neuen temporären Kontext über dem Closure-Kontext der Funktion an und definiert darin die Parameter der Funktion samt den aktuellen Werten. Dann lässt man den Interpreter über den Visitor-Dispatch den Funktionskörper evaluieren und schaltet wieder auf den Kontext vor der Funktionsauswertung zurück.

6.2.4. Funktionsaufruf: Rückgabewerte

```

1  def funcCall(self, AST t):
2      ...
3
4      eval(fn.decl.block())
5
6      ...
7      return None # (Wirkung)

```

```

1  class ReturnEx(RuntimeException):
2      __init__(self, v): self.value = v
3
4  def return(self, AST t):
5      raise ReturnEx(eval(t.expr()))
6
7  def funcCall(self, AST t):
8      ...
9      erg = None
10     try: eval(fn.decl.block())
11     except ReturnEx as r: erg = r.value
12     ...
13     return erg;

```

Quelle: Rückgabewerte: Eigener Code basierend auf einer Idee nach [Return.java](#) und [LoxFunction.java](#) by Bob Nystrom on Github.com (MIT)

Rückgabewerte für den Funktionsaufruf werden innerhalb von `block` berechnet, wo eine Reihe von Anweisungen interpretiert werden, weshalb `block` ursprünglich keinen Rückgabewert hat. Im Prinzip könnte man `block` etwas zurück geben lassen, was durch die möglicherweise tiefe Rekursion relativ umständlich werden kann.

An dieser Stelle kann man den Exceptions-Mechanismus **missbrauchen** und bei der Auswertung eines `return` mit dem Ergebniswert direkt zum Funktionsaufruf zurück springen. In Methoden, wo man einen neuen lokalen Kontext anlegt und die globale `env`-Variable temporär damit ersetzt, muss man dann ebenfalls mit `try/catch` arbeiten und im `finally`-Block die Umgebung zurücksetzen und die Exception erneut werfen.

6.2.5. Native Funktionen

```

1  class Callable:
2      def call(self, Interpreter i, List<Object> a): pass
3  class Fun(Callable): ...
4  class NativePrint(Fun):
5      def call(self, Interpreter i, List<Object> a):
6          for o in a: print a # nur zur Demo, hier sinnvoller Code :-))
7
8  # Im Interpreter (Initialisierung):
9  self.env.define("print", NativePrint())
10
11 def funcCall(self, AST t):
12     ...
13     # prev = self.env; self.env = Environment(fn.closure)
14     # for i in range(args.size()): ...
15     # eval(fn.decl.block()); self.env = prev
16     fn.call(self, args)

```

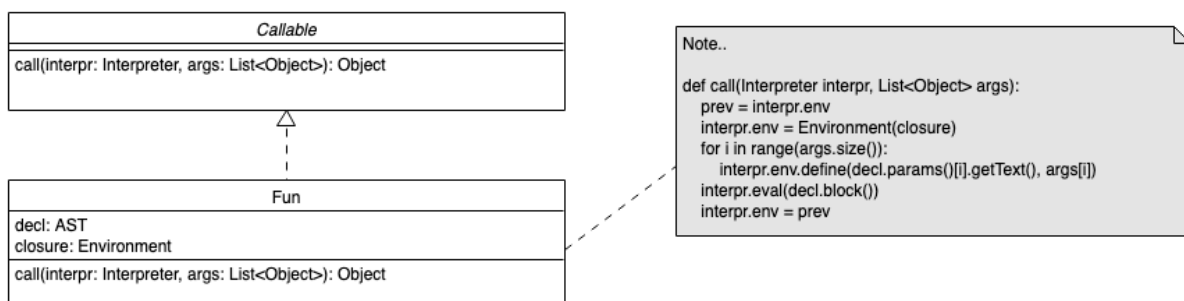
17 ...

Quelle: Native Funktionen: Eigener Code basierend auf einer Idee nach [LoxCachable.java](#) und [LoxFunction.java](#) by Bob Nystrom on Github.com (MIT)

Normalerweise wird beim Interpretieren eines Funktionsaufrufs der Funktionskörper (repräsentiert durch den entsprechenden AST-Teilbaum) durch einen rekursiven Aufruf von `eval()` ausgewertet.

Für native Funktionen, die im Interpreter eingebettet sind, klappt das nicht mehr, da hier kein AST vorliegt.

Man erstellt ein neues Interface `Callable` mit der Hauptmethode `call()` und leitet die frühere Klasse `Fun` davon ab: `class Fun(Callable)`. Die Methode `funcCall()` des Interpreters ruft nun statt der `eval()`-Methode die `call()`-Methode des Funktionsobjekts auf und übergibt den Interpreter (== Zustand) und die Argumente. Die `call()`-Methode der Klasse `Fun` muss nun ihrerseits im Normalfall den im Funktionsobjekt referenzierten AST-Teilbaum des Funktionskörpers mit dem Aufruf von `eval()` interpretieren ...



Für die nativen Funktionen leitet man einfach eine (anonyme) Klasse ab und speichert sie unter dem gewünschten Namen im globalen Kontext des Interpreters. Die `call()`-Methode wird dann entsprechend der gewünschten Funktion implementiert, d.h. hier erfolgt kein weiteres Auswerten des AST.

6.2.6. Klassen und Instanzen I

```
1 classDef : "class" ID "{" funcDecl* "}" ;
```

```

1 def classDef(self, AST t):
2     methods = HashMap<String, Fun>()
3     for m in t.funcDecl():
4         fn = Fun(m, self.env)
5         methods[m.ID().getText()] = fn
6
7     clazz =Clazz(methods)
8     self.env.define(t.ID().getText(), clazz)
  
```

Quelle: Klassen: Eigener Code basierend auf einer Idee nach [Interpreter.java](#) by Bob Nystrom on Github.com (MIT)

Anmerkung: In dieser Darstellung wird der Einfachheit halber nur auf Methoden eingegangen. Für Attribute müssten ähnliche Konstrukte implementiert werden.

6.2.7. Klassen und Instanzen II

```

1  classClazz(Callable):
2      __init__(self, Map<String, Fun> methods):
3          self.methods = methods
4
5      def call(self, Interpreter i, List<Object> a):
6          return Instance(self)
7
8      def findMethod(self, String name):
9          return self.methods[name]
10
11 class Instance:
12     __init__(self,Clazz clazz):
13         self.clazz = clazz
14
15     def get(self, String name):
16         method = self.clazz.findMethod(name)
17         if method != None: return method.bind(self)
18         raise RuntimeError(name, "undefined method")

```

Quelle: Instanzen: Eigener Code basierend auf einer Idee nach [LoxClass.java](#) und [LoxInstance.java](#) by Bob Nystrom on Github.com (MIT)

Instanzen einer Klasse werden durch den funktionsartigen “Aufruf” der Klassen angelegt (parameterloser Konstruktor). Eine Instanz hält die Attribute (hier nicht gezeigt) und eine Referenz auf die Klasse, um später an die Methoden heranzukommen.

6.2.8. Zugriff auf Methoden (und Attribute)

```

1  getExpr : obj "." ID ;

```

```

1  def getExpr(self, AST t):
2      obj = eval(t.obj())
3
4      if isinstance(obj, Instance):
5          return ((Instance)obj).get(t.ID().getText())
6
7      raise RuntimeError(t.obj().getText(), "no object")

```

Beim Zugriff auf Attribute muss das Objekt im aktuellen Kontext evaluiert werden. Falls es eine Instanz von `Instance` ist, wird auf das Feld per interner Hash-Map zugegriffen; sonst Exception.

6.2.9. Methoden und *this* oder *self*

```

1  class Fun(Callable):
2      def bind(self, Instance i):
3          e = Environment(self.closure)
4          e.define("this", i)
5          e.define("self", i)
6          return Fun(self.decl, e)

```

Quelle: Methodenaufwurf: Eigener Code basierend auf einer Idee nach `LoxFunction.java` by Bob Nystrom on Github.com (MIT)

Nach dem Interpretieren von Klassendefinitionen sind die Methoden in der Klasse selbst gespeichert, wobei der jeweilige `closure` auf den Klassenkontext zeigt.

Beim Auflösen eines Methodenaufrufs wird die gefundene Methode an die Instanz gebunden, d.h. es wird eine neue Funktion angelegt, deren `closure` auf den Kontext der Instanz zeigt. Zusätzlich wird in diesem Kontext noch die Variable “`this`” definiert, damit man damit auf die Instanz zugreifen kann.

In Python wird das in der Methodensignatur sichtbar: Der erste Parameter ist eine Referenz auf die Instanz, auf der diese Methode ausgeführt werden soll ...

6.2.10. Wrap-Up

- Interpreter simulieren die Programmausführung
 - Namen und Symbole auflösen
 - Speicherbereiche simulieren
 - Code ausführen: Read-Eval-Loop
- Traversierung des AST: `eval (AST t)` als Visitor-Dispatcher
- Scopes mit `Environment` (analog zu Symboltabellen)
- Interpretation von Funktionen (Deklaration/Aufruf, native Funktionen)
- Interpretation von Klassen und Instanzen

Zum Nachlesen

- Nystrom (2021): Kapitel: A Tree-Walk Interpreter, insb. 10. Functions u. 12. Classes
- Grune u. a. (2012): Kapitel 6
- Mogensen (2017): Kapitel 4

Lernziele

- k3: Ich kann die Traversierung von Parse-Trees implementieren und dabei mit Hilfe des Visitor-Patterns Aktionen ausführen
- k3: Ich kann Environment-Strukturen analog zu den Symboltabellen aufbauen, um Namen und Werte dynamisch zu speichern
- k3: Ich kann eine Read-Eval-Schleife implementieren und dabei durch Traversierung des AST die dort abgelegten Klassendefinitionen und Methodenaufrufe ausführen

Challenges

- Wie interpretiert man Code?
- Warum kann man die Werte nicht einfach in Symboltabellen ablegen?

- Wie geht man mit Funktionen um? Warum? Kann man diese mehrfach aufrufen?
- Wieso erzeugt man eine neue Environment mit der Closure in der Funktion?
- Wie gehen native Funktionen?

Betrachten Sie folgenden Code-Ausschnitt:

```
1  int x = 42;  
2  int f(int x) {  
3      int y = 9;  
4      return y+x;  
5  }  
6  
7  x = f(x);
```

1. Geben Sie den AST an.
2. Stellen Sie die Strukturen der Symboltabelle dar.
3. Stellen Sie die Strukturen im Interpreter dar.

7. Programmiersprachen und -konzepte

Unterschiedliche Programmiersprachen weisen nicht nur verschiedene Syntaxelemente auf, sondern haben eine teilweise stark unterschiedliche Semantik. Beides hat Auswirkungen auf die Bausteine eines Compilers.

7.1. Einführung in C++ (Erinnerungen an C)

TL;DR

Für C wurde ein paar Jahre nach der Entstehung ein objektorientierter Aufsatz entwickelt: C++. Beide Sprachversionen werden aktiv weiterentwickelt, vor allem in C++ gibt es ca. alle 3 Jahre einen neuen Standard mit teilweise recht umfangreichen Ergänzungen. Hier fließen analog zu Java immer mehr Programmierkonzepte mit ein, die aus anderen Sprachen stammen (etwa funktionale Programmierung). Das macht das Erlernen und Beherrschen der Sprache nicht unbedingt leichter. Die für uns wichtigsten Neuerungen kamen mit C11 und C++11 bzw. C++14.

C und C++ versuchen (im Gegensatz zu Java) ressourcenschonende Sprachen zu sein: Ein korrektes Programm soll so schnell wie möglich ausgeführt werden können und dabei so effizient wie möglich sein (etwa in Bezug auf den Speicherbedarf). Deshalb gibt es keine Laufzeitumgebung, der Quellcode wird direkt in ein ausführbares (und damit Betriebssystem-abhängiges) Binary kompiliert. Beide Sprachen erlauben dem Programmierer den Zugriff auf die Speicherverwaltung und damit viele Freiheiten. Die Kehrseite ist natürlich, dass Programmierfehler (etwa bei der Speicherallokation oder bei Indexberechnungen) nicht von der Laufzeitumgebung entdeckt und abgefangen werden können.

C-Programme sehen auf den ersten Blick Java-Code relativ ähnlich. Das ist nicht verwunderlich, da Java zeitlich nach C/C++ entwickelt wurde und die Syntax und große Teile der Schlüsselwörter von C und C++ übernommen hat. C++ hat die C-Syntax übernommen und fügt neue objektorientierte Konzepte hinzu. Mit gewissen Einschränkungen funktioniert also C-Code auch in C++.

In C++ gibt es Klassen (mit Methoden und Attributen), und zusätzlich gibt es Funktionen. Der Einsprungpunkt in ein Programm ist (analog zu Java) die Funktion `main()`, die ein `int` als Ergebnis zurückliefert. Dieser Integer kann vom Aufrufer ausgewertet werden, wobei der Wert 0 typischerweise als Erfolg interpretiert wird. Achtung: Das ist eine *Konvention*, d.h. es kann Programme geben, die andere Werte zurückliefern. Die Werte müssen dokumentiert werden.

Bevor der Compiler den Quelltext "sieht", wird dieser von einem Präprozessor bearbeitet. Dieser hat verschiedene Aufgaben, unter anderem das Einbinden anderer Dateien. Dabei wird ein `#include "dateiname"`

(sucht im aktuellen Ordner) bzw. `#include <dateiname>` (sucht im Standardverzeichnis) ersetzt durch den Inhalt der angegebenen Datei.

C++-Code muss kompiliert werden. Dabei entsteht ein ausführbares Programm. Mit Make kann man den Kompilervorgang über Regeln automatisieren (denken Sie an ANT in der Java-Welt, nur ohne XML). Eine Regel besteht aus einem Ziel (*Target*), einer Liste von Abhängigkeiten sowie einer Liste mit Aktionen (Anweisungen). Um ein Ziel zu “bauen” müssen zunächst alle Abhängigkeiten erfüllt sein (bzw. falls sie es nicht sind, erst noch “gebaut” werden - es muss entsprechend weitere Regeln geben, um diese Abhängigkeiten “bauen” zu können). Dann wird die Liste der Aktionen abgearbeitet. Ziele und Abhängigkeiten sind in der Regel Namen von Dateien, die existieren müssen bzw. über die Aktionen erzeugt werden sollen. Die Aktionen sind normale Befehlssequenzen, die man auch in einer Konsole eingeben könnte. Make berücksichtigt den Zeitstempel der Dateien: Ziele, die bereits existieren und deren Abhängigkeiten nicht neuer sind, werden nicht erneut gebaut.

Die gute Nachricht: In Bezug auf Variablen, Operatoren und Kontrollfluss verhalten sich C und C++ im Wesentlichen wie Java.

Es gibt in C++ den Typ `bool` mit den Werten `true` und `false`. Zusätzlich werden Integerwerte im boolschen Kontext (etwa in einer `if`-Abfrage) ausgewertet, wobei der Wert 0 einem `false` entspricht und alle anderen Integer-Werte einem `true`. (Dies steht etwas im Widerspruch zu den Werten, die in der `main`-Funktion per `return` zurückgeliefert werden: Hier bedeutet 0 in der Regel, dass alles OK war.)

Die Basisdatentypen sind (bis auf `char` und `bool`) in ihrer Größe maschinenabhängig. Es kann also sein, dass Code, der auf einem 64bit-Laptop ohne Probleme läuft, auf einem Raspberry PI Überläufe verursacht! Um besonders ressourcenschonend zu arbeiten, kann man die Speichergröße für einige Basisdatentypen durch die Typmodifikatoren `short` und `long` beeinflussen sowie die Interpretation von Zahlenwerten mit oder ohne Vorzeichen (`signed`, `unsigned`) einstellen.

Die Anzahl der für einen Typ oder eine Variable/Struktur benötigten Bytes bekommt man mit dem Operator `sizeof` heraus.

Mit `typedef` kann man einen neuen Namen für bereits existierende Typen vergeben.

In C++ gibt es Funktionen (analog zu Methoden in Java), diese existieren unabhängig von Klassen.

Wenn eine Funktion aufgerufen wird, muss dem Compiler die Signatur zur Prüfung bekannt sein. Das bedeutet, dass die Funktion entweder zuvor komplett definiert werden muss oder zumindest zuvor deklariert werden muss (die Definition kann auch später in der Datei kommen oder in einer anderen Datei). Das Vorab-Deklariere einer Funktion nennt man auch “Funktionsprototypen”.

Eine Deklaration darf (so lange sie konsistent ist) mehrfach vorkommen, eine Definition immer nur exakt einmal. Dabei werden alle Code-Teile, die zu einem Programm zusammencompiliert werden, gemeinsam betrachtet. => Das ist auch als **One-Definition-Rule** bekannt.

In C++ gilt beim Funktionsaufruf immer zunächst immer die Parameterübergabe per **call-by-value** (dito bei der Rückgabe von Werten). Wenn Referenzen oder Pointer eingesetzt werden, wird dagegen auch ein *call-by-reference* möglich. (Dazu später mehr.)

Unterscheidung in globale, lokale und lokale statische Variablen mit unterschiedlicher Lebensdauer und unterschiedlicher Initialisierung durch den Compiler.

Videos

- [VL Einführung in C++ \(Erinnerungen an C\)](#)

7.1.1. Warum?

- C++ erlaubt ressourcenschonende Programmierung
- Objektorientierter “Aufsatz” auf C
- Verbreitet bei hardwarenaher und/oder rechenintensiver Software

Sie werden C++ im Modul “Computergrafik” brauchen!

Geschichte

- 1971-73: Ritchie entwickelt die Sprache C
- Ab 1979: Entwicklung von C++ durch Bjarne Stroustrup bei AT&T
 - Erweiterung der prozeduralen Sprache C
 - Ursprünglich “C mit Klassen”, später “C++” (Inkrement-Operator)
- Bis heute: Fortlaufende Erweiterungen: alle 3 Jahre neuer Standard (C++11, C++14, ...)

Tip

C/C++ vs. Java

- Java: Fokus auf Sicherheit und Robustheit
 - Diverse Sicherheitschecks durch Compiler und VM (zb. Array-Zugriff)
 - Speicherverwaltung (Garbage Collection), kein Speicherzugriff über Pointer
 - Automatische Initialisierung von Variablen
- C/C++: Fokus auf Effizienz (Speicher, Laufzeit) für korrekte Programme
 - Vergleichsweise schwache Sicherheitschecks durch Compiler, keine VM (d.h. keine Prüfung von Array-Indizes u.a.)
 - Keine Garbage Collection, Programmierer hat direkten Zugriff auf Speicher
 - Keine automatische Initialisierung von Variablen

7.1.2. Hello World!

```
1  /*
2   * HelloWorld.cpp (g++ -Wall HelloWorld.cpp)
3   */
4
5  #include <cstdio>
```

```
6  #include <iostream>
7  #include <cstdlib>
8
9  using namespace std;
10
11 int main() {
12     printf("Hello World from C++ :-)\n");
13     cout << "Hello World from C++ :-)" << endl;
14     std::cout << "Hello World from C++ :-)" << std::endl;
15
16     return EXIT_SUCCESS;
17 }
```

7.1.2.1. Beobachtungen

Jedes (ausführbare) C++-Programm hat genau eine `main()`-Funktion. Die `main()`-Funktion ist *keine Methode* einer Klasse: In C/C++ gibt es Funktionen auch außerhalb von Klassen.

In C++ gibt es Namespaces (dazu später mehr). Die aus der Standardbibliothek importierten Funktionen sind in der Regel im Namespace `std` definiert. Mit `using namespace std;` können Sie auf die Elemente direkt zugreifen. Wenn Sie das `using namespace std;` weglassen, müssten Sie bei jeder Verwendung eines Symbols den Namensraum explizit dazu schreiben `std::cout << "Hello World from C++ :-)" << std::endl;`.

Sie können im C++-Code auch Funktionen aus C benutzen, d.h. Sie können für die Ausgabe beispielsweise `printf` nutzen (dazu müssen Sie den Header `<cstdio>` importieren). Die "richtige" Ausgabe in C++ ist aber die Nutzung des Ausgabestreams `cout` und des Ausgabeoperators `<<`. Das `endl` sorgt für einen zum jeweiligen Betriebssystem passenden Zeilenumbruch.

Der Rückgabewert signalisiert Erfolg bzw. Fehler der Programmausführung. Dabei steht der Wert 0 traditionell für Erfolg (Konvention!). Besser Makros nutzen: `EXIT_SUCCESS` bzw. `EXIT_FAILURE` (in `cstdlib`).

7.1.2.2. Präprozessor

Der Präprozessor transformiert den Quellcode *vor* dem Compiler-Lauf. Zu den wichtigsten Aufgaben gehören dabei die Makrosubstitution (`#define Makroname Ersatztext`) und das Einfügen von Header-Dateien (und anderen Dateien) per `#include`. Es gibt dabei zwei Formen, die an unterschiedlichen Orten nach der angegebenen Datei suchen:

- `#include "dateiname"` sucht im aktuellen Ordner
- `#include <dateiname>` sucht im Standardverzeichnis

Das `#include` kann wie in C genutzt werden, aber es gibt auch die Form ohne die Dateiendung ".h". Da es in C keine Funktionsüberladung gibt (in C++ dagegen schon), müssen die C-Header speziell markiert sein, um sie in C++ verwenden zu können. Für die Standard-Header ist dies bereits erledigt, Sie finden diese mit einem "c" vorangestellt:

- Include in C: `#include <stdio.h>`
- Include in C++: `#include <cstdio>`

7.1.2.3. Übersetzen, Linken, Ausführen

C++-Dateien werden üblicherweise mit der Endung **“.cpp”** oder **“.cxx”** oder **“.cc”** abgespeichert, Header-Dateien mit den Endungen **“.hpp”** oder **“.hxx”** oder **“.hh”**.

Zum Übersetzen und Linken in einem Arbeitsschritt rufen Sie den Compiler auf: `g++ HelloWorld.cpp` bzw. besser `g++ -Wall -o helloworld HelloWorld.cpp`. Die Option `-Wall` sorgt dafür, dass alle Warnungen aktiviert werden.

Ausführen können Sie das erzeugte Programm in der Konsole mit: `./helloworld`. Der aktuelle Ordner ist üblicherweise (aus Sicherheitsgründen) nicht im Suchpfad für ausführbare Dateien enthalten. Deshalb muss man explizit angeben, dass ein Programm im aktuellen Ordner (.) ausgeführt werden soll.

[Konsole: HelloWorld.cpp](#)

7.1.3. Variablen, Operatoren, Kontrollfluss

Im Wesentlichen wie von C und Java gewohnt ... :-)

- Wichtig(st)e Abweichung:

Im booleschen Kontext wird **int** als Wahrheitswert interpretiert:

Alle Werte ungleich 0 entsprechen **true** (!)

Anmerkung: Dies steht im Widerspruch zu den Werten, die in der `main`-Funktion per **return** zurückgeliefert werden: Hier bedeutet 0 in der Regel, dass alles OK war.

=> Vorsicht mit

```
1 int c;  
2 if (c=4) { ... }
```

7.1.4. Ein- und Ausgabe mit *printf* und *cin/cout*

- `printf(formatstring, ...)`

```
1 string foo = "fluppie";  
2 printf("hello world : %s\n", foo.c_str());
```

– Einbinden über `#include <stdio>`

– Format-String: Text und Formatierung der restlichen Parameter: `%[flags][width][.precision]conversion`

* `flags`: hängt von der konkreten Ausgabe ab

* `width`: Feldbreite

* `precision`: Anzahl der Dezimalstellen

* `conversion`: (Beispiele)

c	Zeichen (Char)
d	Integer (dezimal)
f	Gleitkommazahl

- Standardkanäle: `cin` (Standardeingabe), `cout` (Standardausgabe), `cerr` (Standardfehlerausgabe)
 - Genauer: `cout` ist ein Ausgabestrom, auf dem der Operator `<<` schreibt
 - Einbinden über `#include <iostream>`
 - Implementierung der Ein- und Ausgabeoperatoren (`>>`, `<<`) für Basistypen und Standardklassen vorhanden
 - Automatische Konvertierungen für Basistypen und Standardklassen

```
1 // Ausgabe, auch verkettet
2 string foo = "fluppie";
3 cout << "hello world : " << foo << endl;
4
5 // liest alle Ziffern bis zum ersten Nicht-Ziffernzeichen
6 // (fuehrende Whitespaces werden ignoriert!)
7 int zahl; cin >> zahl;
```

```
1 // Einzelne Zeichen (auch Whitespaces) lesen
2 char c; cin.get(c);
```

Beispiel: `cin.cpp`

7.1.5. Sichtbarkeit und Gültigkeit und Namespaces

Wie in Java:

- Namen sind nur nach Deklaration und innerhalb des Blockes, in dem sie deklariert wurden, gültig
- Namen sind auch gültig für innerhalb des Blockes neu angelegte innere Blöcke
- Namen in inneren Blöcken können Namen aus äußeren Scopes überdecken

Zusätzlich gibt es noch benannte Scopes und einen Scope-Operator.

- C++ enthält den Scope-Operator `::` => Zugriff auf global sichtbare Variablen

```
1 int a=1;
2 int main() {
3     int a = 10;
4     cout << "lokal: " << a << "global: " << ::a << endl;
5 }
```

- Alle Namen aus `XYZ` zugänglich machen: `using namespace XYZ;`

```
1 using namespace std;
2 cout << "Hello World" << endl;
```

- Alternativ gezielter Zugriff auf einzelne Namen: `XYZ::name`

```
1 std::cout << "Hello World" << std::endl;
```

- Namensraum `XYZ` deklarieren

```
1 namespace XYZ {
2     ...
3 }
```

Beispiel: `cppScope.cpp`

7.1.6. Arrays und Vektoren in C++

- Syntax: `Typ Name[AnzahlElemente];`

```
1 int myArray[100];
2 int myArray2[] = {1, 2, 3, 4};
```

- Compiler reserviert sofort Speicher auf dem Stack
=> **statisch**: im Programmlauf nicht änderbar
- Zugriff über den Indexoperator `[]`
- Achtung: "roher" Speicher, d.h. **keinerlei Methoden**
- Größe nachträglich bestimmen mit `sizeof`:

```
1 int myArray[100], i;
2 int cnt = sizeof(myArray)/sizeof(myArray[0]);
```

Guter Stil: Anzahl der Elemente als Konstante deklarieren: Statt `int myArray[100];` besser

```
1 #define LENGTH 100
2 int myArray[LENGTH];
```

- Vordefinierter Vektor-Datentyp `vector`

- Einbinden über `#include <vector>`
- Parametrisierter Datentyp (C++: Templates) - Nutzung analog wie in Java (Erstellung von Template-klassen und -methoden aber deutlich anders!)
- Anlegen eines neuen Arrays mit 10 Elementen für Integer:

```
1 vector<int> v(10);
2 vector<double> meinVektor = {1.1, 2.2, 3.3, 4.4};
3 meinVektor.push_back(5.5);
4 cout << meinVektor.size() << endl;
```

- Zugriff auf Elemente:

```
1 cout << v[0] << endl; // ohne Bereichspruefung!
2 cout << v.at(1000) << endl; // mit interner Bereichspruefung
```

- Zuweisung (mit Kopieren):

```
1 vector<double> andererVektor;
2 andererVektor = meinVektor;
```

- Dynamische Datenstruktur:

```
1 vector<int> meineDaten; // initiale Groesse: 0
2 meineDaten.push_back(123); // Wert anhaengen
3
4 meineDaten.pop_back(); // Wert loeschen
5 meineDaten.empty(); // leer?
```

Vorsicht! `vector<int> arr()`; ist **kein** Vektor der Länge 0, sondern deklariert eine **neue Funktion!**

7.1.7. Alias-Namen für Typen mit *typedef* und *using*

- Syntax: `typedef existTyp neuerName;` (C, C++)

```
1 typedef unsigned long uint32;
2 uint32 x, y, z;
```

Im Beispiel ist `uint32` ein neuer Name für den existierenden Typ `unsigned long`, d.h. die Variablen `x`, `y` und `z` sind `unsigned long`.

- Syntax: `using neuerName = existTyp;` (C++)

```
1 typedef unsigned long uint32; // C, C++
2 using uint32 = unsigned long; // C++11
3
4 typedef std::vector<int> foo; // C, C++
5 using foo = std::vector<int>; // C++11
6
7 typedef void (*fp)(int,double); // C, C++
8 using fp = void (*)(int,double); // C++11
```

Seit C++11 gibt es das Schlüsselwort `using` für Alias-Deklarationen (analog zu `typedef`). Dieses funktioniert im Gegensatz zu `typedef` auch für Templates mit (teilweise) gebundenen Template-Parametern.

7.1.8. Erinnerungen an C - Vergleich mit C++

7.1.8.1. Basisdatentypen

char	Zeichen (ASCII, 8 Bit bzw. 1 Byte)
int	Ganze Zahl (16, 32 oder 64 Bit)
float	Gleitkommazahl (typ. 32 Bit)
double	Doppelt genaue Gleitkommazahl (typ. 64 Bit)
void	Ohne/kein Wert
bool	true, false

Außerdem sind Arrays und Pointer mit diesen Typen möglich.

7.1.8.2. Typmodifikatoren ändern Bedeutung

Vorangestellte Modifikatoren ändern Bedeutung:

1. Länge im Speicher

short	Speicher: halbe Wortlänge
long	Speicher: doppelte/dreifache Wortlänge

2. Vorzeichen

signed	mit Vorzeichen (Default bei Zahlen)
unsigned	ohne Vorzeichen

Anwendung auf ganze Zahlen:

- **short** und **long** sind Synonyme für **short int** und **long int**
- **long long** ist typischerweise eine ganze Zahl mit 8 Byte
- **unsigned char** sind Zahlen von 0, ..., 255 (1 Byte)
- zusätzlich: **long double** (nur diese Form)

Sie können **short**, **long** und **long long** nur für ganze Zahlen (**int**) nutzen, mit der Ausnahme **long double**. Dagegen können **signed** und **unsigned** sowohl für **char** als auch für **int** benutzt werden.

vgl. en.wikipedia.org/wiki/C_data_types

7.1.8.3. Größe eines Datentyps ist maschinenabhängig

Der reservierte Speicherbereich und damit auch der Zahlenbereich für einen einfachen Typ in C/C++ ist maschinenabhängig!

- Zahlenbereiche für konkrete Implementierung in Header-Files definiert

`limits.h` und `float.h`: Konstanten `INT_MAX`, `INT_MIN`, ...

- Alternativ Herausfinden der Größe in Bytes: Operator `sizeof`

Syntax: `sizeof(Typ)`

Es gilt in C/C++:

- `sizeof(unsigned char) = 1`
- `sizeof(short int) = 2`
- `sizeof(short int) ≤ sizeof(int) ≤ sizeof(long int)`
- `sizeof(float) ≤ sizeof(double) ≤ sizeof(long double)`

Hinweis Arrays: `sizeof` gibt immer die Anzahl der Bytes für einen Typ oder eine Variable zurück. Bei Array ist das nicht unbedingt die Anzahl der Elemente im Array!

Beispiel:

```
1 char a[10];
2 double b[10];
```

`sizeof(a)` würde den Wert 10 als Ergebnis liefern, da ein `char` in C/C++ immer exakt ein Byte benötigt und entsprechend 10 `char` 10 Byte. `sizeof(b)` ist maschinenabhängig und liefert die Anzahl der Bytes, die man für die Darstellung von 10 Double-Werten benötigt.

Wenn man die Anzahl der Elemente im Array mit `sizeof` herausfinden will, muss man den Gesamtwert für das Array noch durch den Speicherbedarf eines Elements teilen, also beispielsweise `sizeof(b) / sizeof(b[0])`.

7.1.8.4. (Beispiele für) Schleifen und Kontrollstrukturen in C/C++

```
1 int x=5, y=1;
2
3
4 if (x>5) {
5     x++;
6 } else if (y<=1) {
7     y = y-x;
8 } else {
9     y = 2*x;
10 }
11
12
13 while (y>0) {
14     y--;
15 }
16
17
18 for (x=0; x<10; x++) {
19     y = y*y;
20 }
```

7.1.8.5. Funktionen in C und C++

- Funktionen sind mit Methoden in Java vergleichbar
=> sind aber unabhängig von Klassen bzw. Objekten
- Syntax:

```
1 Rueckgabetyt Funktionsname(Parameterliste) {  
2     Anweisungen (Implementierung)  
3 }
```

- Aufruf: Nennung des Namens (mit Argumenten) im Programmcode

```
1 int x = foo(42);
```

Anmerkung: Unterschied "Parameter" und "Argument":

- Funktion hat "Parameter" in ihrer Parameterliste, auch "formale Parameter" genannt
- Beim Aufruf werden "Argumente" übergeben, auch "aktuelle Parameter" genannt

In der Praxis verwendet man beide Begriffe i.d.R. synonym.

Funktionen: Deklaration vs. Definition

- **Deklaration:** (Funktions-) Prototyp: Festlegen von Signatur (d.h. Funktionsname und Anzahl, Typ, Reihenfolge der Parameter) u. Rückgabetyt

```
1 void machWas(int, int);
```

- **Definition:** Implementierung der Funktion

```
1 void machWas(int a, int b) {  
2     cout << "a: " << a << ", b: " << b << endl;  
3 }
```

- Compiler "liest" Quellcode von oben nach unten
- Funktionen müssen (wie alle anderen Symbole auch) vor ihrer Verwendung zumindest deklariert sein, d.h. es muss zumindest ihre Signatur bekannt sein (siehe nächste Folie)
- Deklaration: Variablenamen können weggelassen werden

Tip

Deklaration vs. Definition

- **Deklaration:** Macht einen Namen bekannt und legt den Typ der Variablen bzw. die Schnittstelle der Funktionen fest.
- **Definition:** Deklaration plus Reservierung von Speicherplatz für die Variable oder Implementierung einer Funktion/Struktur/...

[Konsole: simplefunction.cpp](#)

One Definition Rule (für Funktionen)

Jede Funktion darf im **gesamten** Programm nur **einmal definiert** sein!

Funktionen und Parameter

- Funktionen “ohne” Parameter:

Leere Parameter-Liste¹ oder Schlüsselwort **void**

```
1 void fkt();  
2 void fkt(void);
```

- Funktionen **mit** Parameter:

- Deklaration: Variablennamen können weggelassen werden
- Definition: Variablennamen müssen angegeben werden

```
1 void fkt(int, char);  
2 void fkt(int a, char b);  
3  
4 void fkt(int a, char b) { ... }
```

Leere Parameterliste in C

Wenn eine Funktion keine Parameter hat, können Sie wie in C die Parameterliste entweder einfach leer lassen (**int fkt()**;) oder das Schlüsselwort **void** nutzen (**int fkt(void)**;).

Betrachten Sie folgendes Beispiel:

```
1 // Legal in C  
2 int wuppie(); // Deklaration: "Ich verrate Dir nicht,  
   wieviele Parameter wuppie() hat."  
3 int wuppie(int x) { return x; } // Aufruf mit Argumenten => ist okay  
4  
5 // Fehler in C  
6 int fluppie(void); // Deklaration: fluppie() hat KEINE Parameter!  
7 int fluppie(int x) { return x; } // Aufruf mit Argumenten => Compiler-Fehler
```

Wenn Sie eine mit leerer Parameterliste deklarierte Funktion definieren bzw. aufrufen, akzeptiert der C-Compiler dennoch **alle** übergebenen Parameter. Dies kann zu schwer verständlichen Fehlern führen! Sobald eine Funktion explizit mit dem Schlüsselwort **void** in der Parameterliste deklariert wird, **muss** diese dann auch ohne Parameter aufgerufen werden.

=> **Bevorzugen Sie in C die Variante mit dem Schlüsselwort void!**

¹Achtung: C-Compiler akzeptiert **alle** Parameter!

Leere Parameterliste in C++

Keine Parameter: Leere Liste und Schlüsselwort **void** gleichwertig

```
1 void fkt();  
2 void fkt(void);
```

Defaultparameter in C++

- Parameter mit Defaultwerten am **Ende** der Parameterliste
- Bei Trennung von Deklaration und Definition: Defaultparameter **nur** in Deklaration

```
1 // Deklaration  
2 void f(int i, int j=1, int k=2);  
3  
4 // Definition  
5 void f(int i, int j, int k) { ... }
```

Überladen von Funktionen

- Funktionen im **gleichen Gültigkeitsbereich** können überladen werden
- Zu beachten:
 1. Funktionsname identisch
 2. Signatur (Anzahl, Typen der Parameter) muss unterschiedlich sein
 3. Rückgabewert darf variieren

=> Warnung: Überladene Funktionen sollten gleichartige Operationen für unterschiedliche Datentypen bereitstellen!

Probleme beim Überladen von Funktionen

1. Defaultparameter

```
1 int maximum(int, int);  
2 int maximum(int, int, int=10);
```

2. Identische Signatur, Unterschied nur im Rückgabewert

```
1 int maximum(int, int);  
2 double maximum(int, int);
```

3. Überladen nur für Funktionen des selben Gültigkeitsbereichs!

```
1 #include <iostream>  
2 using namespace std;  
3  
4 void f(char c) {  
5     cout << "f(char): " << c << endl;  
6 }  
7 void f(int i) {
```

```

8      cout << "f(int): " << i << endl;
9  }
10
11
12  int main() {
13      void f(int i); // f(char) nicht mehr sichtbar!
14      f('a');
15
16      return 0;
17  }

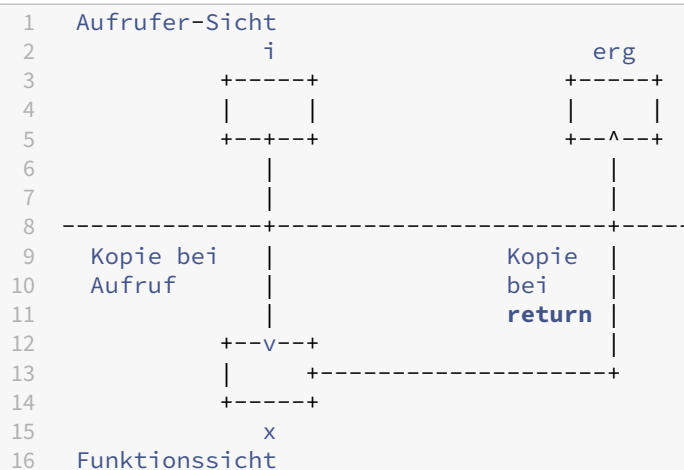
```

Parameterübergabe in C/C++: Call-by-Value

```

1  int add_5(int x) {
2      x += 5;
3      return x;
4  }
5
6  int main() {
7      int erg, i=0;
8      erg = add_5(i);
9  }

```



- Default in C/C++ ist die call-by-value Semantik:
 - Argumente werden bei Übergabe kopiert
 - Ergebniswerte werden bei Rückgabe kopiert
- Folgen:
 - Keine Seiteneffekte durch Verändern von übergebenen Strukturen
 - Negative Auswirkungen auf Laufzeit bei großen Daten

Ausnahme: Übergabe von C++-Referenzen oder Pointern (wobei Pointer streng genommen auch kopiert werden, also per call-by-value übergeben werden ...)

7.1.8.6. Unterschiedliche Variablenarten

Lokale Variablen (“automatische Variablen”)

```
1  int b = 1;
2
3  void f() {
4      int b = 42;
5  }
6
7  int main() {
8      int b = 3;
9
10     {
11         int b = 7;
12     }
13 }
```

- Innerhalb einer Funktion (oder Blockes) definierte Variablen
- Gilt auch für Variablen aus Parameterliste
- Überdecken **globale Variablen** gleichen Namens
- Sichtbarkeit:
 - Außerhalb der Funktion/Blockes nicht zugreifbar
 - Beim Betreten der Funktion Reservierung von Speicherplatz für lokale Variablen
 - Dieser wird beim Verlassen des Blockes/Funktion automatisch wieder freigegeben
 - Namen sind nur nach Deklaration und innerhalb des Blockes, in dem sie deklariert wurden, gültig
 - Namen sind auch gültig für innerhalb des Blockes neu angelegte innere Blöcke

Software Engineering: Vermeiden Sie lokale Namen, die Namen aus einem äußeren Scope überdecken!

=> Werden auch als **automatische Variablen** bezeichnet

Globale Variablen (“externe Variablen”)

```
1  /* ===== Datei main.cpp (einzeln kompilierbar) ===== */
2  int main() {
3      extern int global;  // Deklaration
4  }
5
6  int global;            // Definition
```

```
1  /* ===== Datei foo.cpp (einzeln kompilierbar) ===== */
2  extern int global;    // Deklaration
3
4  void foo() {
5      global = 45;
6  }
```

- Globale Variablen: Außerhalb **jeder** Funktion definierte Variablen
- Globale Variablen gelten in allen Teilen des Programms

- Auch in anderen Dateien! => müssen bei *Nutzung* in Funktionen als `extern` deklariert werden
- Existieren die **gesamte** Programmlebensdauer über

=> Werden auch als **externe Variablen** bezeichnet

Die Dateien sind einzeln kompilierbar (`extern` sagt dem Compiler, dass die Variable woanders definiert ist) => erst der Linker löst das auf.

Hinweis: Bei globalen Konstanten in C++ brauchen Sie zusätzlich auch bei der Definition ein `extern`, da die Konstante sonst nur in ihrer Datei sichtbar ist.

Statische lokale Variablen

```
1 void foo() {
2     static int x = 42;
3     x++;
4 }
5
6 int main() {
7     foo(); foo(); foo();
8 }
```

- Lokale Variablen mit “Gedächtnis”: Definition mit dem vorangestellten Schlüsselwort “static”

```
1 static int callCount;
```

- Eigenschaften:
 - Wert bleibt für die folgenden Funktionsaufrufe erhalten
 - Wert kann in der Funktion verändert werden
 - Dennoch: lokale Variable, d.h. von außen nicht sichtbar/gültig

Hinweis: `static` für globale Variablen bedeutet etwas anderes! (s.u. “Sichtbarkeit”)

Initialisierung von Variablen

(Automatische) Initialisierung von Variablen hängt von ihrer Speicherklasse ab!

- **Automatisch**
 - Werden nicht automatisch initialisiert (!)
 - Bei vorgegebenem Wert ab Aufruf der Funktion
- **Extern**
 - Mit dem Wert 0 oder vorgegebenem Wert
 - Bereits vor Programmstart (im Code enthalten)
- **Statisch**
 - Mit dem Wert 0 oder vorgegebenem Wert
 - Ab erstem Aufruf der Funktion

Sichtbarkeit globaler Variablen (und Funktionen) beschränken

- Beschränkung der Gültigkeit von **globalen Variablen** auf die Datei, wo sie definiert sind: **Schlüsselwort `static`**
 - werden (weiterhin) automatisch mit 0 initialisiert
 - sind nun nur in der Datei sichtbar/gültig, wo sie definiert sind
 - dient zur Vermeidung von Namenskonflikten bei globalen Variablen
- Sichtbarkeitsbeschränkung gilt auch für **Funktionen**

static für globale Variablen beschränkt deren Sichtbarkeit auf die Datei, wo sie definiert sind. D.h. man kann diese dann nicht in einer anderen Datei nutzen, nicht mal mit `extern`...

static für Funktionen beschränkt deren Sichtbarkeit ebenfalls auf die Datei, wo sie definiert sind. Man kann sie dann nur in anderen Funktionen, die ebenfalls in der selben Datei definiert werden, nutzen. In anderen Dateien sind die **static** Funktionen *nicht* sichtbar. D.h. es macht auch keinen Sinn, sie in einer Header-Datei zu deklarieren! (In der Praxis liefert der gcc dann sogar einen Fehler!). Das ist mit **private** Methoden vergleichbar.

7.1.8.7. Globale Konstanten

In C funktionieren globale Konstanten wie globale Variablen

- **Definition** in einer Übersetzungseinheit ohne `extern`
 - => Definition als `extern` wird in C mit einer Warnung quittiert!
- Nutzung in anderen Übersetzungseinheiten durch (erneute) **Deklaration** als `extern`
- Beispiel:

```
1 /* ===== Datei main.c ===== */
2 const int PI=123;           // Definition OHNE "extern" (C)
3
4 int main() {
5     fkt_a1();
6     int x = PI;
7     ...
8 }
```

```
1 /* ===== Datei a.c ===== */
2 extern const int PI;        // (erneute) Deklaration mit "extern"
3 void fkt_a1() {
4     int x = PI;
5     ...
6 }
```

In C++ sind globale Konstanten per Default nur in ihrer Definitionsdatei sichtbar!

- Abhilfe: Definieren *und* Deklarieren mit `extern`
- Beispiel:

```

1  /* ===== Datei main.cpp ===== */
2  extern const int PI=123;    // Definition MIT "extern" (C++)
3
4  int main() {
5      fkt_a1();
6      int x = PI;
7      ...
8  }

```

```

1  /* ===== Datei a.cpp ===== */
2  extern const int PI;        // (erneute) Deklaration mit "extern"
3  void fkt_a1() {
4      int x = PI;
5      ...
6  }

```

Alternativ: In beiden Sprachen Konstanten vorwärts deklarieren

Folgende Definition und (Vorwärts-) Deklaration der Konstanten `PI` funktioniert sowohl in C als auch in C++:

```

1  /* ===== Datei main.c ===== */
2  extern const int PI;        // (Vorwärts-) Deklaration mit "extern"
3  const int PI=123;          // Definition OHNE "extern"
4
5  int main() {
6      fkt_a1();
7      int x = PI;
8      ...
9  }

```

```

1  /* ===== Datei a.c ===== */
2  extern const int PI;        // (erneute) Deklaration mit "extern"
3  void fkt_a1() {
4      int x = PI;
5      ...
6  }

```

7.1.8.8. Automatisieren der Buildvorgänge: GNU Make

Makefile: Textdatei mit Regeln für das Programm make

- Abläufe automatisieren: Kompilieren, testen, Pakete bauen, aufräumen, ...
- Java: `ant`, C/C++: `make`
- Achtung: Verschiedene Make-Dialekte! Wir nutzen *GNU Make*!

```

1  # Kommentar
2  Ziel1: AbhaengigkeitenListe1
3      Aktionen1
4
5  Ziel2: AbhaengigkeitenListe2
6      Aktionen2

```

```

7
8 # ... und so weiter :-)
9 # ACHTUNG:
10 # Vor den Aktionen <TAB> benutzen, keine Leerzeichen!!!
11 # Vorsicht mit Editor-Einstellungen!

```

Bedeutung: Um das Ziel `Ziell` zu erzeugen, müssen alle Abhängigkeiten der Liste `AbhaengigkeitenListel` erfüllt sein. Dann werden die Aktionen in `Aktionen1` durchgeführt, um `Ziell` zu erzeugen. Aber nur, falls das Ziel `Ziell` nicht existiert oder veraltet ist!

Falls die Abhängigkeiten nicht erfüllt sind, wird nach Regeln gesucht, um diese zu erzeugen. Das bedeutet, dass u.U. zunächst weitere Targets “gebaut” werden, bevor die Aktionenliste ausgeführt wird.

Die Ziele und Abhängigkeiten sind i.d.R. Dateien (müssen es aber nicht sein).

Makefiles: Fiktives Beispiel

- Annahme: Projekt besteht aus der Datei `main.cpp`, daraus soll das Programm “tollesProgramm” erzeugt werden
- Passendes Makefile:

```

1 CXXFLAGS = -Wall
2
3 .PHONY: all
4 all: tollesProgramm
5
6 tollesProgramm: main.o
7     $(CXX) $(LDFLAGS) $< $(LDLIBS) -o $@
8
9 %.o: %.cpp
10    $(CXX) $(CXXFLAGS) -c $< -o $@
11
12 .PHONY: clean
13 clean:
14     rm -rf tollesProgramm *.o *.~

```

Bedeutung: Um das Ziel `all` zu erzeugen, muss die Abhängigkeit `tollesProgramm` erfüllt sein. Beachten Sie, dass im Beispiel `all` kein Dateiname ist, `tollesProgramm` dagegen schon.

Um `tollesProgramm` zu erzeugen, muss die Datei `main.o` vorhanden sein. Falls sie es nicht ist, wird sie mit Hilfe des dritten Targets erzeugt. Das `%` ist dabei ein Patternmatcher, d.h. wenn nach einem `main.o` gesucht ist, matcht `%.o` (das `%` bindet sich dabei an “main”) und auf der rechten Seite des Targets steht als Abhängigkeit `main.cpp`.

Die Variablen `CXX`, `CXXFLAGS`, `LDFLAGS` und `LDLIBS` sind vordefinierte Variablen:

- `CXX`: C++-Compiler, Default: `g++`
- `CXXFLAGS`: Extra Flags für den C++-Compiler (nur für Kompilieren)
- `LDFLAGS`: Extra Flags, die für das Linken genutzt werden (Beispiel: `-L.`; **nicht** `-lm`)
- `LDLIBS`: Bibliotheken, die für das Linken genutzt werden (Beispiel: `-lm -lfoo`; **nicht** `-L.`)

Die Variablen `$<`, `^` und `$@` lösen auf das Ziel bzw. die Abhängigkeiten eines Targets auf:

- `$<=>` gibt die erste Abhängigkeit an
- `$^=>` gibt alle Abhängigkeiten an
- `$@=>` gibt das Ziel an

Falls die Datei `tollesProgramm` nicht existiert oder aber älter ist als `main.o`, wird die Regel des Targets `tollesProgramm` ausgeführt, um die Datei `tollesProgramm` zu erzeugen: `g++ main.o -o tollesProgramm`.

Hinweis: Das Beispiel entspricht den minimalen Kenntnissen, die Sie über Make haben müssen.

Makefiles: Typische Aufrufe

- `make`
Sucht nach Datei mit dem Namen “GNUmakefile”, “makefile” oder “Makefile” und erzeugt das erste Ziel in der Datei

Konvention: Das erste Ziel hat den Namen `all`
- `make -f <datei>`
Sucht die Datei mit dem angegebenen Namen, erzeugt das erste Ziel in der Datei
- `make -f <datei> <ziel>`
Sucht die Datei mit dem angegebenen Namen, erzeugt das Ziel `<ziel>`
- `make <ziel>`
Sucht nach Datei mit dem Namen “GNUmakefile”, “makefile” oder “Makefile” und erzeugt das Ziel `<ziel>`

7.1.9. Wrap-Up

- C/C++ sind enge Verwandte: kompilierte Sprachen, C++ fügt OO hinzu
- Funktionsweise einfachster Make-Files
- Wichtigste Unterschiede zu Java
 - Kontrollfluss wie in Java
 - Basisdatentypen vorhanden
 - Typ-Modifikatoren zur Steuerung des Speicherbedarfs/Wertebereich
 - Integer können im booleschen Kontext ausgewertet werden
 - Operator `sizeof` zur Bestimmung des Speicherbedarfs
 - Alias-Namen für existierende Typen mit `typedef` definierbar
 - Funktionen mit Default-Parametern und Überladung

Zum Nachlesen

- Breyman (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k1: Ich kenne den Zusammenhang zwischen C und C++
- k1: Ich kenne die wichtigsten Unterschiede und Gemeinsamkeiten zu Java
- k1: Ich kenne die wichtigsten Aufgaben des Präprozessors
- k3: Ich kann einfache C++-Programme schreiben, kompilieren und starten
- k3: Ich kann die Standard-Ein-/Ausgabe-Kanäle in C++ sowie die Operatoren >> und << einsetzen
- k3: Ich kann die Basisdatentypen einschließlich der Modifikatoren nutzen
- k3: Ich kann Variablen deklarieren und Kontrollstrukturen und Operatoren einsetzen
- k2: Ich habe verstanden, dass Integerwerte im booleschen Kontext ausgewertet werden (können)
- k3: Ich kann den Scope-Operator :: und Namensräume einsetzen
- k3: Ich kann `sizeof` zur Bestimmung des Speicherbedarfs einsetzen
- k3: Ich kann `typedef` zur Definition neuer Typen (Aliase bestehender Typen) einsetzen
- k3: Ich kann einfache Makefiles schreiben, um das Übersetzen und Starten von Programmen mit von GNU Make zu automatisieren
- k2: Ich kann den Unterschied zwischen Deklaration und Definition an einem Beispiel erklären und dabei auf One Definition Rule eingehen
- k2: Ich kann die Problematik bei der Deklaration parameterloser Funktionen erklären
- k2: Ich verstehe die Call-by-Value-Semantik bei der Parameterübergabe
- k2: Ich kann die unterschiedliche Sichtbarkeit und Initialisierung von Variablen an einem Beispiel erklären
- k3: Ich kann Funktionen deklarieren und definieren und aufrufen
- k3: Ich kann lokale und globale und lokale statische Variablen nutzen

Challenges

- Wie groß ist der Bereich der Basisdatentypen (Speicherbedarf, Zahlenbereich)? Wie können Sie das feststellen?

```
1 unsigned char a;  
2 int b;  
3  
4 long long x[10];  
5 long long y[] = {1, 2, 3};  
6 long long z[7] = {3};
```

- Erklären Sie den Unterschied `sizeof(x)` vs. `sizeof(x)/sizeof(x[0])!`
- Warum ist der folgende Code-Schnipsel gefährlich?

```
1 if (i=3)  
2     printf("Vorsicht");  
3 else  
4     printf("Vorsicht (auch hier)");
```

- Limits kennen: Datentypen, Wertebereiche

Schreiben Sie ein C-Programm, welches die größtmögliche **unsigned int** Zahl auf Ihrem System berechnet.

Verwenden Sie hierzu **nicht** die Kenntnis der systemintern verwendeten Bytes (`sizeof, ...`). Nutzen Sie auch nicht die Konstanten/Makros/Funktionen aus `limits.h` oder `float.h` oder anderen Headerdateien!

- Erklären Sie die Probleme bei folgendem Code-Schnipsel:

```
1 int maximum(int, int);
2 double maximum(int, int);
3 char maximum(int, int, int=10);
```

- Erklären Sie die Probleme bei folgendem Code-Schnipsel:

```
1 int maximum(int, int);
2 double maximum(double, double);
3
4 int main() {
5     cout << maximum(1, 2.2) << endl;
6 }
```

- Erklären Sie den Unterschied zwischen

```
1 int a=1;
2 int main() {
3     extern int a;
4     return 0;
5 }
```

und

```
1 int a=1;
2 int main() {
3     int a = 4;
4     return 0;
5 }
```

7.2. C++: Pointer und Referenzen

TL;DR

Es gibt viele Arten Speicher, die sich vor allem in der Größe und Geschwindigkeit unterscheiden (Cache, RAM, SSD, Festplatte, ...). Der Kernel stellt jedem Prozess einen linearen Adressraum bereit und abstrahiert dabei von den darunter liegenden physikalischen Speichermedien (es gibt eine Abbildung auf die jeweiligen Speichermedien durch die MMU, dies ist aber nicht Bestandteil dieses Kurses).

Den virtuellen Speicher kann man grob in drei Segmente aufteilen: Text (hier befindet sich der Programmcode des Prozesses), Stack (automatische Verwaltung, für Funktionsaufrufe und lokale Variablen) und Heap

(Verwaltung durch den Programmierer, dynamische Bereitstellung von Speicher während der Laufzeit des Programms).

Pointer sind Variablen, deren **Wert als Adresse** (im virtuellen Speicher) interpretiert wird. Pointer können auf andere Objekte bzw. Variablen zeigen: Der Adressoperator "&" liefert die Adresse eines Objekts im virtuellen Speicher, diese kann einem Pointer zugewiesen werden (der Wert des Pointers ist dann die zugewiesene Adresse). Pointer können mit "*" dereferenziert werden, d.h. es wird an der Speicherstelle im virtuellen Speicher nachgeschaut, deren Adresse im Pointer gespeichert ist. Dadurch erfolgt der Zugriff auf das verwiesene Objekt. (Dies hat noch nichts mit *dynamischer Speicherverwaltung* zu tun!) Die Deklaration eines Pointers erfolgt mit einem * zwischen Typ und Pointername: `int *p;`. Da Pointer normale Variablen sind, unterliegen Pointer-Variablen den üblichen Gültigkeitsbedingungen (Scopes).

In C++ gibt es zusätzlich **Referenzen**. Diese stellen Alias-Namen für ein Objekt (oder eine Variable) dar, d.h. ein Zugriff auf eine Referenz bewirkt den direkten Zugriff auf das verbundene Objekt. Referenzen müssen bei der Deklaration initialisiert werden (`Typ &ref = obj;`) und sind dann fest mit diesem Objekt verbunden.

In C und C++ werden Funktionsparameter immer per Call-by-Value übergeben: Der Wert des Arguments wird in die lokale Variable des Funktionsparameters kopiert. Wenn ein Pointer übergeben wird, wird entsprechend der Wert des Pointers kopiert, also die gespeicherte Adresse. Mit der Adresse eines Objekts kann man aber auch in der Funktion direkt auf dieses Objekt zugreifen und dieses auslesen und verändern, d.h. durch die Übergabe eines Pointers hat man zwar immer noch Call-by-Value (die Adresse wird kopiert), die Wirkung ist aber wie bei Call-by-Reference (also als ob eine Referenz auf das Objekt übergeben wurde). Bei der Verwendung von C++-Referenzen hat man dagegen echtes Call-by-Reference.

Zur Laufzeit kann man Speicher auf dem Heap reservieren (*allozieren*). Im Gegensatz zu Speicher auf dem Stack ist man selbst auch für die Freigabe des reservierten Speichers zuständig - wenn man dies nicht beachtet, läuft irgendwann der Heap voll. Allokation und Freigabe kann entweder mit den C-Funktionen `malloc` und `free` erfolgen oder mit den C++-Operatoren `new` und `delete`. Mischen Sie niemals nie `malloc()/free()` mit `new/delete`!

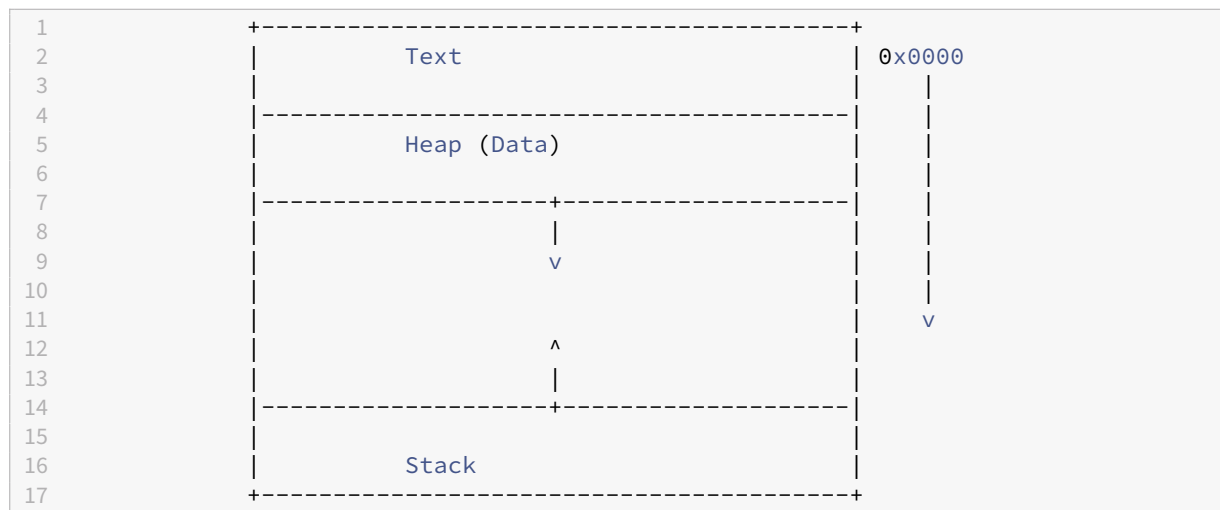
Zwischen Pointern und Arrays gibt es eine enge Verwandtschaft. Die einzelnen Elemente eines Arrays werden vom Compiler direkt aufeinanderfolgend im Speicher angeordnet, der Array-Name ist wie ein (konstanter) Pointer auf das erste Element. Tatsächlich übersetzt der Compiler Indexzugriffe für ein Array in die passende Pointerdereferenzierung: `a[i]` wird zu `*(a+i)`. Ein Pointer kann wiederum auch auf das erste Element eines zusammenhängenden Speicherbereichs zeigen, etwa wenn man über `malloc` Speicherplatz für mehrere Elemente anfordert. Da der Compiler aus einem Indexzugriff ohnehin die Pointerdereferenzierung macht, könnte man so einen Pointer auch per Indexzugriff abfragen. Dies ist aber gefährlich: Es funktioniert auch, wenn der Pointer nur auf *ein* anderes Objekt zeigt und nicht auf einen Speicherbereich... Ein Arrayname wird vom Compiler fest der ersten Speicheradresse des Arrays zugeordnet und kann nicht verändert werden, der Inhalt eines (nicht-konstanten) Pointer dagegen schon (der Pointer selbst wird auch fest im Speicher angelegt).

Pointer haben einen Typ: Die Pointerarithmetik berücksichtigt die Speicherbreite des Typs! Damit springt man mit `ptr+1` automatisch zum nächsten Objekt und nicht notwendigerweise zum nächsten Byte.

Videos

- [C++: Pointer und Referenzen](#)

7.2.1. Virtueller Speicher



- Kernel weist jedem Prozess seinen eigenen virtuellen Speicher zu
- Linearer Adressbereich, beginnend mit Adresse 0 bis zu einer maximalen Adresse
- Verwaltung durch MMU (*Memory Management Unit*)
 - MMU bildet logische Adressen aus virtuellem Speicher auf den physikalischen Speicher ab
 - Transparent für den Prozess

7.2.1.1. Segmente des virtuellen Speichers: Text (read-only)

- Programm Code
- Konstanten, String Literale

zusätzlich (nicht in Abbildung dargestellt):

- Bereich initialisierter Daten (globale und static Variablen (explizit initialisiert))
- Bereich uninitialisierter Daten (globale und static Variablen (uninitialisiert) => Wert 0)

7.2.1.2. Segmente des virtuellen Speichers: Stack

- Dynamisch wachsend und schrumpfend
- Stackframe je Funktionsaufruf:
 - Lokale Variablen ("automatische" Variablen)
 - Argumente und Return-Werte

- **Automatische Pflege**

- Nach Funktionsrückkehr wird der Stackpointer ("Top of Stack") weiter gesetzt
- Dadurch "Bereinigung": Speicher der lokalen Variablen wird freigegeben

7.2.1.3. Segmente des virtuellen Speichers: Data (Heap)

- Dynamisch wachsend und schrumpfend
- Bereich für dynamischen Speicher (Allokation während der Laufzeit)
- Zugriff und Verwaltung aus laufendem Programm => **Pointer**
 - `malloc()/calloc()/free()` (C)
 - `new/delete` (C++)
 - typischerweise **Pointer**
- **KEINE automatische Pflege - Programmierer ist selbst verantwortlich!**

7.2.2. Konzept eines Pointers

```

1  int i = 99;
2  int *iptr;
3
4  iptr = &i; /* Wert von iptr ist gleich Adresse von i */
5  *iptr = 2; /* Deferenzierung von iptr => Veränderung von i */

```

	Variable	Speicheradresse	Inhalt	
1				
2				
3				
4			+-----+	
5	i	10125	99	<---+
6			+-----+	
7				
8				
9				
10			+-----+	
11	iptr	27890	10125	---+
12			+-----+	
13				

7.2.2.1. Pointer sind Variablen

- haben Namen und Wert
- können mit Operatoren verändert werden
- sind einer Speicheradresse im virtuellen Speicher zugeordnet

Im Beispiel:

- Variable `i`:

- Name: "i"
- Wert: 99
- Speicherzelle (Adresse): 10125
- Variable `iptr`:
 - Name: "iptr"
 - Wert: 10125
 - Speicherzelle (Adresse): 27890

7.2.2.2. Pointer sind besondere Variablen

Der Wert eines Pointers wird als **Adresse** im Speicher behandelt

Der Wert von `iptr` ist nicht ein beliebiger Integer, sondern eine Adresse. In diesem Fall handelt es sich um die Adresse im virtuellen Speicher, wo die Variable `i` abgelegt ist.

Wirkung/Interpretation: Variable `iptr` "zeigt" auf die Adresse von Variable `i`.

7.2.2.3. Pointer und Adressen (Syntax)

- Deklaration

```
1 Typ * Name;
```

- Zuweisung einer Adresse über den &-Operator:

```
1 int i = 99;  
2 int *iptr;  
3  
4 iptr = &i; /* Wert von iptr ist gleich Adresse von i */
```

- `iptr` ist ein Pointer auf eine (beliebige) Speicherzelle mit Inhalt vom Typ `int`
- Nach Zuweisung: `iptr` ist ein Pointer auf die Speicherzelle der Variablen `i`

7.2.2.4. Dereferenzierung: Zugriff auf Ziel

- Dereferenzierung mit `*`:

```
1 int i = 99;  
2 int *iptr;  
3  
4 iptr = &i;  
5  
6 *iptr = 2; // Zugriff auf verwiesene Speicherzelle i
```

7.2.2.5. Pointer: Schreibweisen

- Position des * zwischen Typ und Name beliebig

```
1 /* äquivalente Schreibweisen */
2 int* iptr;
3 int * iptr;
4 int *iptr;
```

```
1 /* Vorsicht Mehrfachdeklaration */
2 int* iptr, ptr2;      /* ptr2 ist nur ein int! */
```

- Dereferenzierung von Pointern auf Klassen/Structs: Operator ->

```
1 /* äquivalente Schreibweisen */
2 (*iptr).attribut;
3 iptr->attribut;
```

7.2.2.6. Pointer: Zuweisungen an andere Pointer

```
1 int i=99, *iptr, *ptr2;
2
3 iptr = &i;
4
5 ptr2 = iptr;
6
7 *ptr2 = 2;
```

Jetzt zeigen zwei Pointer auf die Speicherzelle von Variable `i`: `iptr` (wegen `iptr = &i`), und weil der Wert von `iptr` in `ptr2` kopiert wurde (`ptr2 = iptr`), zeigt nun auch `ptr2` auf `i`.

Der Wert von `iptr` ist die Adresse von `i`. Wenn dieser Wert kopiert oder zugewiesen wird, ändert sich an dieser Adresse nichts. `ptr2` bekommt diesen Wert zugewiesen, d.h. bei einer Dereferenzierung von `ptr2` würde auf die Adresse von `i` zugegriffen werden und dort gelesen/geschrieben werden.

7.2.2.7. Pointer und Scopes

Nicht auf Variablen außerhalb ihres Scopes zugreifen!

```
1 int i=9;
2 int *ip = &i;
3
4 *ip = 8;
5 { /* neuer Block */
6     int j=7;
7     ip = &j;
8 }
9 *ip = 5; /* AUTSCH!!! */
```

```
1 int* murks() {  
2     int i=99;  
3     return &i; /* AUTSCH!!! */  
4 }
```

7.2.2.8. Hotelzimmer-Analogie

- Wenn Sie in ein Hotel einchecken, bekommen Sie den Schlüssel zu **Ihrem** Zimmer
 - *Pointer* == Schlüssel
 - *Variable auf die Pointer zeigt* == Zimmer
- Wenn Sie auschecken, geben Sie normalerweise Ihr Zimmer auf und den Schlüssel ab
 - Pointer wird ungültig
 - Variable wird ungültig
- Wenn Sie beim Auschecken den Schlüssel nicht abgeben, gehört das Zimmer dennoch nicht mehr Ihnen
 - Sie haben noch den Pointer
 - Die Variable, auf die der Pointer zeigt, ist ungültig
- Wenn Sie jetzt auf das Zimmer gehen, kommen Sie (evtl.) noch rein
 - Evtl. ist das Zimmer noch nicht wieder belegt, und Sie finden Ihr vergessenes Handy
 - Bei Dereferenzierung erhalten Sie noch den alten Wert der Variablen
 - * Evtl. wurde das Zimmer bereits wieder vergeben => Sie “brechen” bei einem Fremden ein!
 - * Bei Dereferenzierung greifen Sie auf “fremde” Variablen (Speicherbereiche) zu!

7.2.2.9. Pointer und Initialisierung

Pointer werden vom Compiler **nicht** initialisiert!

- Zeigen ohne explizite Initialisierung auf **zufällige** Adresse
- Dereferenzierung uninitialisierter Pointer problematisch

Explizite Null-Pointer:

- Wert 0 zuweisen
- Besser: Symbolische Konstante `NULL` aus `stdio.h` bzw. `cstdio` bzw. in C++ `nullptr`

7.2.3. Speicherverwaltung

- C: **Funktionen** zur Verwaltung dynamischen Speichers: `malloc()`, `free()`, ... (in `<stdlib.h>`)

```
1 void* malloc(size_t size)
```

- Alloziert `size` Bytes auf dem Heap und liefert Adresse zurück

- Pointer auf **void**, da Typ unbekannt - vor Nutzung auf korrekten Typ umcasten
- Im Fehlerfall wird ein Null-Pointer zurückgeliefert: **NULL**
- Achtung: Speicher ist nicht initialisiert!

```

1  int *p = (int*) malloc(sizeof(int));
2  int *pa = (int*) malloc(4*sizeof(int));
3
4  free(p);
5  free(pa);

```

• C++: **Operatoren: new und delete**

- Direkte Angabe des Zieltyps
- Rückgabe eines Pointers auf diesen Typ
- Exception, wenn kein Speicher verfügbar
- Form mit []-Operator für Arrays
- Mit **new** allozierter Speicher muss mit **delete** freigegeben werden
- Mit **new []** allozierter Speicher muss mit **delete []** freigegeben werden

```

1  int *p = new int;
2  int *pa = new int[4];
3
4  delete p;
5  delete [] pa;

```

7.2.3.1. Speicher allozieren: Standardidiom

In C müssen Sie die Rückgabe von **malloc** prüfen:

```

1  int *i, *x;
2
3  i = (int *) malloc(sizeof(int));
4  x = (int *) malloc(sizeof(*x)); /* Stern wichtig */
5
6  if (!i) {
7      /* Fehlerbehandlung */
8  } else {
9      /* mach was */
10 }

```

In C++ bekommen Sie eine Exception, falls **new** nicht erfolgreich war:

```

1  int *i;
2
3  try {
4      i = new int;
5      /* mach was */
6  } catch (...) { /* Fehlerbehandlung */ }

```

Hinweis: Pointer-Variablen **i** und **x** liegen auf Stack, angeforderter Speicher im Heap!

7.2.3.2. Pointer und Typen

- Typ eines Zeigers relevant, wird vom Compiler geprüft
- Zuweisung ohne expliziten Cast nur an allgemeinere Typen/Oberklassen
 - Jeder Zeiger auf Typ **T** kann automatisch zum **void**-Pointer konvertiert werden
 - Für Zuweisung von **void**-Pointern an Pointer auf Typ **T** expliziter Cast nach **T*** nötig (siehe auch nachfolgenden Hinweis zu C11)

```

1 char *cp;
2 void *vp;
3
4 vp = cp;           /* OK */
5 cp = vp;           /* problematisch */
6 cp = (char *) vp; /* OK */

```

7.2.3.3. Fallstricke dynamischer Speicherverwaltung

Nur new und delete kombinieren bzw. malloc und free

- **delete** darf nur auf mit **new** erzeugte Objekte angewendet werden
 - Vorsicht bei Pointern auf Stack-Variablen!
 - **NIE** mischen mit **malloc()/calloc()/free()**!

```

1 int *p = (int *) malloc(sizeof(int));
2 delete p; // FEHLER! Absturzgefahr

```

delete[] genau nur bei **new[]**

- **delete[]** darf nur auf mit **new[]** erzeugte Objekte angewendet werden (und **muss** dort auch angewendet werden)

delete auf mit **new[]** erzeugtes Array würde nur erstes Element freigeben!

Vorsicht mit Pointern auf lokale Variablen

- Funktioniert technisch, ist aber gefährlich:

```

1 int* murks() {
2     int i=99;
3     return &i; /* SO NICHT: Pointer auf lokale Variable! */
4 }

```

- Etwas besser:

```

1 int* wenigerMurks() {
2     int *p = (int *) malloc(sizeof(int)); /* neuer Speicher */
3     *p=99;

```

```

4     return p; /* das geht */
5 }

```

Warum nur “etwas besser”?

Jetzt haben Sie aber ein neues Problem: Der Aufrufer der Funktion muss wissen, dass diese Speicher alloziert und muss sich selbst um die Freigabe kümmern. Dies ist unschön, da die Allokation und Freigabe in unterschiedlicher Verantwortung liegen! Dadurch können sehr schnell Fehler passieren.

Besser wäre, wenn der Aufrufer einen Pointer übergibt, mit dem dann in der Funktion gearbeitet wird. Dann liegt die Verantwortung für die Erstellung und Freigabe des Pointers komplett in der Hand des Aufrufers.

Memory Leaks

- Pointer-Variablen unterliegen den Gültigkeitsregeln für Variablen
- Mit `malloc()` reservierter Speicher existiert bis Programmende

```

1 {
2     int *i;
3     i = (int *) malloc(sizeof(*i));
4     *i = 99;
5 }
6 /* hier existiert die Variable i nicht mehr */
7 /* aber der Speicher auf dem Heap bleibt belegt */
8 /* ist aber nicht mehr zugreifbar -> SPEICHERLOCH! */

```

Double Free und Stale Pointer

- `free()` darf nur einmal pro Objekt aufgerufen werden
 - Hintergrund: Intern wird eine Freispeicherliste verwaltet
- Nach `free()` ist der Zeiger undefiniert:
 - Zeigt immer noch in den Heap (alte Adresse!)
 - Ist nicht gleich `NULL` oder 0
 - Zugriff ist möglich, aber gefährlich: Speicher kann wieder vergeben und überschrieben werden (*Hotelzimmer-Analogie*)
- Mehrere Pointer auf ein Objekt: Einmal `free()` reicht!
 - Die anderen Pointer dürfen anschließend aber auch nicht mehr dereferenziert werden (stale/dangling pointer)

Beispiel Stale Pointer

```

1     int *i, *k; i = (int *) malloc(sizeof(*i)); k = i;
2
3     free(i);
4     free(i); /* EINMAL reicht! */
5     *k = 42; /* Speicher ist bereits frei - stale pointer */

```

```
6     free(k); /* Speicher ist bereits frei - double free */
7     *i = 99; /* Speicher ist bereits frei */
```

Anmerkung: Anwendung auf **NULL**-Pointer bewirkt nichts und ist unschädlich

Dereferenzieren von “Bad Pointern”

Der klassische Scanf-Bug :)

```
1  int i;
2  scanf("%d", i);
```

Tipp: **i** ist **kein** Pointer :)

Auslesen von nicht-initialisiertem Speicher

Wenn Programmierer denken, dass irgendwer den Heap zwischendurch immer mal wieder auf 0 setzt ...

```
1  /* return y = Ax */
2  int *matvec(int **A, int *x, int N) {
3      int *y = malloc(N*sizeof(int));
4      for (int i=0; i<N; i++) {
5          for (int j=0; j<N; j++) {
6              y[i] += A[i][j] * x[j];
7          }
8      }
9      return y;
10 }
```

Tipp: `y[i] += ...` setzt sinnvolle Werte in `y[i]` voraus ...

Überschreiben von Speicher I

Allokation von falschen Größen

```
1  int *p;
2
3  p = malloc(N*sizeof(int));
4
5  for (int i=0; i<N; i++) {
6      p[i] = malloc(M*sizeof(int));
7  }
```

Tipp: Jedes `p[i]` kann einen **int** speichern, bekommt aber einen Pointer zugewiesen (könnte deutlich breiter im Speicher sein als ein **int**) ...

Überschreiben von Speicher II

Indexberechnung kaputt, sogenannte “*off-by-one-errors*”

```

1  int **p;
2
3  p = malloc(N*sizeof(int));
4
5  for (int i=0; i<=N; i++) {
6      p[i] = malloc(M*sizeof(int));
7  }

```

Tipp: Hier läuft `i` um einen Platz zu weit ...

Überschreiben von Speicher III

Einlesen von Strings, zu kleine Buffer

```

1  char s[8];
2  gets(s);

```

Tipp: Wenn hier mehr als 7 Zeichen eingegeben werden, gibt es Probleme :)

Überschreiben von Speicher IV

Pointerarithmetik falsch verstanden

```

1  int *search(int *p, int val) {
2      while (*p && *p != val)
3          p += sizeof(int);
4      return p;
5  }

```

Tipp: Jeder Pointer hat einen Typ, und der Ausdruck "Pointer + 1" rutscht um so viele Bytes im Speicher weiter, wie der Typ breit ist. D.h. mit einem "Pointer + 1" gelangt man zum nächsten Element, während der obige Ausdruck `p += sizeof(int)`; um `sizeof(int)` Elemente weterspringt!

7.2.4. Pointer und Arrays

Ein Array-Name ist wie ein *konstanter* Pointer auf Array-Anfang: `a[i] == *(a+i)`

Ein **Array-Name** ist nur ein Label, welches der **Adresse des ersten Array-Elements** entspricht. Die Wirkung ist entsprechend die eines konstanten Pointers auf den Array-Anfang.

=> Der Compiler übersetzt Array-Zugriffe per Indexoperator in Pointerarithmetik: `a[i]` wird zu `*(a+i)` ...

Vgl. auch die Diskussion in eli.thegreenplace.net/2009/10/21/are-pointers-and-arrays-equivalent-in-c

```

1  char a[6], c, *cp;
2
3  &a[0] == a;
4  cp = a;
5
6  c = a[5];
7  c = *(a+5);

```

```

8  c = *(cp+5);
9  c = cp[5];
10
11 a = cp;    /* FEHLER */
12 a = &c;    /* FEHLER */

```

7.2.4.1. Iteration durch Arrays (Varianten)

```

1  int a[10], *pa=a;
2
3  for (int k=0; k<10; k++)    /* Iteration, Variante 1 */
4      printf("%d ", a[k]);
5
6  for (int k=0; k<10; k++)    /* Iteration, Variante 2 */
7      printf("%d ", *(a+k));
8
9  pa = a;
10 for (int k=0; k<10; k++)    /* Iteration, Variante 3 */
11     printf("%d ", *pa++);
12
13
14 /* Iteration, KEINE Variante */
15 for (int k=0; k<10; k++)
16     printf("%d ", *a++);    /* DAS GEHT NICHT */

```

`*pa++`: Operator `++` hat Vorrang vor `*`, ist aber die Postfix-Variante. D.h. `++` wirkt auf `pa` (und nicht auf `*pa`), aber zunächst wird für die Ausgabe `*pa` ausgewertet ...

`*a++` ist nicht erlaubt, weil dadurch der Name des Arrays (== Adresse des ersten Array-Elements == konstanter Zeiger auf den Anfang des Arrays) verändert würde.

7.2.4.2. Array-Namen sind wie konstante Pointer

Array-Namen können NICHT umgebogen werden!

```

1  int a[], *pa=a, k;
2
3  /* erlaubt */
4  a + k;
5  pa++;
6
7  /* VERBOTEN */
8  a++;

```

7.2.4.3. Selbsttest: Was bedeutet was, was ist erlaubt/nicht erlaubt, was kommt raus? Warum?

```

1  int a[10], *pa, *pb, x;
2  pa = a;    pb = (int*) malloc(sizeof(int));
3

```

```

4 x = a[1];
5 x = *(a+1);
6 x = *(a++);
7
8 x = pa[1];
9 x = *(pa+1);
10 x = *(pa++);
11
12 x = pb[1];
13 x = *(pb+1);
14 x = *(pb++);

```

=> Arrays können wie konstante Pointer behandelt werden.

=> Pointer dürfen **nicht immer wie Arrays** behandelt werden! (Syntaktisch zulässig, semantisch normalerweise nicht!)

7.2.4.4. Pointerarithmetik: Typen beachten

- Pointer zeigen auf Objekte mit einem bestimmten Typ
- Typen haben unterschiedliche Speicherbreite
- Inkrementierung/Dekrementierung: Pointer zeigt nicht auf nächste Speicheradresse, sondern auf die Adresse des nächsten Werts!

```

1 double d[10];
2 double *d1 = &d[2];
3 double *d2 = d1;
4
5 d2++;
6
7 printf("%ld\n", d2-d1);           // ergibt 1
8 printf("%ld\n", (long)d2 - (long)d1); // double -> zB. 8 Bytes
9
10 printf("%ld\n", sizeof(d1)); // Breite Pointervariable
11 printf("%ld\n", sizeof(*d1)); // Breite Pointerdatentyp

```

7.2.5. Referenzen in C++

Typ & Name = Objekt;

```

1 int i=2;
2 int j=9;
3
4 int &r=i;    // Referenz: neuer Name fuer i
5 r=10;       // aendert i: i=10
6 r=j;        // aendert i: i=9
7
8 int &s=r;    // aequivalent zu int &s = i;

```

7.2.5.1. Referenzen bilden Alias-Namen

```

1  int i = 99;
2  int *iptr = &i;
3
4  int &ieref = i;    // Referenz: neuer Name fuer i

```

1	Variable	Speicheradresse	Inhalt	
2				
3				
4			+-----+	
5	i, ieref	10125	99	<---+
6			+-----+	
7				
8				
9				
10			+-----+	
11	iptr	27890	10125	---+
12			+-----+	
13				

- Referenz bildet Alias-Namen für ein Objekt
- Objekt hat damit mehrere Namen, über die es ansprechbar ist
- Referenzen in C++ mit Hilfe des &-Operators deklarieren

7.2.5.2. Eigenschaften von Referenzen in C++

- Referenzen müssen bei Deklaration initialisiert werden
- Referenzen können nicht um-assigned werden
- Referenzen brauchen keinen eigenen Speicherplatz
- Vorsicht bei gleichzeitiger Deklaration mehrerer Referenzen:

```

1  int i=2;
2  int j=9;
3
4  int& r=i, s=j;    // SO NICHT!!!
5  int &r=i, &s=j;    // korrekt

```

7.2.5.3. Referenzen als Funktionsparameter

- Signatur:

```

1  void fkt(int&, char);
2  void fkt(int &a, char b);    // a per Referenz

```

- Aufruf: ganz normal (ohne extra &) ...

```

1  int x=3;
2  char y='a';

```

```
3 fkt(x, y); // x per Referenz
```

Im Beispiel werden die Variablen `x` und `y` an die Funktion `fkt` übergeben. Der erste Parameter wird per Referenz (call-by-reference), der zweite per Kopie (call-by-value) übergeben.

Der Funktionsparameter `a` bindet sich an `x`, ist eine Referenz auf/für `x` - jeder Zugriff auf `a` ist wie ein Zugriff auf `x`. Änderungen von `a` sind also Änderungen von `x`.

Der zweite Parameter bindet sich an den Wert von `y`, d.h. `b` hat den Wert `'a'`. Zwar kann auch `b` verändert werden, das hat dann aber nur Auswirkungen innerhalb der Funktion und nicht auf die Variable `y` im äußeren Scope.

7.2.6. Call-by-Reference Semantik in C++

7.2.6.1. Variante A: Pointer (C und C++)

Mit Hilfe von Pointern lässt sich die Call-by-Reference Semantik in C und in C++ simulieren.

Bei der Übergabe eines Pointers wird der Wert des Pointers *kopiert* (call-by-value!). Im Inneren der Funktion kann diese Adresse dereferenziert werden und so auf das außerhalb der Funktion "lebende" Objekt zugegriffen werden. Damit bekommt man in der Wirkung call-by-reference.

```
1 void add_5(int *x) {  
2     *x += 5;  
3 }  
4  
5 int main() {  
6     int i=0, *ip=&i;  
7  
8     add_5(ip);  
9  
10    add_5(&i);  
11 }
```

- Pointer wird nach wie vor per call-by-value übergeben:
 - Wert wird bei Übergabe kopiert (hier Adresse von `i`)
 - Kopierter Wert ist immer noch ein Pointer (hier Pointer auf `i`, da Adresse von `i`)
 - Dereferenzierung des kopierten Pointers: Zugriff auf das Original-Objekt (hier `i`)

7.2.6.2. Variante B: Referenzen (nur C++)

Referenzen müssen bei der Deklaration initialisiert werden und binden sich an das dabei genutzte Objekt. Sie stellen letztlich lediglich einen neuen Namen für das Objekt dar.

Bei der Übergabe von Variablen an Referenz-Parameter einer Funktion binden sich diese Parameter an die übergebenen Objekte. Jeder Zugriff innerhalb der Funktion auf einen Referenz-Parameter bewirken einen Zugriff auf das ursprüngliche Objekt.

```

1  int add_5(int &x) {
2      x += 5;
3      return x;
4  }
5
6  int main() {
7      int i=0, erg;
8      erg = add_5(i);
9  }

```

- Funktionsparameter `x` ist eine Referenz
- Bei Aufruf der Funktion wird dieser Parameter initialisiert - die Referenz `x` bindet sich im Beispiel an die Variable `i`
- Zugriffe auf `x` in der Funktion sind also Zugriffe auf das Original-Objekt `i` - `x += 5` ist nichts anderes als `i += 5`
- Bei weiteren Aufrufen wird `x` dann neu gebunden

7.2.6.3. Call-by-Reference: const

- Nachteil bei Call-by-Reference:
Übergebenes Objekt könnte durch die Funktion (unbeabsichtigt) verändert werden
- Abhilfe: Deklaration der Parameter als konstant (Schlüsselwort **const**):

```

1  void fkt(const int&, char);
2  void fkt(const int &a, char b);
3  // a wird per Referenz uebergeben, darf aber in der Funktion nicht
   veraendert werden

```

=> **const**-heit ist Bestandteil der Signatur!

Arbeiten Sie (wo möglich/sinnvoll) mit (konstanten) Referenzen!

7.2.6.4. Rückgabe von Werten per Referenz

- Normalerweise per call-by-value (Kopie)
- Mit Referenzen oder Pointern auch als call-by-reference

```

1  int &fkt1(const int &, const char *);
2  int *fkt2(const int &, const char *);

```

- Vorsicht mit lokalen Variablen (Gültigkeit)!

```

1  int &fkt1(const int &i, const char *j) {
2      int erg = i+1;
3      return erg;    // Referenz auf lokale Variable!
4  }

```

```

5  int *fkt2(const int &i, const char *j) {
6      int erg = i+2;
7      return &erg; // Pointer auf lokale Variable!
8  }
9  int main() {
10     int &x = fkt1(2, "a"); // AUTSCH!!!
11     int *y = fkt2(2, "b"); // AUTSCH!!!
12     int z = fkt1(2, "c"); // OK
13 }

```

Die Zuweisung `int &x = fkt1(2, "a");` ist syntaktisch erlaubt. Semantisch aber nicht: Die Referenz `x` bindet sich an das zurückgelieferte lokale `erg` - dieses existiert aber nicht mehr, da der Scope von `erg` beendet ist ...

=> Nur Pointer auf Speicher zurückliefern, der nach Beendigung des Funktionsaufrufes noch existiert! (Dies könnte beispielsweise Speicher aus `malloc` oder `new` oder ein Pointer auf das eigene Objekt (`*this`) sein.)

Die Zuweisung `int *y = fkt2(2, "b");` ist syntaktisch erlaubt. Semantisch aber nicht: Der Pointer `y` übernimmt die zurückgelieferte Adresse des lokalen `erg` - dieses existiert aber nicht mehr, da der Scope von `erg` beendet ist ...

=> Nur Referenzen zurückliefern, die nach Beendigung des Funktionsaufrufes noch gültig sind! (Dies könnten beispielsweise Referenz-Inputparameter oder eine Referenz auf das eigene Objekt (`*this`) sein.)

Die Zuweisung `int z = fkt1(2, "c");` ist unbedenklich, da `z` eine normale Integervariable ist und hier das übliche Kopieren der Rückgabe von `fkt1` in die Variable stattfindet.

7.2.6.5. Diskussion

In C++ können Sie Call-by-Reference über Pointer und/oder über Referenzen erreichen.

In den obigen Beispielen wurde dies für die Parameter einer Funktion gezeigt - es sind aber auch Pointer und/oder Referenzen als Rückgabetypen möglich. Beachten Sie dabei, ob das jeweils wirklich Sinn ergibt! Eine Referenz oder ein Pointer auf eine lokale Variable ist eine große Fehlerquelle.

In C++ werden Referenzen über Pointer bevorzugt. Wenn Sie die Wahl zwischen den beiden Signaturen `bar foo(wuppie&, bar)` und `bar foo(wuppie*, bar)` haben, sollten Sie sich für `bar foo(wuppie&, bar)` entscheiden.

7.2.7. Vergleich Pointer mit Referenzen

Referenzen	Pointer
Alias-Name für Objekte/Variablen, kein eigener Speicherplatz	“Echte” Variablen mit eigenem Speicherplatz (für den Wert des Pointers)
Können nicht auf andere Objekte “umgebogen” werden	Können auf andere Objekte zeigen (falls nicht const)

Referenzen	Pointer
Operationen agieren direkt auf dem referenzierten Objekt	Operationen auf referenzierten Objekt als auch auf dem Pointer selbst
Nur in C++	In C und in C++ Mit Pointern ist dynamische Speicherverwaltung möglich: Manipulation von Speicherbereichen im Heap

7.2.8. Wrap-Up

- Virtueller Speicher: Kernel stellt Prozessen linearen Adressraum bereit, Segmente: Text, Stack, Heap
- Pointer sind Variablen, deren **Wert als Adresse** interpretiert wird
 - Deklaration mit * zwischen Typ und Name
 - Adressoperator & liefert die Adresse eines Objekts
 - Dereferenzierung eines Pointers mit * vor dem Namen
- Verwandtschaft zw. Arrays und Pointern: Array-Name ist konstanter Pointer auf Array-Anfang
- Pointer haben Typ: Pointerarithmetik berücksichtigt Speicherbreite des Typs
- C++-Referenzen als Alias-Namen für ein Objekt
 - Deklaration: `Typ &ref = obj;`
 - Fest mit Objekt verbunden
 - Zugriff auf Referenz: Direkter Zugriff auf das Objekt

Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k1: Ich kenne das Konzept des virtuellen Speichers und die Aufteilung in die Segmente: Text, Data, Stack
- k2: Ich kann an einem Beispiel erklären, dass Pointer Variablen sind, deren Wert wird als Adresse interpretiert wird
- k2: Ich kann die Initialisierung und Scopes bei Pointern erklären
- k3: Ich kann in meinem Programm eine Adresse an einen Pointer zuweisen
- k3: Ich kann in meinem Programm Pointer dereferenzieren und auf das referenzierte Element zugreifen

- k3: Ich kann Pointer als Funktionsparameter nutzen und so Call-by-Reference realisieren
- k2: Ich kann die Problematik der Memory Leaks und Stale Pointer erklären und weiss, wie ich diese vermeide
- k3: Ich kann die C++-Operatoren **new** und **delete** praktisch einsetzen und ich kann den Unterschied zu **malloc()** und **free()** erklären
- k3: Ich kann Referenzen in C++ einsetzen (Deklaration, Initialisierung, Nutzung)
- k2: Ich kann den Zusammenhang und den Unterschied zwischen Pointer und Arrays erklären
- k3: Ich kann mit Pointern rechnen und dabei den Typ berücksichtigen

Challenges

Pointer

- Erklären Sie das Problem bei folgender Deklaration: **int*** *xptr*, *yptr*;
- Seien *p1* und *p2* Pointer auf **int**. Was ist der Unterschied zwischen den beiden Code-Zeilen?

```
1  p2 = p1;
2  *p2 = *p1;
```

- Ist ***&x** immer identisch mit **x**?
- Ist **&*x** immer identisch mit **x**?
- Wann kann die Funktion **void f(int*)** so aufgerufen werden: **f(&x)** ;?

Swap ...

- Warum funktioniert die folgende **swap()**-Funktion nicht? Wie müsste sie korrigiert werden?

```
1  void swap(int x, int y) {
2      int tmp; tmp=x; x=y; y=tmp;
3  }
```

- Was ist mit dieser Version dieser **swap()**-Funktion?

```
1  void swap(int *x, int *y) {
2      int *tmp;
3      tmp=x; x=y; y=tmp;
4  }
```

C++: new und delete

Betrachten Sie folgende Code-Schnipsel. Erklären Sie die Wirkung der jeweiligen Anweisungen.

```
1  void fkt() {
2      char *cp = new char[100];
3      cp[0] = 'a';
4  }
```

```
1  int i=10;
2  int *p = &i;
3  delete p;
```

```

1 char *p;
2 {
3     char *cp = new char[100];
4     p = cp;
5     free(cp);
6 }
7 delete p;

```

Referenzen vs. Pointer: Welche der Aufrufe sind zulässig?

```

1 void f1(int*);
2 void f2(int&);
3
4 int main() {
5     int i=0, *ip=&i, &ir=i;
6
7     f1(i);      f1(&i);      f1(*i);
8     f1(ip);     f1(&ip);     f1(*ip);
9     f1(ir);     f1(&ir);     f1(*ir);
10
11    f2(i);      f2(&i);      f2(*i);
12    f2(ip);     f2(&ip);     f2(*ip);
13    f2(ir);     f2(&ir);     f2(*ir);
14 }

```

C++-Referenzen und Pointer

Betrachten Sie folgende Code-Schnipsel. Erklären Sie die Wirkung der jeweiligen Anweisungen.

```

1 int x=5, &y=x;
2 int *ptr1 = &x;
3 int *ptr2 = &y;
4
5 *ptr1 += 1;
6 *ptr1++;
7
8 ptr2 = ptr1;
9 *ptr2 = *ptr1;
10
11 ptr1 == ptr2;
12 *ptr1 == *ptr2;

```

Fallstricke mit C++-Referenzen

Betrachten Sie folgende Code-Ausschnitte. Welchen Wert haben die Variablen nach der Ausführung? Begründen Sie Ihre Antwort.

```

1 int i=2, j=9;
2 int &r=i, &s=r;
3 s=200;

```

```

1 int &versuch(int i, int j) {
2     int erg = i+j;
3     return erg;
4 }

```

```
5 int main() {  
6     int &z = versuch(2, 10);  
7     return 0;  
8 }
```

1.

```
1 int &versuch(int&, int&);  
2  
3 int main() {  
4     int a=10, b=20;  
5     int &z = versuch(a, b);  
6     return 0;  
7 }  
8  
9 int &versuch(int &i, int &j) {  
10    int &erg = i+j;  
11    return erg;  
12 }
```

2.

```
1 int &versuch(int&, int&);  
2  
3 int main() {  
4     int a=10, b=20;  
5     int &z = versuch(a, b);  
6     return 0;  
7 }  
8  
9 int &versuch(int &i, int &j) {  
10    int erg = i+j;  
11    return erg;  
12 }
```

3.

```
1 int &versuch(int&, int&);  
2  
3 int main() {  
4     int a=10, b=20;  
5     int &z = versuch(a, 10);  
6     return 0;  
7 }  
8  
9 int &versuch(int &i, int &j) {  
10    j += i;  
11    return j;  
12 }
```

- Erklären Sie die Unterschiede folgender Anweisungen. Welche sind erlaubt, welche nicht? Welche führen möglicherweise zu Fehlern?

```

1  int a[10], *pa, *pb, x;
2  pa = a;
3  pb = new int;
4
5  x = a[1];
6  x = *(a+1);
7  x = *(a++);
8
9  x = pa[1];
10 x = *(pa+1);
11 x = *(pa++);
12
13 x = pb[1];
14 x = *(pb+1);
15 x = *(pb++);

```

Typ eines Pointers bei Adressarithmetik

- Was ist der Unterschied zwischen den beiden folgenden Statements?

```

1  ((char *)ptr)+1
2  ((double *)ptr)+1

```

7.3. C++: Klassen

TL;DR

Klassen werden in C++ mit dem Schlüsselwort **class** definiert. Dabei müssen Klassendefinitionen immer mit einem Semikolon abgeschlossen werden(!). Bei Trennung von Deklaration und Implementierung muss die Definition der Methoden mit dem Namen der Klasse als Namespace erfolgen:

```

1  // .h
2  class Fluppie {
3  public:
4      int wuppie(int c=0);
5  };
6
7  // .cpp
8  int Fluppie::wuppie(int c) { ... }

```

Die Sichtbarkeiten für die Attribute und Methoden werden blockweise definiert. Für die Klassen selbst gibt es keine Einstellungen für die Sichtbarkeit.

Objekt-Layout: Die Daten (Attribute) liegen direkt im Objekt (anderenfalls Pointer nutzen). Sofern der Typ der Attribute eine Klasse ist, kann man diese Attribute nicht mit **NULL** initialisieren (kein Pointer, keine Referenz).

Für den Aufruf eines Konstruktors ist kein **new** notwendig, es sei denn, man möchte das neue Objekt auf dem Heap haben (inkl. Pointer auf das Objekt).

Beachten Sie den Unterschied der Initialisierung der Attribute bei einer Initialisierung im Body des Konstruktors vs. der Initialisierung über eine **Initialisierungsliste**. (Nutzen Sie in C++ nach Möglichkeit Initialisierungslisten.)

Videos

- [VL C++: Klassen](#)

7.3.1. OOP in C++

```
1 public abstract class Dummy {
2     public Dummy(int v) { value = v; }
3     public abstract int myMethod();
4
5     private int value;
6 }
```

```
1 class Dummy {
2 public:
3     Dummy(int v = 0);
4     int myMethod();
5     virtual ~Dummy();
6 private:
7     int value;
8 };
```

7.3.1.1. OOP in C++: Unterschiede zu Java

- Klassendefinition muss mit Semikolon beendet werden
- Sichtbarkeit wird immer blockweise eingestellt (per Default immer **private**)
- Wie bei Funktionen: Deklaration muss vor Verwendung (= Aufruf) bekannt sein
- **this** ist keine Referenz, sondern ein **Pointer** auf das eigene Objekt

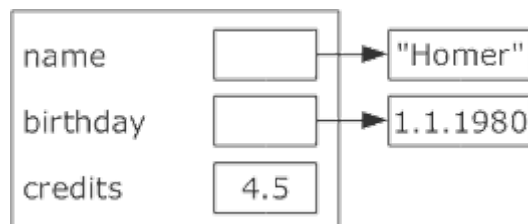
7.3.1.2. Objektlayout: Java vs. C++

Java: Referenzen auf Objekte

```

1 class Student {
2     String name;
3     Date birthday;
4     double credits;
5 }

```



In Java werden im Objektlayout lediglich die primitiven Attribute direkt gespeichert.

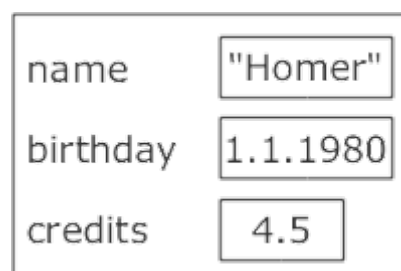
Für Objekte wird nur eine Referenz auf die Objekte gehalten. Die Attribute selbst liegen aber außerhalb der Klasse, dadurch benötigt das Objekt selbst nur relativ wenig Platz im Speicher.

C++: Alles direkt im Objekt

```

1 class Student {
2     string name;
3     Date birthday;
4     double credits;
5 };

```



In C++ werden alle Attribute innerhalb des Objektlayouts gespeichert. Ein Objekt mit vielen oder großen Feldern braucht also auch entsprechend viel Platz im Speicher.

Wollte man eine Java-ähnliche Lösung aufbauen, müsste man in C++ entsprechend Pointer einsetzen:

```

1 class Student {
2     private:
3         string *name;
4         Date *birthday;
5         double credits;
6 }

```

Warum nicht Referenzen?

7.3.2. Objekte erzeugen mit Konstruktoren

```
1 class Dummy {
2 public:
3     Dummy(int c=0) { credits = c; }
4 private:
5     int credits;
6 };
```

Erzeugen neuer Objekte:

```
1 Dummy a;
2 Dummy b(37);
3 Dummy c=99;
```

=> Kein Aufruf von **new**!

(**new** würde zwar auch ein neues Objekt anlegen, aber **auf dem Heap**!)

7.3.3. Default-Konstruktoren

Der C++-Compiler generiert einen **parameterlosen Defaultkonstruktor** - sofern man nicht selbst mindestens einen Konstruktor definiert.

Dieser parameterlose Defaultkonstruktor wendet für jedes Attribut dessen parameterlosen Konstruktor an, für primitive Typen erfolgt keine garantierte Initialisierung!

Achtung: Default-Konstruktor wird ohne Klammern aufgerufen!

```
1 Dummy a;    // Korrekt
2 Dummy a();  // FALSCH!!! (Deklaration einer Funktion `a()`, die ein `Dummy`
                zurueckliefert)
```

7.3.4. C++: Trennung .h und .cpp

```
1 // .h
2 class Dummy {
3 public:
4     Dummy(int c=0);
5 private:
6     int credits;
7 };
```

```
1 // .cpp
2 Dummy::Dummy(int c) {
3     credits = c;
4 }
```

Klassenname ist der Scope für die Methoden

7.3.5. Konstruktoren: Normale (Java-like) Initialisierung

```
1 class Student {
2 public:
3     Student(const string &n, const Date &d, double c) {
4         name = n;
5         birthday = d;
6         credits = c;
7     }
8 private:
9     string name;
10    Date birthday;
11    double credits;
12 };
```

Hier erfolgt die Initialisierung in **zwei** Schritten:

1. Attribut wird angelegt und mit Defaultwert/-konstruktor des Datentyps initialisiert
2. Anschließend wird die Zuweisung im Body des Konstruktors ausgeführt

Das klappt natürlich nur, wenn es einen parameterlosen Konstruktor für das Attribut gibt.

Beispiel oben: Beim Anlegen von `birthday` im Speicher wird der **Defaultkonstruktor** für `Date` aufgerufen. Danach wird im Body der übergebene Datumswert **zugewiesen**.

[Konsole: studiInitBody.cpp](#)

7.3.6. Konstruktoren: Initialisierungslisten

```
1 class Student {
2 public:
3     Student(const string &n, const Date &d, double c)
4         : name(n), birthday(d), credits(c)
5         {}
6 private:
7     string name;
8     Date birthday;
9     double credits;
10 };
```

In diesem Fall erfolgt die Initialisierung in nur einem Schritt:

1. Attribut wird angelegt und direkt mit übergebenen Wert (Kopie) initialisiert

Das klappt natürlich nur, wenn ein passender Konstruktor für das Attribut existiert.

Achtung: Die Reihenfolge der Auswertung der Initialisierungslisten wird durch die Reihenfolge der Attribut-Deklarationen in der Klasse bestimmt!!!

Beispiel oben: Beim Anlegen von `birthday` im Speicher wird direkt der übergebene Wert **kopiert**.

Konsole: [studilnitListe.cpp \(ohne/mit -Wall\)](#)

7.3.7. Zwang zu Initialisierungslisten

In manchen Fällen **muss** man die Initialisierung der Attribute per Initialisierungsliste durchführen.

Hier einige Beispiele:

- Attribut **ohne parameterfreien Konstruktor**

Bei “normaler” Initialisierung würde zunächst der parameterfreie Konstruktor für das Attribut aufgerufen, bevor der Wert zugewiesen wird. Wenn es keinen parameterfreien Konstruktor für das Attribut gibt, bekommt man beim Kompilieren einen Fehler.

- **Konstante** Attribute

Bei “normaler” Initialisierung würde das Attribut zunächst per parameterfreiem Konstruktor angelegt (s.o.), danach existiert es und ist konstant und darf nicht mehr geändert werden (müsste es aber, um die eigentlich gewünschten Werte im Body zu setzen) ...

- Attribute, die **Referenzen** sind

Referenzen müssen direkt beim Anlegen initialisiert werden.

7.3.8. C++11 und delegierende Konstrukturen

```

1  class C {
2      // 1: Normaler Konstruktor
3      C(int x) { }
4
5      // 2: Delegiert zu (1)
6      C() : C(42) { }
7
8      // 3: Rekursion mit (4)
9      C(char c) : C(42.0) { }
10
11     // 4: Rekursion mit (3)
12     C(double d) : C('a') { }
13 };

```

Delegierende Konstrukturen gibt es ab C++11:

- Vor C++11: Ein Objekt ist fertig konstruiert, wenn der Konstruktor durchgelaufen ist
- Ab C++11: Ein Objekt ist fertig konstruiert, wenn der **erste** Konstruktor fertig ausgeführt ist => Jeder weitere aufgerufene Konstruktor agiert auf einem “fertigen” Objekt.
- Vorsicht mit rekursiven Aufrufen: Compiler *kann* warnen, muss aber nicht.

7.3.9. C++ und explizite Konstruktoren

- Implizite Konvertierung mit einelementigen Konstruktoren:

```
1 class Dummy {
2 public:
3     Dummy(int c=0);
4 };
5
6 Dummy a;
7 a = 37;    // Zuweisung(!)
```

Auf der linken Seite der Zuweisung steht der Typ `Dummy`, rechts ein `int`. Der Compiler sucht nach einem Weg, aus einem `int` einen `Dummy` zu machen und hat durch die Gestaltung des Konstruktors von `Dummy` diese Möglichkeit. D.h. in dieser Zuweisung wird implizit aus der 37 ein Objekt vom Typ `Dummy` gebaut (Aufruf des Konstruktors) und dann die Zuweisung ausgeführt.

Dieses Verhalten ist in vielen Fällen recht praktisch, kann aber auch zu unerwarteten Problemen führen. Zur Abhilfe gibt es das Schlüsselwort `explicit`.

- Falls unerwünscht: Schlüsselwort `explicit` nutzen

```
1 explicit Dummy(int c=0);
```

7.3.10. Wrap-Up

- Klassendefinition mit Semikolon abschließen (!)
- Sichtbarkeiten blockweise, keine für Klasse
- Daten liegen direkt im Objekt (anderenfalls Pointer nutzen)
- Attribute sind echte Objekte: Initialisieren mit `NULL` nicht möglich
- Konstruktoren: Kein `new` nötig (würde Objekt auf Heap anlegen und Pointer liefern)

Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Ich kann den Aufbau von C++-Klassen im Speicher an einem Beispiel erklären
- k2: Ich kann die Problematik expliziter Konstruktoren erklären
- k2: Ich kann die Problematik mit Defaultkonstruktoren/-operatoren im Zusammenhang mit Pointern erklären und ein Beispiel angeben
- k3: Ich kann Konstruktoren definieren und einsetzen (eigene, Default)

- k3: Ich kenne den Unterschied zw. Initialisierungslisten vs. Initialisierung im Body und kann das in eigenen Programmen einsetzen

Challenges

C++: Klassen

Erklären Sie die Unterschiede zwischen den Klassendefinitionen (Java, C++):

```
1 class Student {
2     private String name;
3     private Date birthday;
4     private double credits;
5 }
```

```
1 class Student {
2     private:
3         string name;
4         Date birthday;
5         double credits;
6 };
```

Konstruktoren

- Wie kann der implizite Aufruf eines Konstruktors verhindert werden (beispielsweise in `Dummy b; b = 3;`)?
- In welchen Fällen muss eine Initialisierung von Attributen in der Initialisierungsliste stattfinden?
- Wie können/müssen **static** Attribute initialisiert werden?

7.4. C++: Big 3

TL;DR

Für C++-Klassen kann man Destruktoren, Copy-Konstruktoren und Zuweisungsoperatoren definieren. Wenn man keine eigenen definiert, erzeugt C++ Default-Varianten. Diese bereiten u.U. Probleme, wenn man Pointertypen für die Attribute verwendet: Dann werden u.U. nur flache Kopien erzeugt bzw. es wird u.U. der Platz auf dem Heap nicht freigegeben.

Der Default-Destruktor ruft die Destruktoren der Objekt-Attribute auf. Der Copy-Konstruktor wird aufgerufen, wenn die linke Seite (einer scheinbaren "Zuweisung") ein unfertiges Objekt ist (noch zu bauen) und die rechte Seite ein fertiges Objekt ist. Der Zuweisungs-Operator wird dagegen aufgerufen, wenn auf beiden Seiten ein fertiges Objekt vorliegt.

Innerhalb einer Klasse kann man über den **Pointer this** auf das eigene Objekt zugreifen (analog zu `self` in Python oder `this` in Java, dort aber Referenzen).

Bei statischen Methoden und Attributen wird die Deklaration als **static** nicht in der Implementierung wiederholt! Statische Attribute müssen außerhalb der Klassendefinition einmal initialisiert werden!

Methoden können als “konstant” ausgezeichnet werden (**const** rechts von der Parameterliste). Das **const** gehört zur Signatur der Methode! Konstante Methoden dürfen auf konstanten Objekten/Referenzen aufgerufen werden.

Videos

- [VL C++: Big 3](#)

7.4.1. Big Three

Neben dem eigentlichen Konstruktor existieren in C++ weitere wichtige Konstruktoren/Operatoren: die sogenannten “Big Three”:

- Copy-Konstruktor
- Destruktor: Gegenstück zum Konstruktor
- Zuweisungsoperator (**operator=**)

Anmerkung: Für Fortgeschrittenere sei hier auf die in C++11 eingeführte und den Folgeversionen verbesserte und verfeinerte [Move-Semantik](#) und die entsprechenden Varianten der Konstruktoren und Operatoren verwiesen. Man spricht deshalb mittlerweile auch gern von den “Big Five” bzw. der “[rule of five](#)”.

```
1 class Dummy {
2 public:
3     Dummy(int a=0);
4     Dummy(const Dummy &d);
5     ~Dummy();
6     Dummy &operator=(const Dummy &d);
7 private:
8     int value;
9 };
```

```
1 Dummy::Dummy(int a): value(a) {}
2 Dummy::Dummy(const Dummy &d): value(d.value) {}
3 Dummy::~~Dummy() {}
4 Dummy::Dummy &operator=(const Dummy &d) {
5     if (this != &d) { value = d.value; }
6     return *this;
7 }
```

7.4.1.1. Big Three: Destruktor

- Syntax: `Dummy::~~Dummy()` ;
(Konstruktor mit vorgesetzter Tilde)
- Wird aufgerufen:

- wenn ein Objekt seinen Scope verlässt, oder
- wenn explizit `delete` für einen Pointer auf ein Objekt (auf dem Heap!) aufgerufen wird
- Default-Destruktor ruft Destrukturen der Objekt-Attribute auf

[Konsole: destruktork.cpp](#)

7.4.1.2. Big Three: Copy-Konstruktor

- Syntax: `Dummy::Dummy(const Dummy &);`
- Wird aufgerufen bei:
 - Deklaration mit Initialisierung mit Objekt
 - Objektübergabe und -rückgabe mit Call-by-Value
 - Nicht bei Zuweisung
- Default-Copy-Konstruktor kopiert einfach elementweise
=> bei Pointern also nur **flache Kopie**

“**Merkregel**”: Linke Seite unfertiges Objekt (noch zu bauen), rechte Seite fertiges Objekt.

[Konsole: copyKonstruktor.cpp](#)

7.4.1.3. Big Three: Zuweisungsoperator

- Syntax: `Dummy &Dummy::operator=(const Dummy &)`
- Wird aufgerufen:
 - bei Zuweisung bereits initialisierter Objekte
- Default-Zuweisungsoperator kopiert einfach elementweise
=> bei Pointern also nur **flache Kopie**

“**Merkregel**”: Linke Seite fertiges Objekt, rechte Seite fertiges Objekt.

[Konsole: zuweisungsOperator.cpp](#)

7.4.1.4. Big Three: Defaults

Analog zum Default-Konstruktor kann der Compiler auch Defaults für die Big Three (Copy-Konstruktor, Destruktor, Zuweisungsoperator) generieren. Das funktioniert nur, so lange Sie nicht selbst einen Copy-Konstruktor, Destruktor oder Zuweisungsoperator definiert haben.

Diese Defaults passen normalerweise, wenn die Data-Member vom Typ `int`, `double`, `vector<int>`, `string`, `vector<string>` o.ä. sind.

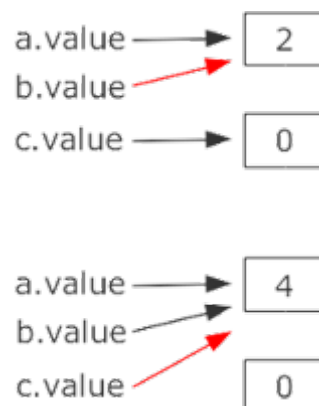
Problematisch wird es, wenn Pointer dabei sind: Dann werden flache Kopien erzeugt bzw. Speicher auf dem Heap nicht oder mehrfach freigegeben! Sobald Sie für die Attribute Pointer verwenden, sollten Sie eigene Konstruktoren, Copy-Konstruktoren, Destrukturen und Zuweisungsoperatoren definieren!

Hier ein Beispiel für die Wirkung:

```

1  class Dummy {
2  public:
3      Dummy(int initValue = 0) {
4          value = new int(initValue);
5      }
6
7      int getValue() {
8          return *value;
9      }
10     void setValue(int a) {
11         *value = a;
12     }
13 private:
14     int *value;
15 };
16
17 void main() {
18     // oberer Teil der Abbildung
19     Dummy a(2);
20     Dummy b = a;
21     Dummy c;
22
23     // unterer Teil der Abbildung
24     c=b;
25     a.setValue(4);
26 }

```



Analyse:

1. Es sind Pointer im Spiel. Es wurde ein eigener Konstruktor definiert, aber kein Copy-Konstruktor, d.h. diesen "spendiert" der Compiler.
2. Beim Anlegen von `a` wird auf dem Heap Speicher für einen `int` reserviert und dort der Wert 2 hineingeschrieben.
3. Beim Anlegen von `b` wird der Default-Copy-Konstruktor verwendet, der einfach elementweise kopiert. Damit zeigt der Pointer `value` in `b` auf den selben Speicher wie der Pointer `value` in `a`.
4. Der Ausdruck `c=b` ist eine Zuweisung (warum?). Auch hier wird der vom Compiler bereitgestellte Default genutzt (elementweise Zuweisung). Damit zeigt nun auch der Pointer `value` in `c` auf den selben Speicher wie die `value`-Pointer in `a` und `b`.

7.4.1.5. Hinweis Abarbeitungsreihenfolge

```
1 Dummy a(0); Dummy b(1); Dummy c(2); Dummy d(3);
2 a = b = c = d; // entspricht: a.operator=(b.operator=(c.operator=(d)));
```

7.4.2. delete this?

Erinnerung:

- **this** ist ein Pointer auf das eigene Objekt
- **delete** darf nur für Pointer auf Objekte, die mit **new** angelegt wurden, aufgerufen werden => Freigabe von Objekten auf dem Heap!
- **delete** ruft den Destruktor eines Objekts auf ...

Frage: Ist das folgende Konstrukt sinnvoll? Ist es überhaupt erlaubt? Was passiert dabei?

```
1 class Foo {
2 public:
3     ~Foo() {
4         delete this;
5     }
6 };
```

Analyse: Wir haben hier gleich zwei Probleme:

1. **delete** ruft den Destruktor des verwiesenen Objekts auf. Da **this** ein Pointer auf das eigene Objekt ist, ruft **delete this;** den eigenen Destruktor auf, der dann wiederum **delete this;** aufruft und so weiter. => Endlosschleife!
2. Außerdem wissen wir im Destruktor bzw. im Objekt gar nicht, ob das Objekt wirklich mit **new** auf dem Heap angelegt wurde! D.h. wenn wir nicht in die Endlosschleife eintreten würden, würde das Programm abstürzen.

Der Destruktor wird aufgerufen, wenn ein Objekt zerstört wird, d.h. wenn ein Objekt seine Lebensdauer beendet (Verlassen des Scopes, in dem das Objekt definiert wurde) bzw. wenn explizit ein **delete** auf das Objekt aufgerufen wird (d.h. **delete** auf einen Pointer auf das Objekt, wobei dieses mit **new** angelegt wurde).

Im Destruktor sollten **durch das Objekt verwaltete Ressourcen freigegeben** werden, d.h. sämtliche im Objekt mit **new** oder **malloc** allozierten Ressourcen auf dem Heap müssen freigegeben werden. Außerdem sollten ggf. offene Verbindungen (offene Dateien, Datenbankverbindungen, Kommunikation, ...) geschlossen werden, wenn sie durch das Objekt geöffnet wurden bzw. in der Verantwortung des Objekts stehen. Einfache Datentypen oder Objekte, die nicht per Referenz oder Pointer im Objekt verwaltet werden, werden automatisch freigegeben (denken Sie an das Speichermodell - diese Daten "stehen" direkt im Speicherbereich des Objekts).

Der Speicherbereich für das Objekt selbst wird nach Beendigung des Destruktors automatisch freigegeben (auf dem Stack wegen des Verlassen des Scopes => automatische Variable), auf dem Heap durch das vorherige Aufrufen von **delete** auf den Pointer auf das Objekt im Heap), d.h. Sie brauchen im Destruktor **kein delete** auf "sich selbst" (das ist wie oben demonstriert sogar schädlich)!

Warning

Auch wenn es zunächst irgendwie sinnvoll aussieht - rufen Sie **niemals nie** `delete this` im Destruktor auf!

Konsole: [deletethis.cpp](#)

7.4.3. C++11: default und delete

```
1 class Dummy {
2 public:
3     Dummy() = default;
4     Dummy(int a) { value = a; }
5     Dummy(const Dummy &a) = delete;
6
7 private:
8     int value;
9     Dummy &operator=(const Dummy &d);
10 };
```

- C++ erzeugt etliche triviale Methoden/Operatoren, sofern man diese nicht selbst definiert:
 - Methoden:
 - * Standardkonstruktor
 - * Copy-Konstruktor
 - * Zuweisungsoperator
 - * Destruktor
 - Operatoren:
 - * Operator `new`
 - * Operator `delete`
 - * Adresse von
 - * Indirektion
 - * Elementzugriff
 - * Elementindirektion
- Vor C++11: Default-Methode/-Operator verbieten: Sichtbarkeit auf `private` setzen (Definition nicht nötig)
- Ab C++11: Schlüsselwort `delete`: Entfernt Default-Methode/-Operator
- C++11: Default-Methode/-Operator zusätzlich zu selbst implementierten: Schlüsselwort `default`

7.4.4. Statische Methoden und Attribute

```
1 class Studi {
2     static int getCount();
3     static int count;
4 };
```

```

1  int Studi::count = 0;
2
3  int Studi::getCount() {
4      return Studi::count;
5  }

```

- Deklaration als **static nicht** in Implementierung wiederholen
- Statische Attribute: Initialisierung **immer** außerhalb der Klasse!

[Konsole: Studi.cpp \(static\)](#)

7.4.5. Konstante Methoden und Kontexte

```

1  class Studi {
2      int getCredits() const;
3      int getCredits();
4  };

```

```

1  int Studi::getCredits() const {
2      return credits;
3  }
4
5  int Studi::getCredits() {
6      return credits;
7  }

```

Das **const** gehört zur Signatur der Methode!

So wie im Beispiel gezeigt, gibt es jetzt zwei Methoden `getCredits()` - eine davon ist konstant. Konstante Methoden dürfen auf konstanten Objekten/Referenzen aufgerufen werden.

Was passiert, wenn das **const** auf der linken Seite steht? Dann bezieht es sich auf den Rückgabewert:

```

1  const foo wuppie(foo&, foo&);

```

Hier darf der Rückgabewert nicht als L-Wert benutzt werden: `wuppie(a,b) = c`; ist verboten.

[Konsole: Studi.cpp \(const\)](#)

7.4.6. Wrap-Up

- Klassen: Destruktoren, Copy-Konstruktor, Zuweisungsoperator
- Vorsicht mit Default-*strukturen/-operatoren
- Statische Methoden und Attribute:
 - Deklaration als **static nicht** in Implementierung wiederholen
 - Statische Attribute: Initialisierung außerhalb der Klasse!
- Konstante Methoden und Kontexte

- **const** gehört zur Signatur der Methode!
- Konstante Methoden dürfen auf konstanten Objekten/Referenzen aufgerufen werden

Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Ich kann die Problematik mit Defaultkonstruktoren/-operatoren bei der Verwendung von Pointern an einem Beispiel erklären
- k2: Ich kann die Problematik konstanter Funktionen erklären und weiss, wann diese aufgerufen werden
- k3: Ich kann die ‘Big Three’ in eigenen Programmen definieren und aufrufen

Challenges

Konstruktor, Copy-Konstruktor, Zuweisungsoperator?

- Erklären Sie die folgenden Anweisungen, worin liegt der Unterschied?

```
1 Dummy a;  
2 Dummy b = 3;  
3 Dummy c(4);
```

- Erklären Sie die folgenden Anweisungen:

```
1 Dummy a;  
2 Dummy b = 3;  
3 Dummy c(4);  
4 Dummy d = b;  
5 Dummy e(b);  
6 Dummy f;  
7 f = b;
```

Destruktor

- Erklären Sie die Wirkungsweise eines Destruktors.
- Wann wird ein Destruktor aufgerufen?
- Warum ist `delete this` keine gute Idee (nicht nur im Destruktor)?!
- Was sollten Sie im Destruktor aufräumen, was nicht?

Die “Großen Drei”

1. Beschreiben Sie den Unterschied der folgenden beiden Codeblöcke (A sei eine beliebige Klasse):

```
1 A a, b = a;
```

```
1 A a, b; b = a;
```

2. Erläutern Sie an einem **Beispiel** die Regel der “Big Three”:

Ist ein Copy-Konstruktor, ein Destruktor oder ein eigener Zuweisungsoperator notwendig, muss man in der Regel die jeweils anderen beiden ebenfalls bereit stellen.

3. Beim Zuweisungsoperator werden Selbstzuweisungen, d.h. ein Objekt soll an sich selbst zugewiesen werden, üblicherweise durch eine entsprechende Prüfung vermieden.

Begründen Sie diese Praxis, indem Sie ein **Beispiel konstruieren**, bei dem es zu Datenverlust kommt, wenn die Selbstzuweisung nicht unterbunden wird.

Wenn vor der Wertzuweisung der alte Inhalt freigegeben werden muss, führt Selbstzuweisung zum Fehler.

Können Sie ein konkretes Beispiel angeben?

Quiz: Was passiert bei den folgenden Aufrufen?

```
1 class Foo {
2 public:
3     const Foo &bar(const vector<Foo> &a) { return a[0]; }
4 };
5
6 int main() {
7     Foo f; vector<Foo> a = {"hello", "world", ":",});
8
9     Foo s1 = f.bar(a);
10    const Foo &s2 = f.bar(a);
11    Foo &s3 = f.bar(a);
12    Foo s4;
13    s4 = f.bar(a);
14
15    return EXIT_SUCCESS;
16 }
```

7.5. C++: Operatoren

TL;DR

In C++ können existierende Operatoren überladen werden, etwa für die Nutzung mit eigenen Klassen. Dabei kann die Überladung innerhalb einer Klassendefinition passieren (analog zur Implementierung einer Methode) oder außerhalb der Klasse (analog zur Definition einer überladenen Funktion).

Beim Überladen in einer Klasse hat der Operator nur einen Parameter (beim Aufruf das Objekt auf der rechten Seite) und man kann auf die Attribute der Klasse direkt zugreifen. Bei der Überladung außerhalb der Klasse hat der Operator zwei Parameter und darf nicht auf die Attribute der Klasse zugreifen.

Man kann Funktionen, Methoden/Operatoren und Klassen als **friend** einer Klasse deklarieren. Damit bricht man die Kapselung auf und erlaubt den Freunden den direkten Zugriff auf die internen Attribute einer Klasse.

Um bei der Implementierung von Post- und Präfix-Operatoren die Variante für den Compiler unterscheidbar zu machen, hat die Signatur der Postfix-Variante einen Dummy-Parameter vom Typ **int**. Dieser wird beim Aufruf aber nicht genutzt.

Videos

- [VL C++: Operatoren](#)

7.5.1. Überladen von Operatoren *in* Klassen

```
1 MyString a, b("hallo");
2 a = b;           // ???
```

```
1 a.operator=(b);
```

Aufruf `a=b` ist äquivalent zu `a.operator=(b)`

Überladen ähnlich wie bei Methoden:

```
1 class MyString {
2     MyString &operator=(const MyString &s) {
3         if (&this != &s) {
4             // mach was :-))
5         }
6         return *this;
7     }
8 };
```

Analog weitere Operatoren, etwa `operator==`, `operator+`, ... überladen

7.5.2. Überladen von Operatoren *außerhalb* von Klassen

```
1 MyString a("hallo");
2 cout << a << endl;
```

```
1 class MyString {
2     ostream &operator<<(ostream &o) { return o << str; }
3 };
```

So funktioniert das leider nicht!

- Erinnerung: `cout << a` entspricht `cout.operator<<(a)`
 - Operator kann nicht in `MyString` überladen werden!
 - Klasse `ostream` müsste erweitert werden
 - => Geht aber nicht, da System-weite Klasse!

=> Lösung: Operator **außerhalb** der Klasse überladen => 2 Parameter

7.5.3. Überladen von Operatoren *außerhalb* von Klassen (cnt.)

Operator außerhalb der Klasse überladen => 2 Parameter

```
1 ostream &operator<<(ostream &out, const MyString &s) {
2     return out << s.str;
3 }
```

- Nachteil: Benötigt Zugriff auf Klassen-Interna
 - entweder umständlich über Getter-Funktionen
 - oder als `friend` der Klasse `MyString` deklarieren
 - Alternativ Zugriffsmethoden (aka *Getter*) nutzen wie `toString()` ...

Anmerkung: Rückgabe der Referenz auf den Stream erlaubt die typische Verkettung: `cout << s1 << s2 << endl;`

7.5.4. Meine Freunde dürfen in mein Wohnzimmer

```
1 void test();
2
3 class TestDummy {
4     int ganzTolleMethode();
5 };
6
7
8 class Dummy {
9     private:
10        int *value;
11
12        friend class TestDummy;
13        friend int TestDummy::ganzTolleMethode();
14        friend void test();
15 };
```

7.5.5. (Fast) alle Operatoren lassen sich überladen

- Alle normalen arithmetischen Operatoren
- Zuweisung, Vergleich, Ein-/Ausgabe
- Index-Operator `[]`, Pointer-Dereferenzierung `*` und `->`, sowie `()`, **new** und **delete** (auch in `[]`-Form)

- **Ausnahmen:**

1. `.`
2. `::`
3. `?:`
4. **sizeof**

- Anmerkungen:

- Beim Überladen muss die Arität erhalten bleiben
- Nur *existierende* Operatoren lassen sich überladen
=> Es lassen sich keine neuen Operatoren erschaffen

Vgl. Tabelle 9.1 (S. 318) im Breymann (2011)

7.5.6. Implizite Typkonvertierungen bei Aufruf

```
1 MyString s;
2 s != "123";    // ???
3 "123" != s;    // ???
```

- Operatoren **in** Klasse überladen: Typ der linken Seite muss **exakt** passen

```
1 class MyString {
2 public:
3     MyString(const char *s = "");
4     bool operator!=(const MyString&);
5 };
6
7 MyString s;
8 s != "123";    // impliziter Aufruf des Konstruktors, danach MyString::
9               operator!=
10 "123" != s;    // KEIN operator!=(char*, MyString&) vorhanden!
```

Das ist letztlich wie bei einem Methodenaufruf: Um die richtige Methode aufzurufen, muss der Typ (die Klasse) des Objekts bekannt sein.

- Operatoren **außerhalb** überladen: Konvertierung auf *beiden* Seiten möglich

```

1 class MyString {
2 public:
3     MyString(const char *s = "");
4 };
5 bool operator!=(const MyString&, const MyString&);

```

NIEMALS beide Formen gleichzeitig für einen Operator implementieren!

7.5.7. Anmerkung zu “++” und “--” Operatoren: Präfix und Postfix

- Präfix: `o1 = ++o2;`
 - Objekt soll **vor Auswertung** inkrementiert werden
 - Signatur: `Typ &operator++()`
- Postfix: `o1 = o2++;`
 - Objekt soll erst **nach Auswertung** inkrementiert werden
 - Signatur: `Typ operator++(int)` ($\Rightarrow$ `int` dient nur zur Unterscheidung der Präfix-Variante, wird **nie** benutzt)

7.5.8. Weitere Anmerkungen

- Operatoren werden **nicht** vom System zusammengesetzt
 - `operator+` und `operator+=` sind zwei verschiedene Operatoren!
 - Implementierung ist prinzipiell unabhängig!
 - $\Rightarrow$ Erwartung: `operator+= == (operator+ + operator=)`
- Operatoren lassen sich in C++ verketten:

```

1 Dummy a(0); Dummy b(1); Dummy c(2);
2 a = b = c; // a.operator=(b.operator=(c));

```

- Übertreiben Sie nicht!

```

1 Firma f;
2 Person p;
3 f += p; // ??!

```

Nutzen Sie im Zweifel lieber Methoden mit aussagekräftigen Namen!

7.5.9. Wrap-Up

- Überladen von Operatoren (innerhalb und außerhalb einer Klasse)
 - Innerhalb: 1 Parameter (Objekt auf der rechten Seite)
 - Außerhalb: 2 Parameter
- Zugriff auf Attribute: `friend` einer Klasse
- Implementierung von Post- und Präfix-Operatoren

Zum Nachlesen

- Breyman (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Ich kann implizite Typkonvertierungen bei Operatoren an einem Beispiel erklären
- k3: Ich kann in meinem Programm Operatoren überladen (innerhalb bzw. außerhalb einer Klasse)
- k3: Ich kann die Deklaration als `friend` gezielt anwenden
- k3: Ich kann Post- und Präfix-Operatoren implementieren und kenne die Unterschiede in der Signatur

Challenges

Operator “++”

Betrachten Sie die folgende Klasse:

```
1 class Studi {
2 public:
3     Studi(int credits);
4     ~Studi();
5 private:
6     int *credits;
7 };
```

Implementieren Sie den `operator++` sowohl in der Präfix- als auch in der Postfix-Variante.

C'toren und Operatoren: Was muss noch *deklariert* werden?

```
1 class Studi {
2 public:
3     Studi(int credits);
4 private:
5     int *credits;
6 };
7
8 int main() {
9     Studi a(1), b, *c = new Studi(99);
```

```
10     b = *c+a+1;
11     std::cout << "b: " << b << " credits" << std::endl;
12
13     return 0;
14 }
```

```
1 test wuppie;
2 bool fluppie = wuppie(3);
```

7.6. C++: Vererbung und Polymorphie

TL;DR

Vererbung analog zu Java passiert in C++ über die “**public**-Vererbung”: **Subklasse** : **public Superklasse**. Dabei gibt es in C++ **keine** gemeinsame Oberklasse wie **Object** und entsprechend kein **super**. (Es kann auch private Vererbung geben.)

Operatoren und *strukturen werden in den vom Compiler erzeugten Defaults richtig verkettet. Bei der eigenen Implementierung von Operatoren und Konstruktoren muss zunächst der Operator/Konstruktor der Basisklasse aufgerufen werden (Basisklassen-Konstruktoren dabei in der Initialisierungsliste!), danach erfolgt die Implementierung für die eigenen Attribute der abgeleiteten Klasse. Der Zugriff auf die Elemente der Elternklasse erfolgt dabei über den Namen der Elternklasse und den Scope-Operator (nicht mit **super**!). Destrukturen von abgeleiteten Klassen müssen sich dagegen nur um die zusätzlichen Attribute der abgeleiteten Klasse kümmern, der Basisklassendestruktor wird automatisch verkettet bzw. aufgerufen.

Abstrakte Klassen in C++ haben mindestens eine abstrakte Methode. Eine Methode ist abstrakt, wenn sie als “**virtual**” deklariert ist **und** der Deklaration ein “=0” folgt.

In C++ hat man aus Effizienzgründen per Default statische Polymorphie. Bei der Zuweisung eines Objekts einer abgeleiteten Klasse (rechte Seite) an ein Objekt vom Typ der Oberklasse (linke Seite) erfolgt dabei “Slicing”, d.h. alle zusätzlichen Eigenschaften der abgeleiteten Klasse gehen dabei verloren. Dynamische Polymorphie kann man in C++ nutzen, indem man (a) die gewünschten Methoden in der Basisklasse als **virtual** deklariert und (b) für den Zugriff auf die Objekte der abgeleiteten Klasse Pointer oder Referenzen vom Basisklassen-Typ benutzt.

In C++ ist Mehrfachvererbung möglich, d.h. eine Klasse kann von mehreren anderen Klassen erben. Damit erbt sie auch das Objekt-Layout aller Elternklassen.

Bei rautenförmigen Vererbungsbeziehung führt dies zu Problemen, da Attribute und Methoden der gemeinsamen Basisklasse mehrfach vorhanden (über jeden Zweig der Raute).

Zur Umgehung des Problems kann man die gemeinsam genutzten Basisklassen “**virtual**” deklarieren. Dadurch sind gemeinsam genutzte Attribute und Methoden nur noch einfach vorhanden. Da die Klassen “in der Raute” ihrerseits den Konstruktor der Basisklasse aufrufen (könnten) und es dadurch zu Konflikten

beim Setzen der Attribute der Basisklasse kommen kann, gelten bei virtueller Ableitung Sonderregeln: Für die virtuelle Basisklasse wird die Weiterleitung der Werte aufgehoben (es muss also ein parameterloser Konstruktor existieren, der durch die direkten Unterklassen aufgerufen wird) und die Klasse am “unteren Ende der Raute” kann direkt den Konstruktor der virtuellen Basisklasse am “oberen Ende der Raute” aufrufen.

Videos

- [VL C++: Vererbung und Polymorphie](#)

7.6.1. Vererbung: “IS-A”-Beziehung zw. Klassen

```
1 class Student : public Person { ... }
```

```
1 Student(const string &name = "", double c = 0.0)
2 : Person(name), credits(c) { }
3
4 Student(const Student &s)
5 : Person(s), credits(s.credits) { }
```

Analog zu Java:

- **Student**: abgeleitete Klasse
- **Person**: Basisklasse
- **: public**: Vererbungsbeziehung (analog zu **extends** in Java)
- **public**-Vererbung: Verhalten wie in Java
- Hinweis: Es gibt weitere Spielarten (**protected**, **private**), vgl. Semesterliteratur
- Ab C++11:
 - Schlüsselwort **override**: Die Methode muss eine virtuelle Methode der Klassenhierarchie überschreiben.
 - Schlüsselwort **final**: Die virtuelle Methode darf nicht in abgeleiteten Klassen überschrieben werden.

7.6.1.1. Vererbung und Konstruktoren

- Defaultkonstruktoren werden automatisch richtig verkettet
 - zuerst Aufruf des Basisklassen-Konstruktors
 - anschließend Behandlung der zusätzlichen Attribute
- Eigene Konstruktoren verketteten:
 - Zuerst Basisklassen-Konstruktor aufrufen (in Initialisierungsliste!)
 - => Konkreten Konstruktor nehmen, nicht **super** wie in Java

7.6.1.2. Vererbung und Destruktoren

- Defaultdestruktoren werden automatisch richtig verkettet
 - zuerst werden die Destruktoren der zusätzlichen Attribute aufgerufen
 - dann der Destruktor der Basisklasse
- Eigene Destruktoren werden automatisch verkettet
- Destruktor abgeleiteter Klasse muss sich nur um zusätzliche Attribute kümmern

7.6.1.3. Vererbung und Operatoren

- Defaultoperatoren werden automatisch richtig verkettet
 - zuerst Aufruf des Basisklassen-Operators
 - anschließend Behandlung der zusätzlichen Attribute
- Eigene Operatoren am Beispiel Zuweisungsoperator:
 - Zuerst den Zuweisungsoperator der Basisklasse aufrufen
 - Zugriff über Superklassennamen und Scope-Operator (nicht mit **super**!)

```
1  const Student &operator=(const Student &s) {
2      if (&this != &s) {
3          Person::operator=(s);
4          credits = s.credits;
5      }
6      return *this;
7  }
```

7.6.1.4. Vererbung von Freundschaften

- Freundschaften werden nicht vererbt!
- **friends** der Basisklasse haben keinen Zugriff auf zusätzliche private Attribute/Methoden der Unterklassen
- Aber: weiterhin Zugriff auf die geerbten privaten Elemente!

7.6.2. Abstrakte Klassen

- Eine **Klasse** ist abstrakt, wenn sie mindestens eine abstrakte Methode hat
- Eine **Methode** ist in C++ abstrakt, wenn sie
 1. als virtuell deklariert ist, **und**
 2. der Deklaration ein “=0” folgt

Abstrakte Methoden können Implementierung haben! => Implementierung außerhalb der Klassendeklaration

```

1 class Person {
2 public:
3     virtual string toString() const = 0;
4     ...
5 };
6
7 string Person::toString() const { ... } // Implementierung :-)
```

7.6.3. Polymorphie: Was passiert im folgenden Beispiel?

IS-A Beziehung: Objekte können als Objekte ihrer Oberklasse behandelt werden

```

1 class Person { ... }
2 class Student : public Person { ... }
3
4 Student s("Heinz", "heizer");
5 Person &p = s;
6
7 cout << s.toString() << endl;
8 cout << p.toString() << endl;
```

Konsole: polyStat.cpp

Antwort: Es wird die falsche Methode aufgerufen!

- `s.toString()` => `Student::toString()` => wie erwartet
- `p.toString()` => `Person::toString()` => **unerwartet!**

7.6.4. Polymorphie: statisch und dynamisch

- C++ entscheidet zur **Kompilierzeit**, welche Methode aufgerufen wird
 - `p` ist vom Typ `Person` => `p.toString()` => `Person::toString()`
 - Dieses Verhalten wird **statisches Binden** genannt.
- Von Java her bekannt: **dynamisches Binden**
 - Typ eines Objektes wird zur **Laufzeit** ausgewertet

7.6.5. Dynamisches Binden geht auch in C++ ...

Für dynamische Polymorphie müssen in C++ drei Bedingungen erfüllt sein:

1. Methoden in **Basisklasse** als **virtuelle Funktion** deklarieren
=> Schlüsselwort `virtual`

2. Virtuelle Methoden in Subklasse normal überschreiben (gleiche Signatur)

Zusätzlich muss der Rückgabotyp exakt übereinstimmen

(Ausnahme: Rückgabe Pointer/Referenz auf *abgeleitete* Klasse)

3. Objekte mittels Basisklassen-Referenzen bzw. -Pointer zugreifen (siehe nächste Folie)

```
1 class Person {
2     virtual string toString() const { ... }
3 };
```

Konsole: polyDyn.cpp

7.6.6. Vorsicht Slicing

```
1 Student s("Heinz", 10.0);
2 Person p("Holger");
3
4 p = s;
5 cout << "Objekt s (Student): " << s.toString() << endl;
6 cout << "Objekt p (Person): " << p.toString() << endl;
```

Konsole polySlicing.cpp

=> **p** ist vom Typ **Person**

- Zuweisung von Objekten vom Typ **Student** ist erlaubt (Polymorphie)
- **p** hat aber nur Speicherplatz für genau eine **Person** => "Abschneiden" aller Elemente, die nicht Bestandteil von **Person** sind!
- Slicing passiert immer beim Kopieren/Zuweisen von Objekten

=> **Dyn. Polymorphie** in C++ immer über **Referenzen** (bzw. Pointer) **und virtuelle Methoden**

Wir hatten die Methode **toString** in der Basisklasse **Person** zwar als **virtual** deklariert, und wir hatten diese Methode in der ableitenden Klasse **Studi** passend überschrieben.

Damit haben wir aber nur zwei der drei Bedingungen für dynamische Polymorphie in C++ erfüllt. Wenn wir Objekte vom Typ **Studi** über eine normale Variable vom Typ **Person** handhaben, haben wir immer noch statische Polymorphie - uns stehen also nur die Methoden aus und in **Person** zur Verfügung.

Zusätzlich haben wir durch die Zuweisung **p = s**; das Objekt **s** in den Speicherbereich von **p** "gequetscht". Dieses ist vom Typ **Person** und hat auch nur (Speicher-) Platz für Elemente dieses Typs. Alles andere wird bei der Zuweisung "abgeschnitten", d.h. **p** ist immer noch ein Objekt vom Typ **Person**, der zusätzliche Rest aus **Studi** fehlt ...

Wir könnten das durch Pointer oder Referenzen heilen:

```
1 // Variante mit Basisklassen-Pointer
2 Student s("Heinz", 10.0);
3 Person *p;
```

```

4
5 p = &s;
6 cout << "Objekt s (Student): " << s.toString() << endl;
7 cout << "Objekt p (Person): " << p->toString() << endl;

```

Anmerkung: Der Operator `->` ist die zusammengefasste Dereferenzierung des Pointers und der nachfolgende Zugriff auf Methoden oder Attribute. Man könnte also entsprechend auch `(*p).toString()` statt `p->toString()` schreiben.

```

1 // Variante mit Basisklassen-Referenz
2 Student s("Heinz", 10.0);
3 Person &p = s;
4
5 cout << "Objekt s (Student): " << s.toString() << endl;
6 cout << "Objekt p (Person): " << p.toString() << endl;

```

Erst damit erfüllen wir die dritte Bedingung und haben echte dynamische Polymorphie in C++.

7.6.7. Anmerkungen zu Polymorphie in C++

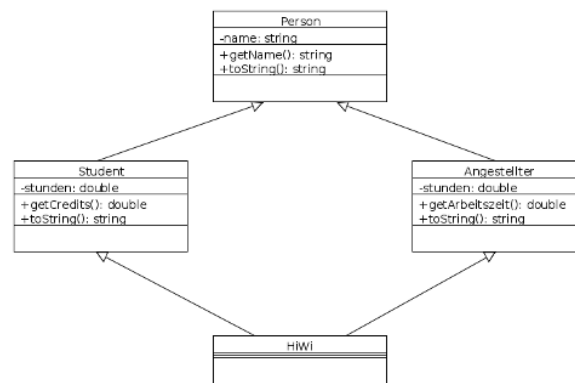
- **Gestaltung der API:**
 - Zum Überschreiben gedachte Methoden als virtuell deklarieren
 - Nicht virtuelle Methoden aus der Basisklasse nicht überschreiben
- Trennung von Deklaration und Implementierung:
 - Deklaration als virtuelle Funktion nur im Deklarationsteil
 - Keine Wiederholung im Implementierungsteil (analog zu Defaultwerten)
- “Virtualität vererbt sich”:
 - Virtuelle Funktionen sind virtuell in der Vererbungshierarchie hinab ab der ersten Deklaration als virtuell
- Virtualität ist “teuer”: Es muss eine Tabelle aller virtuellen Funktionen aufgebaut werden und zur Laufzeit geprüft werden, welche Funktion genommen werden soll

7.6.8. Mehrfachvererbung in C++

```

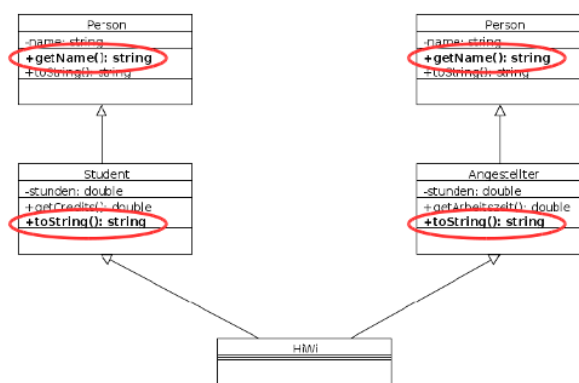
1 class HiWi: public Student, public Angestellter {...};

```



Hinweis Speicherlayout ...

7.6.8.1. Problem 1: Gleichnamige Methoden aus Basisklassen geerbt



Namenskollision bei Mehrfachvererbung auflösen:

- Scope-Operator :: nutzen:

```

1 HiWi h("Anne", 23.0, 40.0);
2
3 cout << h.Student::toString() << endl;
4 cout << h.Angestellter::toString() << endl;
5 cout << h.Student::getName() << endl;
6 cout << h.Angestellter::getName() << endl;

```

- Methode in abgeleiteter Klasse überschreiben

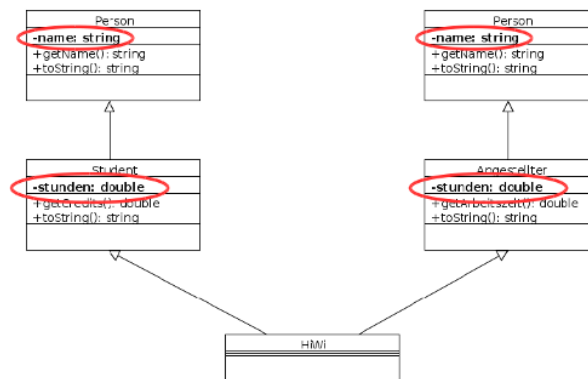
```

1 HiWi h("Anne", 23.0, 40.0);
2
3 cout << h.toString() << endl;
4 cout << h.Student::toString() << endl;
5 cout << h.Angestellter::toString() << endl;

```

Konsole vererbungMultiMethoden.cpp

7.6.8.2. Problem 2: Gemeinsam geerbte Attribute sind mehrfach vorhanden



Konsole vererbungMultiAttribute.cpp

7.6.9. Mehrfachvererbung in C++: Virtuelle Basisklassen

```

1 class Angestellter: virtual public Person {...};
2 class Student: virtual public Person {...};
3
4 class HiWi: public Student, public Angestellter {...};
  
```

- `Person` ist jetzt eine virtuelle Basisklasse
- Auswirkungen erst in Klasse `HiWi`
- Dadurch sind gemeinsam genutzte Anteile nur einfach vorhanden

```

1 Student s("Heinz", 10.0); // wie vorher: nur EIN name-Feld
2 Angestellter a("Holger", 80.5); // wie vorher: nur EIN name-Feld
3
4 HiWi h("Anne", 23.0, 40.0); // jetzt auch nur EIN name-Feld
  
```

Konsole vererbungMultiVirtual.cpp

7.6.10. Sonderregeln bei virtueller Ableitung

Virtuelle Ableitung: Potentiell Konflikte zwischen Konstruktoren!

- Gemeinsam geerbtes Attribut nur noch einmal vorhanden
- Konstruktoren werden nacheinander aufgerufen, alle wollen das gemeinsame Attribut initialisieren (durch Aufruf des Konstruktors der jeweiligen Basisklasse)
- Zuletzt aufgerufener Konstruktor würde "gewinnen"

Deshalb gibt es bei virtueller Ableitung folgende Sonderregeln:

1. Für virtuelle Basisklassen ist **Mechanismus des Weiterreichens** von Initialisierungswerten **deaktiviert**

2. Konstruktor einer virtuellen Basisklasse kann in Initialisierungsliste von indirekten Unterklassen aufgerufen werden

Sonst wird der Defaultkonstruktor der virtuellen Basisklasse genutzt!

[Konsole vererbungMultiVirtual.cpp \(Basiskonstruktor\)](#)

7.6.11. Mehrfachvererbung in C++ ist ein recht kompliziertes Thema

Warum ist die Möglichkeit dennoch nützlich?

- In Java kann man nur von einer Klasse erben, aber viele Interfaces implementieren. In C++ gibt es keine Interfaces ...
 - => Interfaces mit abstrakten Klassen Interfaces simulieren
 - => Mehrfachvererbung!

Tatsächlich dürfen Java-Interfaces mittlerweile auch Verhalten implementieren und vererben, wodurch eine ähnliche Situation wie hier in C++ entsteht und es ausgefeilte Regeln für die Konfliktauflösung braucht. Allerdings ist das in Java auf Verhalten beschränkt, d.h. Attribute (Zustand) ist in Java-Interfaces (noch) nicht erlaubt.

7.6.12. Wrap-Up

- **public**-Vererbung in C++: `Subklasse : public Superklasse`
- Keine gemeinsame Oberklasse wie `Object`, kein `super`
- Verkettung von Operatoren und *strukturen
- Abstrakte Klassen in C++
- Statische und dynamische Polymorphie in C++
 - Methoden in Basisklasse als `virtual` deklarieren
 - Dyn. Polymorphie nur mittels Pointer/Referenzen
 - Slicing in C++ (bei Call-by-Value)
- Konzept der Mehrfachvererbung
- Problem bei rautenförmiger Vererbungsbeziehung: Attribute und Methoden mehrfach vorhanden
- Virtuelle Basisklassen: Gemeinsam genutzte Attribute nur noch einfach vorhanden

Zum Nachlesen

- Breymann (2011)
- „C and C++ Reference“ (o. J.)
- Allain und Hoffer (o. J.)

Lernziele

- k2: Ich kann den Unterschied zw. Überladen und Überschreiben an einem Beispiel erklären
- k2: Ich kann das "Slicing" in C++ erklären
- k2: Ich kann typische Probleme bei Mehrfachvererbung erklären
- k2: Ich kann den Einsatz und die Wirkung virtueller Basisklassen erklären
- k3: Ich kann **public**-Vererbung in C++ in eigenen Programmen nutzen
- k3: Ich kann Operatoren und *strukturen verketteten und kenne die Defaultregeln
- k3: Ich kann statische und dynamische Polymorphie in C++ einsetzen
- k3: Ich kann abstrakte Klassen in C++ definieren

Challenges

Destruktoren und Vererbung

Welcher Destruktor würde im folgenden Beispiel aufgerufen?!

```
1 Student *s3 = new Student("Holger", 1.0);  
2 Person *p = s3;  
3  
4 delete p;
```

Vererbung

- Welche Formen der (einfachen) Vererbung gibt es in C++ neben der **public**-Form noch? Was bewirken diese Formen?
- Warum wird in C++ die **public**-Form der Vererbung vorgezogen (zumindest, wenn man dynamische Polymorphie nutzen will)?
- Wie müssen Konstruktoren/Destruktoren richtig verkettet werden?
- Arbeiten Sie das Beispiel auf S. 274 im Breymann (2011): "Der C++ Programmierer" durch.

Virtuelle Methoden, Dynamische Polymorphie in C++

1. Was sind virtuelle Methoden und wie setze ich diese ein?
2. Wozu brauche ich in C++ virtuelle Klassen? Was muss beachtet werden?
3. Was passiert in C++, wenn eine virtuelle Methode innerhalb von Konstruktoren verwendet wird? Schreiben Sie ein kurzes Programm zur Verdeutlichung.
4. Wie verhält es sich mit der Problematik aus (a) in Java?
5. Wie unterscheiden sich in C++ virtuelle und nicht virtuelle Destruktoren? Schreiben Sie ein kurzes Programm zur Verdeutlichung.
6. Was passiert, wenn in C++ aus einem Destruktor heraus eine virtuelle Methode aufgerufen wird?

Hinweis: Möglicherweise müssen jeweils mehrere Fälle betrachtet werden!

Teil III.

Praktikum

Hier finden Sie die Übungsblätter.

8. Blatt 01: Reguläre Sprachen

8.1. A1.1: Sprachen von regulären Ausdrücken (1P)

Welche Sprache wird von dem folgenden regulären Ausdruck beschrieben?

$$a + a(a + b)^* a$$

8.2. A1.2: Bezeichner in Programmiersprachen (3P)

Betrachten Sie eine Programmiersprache, in der die Bezeichner (= Namen für Variablen, Funktionen, Klassen, Methoden, ...) folgenden Aufbau haben:

- Alle Variablennamen beginnen mit **V** oder **v**
- Handelt es sich um globale Variablen, beginnen Sie mit **V**, lokale beginnen mit **v**
- Funktions- und Methodenparameter beginnen mit **p**, Klassenparameter (bei der Definition von Vererbung) beginnen mit **P**
- Weitere Bezeichner müssen mit einem Buchstaben (a-z, A-Z) beginnen
- Die folgenden Zeichen dürfen Buchstaben, Ziffern und ein Unterstrich sein
- Bezeichner dürfen nicht mit einem Unterstrich enden
- Alle Bezeichner müssen aus mindestens zwei Zeichen bestehen

Entwickeln Sie einen regulären Ausdruck, der den Aufbau der Bezeichner beschreibt. Beachten Sie, dass Ihr regex alle zulässigen Bezeichner beschreiben muss, aber keinen einzigen unzulässigen beschreiben darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom regex gematcht werden.

Entwickeln Sie einen DFA, der diese Bezeichner akzeptiert. Beachten Sie, dass Ihr DFA alle zulässigen Bezeichner akzeptieren muss, aber keinen einzigen unzulässigen akzeptieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie, wie sie vom Automaten zeichenweise gelesen und akzeptiert werden.

Entwickeln Sie eine reguläre Grammatik, die diese Bezeichner generiert. Beachten Sie, dass Ihre Grammatik alle zulässigen Bezeichner generieren können muss, aber keinen einzigen unzulässigen generieren darf. Wählen Sie zwei Bezeichner aus der Sprache und zeigen Sie die Ableitungsbäume dazu.

8.3. A1.3: Gleitkommazahlen in Programmiersprachen (2P)

Recherchieren Sie zunächst den Aufbau von Gleitkommazahlen in Python und Java.

Erstellen Sie für jede der beiden Programmiersprachen reguläre Ausdrücke, DFAs und reguläre Grammatiken wie in Aufgabe A1.2. Verifizieren Sie Ihre Lösungen wie in Aufgabe A1.2. Vorgaben, die sich auf Längen oder Werte von Teilen der Zahlen beziehen, ignorieren Sie bitte.

8.4. A1.4: Mailadressen? (1P)

Warum ist der folgende regex ungeeignet für die Verarbeitung von Mailadressen?

$(a - z)^+ @ (a - z) . (a - z)$

Bitte beachten Sie, dass die Schreibweise a-z nicht unserer Definition genügt. Eigentlich müsste jedes Zeichen aufgeführt werden:

$a + b + c + c + \dots + z$ ist besser, aber immer noch nicht richtig. Warum?

Anmerkung: Diese Darstellung wird ab jetzt akzeptiert.

Verbessern Sie den gegebenen regulären Ausdruck.

8.5. A1.5: Der zweitletzte Buchstabe (1P)

Entwickeln Sie einen DFA, der nur Wörter über $\Sigma = \{1, 2, 3\}$ akzeptiert, deren zweitletztes Zeichen dasselbe ist wie das zweite.

8.6. A1.6: Sprache einer regulären Grammatik (2P)

Welche Sprache generiert die folgende Grammatik?

$$S \rightarrow aA$$

$$A \rightarrow dB \mid bA \mid cA$$

$$B \rightarrow aC \mid bC \mid cA$$

$$C \rightarrow \epsilon$$

Können Sie einen regulären Ausdruck oder einen DFA dafür angeben?

9. Blatt 02: CFG

9.1. A2.1: PDA (3P)

Erstellen Sie einen deterministischen PDA, der die Sprache

$$L = \{w \in \{a, b, c\}^* \mid w \text{ hat doppelt so viele a's wie c's}\}$$

akzeptiert.

Beschreiben Sie Schritt für Schritt, wie der PDA die Eingaben *bcaba* und *bccac* abarbeitet.

9.2. A2.2: Akzeptierte Sprache (2P)

Ist der folgenden PDA deterministisch? Warum (nicht)?

q_4 sei der akzeptierende Zustand.

$$\begin{aligned}\delta(q_0, a, \perp) &= (q_0, A \perp) \\ \delta(q_0, a, A) &= (q_0, AA) \\ \delta(q_0, b, A) &= (q_1, BA) \\ \delta(q_1, b, B) &= (q_1, BB) \\ \delta(q_1, c, B) &= (q_2, \epsilon) \\ \delta(q_2, c, B) &= (q_2, \epsilon) \\ \delta(q_2, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, \epsilon) \\ \delta(q_3, d, A) &= (q_3, AA) \\ \delta(q_3, \epsilon, \perp) &= (q_4, \epsilon)\end{aligned}$$

Zeichnen Sie den Automaten. Geben Sie das 7-Tupel des PDA an. Welche Sprache akzeptiert er?

9.3. A2.3: Kontextfreie Sprache (2P)

Welche Sprache generiert die folgende kontextfreie (Teil-) Grammatik?

$$G = (\{\text{Statement}, \text{Condition}, \dots\}, \{\text{"if"}, \text{"else"}, \dots\}, P, \text{Statement})$$

mit

$$P = \{ \begin{array}{ll} \text{Statement} & \rightarrow \text{"if"} \text{ Condition Statement} \mid \text{"if"} \text{ Condition Statement "else"} \text{ Statement} \\ \text{Condition} & \rightarrow \dots \end{array} \}$$

Ist die Grammatik mehrdeutig? Warum (nicht)?

9.4. A2.4: Kontextfreie Grammatik (3P)

Entwickeln Sie eine kontextfreie Grammatik für die Sprache

$$L = \{a^i b^j c^k \mid i = j \vee j = k\}$$

Zeigen Sie, dass die Grammatik mehrdeutig ist. Entwickeln Sie einen PDA für diese Sprache.

10. Blatt 03: ANTLR

10.1. Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung eines einfachen *Pretty Printers* für eine einfache fiktive Sprache mit Expressions und Kontrollstrukturen.

Dazu werden Sie eine passende kontextfreie Grammatik definieren mit Lexer- und Parser-Regeln und dabei auch übliche Vorrangregeln beachten.

Für diese Grammatik erstellen Sie mit Hilfe von ANTLR einen Lexer und einen Parser, die zu einem Eingabeprogramm einen Parse-Tree erzeugen.

Den im Parse-Tree repräsentierten Code des Eingabeprogramms können Sie mit Hilfe einer Traversierung konsistent eingerückt wieder auf der Standardausgabe ausgeben - das ist der *Pretty Printer*.

Sie werden merken, dass viele Strukturen im Parse-Tree für diese Aufgabe nicht relevant sind und den Baum mit einer weiteren Traversierung in einen vereinfachten Baum, den sogenannten Abstract-Syntax-Tree (AST), transformieren und diesen erneut als formatierten Code auf der Konsole ausgeben.

10.2. Methodik

Nutzen Sie das [Starter-Projekt](#) in der Vorgabe.

Laden Sie sich das Projekt herunter, binden Sie es in Ihre IDE ein und vergewissern Sie sich, dass alles funktioniert: Führen Sie das enthaltene Programm aus, ändern Sie die mitgelieferten Beispielgrammatik.

Bauen Sie dann Ihre Grammatik für die Aufgabe schrittweise auf. Testen Sie diese mit Hilfe der Beispielprogramme der Zielsprache (s.u.) und überlegen Sie sich selbst weitere Code-Schnipsel, die Sie mit Ihrem Parser einlesen bzw. die Ihr Parser zurückweisen sollte.¹ Es empfiehlt sich, in dieser Phase mit dem [ANTLR-Plugin für IntelliJ](#) zu arbeiten.

Erkunden Sie dann die Strukturen Ihres Parse-Trees. Diese sind an die Regeln Ihrer Grammatik gekoppelt und sind deshalb so individuell wie Ihre Grammatik. Mit einer Traversierung des Baumes können Sie die gewünschte Ausgabe programmieren und auch die Erstellung des vereinfachten Baumes (AST).

¹Um den Text lesbar zu halten, wird hier oft nur von "Parser" gesprochen - gemeint ist aber die gesamte auf diesem Blatt zu erstellende Toolchain: Lexer - Parser - AST - Ausgabe.

10.3. Sprachdefinition

Ein Programm besteht aus einer oder mehreren Anweisungen (*Statements*).

10.3.1. Anweisungen (*Statements*)

Eine Anweisung ist eine einzeilige Befehlsfolge, beispielsweise eine Zuweisung oder eine Operation. Sie muss immer mit einem Newline abgeschlossen werden. Eine Anweisung hat keinen Wert.

```
1 a := 10 - 5 # Zuweisung des Ausdrucks 10-5 (Integer-Wert 5) an die Variable a
2 b := "foo"  # Zuweisung des Ausdrucks "foo" (String) an die Variable b
```

Kontrollstrukturen (s.u.) zählen ebenfalls als Anweisungen.

10.3.2. Ausdrücke (*Expressions*)

Die einfachsten Ausdrücke sind Integer- oder String-Literale. Variablen sind ebenfalls Ausdrücke. Komplexere Ausdrücke werden mit Hilfe von Operationen gebildet, dabei sind die Operanden jeweils auch wieder Ausdrücke. Ein Ausdruck hat/ergibt immer einen Wert.

Die Operatoren besitzen eine Rangfolge, um verschachtelte Operationen aufzulösen. Sie dürfen daher nicht einfach von links nach rechts aufgelöst werden. Die Rangfolge der Operatoren entspricht der üblichen Semantik (vgl. Java, C, Python).

Es gibt in unserer Sprache folgende Operationen mit der üblichen Semantik:

10.3.2.1. Vergleichsoperatoren

Operation	Operator
Gleichheit	==
Ungleichheit	!=
Größer	>
Kleiner	<

10.3.2.2. Arithmetische Operatoren

Operation	Operator
Addition / String-Literal-Verkettung	+
Subtraktion	-

Operation	Operator
Multiplikation	*
Division	/

10.3.2.3. Beispiele für Ausdrücke

```

1  10 - 5  # Der Integer-Wert 5
2  "foo"   # Der String "foo"
3  a       # Wert der Variablen a
4  a + b   # Ergebnis der Addition der Variablen a und b

```

10.3.3. Bezeichner

Werden zur Bezeichnung von Variablen verwendet. Sie bestehen aus einer Zeichenkette der Zeichen `a-z`, `A-Z`, `0-9`, `_`. Bezeichner dürfen nicht mit einer Ziffer `0-9` beginnen.

10.3.4. Variablen

Variablen bestehen aus einem eindeutigen Bezeichner (Variablennamen). Den Variablen können Werte zugewiesen werden und Variablen können als Werte verwendet werden. Die Zuweisung erfolgt mithilfe des `:=`-Operators. Auf der rechten Seite der Zuweisung können auch Ausdrücke stehen.

```

1  a := 5      # Zuweisung des Wertes 5 an die Variable a
2  a := 2 + 3  # Zuweisung des Wertes 5 an die Variable a

```

10.3.5. Kommentare

Kommentare werden durch das Zeichen `#` eingeleitet und umfassen sämtliche Zeichen bis zum nächsten Newline.

10.3.6. Kontrollstrukturen

10.3.6.1. While-Schleife

While-Schleifen werden mit dem Schlüsselwort `while` eingeleitet. Sie bestehen im Weiteren aus einer Bedingung, die durch ein `do` abgeschlossen wird, einer Folge von Anweisungen und werden mit dem Schlüsselwort `end` abgeschlossen.

Die Bedingung kann aus einem Vergleichsausdruck bestehen.

```

1 while <Bedingung> do
2     <Anweisung_1>
3     <Anweisung_2>
4 end

```

```

1 a := 10
2 b := 0
3 while a >= 0 do
4     a := a - 1
5     b := b + 9
6 end

```

10.3.6.2. Bedingte Anweisung (If-Else)

Eine bedingte Anweisung besteht immer aus genau einer **if**-Anweisung, gefolgt von einer Bedingung, die mit einem **do** abgeschlossen wird und einer Folge von Anweisungen.

Danach wird die bedingte Anweisung entweder mit dem Schlüsselwort **end** abgeschlossen oder es folgt genau ein **else**-Teil.

Ein **else**-Teil wird mit dem Schlüsselwort **else** eingeleitet. Darauf folgt ein **do** und eine Folge von Anweisungen. Der **else**-Teil wird mit dem Schlüsselwort **end** abgeschlossen.

```

1 if <Bedingung> do
2     <Anweisung_1>
3     <Anweisung_2>
4 end

```

```

1 if <Bedingung> do
2     <Anweisung>
3 else do
4     <Anweisung>
5 end

```

```

1 a := "abc"
2 if a < "abc" do
3     a := "wuppie"
4 else do
5     a := "nope"
6 end

```

10.3.7. Datentypen

Unsere Sprache hat zwei eingebaute Datentypen, für die entsprechende Literale erkannt werden müssen:

Datentyp	Definition der Literale
int	eine beliebige Folge der Ziffern 0–9

Datentyp	Definition der Literale
----------	-------------------------

<code>string</code>	eine beliebige Folge von ASCII-Zeichen, eingeschlossen in "
---------------------	---

Die Sprache ist dynamisch typisiert, d.h. beim Parsen werden Ihnen keine Typ-Angaben im Code begegnen. Aber Sie müssen die entsprechenden Werte (Literale) parsen können.

10.3.8. Beispiele

```
1 a := "wuppie fluppie"
```

```
1 a := 0
2 if 10 < 1 do
3     a := 42
4 else do
5     a := 7
6 end
```

10.4. Aufgaben

10.4.1. A3.1: Grammatik (4P)

Definieren Sie für die obige Sprache eine geeignete ANTLR-Grammatik.

Sie werden sowohl Lexer- als auch (rekursive) Parser-Regeln benötigen. Beachten Sie die üblichen Vorrangregeln für die Operatoren, orientieren Sie sich hier an Sprachen wie Java oder Python oder C.

10.4.2. A3.2: Pretty Printer (3P)

Erzeugen Sie mithilfe der Grammatik und ANTLR einen Lexer und Parser. Damit können Sie syntaktisch korrekte Eingabe-Programme in einen Parse-Tree überführen.

Programmieren Sie eine Traversierung Ihres Parse-Trees, in der Sie syntaktisch korrekte Programme konsistent eingerückt ausgeben können.

Jede Anweisung soll auf einer eigenen Zeile stehen. Die Einrückung soll mit Leerzeichen erfolgen und konsistent sein. Sie brauchen keine Begrenzung der Zeilenlänge implementieren.

Demonstrieren Sie die Fähigkeiten an mehreren Beispielen mit unterschiedlicher Komplexität.

Beispiel:

Aus

```

1  a      := 0
2  if 10 < 1 do
3  a      := 42      # Kommentar wird ignoriert
4  else do
5      a := 7
6  if 1>0do
7      while c do
8          if 8 do
9              c:=b+ "foo ."
10         end
11         b:=9+ 4*3 -7
12         c:=b+ "bar ."
13     end
14 end
15 end

```

soll

```

1  a := 0
2  if 10 < 1 do
3      a := 42
4  else do
5      a := 7
6      if 1 > 0 do
7          while c do
8              if 8 do
9                  c := b + "foo ."
10             end
11             b := 9 + 4 * 3 - 7
12             c := b + "bar ."
13         end
14     end
15 end

```

werden.

Hinweis: Es geht nur um die Ausgabe syntaktisch korrekter Programme. Sie brauchen sich um die Semantik (z.B. passende Typen wie etwa keine Multiplikation von Strings mit Integern o.ä.) noch keine Gedanken machen! Achten Sie auf die korrekten Einrücktiefen. Die Zeilenlänge spielt hier keine Rolle, es wird einfach direkt nach jedem Statement umgebrochen (bzw. wie bei den Kontrollstrukturen gezeigt).

Hinweis: Das Thema Pretty Printing ist interessant und kann recht schnell ziemlich aufwändig werden. Sie finden im Paper [“A prettier printer”](#) von Philip Wadler (Wadler 2003) und im Blog [“The Hardest Program I’ve Ever Written”](#) von Bob Nystrom (Nystrom 2015) gut geschriebene Beiträge, um tiefer in die Materie einzusteigen.

10.4.3. A3.3: AST (3P)

Beim Parsen bekommen Sie von ANTLR einen Parse-Tree zurück, der direkt die Struktur Ihrer Grammatik widerspiegelt. Die einzelnen Zweige sind damit in der Regel aber auch viel zu tief verschachtelt.

Überlegen Sie sich, welche Informationen/Knoten Sie für die formatierte Ausgabe wirklich benötigen - das ist Ihr

Abstract-Syntex-Tree (AST) für diesen Task.²

Programmieren Sie eine Transformation des Parse-Tree in die von Ihnen hier formulierten AST-Strukturen. Dies können Sie beispielsweise mit einer passenden Traversierung (Visitor-Pattern oder über Pattern-Matching auf Klassen) erreichen.

Passen Sie den Pretty-Printer so an, dass er auch den AST ausgeben kann. (Alternativ können auch einen zweiten Pretty-Printer für den AST implementieren.)

²In dieser Teilaufgabe geht es lediglich um das Pretty-Printing, nicht um das semantische Prüfen oder sogar Interpretieren der Programme. Die Aufgabe lautet also: Welche Strukturen benötigen Sie, um das Pretty-Printing des erfolgreich geparsten Programms erledigen zu können? Alle anderen Strukturen sind hier unnötiger Ballast...

11. Blatt 04: Lexer und Parser selbst implementiert

11.1. Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung einer Grammatik sowie daraus ableitend die manuelle Implementierung eines Lexers und Parsers mit *recursive descent* für eine Lisp-artige Sprache.

11.2. Methodik

Sie finden nachfolgend eine Beschreibung der Zielsprache für dieses Blatt.

Erstellen Sie zunächst eine Grammatik für diese Sprache. Implementieren Sie dann basierend auf dieser Grammatik einen Lexer und einen *recursive descent* Parser (manuelle Implementierung, kein ANTLR). Überlegen Sie sich, welche Strukturen ein AST für diese Sprache haben sollte und implementieren Sie diese Strukturen und schreiben Sie eine Transformation des Parse-Trees in diesen AST.

Es ist empfehlenswert, die Implementierung mehrstufig zu realisieren:

1. Einlesen aus einer Datei mit Lisp-artigem Code
2. Lexen des eingelesenen Code
3. Parsen des Tokenstroms
4. Erzeugen und Ausgeben des AST

11.3. Sprachdefinition

Ein Programm besteht aus einem oder mehreren Ausdrücken (*Expressions*). Die Ausdrücke haben eine spezielle Form: Sie sind sogenannte **S-Expressions**. Dies sind entweder Literale der Form `x` oder einfache listenartige Gebilde der Form `(. x y)`, wobei der `.` eine Operation (oder Funktion oder ein Keyword) darstellt und `x` und `y` selbst wieder S-Expressions sind.

Tip

Anmerkung: Die Anzahl der S-Expressions in einem Klammer-Ausdruck ist nicht näher definiert - `x` und `y` sind nur Beispiele. Es könnten auch mehr oder weniger S-Expressions nach der Operation/Funktion/Keyword auftauchen (vgl. nachfolgende Sprachdefinition).

Die einfachste Form sind dabei Literale mit konkreten Werten der drei Datentypen `Integer`, `String` und `Boolean`:

```
1 42      ;; Integer
2 "hello"  ;; String
3 true     ;; Boolean
4 false    ;; Boolean
```

Für eine Variable `foo` wäre das Folgende ebenfalls eine S-Expression:

```
1 foo      ;; Variable foo
```

(Über `;;` wird ein Kommentar eingeleitet, der bis zum Ende der Zeile geht.)

Komplexere Ausdrücke werden über die Listenform gebildet:

```
1 (+ 1 1)      ;; 1 + 1
2 (/ 10 3)     ;; 10 / 3
3 (+ 1 2 3 4)  ;; 1 + 2 + 3 + 4
4 (+ (+ (+ 1 2) 3) 4) ;; (((1 + 2) + 3) + 4)
5 (/ (+ 10 2) (+ 2 4)) ;; ((10 + 2) / (2 + 4))
```

In der listenartigen Form ist der erste Eintrag der Liste immer eine Operation (oder ein Funktionsname), danach kommen je nach Operation/Funktion (die Arität muss passen!) entsprechende Einträge, die als Parameter für die Operation oder Funktion zu verstehen sind.

Die Ausdrücke sind implizit von links nach rechts geklammert, d.h. der Ausdruck `(+ 1 2 3 4)` ist *syntactic sugar* für `(+ (+ (+ 1 2) 3) 4)`.

11.3.1. Eingebaute Funktionen

Es gibt zwei Funktionen, die fest in der Sprache integriert sind.

Mit der eingebauten Funktion `print` kann der Wert eines Ausdrucks auf der Konsole ausgegeben werden:

```
1 (print "hello world")
2 (print "wuppie\nfluppie\nfoo\nbar")
```

Die eingebaute Funktion `str` verknüpft ihre Argumente und bildet einen String. Falls nötig, werden die Argumente vorher in einen String umgewandelt.

```
1 (str 42)                ;; liefert "42" zurück
2 (str "wuppie" "fluppie" "foo" "bar") ;; liefert "wuppiefluppiefoobar" zurück
3 (str "one: " 1 " ", two: " 2")      ;; liefert "one: 1, two: 2" zurück
```

11.3.2. Operatoren

Es gibt nur wenige vordefinierte Operatoren, diese mit der üblichen Semantik.

11.3.2.1. Vergleichsoperatoren

Operation	Operator
Gleichheit	=
Größer	>
Kleiner	<

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist immer vom Typ `Boolean`.

11.3.2.2. Arithmetische Operatoren

Operation	Operator
Addition	+
Subtraktion	-
Multiplikation	*
Division	/

Die Operanden müssen jeweils beide den selben Typ haben. Dabei sind `String` und `Integer` zulässig. Das Ergebnis ist vom Typ der Operanden.

11.3.3. Kontrollstrukturen (If-Else)

Die `if-then-else`-Abfrage gibt es mit und ohne den `else`-Zweig:

```
1 (if boolean-form
2   then-form
3   optional-else-form)
```

```
1 (if (< 1 2)
2   (print "true")    ;; then
3   (print "false")) ;; else
```

Dabei kann jeweils nur genau eine S-Expression genutzt werden. Wenn man mehrere Dinge berechnen möchte, nutzt man `do`:

```
1 (do
2   (print "wuppie")
3   (print "fluppie")
4   (print "foo")
5   (print "bar"))
```

Beispiel:

```
1 (if (< 1 2) (do (print "true") (print "WUPPIE")) (print "false"))
```

oder anders formatiert:

```
1 (if (< 1 2)
2   (do (print "true")
3       (print "WUPPIE"))
4   (print "false"))
```

11.3.4. Variablen: Bindings mit *def* anlegen

```
1 (def x 42) ;; definiert eine neue Variable mit dem Namen "x" und dem Wert 42
2
3 x          ;; liefert 42
4 (+ x 7)    ;; liefert 49
```

11.3.5. Funktionen mit *defn* definieren

```
1 ;;      name  params  body
2 (defn hello (n) (str "hello " n)) ;; Definition einer Funktion "hello"
   mit einem Parameter
3
4 (hello "world") ;; Aufruf der Funktion "hello" mit dem
   Argument "world"
```

11.3.6. Lokale Scopes mit *let*

```
1 ;;      bindings      use names here
2 (let (name value) (code that uses name))
3
4 (def x 99) ;; globale Variable x
5 (def y 101) ;; globale Variable y
6 (def z 42) ;; globale Variable z
7 (let (x 1 ;; lokales x mit Wert 1(verdeckt globales x)
8       y 2) ;; lokales y mit Wert 2
9   (+ x y z)) ;; 1+2+42
10
11 (defn hello
12   (n)
13   (let (l 42) ;; l is valid in this scope
14     (str "hello " n ": " l))
15   ) ;; end of local scope
16 ) ;; end of function definition
```

Mit *let* können lokale Variablen erzeugt werden, die dann in dem jeweiligen Scope genutzt werden können. Dies funktioniert wie in anderen Sprachen mit Scopes.

11.3.7. Rekursion

```
1 (defn fac (n)
2   (if (< n 2)
3     1
4     (* n (fac (- n 1)))))
```

Da es kein **while** oder **for** gibt, müssen Schleifen über rekursive Aufrufe abgebildet werden.

11.3.8. Datenstrukturen

In unserer Sprache gibt es Listen:

```
1 (1 2 3)          ;; Fehler!
2 (def v (1 2 3))  ;; Fehler!
```

Das Problem daran ist, dass unsere S-Expressions zwar bereits listenartige Strukturen sind, der erste Eintrag aber als Operator oder Funktion interpretiert wird. Der Ausdruck oben würde beim Auswerten versuchen, die "Funktion" 1 auf den Argumenten 2 und 3 aufzurufen ...

Man braucht also eine Notation, die ein sofortiges Auswerten verhindert und nur die Liste an sich zurückliefert. Dies erreicht man durch die Funktion **list**:

```
1 (list 1 2 3)      ;; (1 2 3)
2
3 (def v (list 1 2 3)) ;; v = (1 2 3)
4 v                ;; (1 2 3)
```

Mit der Funktion **nth** kann man auf das n-te Element einer Liste zugreifen:

```
1 (nth (list "abc" false 99) 2) ;; 99
```

Zusätzlich gibt es die beiden Funktionen **head** und **tail**, die das erste Element einer Liste bzw. die restliche Liste ohne das erste Element zurückliefern:

```
1 (head (list 1 2 3)) ;; 1
2 (tail (list 1 2 3)) ;; (2 3)
```

11.4. Aufgaben

11.4.1. A4.1: Kontextfreie Grammatik (1P)

Betrachten sie die folgende Grammatik:

$$G = (\{S, A\}, \{1, 2, 3\}, P, S)$$

mit

$$P = \{ \begin{array}{l} S \rightarrow 1AS \mid 3 \\ A \rightarrow 2AS \mid \epsilon \\ \end{array} \}$$

Berechnen die *First*- und *Follow*-Mengen der Grammatik.

Zeigen Sie, dass die Grammatik LL(1) ist.

11.4.2. A4.2: Grammatik (2P)

1. Erstellen Sie zunächst einige Programme in der Zielsprache. Diese sollten von einfachsten Ausdrücken bis hin zu komplexeren Programmen reichen. Versuchen Sie beispielsweise eine Funktion zu definieren, die rekursiv die Länge einer Liste berechnet.

Definieren Sie neben gültigen Programmen auch solche, die in der syntaktischen Analyse zurückgewiesen werden sollten. Welche Fehlerkategorien könnte es hier geben?

2. Überlegen Sie:

- Was sind die grundsätzlichen syntaktischen Konstrukte der obigen Sprache?
- Was muss man in Lexer/Parser prüfen und was kann man später (semantische Analyse oder Interpreter) auch noch gut tun?
- Wie viel Semantik muss im Parser schon realisiert werden?
- Was brauchen Sie an Sprachumfang, um die Aufgaben auf diesem Blatt zu lösen?

(Da wir noch nicht über die späteren Phasen im Compiler gesprochen haben: Spekulieren Sie.)

3. Definieren Sie nun für die obige Sprache eine geeignete Grammatik.

11.4.3. A4.3: Lexer (2P)

1. Definieren Sie Strukturen, die Sie für die Repräsentation der Token entsprechend Ihrer Grammatik benötigen.
2. Implementieren Sie dann analog zum Vorgehen in der Vorlesung einen Lexer, der den entsprechenden Teil Ihrer Grammatik abbildet. Diesen Lexer sollen Sie manuell implementieren, Sie dürfen also nicht ANTLR oder andere Generatoren benutzen.

Implementieren Sie dabei das Verarbeiten des Lisp-artigen Codes aus einem übergebenen String.

Sie können wie in der Vorlesung demonstriert auf Anforderung das nächste Token berechnen. Sie können aber auch den Eingabecode vollständig verarbeiten und dann einen Tokenstrom geeignet zurückgeben.

Ihr Lexer soll eine rudimentäre Fehlerbehandlung aufweisen und bei Problemen die Abweichung vom erwarteten Zeichen zum tatsächlich eingelesenen Zeichen ausgeben. Darüber hinaus braucht der Lexer keinen Recovery-Modus haben o.ä., d.h. nach der Fehlermeldung darf Ihr Lexer “aussteigen”.

11.4.4. A4.4: Parser mit *recursive descent* (3P)

Implementieren Sie analog zum Vorgehen in der Vorlesung einen Parser mit *recursive descent*, der den entsprechenden Teil Ihrer Grammatik abbildet. Diesen Parser sollen Sie manuell implementieren, Sie dürfen also nicht ANTLR oder andere Generatoren benutzen.

Implementieren Sie dabei das Verarbeiten des Lisp-artigen Codes aus einem übergebenen String.

Ihr Parser soll eine rudimentäre Fehlerbehandlung aufweisen und bei Problemen die Abweichung vom erwarteten Token zum tatsächlich eingelesenen Token ausgeben. Darüber hinaus braucht der Parser keinen Recovery-Modus haben o.ä., d.h. nach der Fehlermeldung darf Ihr Parser “aussteigen”.

11.4.5. A4.5: AST (1P)

Definieren Sie einen AST für die Zielsprache. Welche Informationen aus dem Eingabeprogramm müssen für ein Pretty-Printing des eingelesenen Programms repräsentiert werden?

Implementieren Sie die entsprechenden Datenstrukturen. Programmieren Sie außerdem eine Traversierung des Parse-Trees, die den AST erzeugt.

Geben Sie den AST auf der Konsole aus (“Pretty Printing” für den AST). Testen Sie dies mit Ihren in der ersten Aufgabe entwickelten (syntaktisch korrekten) Beispielprogrammen.

11.4.6. A4.6: Recherche und Diskussion (1P)

Recherchieren Sie, welche Open-Source-Projekte auf handgeschriebene *recursive descent* Parser setzen (oder umgestiegen sind) und welche Gründe es dafür in diesen Projekten gibt. Was spricht aus Ihrer persönlichen Sicht für und gegen ANTLR?

12. Blatt 05: Semantische Analyse

12.1. Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung einer Symboltabelle und einer einfachen Typ-Prüfung für eine fiktive statisch typisierte Sprache mit Expressions, Kontrollstrukturen und Funktionen.

12.2. Methodik

Analysieren Sie die nachfolgend in der Aufgabe gegebene Grammatik. Welche Sprache akzeptiert/generiert diese Grammatik? Erstellen Sie verschiedene syntaktisch zulässige und unzulässige Programme. Erstellen Sie weitere Programme, in denen Sie bewusst die semantischen Regeln (siehe Aufgabe) verletzen.

Erstellen Sie mit dieser Grammatik und ANTLR einen Lexer und Parser. Konvertieren Sie Ihren Parse-Tree in einen AST, Sie können dazu die vorgegebenen Datenstrukturen und Konvertierungsregeln nutzen.

Die semantische Analyse soll zweiphasig realisiert werden:

1. Aufbauen der Symboltabelle und Prüfen von z.B. Deklaration/Definition (Variablen, Funktionen), bei Referenzierung von Variablen auf vorherige Deklaration/Definition im Sichtbarkeitsbereich
2. Prüfen von Funktionsaufrufen auf vorhandene/sichtbare Funktionsdefinitionen

12.3. Sprachdefinition

```
1 grammar MiniC;
2
3 // Parser
4 program : stmt+ EOF ;
5
6 stmt
7   : vardecl
8   | assign
9   | fndekl
10  | expr ';'
11  | block
12  | while
13  | cond
14  | return
15  ;
16
```

```

17 vardecl : type ID ('=' expr)? ';' ;
18 assign  : ID '=' expr ';' ;
19
20 fndekl  : type ID '(' params? ')' block ;
21 params  : type ID (',' type ID)* ;
22 return  : 'return' expr ';' ;
23
24 fncall   : ID '(' args? ')' ;
25 args    : expr (',' expr)* ;
26
27 block    : '{' stmt* '}' ;
28 while    : 'while' '(' expr ')' block ;
29 cond     : 'if' '(' expr ')' block ('else' block)? ;
30
31 expr
32 : fncall
33 | expr ('*' | '/') expr
34 | expr ('+' | '-') expr
35 | expr ('>' | '<') expr
36 | expr ('==' | '!=') expr
37 | ID
38 | NUMBER
39 | STRING
40 | 'T'
41 | 'F'
42 | '(' expr ')'
43 ;
44
45 type : 'int' | 'string' | 'bool' ;
46
47
48 // Lexer
49 ID      : [a-zA-Z] [a-zA-Z0-9]* ;
50 NUMBER  : [0-9]+ ;
51 STRING  : '"' (~[\\n\\r])* '"' ;
52
53 COMMENT : '#' ~[\\n\\r]* -> skip ;
54 WS      : [ \\t\\n\\r]+ -> skip ;

```

12.4. Semantik

Statements ergeben bei der Ausführung keinen Wert. Expressions haben immer einen Typ und ergeben bei der Evaluierung einen Wert.

Jeder Code-Block (beginnend und endend mit geschweiften Klammern) bildet einen eigenen Scope - alle Deklarationen/Definition in diesem Scope sind im äußeren Scope nicht sichtbar. In einem Scope kann auf die Symbole (Namen) des/der umgebenden Scopes zugegriffen werden. Symbole in einem Scope können gleichnamige Symbole im äußeren Scope verdecken. Symbole dürfen in einem Scope nicht mehrfach definiert (*decl-Statements) werden.

Bei allen Operatoren müssen die Operanden jeweils beide den selben Typ haben. Bei Vergleichsoperatoren sind **string** und **int** zulässig, das Ergebnis ist immer vom Typ **bool**. Bei arithmetischen Operatoren sind für +

sowohl `string` als auch `int` zulässig, für die anderen Operatoren (`-`, `*`, `/`) nur `int`, das Ergebnis ist vom Typ der Operanden.

Variablen müssen vor ihrer Benutzung (Zugriff, Zuweisung) definiert und im aktuellen Scope sichtbar sein. Die Initialisierung kann zusammen mit der Definition erfolgen. Variablen können in einem Scope nicht mehrfach definiert werden. Variablen können nicht als Funktion benutzt werden (aufgerufen werden).

Eine Funktionsdefinition macht dem Compiler die Implementierung einer Funktion bekannt. Funktionen können in einem Scope nicht mehrfach definiert werden. Eine aufgerufene Funktion muss im aktuellen Scope sichtbar sein, der Funktionsaufruf muss zur Definition passen. Die aufgerufene Funktion muss (im Gegensatz zum Zugriff auf Variablen) nicht vor dem ersten Aufruf definiert sein. Sie kann also später im selben Scope definiert werden. Einer Funktion kann nichts zugewiesen werden.

12.5. Aufgaben

12.5.1. A5.1: Grammatik und Sprache (1P)

Analysieren Sie die obige ANTLR-Grammatik. Welche Sprache wird durch diese Grammatik akzeptiert/generiert?

Erstellen Sie einige unterschiedlich komplexe Beispiele in der Sprache. Erstellen Sie dabei auch einige syntaktisch unkorrekte Beispiele. Erstellen Sie zusätzlich semantisch unkorrekte Beispiele.

Generieren Sie mit ANTLR einen Lexer und Parser für die obige ANTLR-Grammatik.

12.5.2. A5.1: AST (2P)

Definieren Sie einen AST für die Zielsprache. Sie können folgende Definition nutzen:

```

1  Stmt
2  = VarDecl(type: PrimType, name: string, initializer?: Expr)
3  | Assign(name: string, value: Expr)
4  | FnDecl(returnType: PrimType, name: string, params: Param*, body: Block)
5  | ReturnStmt(value: Expr)
6  | ExprStmt(expr: Expr)
7  | Block(statements: Stmt*)
8  | WhileStmt(condition: Expr, body: Block)
9  | IfStmt(condition: Expr, thenBranch: Block, elseBranch: Block)
10
11 Expr
12 = IntLiteral(value: int)
13 | StringLiteral(value: string)
14 | BoolLiteral(value: bool)
15 | Variable(name: string)
16 | Binary(left: Expr, op: Operator, right: Expr)
17 | Call(name: string, args: Expr*)
18
19 Param = Param(type: PrimType, name: string)
20

```

```

21 PrimType = INT | STRING | BOOL
22
23 Operator = EQ | NEQ | PLUS | MINUS | MUL | DIV | LT | GT

```

Programmieren Sie eine Traversierung des Parse-Trees, die den AST erzeugt. Sie können dafür folgende Transformationen nutzen:

Transformationen

```

1  program: stmt+ EOF
2      -> Liste[Stmt] in Quellreihenfolge (kein eigener Program-Knoten)
3
4  stmt
5      * vardecl: type ID ('=' expr)? ';'
6          -> VarDecl(PrimType(type), ID.text, initializer?)
7              initializer? = Some(exprAST) falls vorhanden, sonst None
8
9      * assign: ID '=' expr ';'
10         -> Assign(ID.text, exprAST)
11
12     * fndecl: type ID '(' params? ')' block
13         -> FnDecl(PrimType(type), ID.text, paramsList, blockAST)
14
15     * expr ';'
16         -> ExprStmt(exprAST)
17
18     * block: '{' stmt* '}'
19         -> Block([AST(stmt1), AST(stmt2), ...])
20
21     * while: 'while' '(' expr ')' block
22         -> WhileStmt(exprAST, blockAST)
23
24     * cond: 'if' '(' expr ')' block ('else' block)?
25         -> IfStmt(exprAST, thenBlockAST, elseBlockOrEmpty)
26             elseBlockOrEmpty = elseBlockAST, falls vorhanden;
27                 sonst Block([])
28
29     * return: 'return' expr ';'
30         -> ReturnStmt(exprAST)
31
32  expr
33     * fncall: ID '(' args? ')'
34         -> Call(ID.text, argsList)
35
36     * expr ('*' | '/') expr
37         -> Binary(leftAST, op=MUL|DIV, rightAST)
38
39     * expr ('+' | '-') expr
40         -> Binary(leftAST, op=PLUS|MINUS, rightAST)
41
42     * expr ('>' | '<') expr
43         -> Binary(leftAST, op=GT|LT, rightAST)
44
45     * expr ('==' | '!=') expr
46         -> Binary(leftAST, op=EQ|NEQ, rightAST)
47
48     * ID

```

```

49     -> Variable(ID.text)
50
51     * NUMBER
52     -> IntLiteral(parseInt(NUMBER.text))
53
54     * STRING
55     -> StringLiteral(unquote(STRING.text))
56
57     * 'T' -> BoolLiteral(true)
58     * 'F' -> BoolLiteral(false)
59
60     * '(' expr ')'
61     -> exprAST (Klammern sind rein syntaktisch)
62
63     type
64     * 'int'    -> PrimType.INT
65     * 'string' -> PrimType.STRING
66     * 'bool'   -> PrimType.BOOL
67
68     params: type ID (',' type ID)*
69     -> paramsList = [ Param(PrimType(type_i), ID_i.text) ] in Quellreihenfolge
70
71     args: expr (',' expr)*
72     -> argsList = [ exprAST_i ] in Quellreihenfolge
73
74     Operator-Zuordnung
75     '*' -> MUL, '/' -> DIV, '+' -> PLUS, '-' -> MINUS, '>' -> GT, '<' -> LT, '=='
        -> EQ, '!=' -> NEQ
76
77     Allgemein
78     * Tokens wie Klammern, Kommas, Semikolons, Schlüsselwörter erscheinen nicht
        im AST.
79     * Präzedenz/Assoziativität kommt aus der Parse-Struktur; der AST ist
        normalisiert zu Binary(left, op, right).

```

Testen Sie Ihre Transformation mit Ihren in der vorigen Aufgabe erstellten Beispielprogrammen.

12.5.3. A5.2: Aufbau der Symboltabelle (5P)

Bauen Sie für den AST eine Symboltabelle auf. Achten Sie darauf, dass Sie für den zweiten Lauf eine Querverbindung zwischen den Symboltabelleinträgen und den AST-Knoten benötigen.

Führen Sie die im ersten Lauf möglichen Prüfungen durch, beispielsweise ob eine Variable oder Funktion in einem Scope mehrfach definiert wird oder ob Variablen als Funktion genutzt (aufgerufen) werden. Für referenzierte Variablen muss geprüft werden, ob sie tatsächlich bereits definiert und im Scope sichtbar sind. Zuweisungen dürfen nur an Variablen erfolgen.

Geben Sie erkannte Fehler auf der Konsole aus.

12.5.4. A5.3: Symboltabelle: Referenzierungen und Funktionsaufrufe (2P)

Implementieren Sie einen zweiten Lauf. Hier soll für Funktionsaufrufe geprüft werden, ob diese Funktionen bereits definiert und im Scope sichtbar sind.

Damit wird so etwas möglich:¹

```
1 // einfacher Funktionsaufruf, ohne Vorwärtsdeklaration
2 bar();
3
4 void bar() {
5     print_char('b');
6 }
```

Geben Sie erkannte Fehler auf der Konsole aus.

¹Anmerkung: Das ist kein gültiges C, sondern dient nur dazu, das Vorgehen bei mehreren Passes über die Symboltabelle zu erkennen und zu üben.

13. Blatt 06: Interpreter

13.1. Zusammenfassung

Ziel dieses Aufgabenblattes ist die Erstellung eines Tree-Walking-Interpreters für eine Lisp-artige Sprache.

13.2. Methodik

Sie haben in Blatt 04 eine Grammatik für die Lisp-artige Sprache definiert und darauf aufbauend einen Lexer und Parser implementiert, d.h. Sie können für eine Eingabe eines Programms mit S-Expressions den entsprechenden AST erzeugen.

Korrigieren Sie die Codebasis von Blatt 04, falls notwendig. Ergänzen Sie sodann einen Tree-Walking-Interpreter und eine REPL für die Lisp-artige Sprache.

Es ist empfehlenswert, den Interpreter mehrstufig zu realisieren:

1. Einlesen von Lisp-Code, Parsen der Inhalte und Erzeugen des AST
2. Ablaufen des AST und dabei
 - Durchführung der semantischen Analyse
 - Auswerten der Ausdrücke (Interpretation)

Hinweis: Wir wollen eine *REPL* (“Read-Eval-Print”-Loop) realisieren und in einer Schleife Lisp-Code von der Konsole einlesen, parsen und interpretieren und das Ergebnis wieder ausgeben. Sie erzeugen also für jede Eingabe immer wieder neu einen AST. Wenn wir die semantische Analyse als eigenen Pass zwischen Parsing und Interpretation realisieren würden, müssten wir immer sämtlichen bisher verarbeiteten Code mitführen (allerdings nur die gültigen, d.h. nicht verworfenen Eingaben) und zusammen mit der aktuellen Eingabe der semantischen Analyse unterwerfen. Da dies recht frickelig werden kann, realisieren wir auf diesem Blatt eine vereinfachte semantische Analyse “*on-the-fly*” auf den Environments im Interpreter.

Hinweis: Wenn Sie Blatt 04 nicht oder nicht erfolgreich bearbeitet haben sollten, können Sie dieses Blatt trotzdem bearbeiten. Erstellen Sie sich zunächst eine Grammatik für die beschriebene Sprache (oder nehmen eine der beiden unten angegebenen Grammatiken). Treten Sie dabei mental einen Schritt zurück und betrachten Sie das ganze Bild: Was wird *syntaktisch* wirklich an Strukturen benötigt? Danach lassen Sie sich mit ANTLR einen Lexer und Parser generieren und steigen ganz normal in die Bearbeitung dieses Blattes ein.

13.3. Sprachdefinition

Siehe Blatt 04.

13.4. Aufgaben

13.4.1. A6.1: AST und Syntactic Sugar (1P)

Lösen Sie die als “*syntactic sugar*” bezeichneten Ausdrücke auf und transformieren Sie den AST entsprechend: $(+ \ 1 \ 2 \ 3 \ 4)$ soll zu $(+ \ (+ \ (+ \ 1 \ 2) \ 3) \ 4)$ umgeformt werden. Analog für die anderen Operatoren der Sprache (Vergleiche, Arithmetik).

13.4.2. A6.2: Tree-Walking-Interpreter (4P)

Bauen Sie einen Tree-Walking-Interpreter in Ihr Projekt ein.

Realisieren Sie die eingebauten Funktionen `print` und `str` dabei als *native* Funktionen.

Realisieren Sie `list`, `nth`, `head` und `tail` sowie `def`, `let`, `defn`, `do` und die Operatoren und die Kontrollstrukturen geeignet.

Lesen Sie den zu interpretierenden Code aus einer Text-Datei ein.

Testen Sie Ihren Interpreter mit Ihren Beispielprogrammen.

13.4.3. A6.3: Semantische Analyse (3P)

Führen Sie eine vereinfachte semantische Analyse während der Interpretation durch:

- Symbole dürfen im selben Scope nicht mehrfach definiert werden
- Referenzierte Symbole müssen sich über den aktuellen Scope (oder dessen Eltern-Scopes) auflösen lassen
- Bei Funktionsaufrufen muss das als Funktionsname verwendete Symbol tatsächlich eine Funktion sein
- Achten Sie auf die Datentypen der Operanden beim Ausführen der Operatoren

13.4.4. A6.4: Interaktiver Interpreter (1P)

Bauen Sie eine *REPL* ein, d.h. geben Sie nach dem Start des Interpreters einen Prompt aus und verarbeiten Sie die Eingaben interaktiv.

Erlauben Sie das Einlesen eines Lisp-Programms aus einer Text-Datei beim Start des Interpreters. Dabei soll zunächst der eingelesene Code interpretiert werden und damit die Environments vorbelegt werden. Danach soll der Interpreter in die REPL gehen, d.h. der in der Konsole eingegebene Code wird dann im Kontext des vorher eingelesenen Programms interpretiert.

13.4.5. A6.5: Auswirkungen der Grammatik auf den Interpreter (1P)

Vergleichen Sie ihre eigene Grammatik mit den folgenden beiden Grammatiken:

Grammatik A

```

1  grammar MiniLispA;
2
3
4  // Parser
5  program : expr+ EOF ;
6
7  expr    : NUMBER
8          | STRING
9          | TRUE
10         | FALSE
11         | ID
12         | '(' (ID | OP) expr* ')'
13         ;
14
15
16 // Lexer
17 TRUE   : 'true' ;
18 FALSE  : 'false' ;
19 ID     : [a-z][a-zA-Z0-9]* ;
20 NUMBER : [0-9]+ ;
21 OP     : '+' | '-' | '*' | '/' | '=' | '>' | '<' ;
22 STRING : '"' (~[\n\r])* '"' ;
23
24 COMMENT : ';;' ~[\n\r]* -> skip ;
25 WS      : [ ,\t\n\r]+ -> skip ;

```

Grammatik B

```

1  grammar MiniLispB;
2
3
4  // Parser
5  program : expr+ EOF ;
6
7  expr    : literal
8          | symbol
9          | list
10         | def
11         | fn
12         | fcall
13         | let
14         ;
15
16 literal : NUMBER
17         | STRING
18         | TRUE
19         | FALSE
20         ;
21
22 symbol  : ID ;
23

```

```
24 list    : '(' 'list' expr* ')' ;
25 def     : '(' 'def' symbol expr ')' ;
26
27 fn      : '(' 'defn' symbol '(' symbol* ')' expr* ')' ;
28 fcall   : '(' (ID | OP) expr* ')' ;
29
30 let     : '(' 'let' '(' binding* ')' expr ')' ;
31 binding : symbol expr ;
32
33
34 // Lexer
35 TRUE    : 'true' ;
36 FALSE   : 'false' ;
37 ID      : [a-z][a-zA-Z0-9]* ;
38 NUMBER  : [0-9]+ ;
39 OP      : '+' | '-' | '*' | '/' | '=' | '>' | '<' ;
40 STRING  : '"' (~[\n\r"])* '"' ;
41
42 COMMENT : ';;' ~[\n\r]* -> skip ;
43 WS      : [ ,\t\n\r]+ -> skip ;
```

Welche Auswirkungen hat die Grammatik auf den Interpreter? Machen Sie ein kleines **Gedankenexperiment**: Überlegen Sie, was Sie alles in Ihrer Implementierung ändern müssten, wenn Sie die jeweils andere Grammatik-Variante (A bzw. B) nutzen würden.

14. Blatt 07: C++

14.1. Zusammenfassung

Ziel dieses Aufgabenblattes ist die praktische Anwendung Ihrer C++-Kenntnisse, insbesondere werden Sie Pointer, Referenzen und Klassen anwenden und vertiefen. Als Anwendungsbeispiel werden Sie bestimmte in der C++-Welt wohlbekannte Smart-Pointer modellieren sowie einen einfachen Ringpuffer programmieren. Sie lernen mit dem *Reference Counting* nebenbei eine verbreitete Technik der *Garbage Collection* kennen.

14.2. Methodik

Sie werden auf diesem Blatt vier einfache Klassen in C++ implementieren.

Es empfiehlt sich, zunächst die Beispiele gründlich zu analysieren, um die gewünschte Funktionsweise der einzelnen Klassen vorab präzise zu verstehen. Sie werden zu einigen Dingen in der C++-Literatur recherchieren müssen.

Implementieren Sie immer eine Klasse vollständig und testen Sie Ihren Code sowohl mit den vorgegebenen Beispielen als auch mit eigenen Beispielen, bevor Sie sich an die nächste Aufgabe/Klasse setzen.

14.3. Speicherverwaltung in C/C++

C und C++ erlauben als hardwarenahe Programmiersprachen den direkten Umgang mit dem Programmspeicher (Heap). Ein Programm kann dynamisch zu jeder Zeit weiteren Speicher anfordern und so beispielsweise mitwachsende Datenstrukturen realisieren.

Da der Heap-Speicher endlich ist, muss man nicht mehr benötigten Speicher auch wieder freigeben. Anderenfalls ist irgendwann der komplette Heap belegt und das Programm kann nicht mehr ordnungsgemäß arbeiten. Für die Freigabe ist man als Programmierer:in selbst zuständig.

14.3.1. Beispiel für eine Tokenizer-Funktion

Im folgenden Programmschnipsel soll eine Funktion `next_token()` das nächste Token berechnen. So eine Funktion findet sich typischerweise im Lexer. Für die Rückgabe des Tokens hat man in C++ drei Möglichkeiten: als Kopie, als Referenz oder als Pointer.

```

1 // Return as copy
2 Token next_token() {
3     Token wuppie = Token("wuppie", 1, 4); // will be deleted automatically
4     Token bar = Token("bar", 7, 10);      // not used, will be deleted
5                                         automatically
6     return wuppie;
7 }
8 int main() {
9     Token x = next_token();                // copy constructor; no need to
10                                         free
11 }

```

```

1 // Return as pointer
2 Token* next_token() {
3     Token* foo = new Token("foo", 9, 35); // will be free'd manually
4     Token* bar = new Token("bar", 7, 10); // leaves a memory hole!!!
5
6     return foo;
7 }
8 int main() {
9     Token* x = next_token();                // only the pointer (i.e. address)
10                                         will be copied
11     ...
12     delete x;                             // caller needs to free this object
13 }

```

```

1 // Return as C++ reference
2 Token& next_token() {
3     Token* foo = new Token("foo", 9, 35); // will be free'd manually
4     Token* bar = new Token("bar", 7, 10); // leaves a memory hole!!!
5
6     return *foo;
7 }
8 int main() {
9     Token& x = next_token();                // no copy, `x` is just a new alias
10                                         for `foo`
11     ...
12     delete &x;                             // caller needs to free this object
13 }

```

Die Rückgabe per Kopie (Standardfall in C/C++) würde ein lokales Objekt auf dem Stack (im Beispiel wäre das **wuppie**) als Kopie zurückgeben.

- Vorteil: Der Compiler kümmert sich um die Freigabe der lokalen Variable **wuppie**, d.h. nach Beendigung des Funktionsaufrufs wird das Objekt automatisch vom Stack entfernt. Da hierbei einfach der Stackpointer zurückgesetzt wird, ist diese "Freigabe" eine sehr preiswerte Operation.¹
- Nachteil: Der Aufrufer darf nicht einfach auf das Objekt auf dem Stack zugreifen (dieses ist ja nach Beendigung der Funktion nicht mehr gültig). Deshalb muss das Objekt bei der Rückgabe kopiert werden (Copy-Konstruktor). Zusätzlich erfolgt beim Aufrufer oft noch eine Zuweisung, bei der die Attribute des Objekts vermutlich erneut kopiert werden. Dies kann (je nach Aufbau der Objekte) sehr teuer sein!

¹Anmerkung: Man spricht trotzdem von "Freigabe" des Objekts, obwohl lediglich der Stackpointer zurückgesetzt wird und das Objekt zunächst auf dem Stack noch vollständig ist. Es kann und wird aber im weiteren Verlauf des Programms überschrieben.

Die Rückgabe per Pointer erfordert ein Objekt, welches innerhalb der Funktion erzeugt wird und dessen Lebensdauer über das Funktionsende hinausreicht. Das Objekt muss in diesem Fall also auf dem Heap angelegt werden.

- Vorteil: Die Rückgabe erfordert lediglich die Kopie der Adresse des Objekts (also des Pointers). Hier handelt es sich vereinfacht betrachtet um einen Integer, d.h. diese Operation ist relativ preiswert.
- Nachteil: Das Objekt muss vom Aufrufer wieder freigegeben werden, sobald es nicht mehr benötigt wird. Dies muss man explizit programmieren!

Die Rückgabe per C++-Referenz erfordert ebenfalls ein Objekt, welches innerhalb der Funktion erzeugt wird und dessen Lebensdauer über das Funktionsende hinausreicht. Das Objekt muss in diesem Fall also wieder auf dem Heap angelegt werden.

- Vorteil: Die Rückgabe erfordert keinerlei Kopien, da sich die Referenz `x` an das Objekt `foo` bindet und lediglich einen neuen Alias für dieses Objekt darstellt.
- Nachteil: Das Objekt muss vom Aufrufer wieder freigegeben werden, sobald es nicht mehr benötigt wird. Dies muss man explizit programmieren!

Es hat sich gezeigt, dass der Umgang mit den Heap-Ressourcen sehr fehleranfällig ist. Ein Aspekt dabei ist, dass man häufig die Freigabe der Objekte vergisst oder dass die Programmpfade so unübersichtlich sind, dass man nicht genau weiss, ob und wann man Objekte freigeben soll (denken Sie an Exceptions).

14.3.2. Smart-Pointer als Lösung

Während man in Sprachen wie Java die Speicherverwaltung komplett dem Compiler überlässt oder wie in Rust mit strikten Ownership-Modellen arbeitet, hat man in C++ die sogenannten **Smart-Pointer** erdacht. Diese ersetzen den direkten Umgang mit den einfachen Pointern (auch als *raw pointer* bezeichnet) und lösen das Problem der Freigabe der verwalteten Ressourcen.² Es gibt verschiedene Modelle, insbesondere gibt es die Variante *unique pointer*, bei der immer nur genau ein Smart-Pointer gleichzeitig eine bestimmte Ressource besitzen darf, und die *shared pointer*, bei der mehrere Smart-Pointer gleichzeitig die selbe Ressource verwalten können. Sobald die Lebensdauer des *unique pointer* oder des letzten *shared pointer* endet, wird die verwaltete Ressource automatisch vom Smart-Pointer freigegeben.

Das folgende Beispiel arbeitet mit einer selbst implementierten Variante der *shared pointers*. Dabei ist die Klasse `SmartToken` ein Smart-Pointer für Objekte vom Typ `Token`:

```
1 void fluppie() {
2     // new smart pointer for token "wuppie":
3     // wuppie lives on the stack, the token lives on the heap (`new`)
4     SmartToken wuppie = SmartToken(new Token("wuppie", 1, 4));
5
6     if (bla==42) {
7         // fluppie shares resource with wuppie
8         SmartToken fluppie = SmartToken(wuppie);
9         // fluppie2 shares resource with wuppie
10        SmartToken fluppie2(wuppie);
11    }
```

²Dereferenzierung von Null-Pointern oder nicht initialisierten Pointern, Nutzung von `delete` für Pointer, die nicht mit `new` erstellt wurden, mehrfaches `delete`, Speicherlöcher durch Vergessen von `delete`, Dangling Pointer, verwitwete Objekte, ...

```

11
12      // now there are 3 smart pointers sharing the same resource (token "
      wuppie")
13
14      // new smart pointer for token "foo"
15      SmartToken foo = SmartToken(new Token("foo", 9, 35));
16  }    // fluppie, fluppie2, foo will be removed from the stack - foo releases
      its resource
17
18      // wuppie is the only smart pointer with shared resource "wuppie"
19
20 } // wuppie will be removed from the stack - wuppie releases its resource

```

Im Beispiel wird mit `new Token("wuppie", 1, 4)` ein neues Token-Objekt auf dem Heap angelegt. Der Smart-Pointer `wuppie` übernimmt die Ressource im Konstruktor und verwaltet den Pointer. Wichtig ist zu beobachten: Das Token wird auf dem Heap angelegt, während der Smart-Pointer `wuppie` eine normale lokale ("automatische") Variable ist und auf dem Stack liegt.

In der Kontrollstruktur werden weitere Smart-Pointer angelegt. Die ersten beiden (`fluppie`, `fluppie2`) teilen sich die Ressource (den Pointer auf das Token) mit `wuppie`. Es wird kein neues Token angelegt oder kopiert. Der dritte Smart-Pointer `foo` verwaltet ein weiteres Token.

Mit der Kontrollstruktur endet auch die Lebensdauer der lokalen Variablen `fluppie`, `fluppie2` und `foo`, sie werden automatisch vom Stack entfernt. Da `foo` der letzte Smart-Pointer ist, der das Token "foo" verwaltet, wird hier die Ressource freigegeben. Bei `fluppie` und `fluppie2` werden nur die Smart-Pointer auf dem Stack entfernt, die verwaltete Ressource (Token "wuppie") bleibt erhalten, da die noch von einem anderen Smart-Pointer verwaltet wird.

Mit dem Ende der Funktion endet auch die Lebensdauer des Smart-Pointers `wuppie`. Er wird automatisch vom Stack entfernt, und da er im Beispiel der letzte Smart-Pointer ist, der das Token "wuppie" verwaltet, wird dabei automatisch der Pointer zu "wuppie" wieder freigegeben.

Ein Smart-Pointer soll entsprechend folgende Eigenschaften haben:

- Verwendung soll analog zu normalen Pointern sein (Operatoren `*` und `->` überladen)
- Smart-Pointer haben niemals einen undefinierten Wert: entweder sie zeigen auf ein Objekt oder auf `nullptr`³
- Kopieren von (*shared*) Smart-Pointern führt dazu, dass sich mehrere Smart-Pointer das verwiesene Objekt *teilen*
- Smart-Pointer löschen sich selbst (und das verwiesene Objekt, falls kein anderer Smart-Pointer mehr darauf zeigt), wenn die Smart-Pointer ungültig werden (bei Verlassen des Scopes bzw. bei explizitem Aufruf von `delete` auf einen Pointer auf einen Smart-Pointer)
- Es gibt keine verwitweten Objekte mehr: Wenn mehrere Smart-Pointer auf das selbe Objekt zeigen, darf erst der letzte Smart-Pointer das Objekt aus dem Heap löschen
- Smart-Pointer funktionieren nur für mit `new` erzeugte Objekte

Weitere übliche Eigenschaften, die wir auf diesem Blatt aber vereinfachend ignorieren⁴:

³Sie müssen für `nullptr` den g++ auf C++11 oder höher umstellen (`--std=c++11`) und den Header `<cstdint>` includen.

⁴Templates haben wir hier noch nicht behandelt, Exceptions werden wir gar nicht betrachten

- Smart-Pointer sollen für beliebige Klassen nutzbar sein (Template-Klasse)
- Dereferenzierung von nicht existierenden Objekten (d.h. der Smart-Pointer zeigt intern auf `nullptr`) führt nicht zum Programmabsturz, sondern zu einer Exception

14.3.3. Reference Counting

Smart-Pointer werden erzeugt, indem sie entweder einen Pointer auf die zu verwaltende Ressource bekommen (Konstruktor) oder eine Referenz auf ein anderes Smart-Pointer-Objekt (Copy-Konstruktor).

Im Smart-Pointer wird entsprechend der Pointer auf die zu verwaltende Ressource gespeichert.

Für die Bestimmung, wie viele Smart-Pointer sich eine Ressource teilen, muss ein Zähler implementiert werden. Sobald sich ein weiterer Smart-Pointer die selbe Ressource teilt, muss dort auch der Zähler (per Pointer!) übernommen werden und entsprechend inkrementiert werden. Im Destruktor muss der Zähler dekrementiert werden. Falls dabei der Zähler den Wert 0 erreicht, werden die Pointer auf die Ressource und den Zähler freigegeben.

Bei einer Zuweisung verfährt man analog.

```

1  class SmartToken {
2  public:
3      /**
4       * Constructor
5       *
6       * Constructs a new smart pointer from a raw pointer, sets the reference
7       * counter to 1.
8       *
9       * @param p is a raw pointer to the token to be shared
10     */
11     SmartToken(Token* p = nullptr);
12
13     /**
14     * Copy constructor
15     *
16     * Constructs a new smart pointer from another smart pointer, increments
17     * the reference counter.
18     *
19     * @param sp is another smart pointer
20     */
21     SmartToken(const SmartToken& sp);
22
23     /**
24     * Destructor
25     *
26     * Decrements the reference counter. If it reaches zero, the shared token
27     * will be free'd.
28     */
29     ~SmartToken();
30
31     /**
32     * Assignment
33     *
34     * Changes the shared token, thus we need first to perform something like

```

```

35     * the destructor, followed by something like the constructor.
36     *
37     * @param sp is another smart pointer
38     */
39     SmartToken& operator=(const SmartToken& sp);
40
41 private:
42     Token* pObj;          ///< Pointer to the current shared token
43     RefCounter* rc;       ///< Pointer to the reference counter (used for the
                           current token)
44 };
45
46 class RefCounter {
47 public:
48     /**
49      * Default constructor
50      */
51     RefCounter();
52
53     /**
54      * Increment count
55      */
56     void inc();
57
58     /**
59      * Decrement count
60      */
61     void dec();
62
63     /**
64      * Compare the counter with zero
65      *
66      * @return true if n==0, false otherwise
67      */
68     bool isZero() const;
69
70     // Hide copy constructor and assignment operator
71     RefCounter(const RefCounter&) = delete;
72     RefCounter& operator=(const RefCounter&) = delete;
73
74 private:
75     unsigned int n;       ///< How many SmartToken share ownership of "our"
                           object?
76 };

```

14.3.4. Dereferenzierung von Smart-Pointern

(Anmerkung: Dies ist ein Vorgriff auf die Lektion "Operatoren". Betrachten und implementieren Sie die vorgegebenen Operatoren einfach wie normale Methoden.)

Pointer lassen sich dereferenzieren, d.h. man greift direkt auf das verwiesene Objekt zu. Dies lässt sich auch für Smart-Pointer erreichen, indem die beiden Dereferenzierungsoperatoren überladen werden.

```

1 class SmartToken {

```

```

2  public:
3      /**
4       * Dereferences the smart pointer
5       *
6       * @return a reference to the shared token
7       */
8      Token& operator*();
9
10     /**
11     * Dereferences the smart pointer
12     *
13     * @return a pointer to the shared token
14     */
15     Token* operator->();
16 };

```

Damit lässt sich das folgende Verhalten realisieren (Vergleich *raw* Pointer vs. Smart-Pointer):

```

1  Token* foo = new Token("foo", 9, 35);           // raw pointer foo
2  SmartToken wuppie = SmartToken(new Token("wuppie", 1, 4)); // smart pointer
   wuppie
3
4  // Access via token pointer
5  cout << (*foo).lexem << endl;    // "foo"
6  cout << foo->lexem << endl;      // "foo"
7
8  // Access via smart pointer
9  cout << (*wuppie).lexem << endl;  // "wuppie"
10 cout << wuppie->lexem << endl;    // "wuppie"

```

Dabei ist die Form “->” eine vereinfachte Darstellung von `(*ptr)` ., d.h. ein Pointer (linke Seite des Ausdrucks) wird dereferenziert und man greift auf Attribute oder Methoden des verwiesenen Objekts zu (rechte Seite des Ausdrucks).

14.4. Aufgaben

14.4.1. A7.1: Klasse für Token (1P)

Implementieren Sie in C++ die Klasse `Token` mit der folgenden Schnittstelle:

```

1  class Token {
2  public:
3      /**
4       * Constructs a new token object.
5       *
6       * @param l is a pointer to the text of the token (to be copied)
7       * @param r is the row in input where this token was found
8       * @param c is the column in input where this token starts
9       */
10     Token(const char* l, int r, int c);
11
12     /**

```

```

13     * Destructs the token object and free's the stored lexem.
14     */
15     ~Token();
16
17 private:
18     char* lexem;    ///< Pointer to the text of the token
19     int row;        ///< Row in input where this token was found
20     int col;        ///< Column in input where this token starts
21 };

```

Trennen Sie Deklaration und Implementierung.

Der Konstruktor muss den übergebenen `char*` kopieren, d.h. Sie müssen die Länge des übergebenen C-Strings bestimmen, ausreichend viel Speicher mit `new` für `char* lexem` reservieren und danach den String kopieren (C-Funktion).

Sorgen Sie dafür, dass der Speicher beim Vernichten eines `Token`-Objekts wieder korrekt freigegeben wird.

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `Token`-Klasse an selbst gewählten Beispielen.

14.4.2. A7.2: Implementierung eines einfachen Tokenizers (1P)

Erstellen Sie eine Funktion `void tokenize(const string& input, vector<Token>& tokens)`, die einen gegebenen String als Eingabe erhält und diesen in Tokens (Wörter) splittet. Nutzen Sie Referenzen, um die Token-Liste zu aktualisieren. Testen Sie die Funktion mit verschiedenen Eingabestrings und geben Sie die Tokens aus.

14.4.3. A7.3: Reference Counter (1P)

Implementieren Sie nun die Klasse `RefCounter` mit der obigen Schnittstelle. Auch hier können Sie bei Bedarf zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `RefCounter`-Klasse an selbst gewählten Beispielen.

14.4.4. A7.4: Smart-Pointer (3P)

Implementieren Sie nun die Smart-Pointer für `Token`-Objekte mit folgender Signatur (wie oben, leicht erweitert):

```

1 class SmartToken {
2 public:
3     /**
4      * Constructor
5      *
6      * Constructs a new smart pointer from a raw pointer, sets the reference
7      * counter to 1.
8      *

```

```

 9      * @param p is a raw pointer to the token to be shared
10      */
11      SmartToken(Token* p = nullptr);
12
13      /**
14       * Copy constructor
15       *
16       * Constructs a new smart pointer from another smart pointer, increments
17       * the reference counter.
18       *
19       * @param sp is another smart pointer
20       */
21      SmartToken(const SmartToken& sp);
22
23      /**
24       * Destructor
25       *
26       * Decrements the reference counter. If it reaches zero, the shared token
27       * will be free'd.
28       */
29      ~SmartToken();
30
31      /**
32       * Assignment
33       *
34       * Changes the shared token, thus we need first to perform something like
35       * the destructor, followed by something like the constructor.
36       *
37       * @param sp is another smart pointer
38       */
39      SmartToken& operator=(const SmartToken& sp);
40
41      /**
42       * Dereferences the smart pointer
43       *
44       * @return a reference to the shared token
45       */
46      Token& operator*();
47
48      /**
49       * Dereferences the smart pointer
50       *
51       * @return a pointer to the shared token
52       */
53      Token* operator->();
54
55      /**
56       * Comparison
57       *
58       * @param sp is another smart pointer
59       * @return true, if `sp` shares the same token
60       */
61      bool operator==(const SmartToken& sp) const;
62
63      private:
64      Token* pObj;          ///< Pointer to the current shared token
65      RefCounter* rc;       ///< Pointer to the reference counter (used for the

```

```

        current token)
66  };

```

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `SmartToken`-Klasse an selbst gewählten Beispielen sowie an den obigen Beispielen.

14.4.5. A7.5: Ringpuffer (4P)

Ein Ringpuffer ist eine Form der abstrakten Datenstruktur “Warteschlange” (*Queue*), die nur eine beschränkte Anzahl n von Elementen (Datensätzen) aufnehmen kann. Die Daten werden nach dem FIFO-Prinzip über die Funktion `writeBuffer()` am Ende der Schlange eingefügt und mit der Funktion `readBuffer()` vom Anfang der Schlange entnommen.

Aus Effizienzgründen wird bei `readBuffer()` nur das erste Element zurückgeliefert, das gelesene Element wird aber (noch) nicht aus dem Ringpuffer entfernt. Über ein Attribut `head` merkt man sich stattdessen, wo das nächste zu lesende Element liegt (auf dem Platz hinter dem aktuell gelesenen). Ist der Puffer voll, wird bei `writeBuffer()` das älteste Element entfernt und das neue Element auf dem frei gewordenen Platz im internen Array `elems` eingefügt.

Unser Ringpuffer ist auf Elemente vom Typ `SmartToken` festgelegt. Es wird davon ausgegangen, dass diese Elemente Smart-Pointer mit der *shared pointer*-Semantik sind.⁵ Da die `SmartToken` selbst (zum Teil) eine Pointersemantik implementiert haben (man kann die Smart-Pointer dereferenzieren), vermeiden wir Pointer auf die Smart-Pointer in der Schnittstelle und arbeiten stattdessen mit C++-Referenzen bzw. Kopien: Bei `writeBuffer()` wird ein `SmartToken` per konstanter C++-Referenz übergeben, und bei `readBuffer()` wird eine Kopie des gelesenen `SmartToken` zurückgeliefert.

Der Puffer kann effizient durch ein zur Laufzeit angelegtes **Array** mit `size` (Template-Parameter) Plätzen zur Speicherung der Pointer auf die Elemente realisiert werden. Die Ringstruktur wird durch Modulo-Operationen auf den Array-Indizes realisiert.

Implementieren Sie nun den Ringpuffer für `SmartToken`-Objekte mit folgender Signatur:

```

1  class RingBuffer {
2  public:
3      /**
4       * Constructor that creates a new ring buffer for max. `size` elements
5       *
6       * Initialises the attributes and allocates memory for `size` elements
7       * of type `SmartToken` and let the pointer `elems` point to this new
8       * array
9       *
10      * @param size is the max. number of elements that can be stored
11      */
12      RingBuffer(unsigned int size);
13
14      /**
15       * Destructor
16       *

```

⁵Wenn Sie die obigen Aufgaben richtig gelöst haben, haben Sie genau diese Semantik vorliegen.

```

17     * free's the dynamically allocated array `elems`
18     */
19     ~RingBuffer();
20
21     /**
22     * Reading the first (oldest) element
23     *
24     * If an element has been read, the `head` points to the next element
25     * and `count` is decremented. The read element is not released.
26     *
27     * @return Returns (a copy of) the first (i.e. oldest) element of the
28     * buffer, but does not (yet) release it; returns an empty `SmartToken`
29     * if buffer is empty
30     */
31     SmartToken readBuffer();
32
33     /**
34     * Adding a new element
35     *
36     * Appends the new element to the end of the queue. If the buffer is
37     * full, the oldest element will be overwritten by the new element. The
38     * old element will take care of releasing its memory (smart pointer).
39     *
40     * @param data is a reference to the element to be added
41     */
42     void writeBuffer(const SmartToken& data);
43
44 private:
45     unsigned int count;    ///< number of elements currently stored in the
46                          buffer
47     unsigned int head;    ///< points to the beginning of the buffer (oldest
48                          element)
49     unsigned int size;    ///< length of array `elems`
50     SmartToken* elems;    ///< array with `size` places of type `SmartToken`,
51                          dynamically allocated
52 };

```

Bei Bedarf können Sie zusätzliche Attribute und Methoden hinzufügen.

Testen Sie Ihre `RingBuffer`-Klasse an selbst gewählten Beispielen. Überzeugen Sie sich insbesondere vom korrekten Zugriff auf die Ringstruktur und prüfen Sie, ob die Smart-Pointer wie gewünscht arbeiten. Prüfen Sie hierzu auch die `RefCounter` der beteiligten Smart-Pointer. Welche Sonderfälle können Sie identifizieren?

15. Blatt 08: Mini-Projekt C++-Interpreter

15.1. Aufgabe

Sie entwickeln in Ihrem 3er-Team eine kleine, C++-ähnliche Sprache und implementieren Lexer, Parser, semantische Analyse, Interpreter und eine einfache REPL.

Der Fokus liegt auf Variablen und C++-Referenzen, auf Funktionen, auf Klassen, Einfach-Vererbung und dem Unterschied zwischen statischem und dynamischem Dispatch (nicht-virtual vs. virtual) - ohne Pointer/Heap und ohne Casts. Die Sprache ist eine echte Teilmenge von C++ und soll (abgesehen von den REPL-Erweiterungen) mit einem C++-Compiler kompilierbar sein. Präprozessor-Zeilen (`#include . . .`) dürfen in Dateien vorkommen; Ihr Interpreter soll sie ignorieren bzw. wie Kommentare behandeln.

Sie *können* ANTLR zur Erstellung Ihres Lexers und Parsers einsetzen, Sie können aber auch gern einen selbst implementierten LL-Parser einsetzen.

Definieren Sie zunächst eine passende Grammatik und den AST, bevor Sie Lexer und Parser umsetzen. Achten Sie auf eine angemessene semantische Analyse, achten Sie dabei auf die in der Vorlesung besprochene sowie auf diesem Blatt skizzierte und über die Tests gegebene C++-Semantik. Der Interpreter selbst soll ein einfacher Tree-walking Interpreter wie in der Vorlesung besprochen sein.

Wichtig: Wir haben die meisten benötigten Techniken besprochen und geübt. An einigen wenigen Stellen müssen Sie selbst kreativ werden, etwas beim Handling von überladenen Funktionen bzw. Vererbung und überschriebenen Methoden. Recherchieren Sie eigenständig zu den Themen.

15.1.1. Betrachtete Sprache: Sub-Dialekt von C++

15.1.1.1. Sprachkern

Unterstützen Sie mindestens folgende C++-Konzepte:

- Typen:
 - `bool` (Werte `true`, `false`), `int` (Ziffernfolgen ohne Dezimalpunkt, etwa 42), `char` (Character in einfachen Anführungsstrichen, etwa `'a'`), `string` (Zeichenkette in doppelten Anführungsstrichen, etwa `"foo"`)
 - `void` für Funktionen ohne Rückgabewert
 - Zusätzlich Klassertypen (s.u.)
 - Escapes in Literalen mit `\`, beispielsweise `'\0'` als einzelnes `char`
- Variablen und Zuweisungen:

- Deklaration `T x;` und Initialisierung `T x = expr;`
- Einfache Zuweisung `=`
- Nur lokale Variablen (keine globalen Variablen)
- Ausdrücke:
 - Arithmetik (nur `int`): `+`, `-`, `*`, `/`, `%` (binäre Ausdrücke); `+`, `-` (unäre Ausdrücke)
 - Zuweisung: `=`
 - Vergleich: `==`, `!=`, `<`, `<=`, `>`, `>=` (`int` und `char`; `bool` und `string` nur `==` und `!=`)
 - Logik: `&&` und `||` (beide mit *Short-Circuit*) und `!` (nur `bool`)
 - Klammern zur Gruppierung von Ausdrücken `(...)`
 - Funktionsaufrufe `f(args)`
 - Feld-/Methodenzugriff: `obj.f`, `obj.m(args)`
- Kontrollfluss:
 - `if-then-else` mit optionalem `else`-Teil, `while`, Block `{ ... }`
 - `return`
- Funktionen/Methoden:
 - Definition und Aufruf
 - Überladung (Overloading) nur per exakt passender Signatur (Name + Anzahl + exakte Typen inkl. &-Markierung)
- C++-Referenzen "light":
 - Deklaration Variable: `T& x = expr;` bzw. Deklaration Parameter: `T& p`
 - Initialisierung für Referenz-Variablen obligatorisch
- Klassen, Einfach-Vererbung (genau eine optionale Basisklasse), Polymorphie:
 - `class A { public: /*Felder + Methoden */ }` mit Feldern und Methoden (alles "public" sichtbar)
 - Felder können vom Typ her Basistypen oder Klassen sein
 - Parameterloser Konstruktor und weitere Konstruktoren (jeweils ohne Initialisierungslisten), Verwendung nur als `T x;` (ruft `T()` auf) oder `T x = T(args);` (kein direkter Aufruf `T x(args);`!)
 - Methoden können als `virtual` deklariert werden
 - `class D : public B { public: /*Felder + Methoden */ }`: Vererbung mit genau einer Basisklasse, keine Zyklen
- Eingebaute Funktionen (Runtime/Standardbibliothek): `print_bool`, `print_int`, `print_char`, `print_string` (Ausgabe eines Werts des jeweiligen Typs)
- Kommentare:
 - Zeilenweise Kommentare: `//` bis zum Zeilenende
 - Block-Kommentare: `/*` bis zum `*/` (kann über mehrere Zeilen gehen)
 - Präprozessor-Direktiven: `#` bis zum Zeilenende (soll als Kommentar behandelt werden)
- Programmorganisation:
 - Ein einzelnes Source-File mit einer optionalen `main()`-Funktion

- Präprozessor-Anweisungen sollen Sie wie Kommentare ignorieren (behandeln Sie `#include` wie einen zeilenweisen Kommentar, d.h. ab einem `#` wird der Input bis zum nächsten Zeilenumbruch ignoriert)
- Main-Funktion: zulässig sind die Formen `int main()` und `void main()`

Hinweis: Polymorphie in dieser Sprache folgt C++-Prinzipien (Slicing bei Wertkopie, dynamischer Dispatch über Referenzen), nicht Java-Semantik.

15.1.1.2. Reservierte Schlüsselwörter

`int, bool, char, string, true, false, if, else, while, return, class, void, public, virtual`

15.1.1.3. Semantik-/Typregeln

- Keine Mehrfachdefinitionen in demselben Scope (Variablen/Funktionen/Klassen)
- Sichtbarkeit:
 - Verwendungen müssen im aktuellen Scope sichtbar (auflösbar) sein
 - Variablen: “define-before-use”, d.h. Variablen müssen bereits im ersten Pass sichtbar sein
 - Funktionen, Methoden, Klassen: “define-after-use” - mehrere PASSES notwendig (nur beim Start des Interpreters)
- LValues: benannte Variablen `a` bzw. Feldzugriffe `obj.f`; alles andere sind RValues
- Variablen sind nicht aufrufbar; Funktionen sind nicht zuweisbar (keine Funktion als LValue)
- Keine impliziten Typkonversionen außer im booleschen Kontext (in `if/while`-Bedingungen werden `int/char/string` wie in C++ implizit in `bool` konvertiert, d.h. `0` bzw. der leere String werden als `false` und alles andere als `true` behandelt)
- Ausdrücke:
 - Bei binären Operatoren müssen beide Seiten den selben Typ haben
 - Zuweisung ist ein Ausdruck; die linke Seite des Zuweisungsoperators muss ein LValue sein
 - Das Ergebnis der Zuweisung ist Wert/Typ der rechten Seite
 - Operatorpräzedenz und Assoziativität: wie in C++ üblich (Reihenfolge: unär, multiplicative, additive, relational, equality, `&&`, `||`, assignment; assignment rechtsassoziativ)
- Funktionen/Methoden:
 - Argumentanzahl muss zur Parameterliste passen
 - Overload-Auflösung: exakter Match Name und Arität und identische Typen inkl. `&`-Markierung; bei Mehrdeutigkeit Fehler
- Referenzen
 - Referenz-Parameter und -Variablen können nur mit LValues initialisiert werden
 - Keine Neubindung: Zuweisung an eine Referenz-Variable schreibt in das referenzierte Ziel
 - Keine Referenzen als Felder/globale Variablen
 - Keine `&`-Rückgaben (Rückgabe nur als Kopie)

- Klassen/Methoden:
 - Parameterloser Default-Konstruktor wird synthetisiert, wenn keiner definiert wurde
 - Initialisierung von Feldern über impliziten (generierten) parameterlosen Default-Konstruktor: `bool: false, int: 0, char: '\0', string: ""`, Felder von Klassentyp werden per Default-Konstruktor initialisiert
 - Bei Vererbung wird von Konstruktoren bei abgeleiteten Klassen jeweils der parameterlose Basiskonstruktor implizit vor dem Body aufgerufen
 - Kein `this`. Unqualifizierte Namen in Methoden binden an lokale Variablen/Parameter vor eigene Members vor geerbte Members vor globale Scope.
 - Variablen vom Klassentyp sind Werte (feldweise Kopie bei Rückgabe/Zuweisung)
 - Bei abgeleiteten Klassen wird beim Konstruktoraufwurf implizit zunächst der parameterlose Default-Konstruktor der Basisklasse aufgerufen
 - Abgeleitete Klassen erben alle Felder und Methoden der Basisklasse
 - Zuweisung `Base b; b = d;` (mit `class D : public B { ... }` und `D d;`) führt zum Slicing (einzige Ausnahme der Regel für selbe Typen auf beiden Seiten der Zuweisung)
 - Polymorphe Nutzung erfolgt ausschließlich über Referenzen: `B& b = d; b.m();` ruft die überschriebene Methode in `D` auf (wenn `B::m()` zusätzlich als `virtual` deklariert ist)
 - Virtuelle Bindung wie in C++: Ist die Methode im statischen Typ `virtual`, erfolgt dynamischer Dispatch bei Aufruf über Referenzen; Overrides in abgeleiteten Klassen sind auch ohne erneutes `virtual` virtuell
- Fehlerbehandlung:
 - Lexer-, Parser-, Typ-Fehler beenden die Analyse mit klarer Meldung
 - Laufzeitfehler (z.B. Division durch 0) sind sauber zu melden
- REPL (**abweichend** von C++): Neue Variablen/Funktionen/Klassen dürfen in der REPL definiert werden; sie werden in den Sitzungs-Scope aufgenommen und stehen danach zur Verfügung. Für Funktionen und Klassen ist in der REPL kein "define-after-use" mehr möglich.

15.1.1.4. Nicht Teil des Umfangs

Weitere, bisher nicht benannte mit C++ verbundene Konzepte wie beispielsweise Präprozessor, Header-Files, Pointer, Arrays, Templates, Sichtbarkeiten in Klassen, Trennung Deklaration/Implementierung bei Klassen (Trennung .h und .cpp) brauchen Sie nicht umsetzen.

Darunter fallen auch (Aufzählung nicht vollständig):

- Pointer/Adressen: `&` (Adressoperator), `*` (Dereferenzierung), `->`/`new/delete`, Speicherverwaltung
- Casts, Inkrement/Decrement, Compound-Assignments, Mehrfachvererbung, Templates, `static`, `const`, `friend`, Namespaces
- `sizeof`, `typedef`, `using`
- Unterscheidung signed und unsigned Datentypen
- Sichtbarkeiten (außer `public`)
- `this`, dadurch auch kein Shadowing von Member-Namen durch lokale Variablen bzw. Parameter
- Arrays (eindimensional, mehrdimensional)

- Kein **break/continue** in Schleifen
- Globale Variablen
- Referenzen als Rückgabetyt, Referenzen als Feld in Klassen
- Reine Funktionsdeklarationen (auch mit namenlosen Parametern)
- Initialisierungslisten, delegierende Konstruktoren, Destruktoren
- Shadowing: In Methoden sollen Parameter und lokale Variablen nicht den selben Namen haben wie die Felder der Klasse

15.1.1.5. Hinweis “*most vexing parse*”

In C++ entsteht durch die üblichen Syntax-Regeln das “*most vexing parse*”-Problem, wo nach dem Muster `T ID ( ... )`; sowohl ein Funktionsprototyp als auch eine Variablendeklaration mit Konstruktor-Syntax möglich wäre.

Da mit den oben definierten Regeln Funktionsprototypen in diesem Projekt nicht erlaubt sind und für Funktionsdefinitionen immer ein anschließender Block auftaucht (`T ID ( ... ) { ... }`), Variablen für Basistypen entweder über die Form `T x`; oder mit Initialisierung als `T x = expr`; deklariert werden, und Variablen für Klassentypen in der Form `T x`; (Aufruf parameterloser Konstruktor) bzw. der Form `T x = T(arg)`; (Erzeugung eines temporären Objekts mit dem passenden Konstruktor und elementweises Kopieren) deklariert werden, tritt das klassische *most vexing parse* in diesem Projekt nicht auf.

15.1.2. REPL-Modell

15.1.2.1. Initialisierung

Der Interpreter soll beim Start eine optional angegebene Datei mit C++-Code einlesen und verarbeiten können. Für die semantische Prüfung des eingelesenen Codes soll eine Mehrpass-Prüfung realisiert werden (Funktionen und Klassen: “define-after-use” erlaubt). Eine optional enthaltene `main()`-Funktion soll in einem “Sitzungs-Scope” ausgeführt werden, welcher vom globalen Scope “abzweigt”. Der Sitzungs-Scope wird offen gehalten.

15.1.2.2. REPL

Danach soll dem User eine REPL angeboten werden, die einen Prompt in der Konsole ausgibt und in der weitere C++-Statements (inklusive Blöcke und Expression-Statements) eingegeben werden können, die dann im Sitzungs-Scope interpretiert werden. Dadurch wird in den neu eingegebenen Statements der Zugriff auf Variablen in der `main()`-Funktion und auf frühere Funktions- und Klassendefinitionen möglich.

In der REPL können zusätzlich auch neue Klassen und Funktionen und Variablen definiert und genutzt werden. Klassen und Funktionen erweitern dabei den globalen Scope und haben keinen Zugriff auf Sitzungsvariablen; neue Variablen landen dagegen im Sitzungs-Scope. Im Unterschied zur Initialisierungsphase gilt in der REPL stets “define-before-use”, d.h. alle benutzten Namen müssen vorher bereits definiert worden sein.

In der REPL wird nur das Ergebnis von Expression-Statements automatisch ausgegeben; alle anderen Statements erzeugen keine Ausgabe. Für explizite Ausgaben nutzen Sie `print_*`(). Fehler (Lexer, Parser, semantische Analyse, Interpreter/Laufzeit) sollen ebenfalls auf der Konsole ausgegeben werden.

Implementieren Sie eine spezielle Eingabe, mit der die REPL beendet werden kann.

Hinweis: Achten Sie darauf, dass Ihr Interpreter die Eingabe nicht zu früh beendet und zu früh mit der Interpretation beginnt! Oft ist ein Zeilenumbruch das korrekte Signal zum Start der Verarbeitung. Es gibt aber auch Situationen (etwa Funktions- oder Klassendefinitionen o.ä.), wo ein Zeilenumbruch noch nicht eine vollständige Eingabe anzeigt. Hier soll nach einem Zeilenumbruch ein Hilfsprompt ausgegeben werden, damit der User weiss, dass die aktuelle Eingabe noch läuft.

15.1.3. Tests

Im [Starter-Projekt](#) finden Sie einige Positiv- und Negativ-Tests. Für die Positivtests ist die erwartete Antwort des Interpreters angegeben, bei der Interpretation der Negativtests sollte es eine entsprechende Fehlermeldung (Lexer, Parser, semantische Analyse, Interpreter/Laufzeit) geben.

Betrachten Sie die Testfälle als ausführbare Ergänzung der obigen Spezifikation.

Entwickeln Sie weitere aussagekräftige Tests, die die Vorgaben ergänzen.

15.2. Projektvorstellung: Walk-Through statt Präsentation

Stellen Sie Ihr Projekt am Semesterende vor (Termine siehe Fahrplan und Ankündigung).

Jedes Team hat dafür **15 Minuten** Zeit.

Gehen Sie dabei am Code durch Ihr Projekt und diskutieren Sie relevante Teile, mindestens aber:

- Spezifikation: kurze Sprachbeschreibung mit allen getroffenen Designentscheidungen
- Implementierung: ANTLR-Grammatik, AST, Scopes/Symboltabellen, semantische Checks, Interpreter mit dynamischem Dispatch, REPL
- Kurze Anleitung: Wie bauen/ausführen, wie REPL nutzen.

Demonstrieren Sie die Funktionsfähigkeit mit Ihren C++-Codebeispielen.

Sie sollen keine Folien erstellen. Die Präsentation soll live in der IDE erfolgen.

15.3. Abgabeformat

Geben Sie wie bei den anderen Blättern auch pro Person ein individuelles Post Mortem im ILIAS ab. Geben Sie dabei im Post Mortem unbedingt die URL zu Ihrem Team-Repo an. Denken Sie daran, dass dieses Repo öffentlich einsehbar sein muss.

Teil IV.

Organisatorisches

16. Prüfungsvorbereitung

TL;DR

Durchführung: Präsenz oder Open-Book (je nach Corona-Lage)

Die Klausur wird dieses Semester elektronisch stattfinden. Dazu werden wir den Prüfungs-ILIAS der HSBI nutzen.

Sofern die Situation dies zulässt, wird die Klausur in den Räumen der HSBI am Campus Minden unter Aufsicht durchgeführt. Hier werden Ihnen Rechner für den Zugang zum Prüfungs-ILIAS zur Verfügung gestellt, Sie benötigen nur Ihre HSBI-Zugangsdaten (User, Passwort), einen Studierendenausweis und Personalausweis sowie Ihren DIN-A4-Spickzettel.

Wenn die Corona-Lage eine Durchführung in Präsenz nicht erlaubt, wird die Klausur stattdessen als *Open-Book-Ausarbeitung* aus dem Home-Office durchgeführt. Sie benötigen dazu einen normalen Rechner oder Laptop mit einem Standardbrowser. Tablets und Handys können wg. der Mobil-Version der Browser problematisch sein. Sie müssen JavaScript aktivieren und Cookies zulassen, der Privacy-Modus ist bitte ebenfalls zu deaktivieren. Sie erreichen den Prüfungs-ILIAS eassessment.hsbi.de nur über VPN.

Die Entscheidung über die konkrete Durchführung wird spätestens zwei Wochen vor der Prüfung getroffen und Ihnen per EMail über das LSF mitgeteilt.

Ablauf der Klausur

Die Prüfung (das ILIAS-Objekt) selbst schalte ich erst zum Start der Prüfung online. Bei der Durchführung als Open-Book-Ausarbeitung wird parallel zur Prüfung eine Zoom-Sitzung laufen, in der Sie Fragen stellen können.

Hilfsmittel und Themen

Bei der Durchführung in Präsenz am Campus Minden ist ein Spickzettel (DIN A4, beidseitig beschrieben) als Hilfsmittel zugelassen.

Bei der Durchführung als "Open-Book-Ausarbeitung" im Home-Office sind alle Hilfsmittel zugelassen.

Die Unterstützung durch Dritte bzw. jegliche Kommunikation mit Dritten ist in keinem Fall zugelassen. Sie sollen die Prüfung selbstständig bearbeiten.

Es wird keines der behandelten Themen ausgeschlossen, allerdings eignen sich manche Themen etwas besser für Klausurfragen als andere ;-)

Videos

- [Hinweise zur Prüfung: Fragetypen-Demo](#)
- [Hinweise zur Prüfung: Technische Vorbereitung](#)

16.1. Elektronische Klausur: Termin, Materialien

16.1.1. Termin

Die schriftliche Prüfung erfolgt durch eine Klausur, die als digitale Prüfung auf einem Prüfungs-ILIAS durchgeführt wird.

Es wird angestrebt, die Klausur in Präsenz in den Rechnerpools am Campus Minden durchzuführen. Falls dies wegen der Corona-Situation oder anderer Umstände nicht möglich sein sollte, wird die Klausur als “Open-Book-Ausarbeitung” im Home-Office durchgeführt.

Es wird in beiden Prüfungszeiträumen ein Termin angeboten. Die Termine werden vom Prüfungsamt bekannt gegeben.

Dauer jeweils 90 Minuten.

- Die konkrete Durchführungsform (in Präsenz am Campus Minden oder im Home-Office) wird Ihnen spätestens zwei Wochen vor der Prüfung über das LSF bekanntgegeben

16.1.2. Zugelassene Hilfsmittel

Präsenz (in Minden)

Zugelassene Materialien: DIN-A4-Spickzettel (beidseitig)

Sie dürfen **einen** Spickzettel im **DIN-A4**-Format benutzen, der beidseitig beschrieben sein kann.

Ich möchte Sie hier noch einmal ermuntern, diesen Zettel tatsächlich manuell zu erstellen (also ganz traditionell zu **schreiben**), da bereits der Schreibvorgang einen gewissen Lerneffekt bewirkt!

Open-Book-Ausarbeitung (Homeoffice)

Falls die Prüfung als Open-Book-Ausarbeitung im Home-Office durchgeführt werden sollte, dürfen Sie alle Unterlagen benutzen.

- Ausnahme: **Keine Hilfe durch Dritte!** (insbesondere keine Zusammenarbeit, keine Kommunikation)
Sie sollen die Prüfung eigenständig bearbeiten. Hilfe von Dritten sowie jegliche Kommunikation mit Dritten ist in keinem Fall zugelassen und wird als Täuschungsversuch gewertet.

16.1.3. Einsicht

- Prüfungseinsicht: Zeitnah; Bekanntgabe per Mail

16.2. Technische Vorbereitungen

Präsenz (in Minden)

Diese Bemerkungen betreffen die Durchführung als Präsenzprüfung in den Räumen am Campus Minden.

- **HSBI-Zugangsdaten:** Username, Passwort

Bei der Durchführung der Prüfung am Campus Minden wird Ihnen ein Rechner zur Verfügung gestellt. Dort läuft voraussichtlich ein Browser im Kiosk-Mode, wo Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie ihre HSBI-Zugangsdaten, mit denen Sie sich auch im “normalen” ILIAS anmelden.

- **Studierendenausweis** und Personalausweis

An der Prüfung dürfen nur Personen teilnehmen, die dafür im LSF angemeldet sind. Es findet eine entsprechende Kontrolle statt. Halten Sie Ihren Studierendenausweis und Personalausweis bereit.

Open-Book-Ausarbeitung (Homeoffice)

Diese Bemerkungen betreffen die Durchführung aus dem Home-Office mit Ihrer Hardware. Bei der Durchführung in Präsenz in den Räumen am Campus Minden werden die technischen Details von uns für Sie vorbereitet sein.

- **Rechner:** Nutzen Sie für die Prüfung einen stationären Rechner oder ein Notebook.

Vermeiden Sie die Verwendung von Tablets und Smartphones! Bei der Verwendung von Tablets kann es unter Umständen zu Darstellungsproblemen kommen. Smartphones sind aufgrund des kleinen Bildschirms für die Prüfungsdurchführung schlicht ungeeignet.

Bei fehlendem Zugang zu einem entsprechenden Endgerät kontaktieren Sie bitte frühzeitig die Prüfungsverantwortlichen.

- **Netz:** Stabil genug? Belastbar genug?

Wenn Sie keinen Zugang zu einer ausreichend stabilen Internetverbindung haben, setzen Sie sich frühzeitig mit Ihren Prüfungsverantwortlichen in Verbindung.

- **VPN:** Der Prüfungs-ILIAS ist nur im HSBI-VPN erreichbar.

Installieren Sie den VPN-Client (Anleitung: hsbi.de/dvz/faq/cat/7) und testen Sie im Vorfeld der Prüfung bei aktivierter VPN-Verbindung den Zugang zur Prüfungsplattform eassessment.hsbi.de. Zugangsdaten wie im normalen ILIAS.

Achtung: Auch wenn Sie sich in den Räumen der HSBI befinden, müssen Sie oft die VPN-Verbindung aktivieren, um Zugang zur Prüfungsplattform zu erhalten.

- **Browser:** Nutzen Sie einen der Standardbrowser (Edge, Firefox, Safari, Chrome/Chromium) in der Standardeinstellung: insbesondere JavaScript und Cookies müssen aktiviert/erlaubt sein.

Deaktivieren Sie sämtliche Browser-Erweiterungen wie z.B. Ad-Blocker (AdBlockPlus, uBlock, ...) oder JavaScript-Blocker (No-Script, Ghostery, ...) für den Prüfungszeitraum.

Nutzen Sie Ihren Browser nicht im Privacy-Modus!

- **HSBI-Zugangsdaten:** Username, Passwort

Bei der Durchführung der Prüfung als Open-Book-Ausarbeitung führen Sie die Prüfung auf Ihrer eigenen Hardware im Home-Office durch. Auch hier müssen Sie sich am Prüfungs-ILIAS anmelden. Dazu benötigen Sie ihre HSBI-Zugangsdaten, mit denen Sie sich auch im “normalen” ILIAS anmelden.

16.3. Bearbeitung des E-Assessment

1. Lesen Sie sich die Hinweise auf der Startseite durch
2. Bearbeiten Sie die Aufgaben in **einem einzigen** Browser-Tab

Öffnen Sie die Aufgaben NICHT in parallelen Tabs! Es kann sonst zu Fehlfunktionen von ILIAS kommen.

Bewegen Sie sich nicht per Browser-Navigation (“vor”, “zurück” im Browser) durch die Aufgaben, sondern nutzen Sie dafür die Buttons “nächste Frage”, “Weiter” oder “Zurück” vom ILIAS!

3. Hinweis zu Anzeige der restlichen Bearbeitungsdauer

Wenn Sie den Browser bzw. das Tab mit der Prüfung im Laufe der Prüfung verlassen, wird Ihnen bei der Rückkehr unter Umständen eine falsche restliche Bearbeitungsdauer angezeigt. Sie können die Anzeige korrigieren/aktualisieren, indem Sie einfach zu einer vorigen oder nächsten Aufgabe navigieren.

Hinweis: Die restliche Bearbeitungsdauer wird im Test nur dann angezeigt, wenn diese Funktion von den Prüfenden aktiviert wurde.

4. Parallel zum E-Assessment läuft eine Zoom-Session, dort können Sie Fragen stellen
5. Verbindungsprobleme (Home-Office):
 - Bei kurzzeitigen Verbindungsabbrüchen loggen Sie sich einfach wieder ein
 - Wenn die Probleme länger dauern, gilt der Versuch als nicht unternommen (außer Sie haben die Probleme aktiv herbeigeführt, dann kann das als Täuschungsversuch gewertet werden, vgl. RPO §22a (4))

16.4. Fragetypen-Demo

In Ihrem ILIAS-Kurs finden Sie eine **Fragetypen-Demo** mit den wichtigsten Fragetypen. Machen Sie sich mit der Mechanik der Fragetypen vertraut und schauen Sie sich die Kommentare bei den einzelnen Aufgaben an. Sie können die Demo bei Bedarf beliebig oft wiederholen.

16.5. Hinweise zu den Inhalten

- Klausurrelevant: Vorlesung und Praktikum
- Für Verständnis u.U. hilfreich: Studium der vertiefenden Literaturangaben

• Fragen:

- Schauen Sie sich die Challenges und/oder Quizzes an ...
- Schauen Sie sich die Praktikumsaufgaben an ...
- Überlegen Sie sich, was zu einem Themengebiet im Rahmen einer Prüfung möglich ist und (wie) gefragt werden könnte :)

Können vor Kennen :-)

16.6. Beispiele für mögliche Fragen

TODO

Quellen

Aho, A. V., M. S. Lam, R. Sethi, J. D. Ullman, und S. Bansal. 2023. *Compilers: Principles, Techniques, and Tools, Updated 2nd Edition by Pearson*. Pearson India. <https://learning.oreilly.com/library/view/compilers-principles-techniques/9789357054881/>.

Allain, A., und A. Hoffer. o. J. „C Programming and C++ Programming“. Zugriffen 16. September 2024. <https://www.cprogramming.com/>.

Breymann, U. 2011. *Der C++ Programmierer*. 2. Auflage. Hanser.

„C and C++ Reference“. o. J. Cppreference.com. Zugriffen 16. September 2024. <https://en.cppreference.com/w/>.

Grune, D., K. van Reeuwijk, H. E. Bal, C. J. H. Jacobs, und K. Langendoen. 2012. *Modern Compiler Design*. Springer.

Hopcroft, J. E., R. Motwani, und J. D. Ullman. 2003. *Einführung in die Automatentheorie, formale Sprachen und Komplexitätstheorie*. I theoretische informatik. Pearson Education Deutschland GmbH.

Mogensen, T. 2017. *Introduction to Compiler Design*. Springer. <https://doi.org/10.1007/978-3-319-66966-3>.

Nystrom, R. 2015. „The Hardest Program I’ve Ever Written“. <https://journal.stuffwithstuff.com/2015/09/08/the-hardest-program-ive-ever-written/>.

Nystrom, R. 2021. *Crafting Interpreters*. Genever Benning. <https://github.com/munificent/craftinginterpreters>.

Parr, T. 2010. *Language Implementation Patterns*. Pragmatic Bookshelf. <https://learning.oreilly.com/library/view/language-implementation-patterns/9781680500097/>.

Parr, T. 2014. *The Definitive ANTLR 4 Reference*. Pragmatic Bookshelf. <https://learning.oreilly.com/library/view/the-definitive-antlr/9781941222621/>.

Torczon, L., und K. Cooper. 2012. *Engineering a Compiler*. Morgan Kaufmann. <https://learning.oreilly.com/library/view/engineering-a-compiler/9780080916613/>.

Wadler, P. 2003. „A Prettier Printer“. *The Fun of Programming, Cornerstones of Computing*, 223–43. <https://homepages.inf.ed.ac.uk/wadler/papers/prettier/prettier.pdf>.



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Exceptions:

- Abzählreim “99 Bottles of Beer” nach [“Lyrics of the song 99 Bottles of Beer”](#) on 99-bottles-of-beer.net
- Screenshot of [“Language Brainfuck”](#) by Michal Wojciech Tarnowski on 99-bottles-of-beer.net
- [“Language Haskell”](#) by Iavor on 99-bottles-of-beer.net
- [A Map of the Territory \(mountain.png\)](#) by [Bob Nystrom](#) on Github.com ([MIT](#))
- [“Language Java”](#) by Sean Russell on 99-bottles-of-beer.net
- [“Language C”](#) by Bill Wein on 99-bottles-of-beer.net
- [“Language Prolog”](#) by M@ on 99-bottles-of-beer.net

Last modified: ea9517d 2026-06-23 build(deps): bump actions/checkout from 6 to 7 (#435)